MindVideo AI

هندسة النماذج اللغوية الكبيرة

من المحول الأول إلى نظام إنتاجي متكامل

مرجع هندسي عربي يجمع الأسس الرياضية، والمعماريات، وبناء النماذج، والتكييف، والتقييم، والتطبيقات، والنشر.



الأسس الرياضية | المعماريات | بناء النماذج | التكييف | التقييم | التطبيقات | النشر

المؤلف

م/ أحمد صالح المغيز

2026

كيف تستخدم هذا الكتاب؟

- **ابدأ بالحدس:** اقرأ السؤال الافتتاحي والخريطة الذهنية في الفصل، ثم ارسم الفكرة بكلماتك.
- **انتقل إلى الآلية:** تتبّع المعادلات والأمثلة والكود خطوة بخطوة؛ لا تكتفِ بتشغيل الشيفرة دون فهم شكل المدخلات والمخرجات.
- **اختر مستوى التعمّق:** صناديق التفاصيل والمراجع مناسبة للباحث أو للقارئ الذي يريد استكشاف الفرضيات والبدائل.
- **نفّذ التجارب:** مجلد `book_code` مستقل، وأمثلته صغيرة ومهيأة للعمل محليًا وفي Colab وKaggle دون تنزيل بيانات خاصة.
- **تحقق من الإصدارات:** مجال LLM سريع التغير؛ ثبّت إصدارات البيئة وسجّلها قبل إعادة إنتاج النتائج.

المتطلبات المقترحة

Python 3.13 أو أحدث، أساسيات Python والجبر الخطي، وPyTorch حديث للتجارب العميقة. تعمل الأمثلة التعليمية على CPU، ويمكن الاستفادة من GPU عند توفره. لا يحتاج تشغيل دفاتر البداية إلى مفاتيح API أو بيانات اعتماد.

فهرس الكتاب

01. الجزء 01 — الأساسيات (Foundations)

- **01.01** — الفصل الأول: مقدّمات رياضية سريعة — اللغة السرّية التي يتحدّثها كل نموذج ذكاء اصطناعي
- **01.02** — الفصل الثاني: الشبكة العصبية الأولى من الصفر — من نقطة واحدة إلى شبكة تتعلّم
- **01.03** — الفصل الثالث: Backpropagation والتحسين (Optimization)
- **01.04** — الفصل الرابع: التقييم والتعميم — Overfitting وRegularization وPerplexity
- **01.05** — الفصل الخامس: أساسيات PyTorch العملية والبنية التحتية الحسابية

02. الجزء 06 — التضمينات (Embeddings)

- **02.06** — الفصل السادس: Tokenization — كيف نُحوّل اللغة إلى أرقام؟
- **02.07** — الفصل السابع: Word2Vec — كيف تتعلّم الآلة معنى الكلمات؟
- **02.08** — الفصل الثامن: من الكلمة إلى الجملة — RNN وLSTM وGRU

03. الجزء 09 — المحوّلات: البنية الأساسية (Transformers Core)

- **03.09** — الفصل التاسع: الدافع وراء Self-Attention
- **03.10** — الفصل العاشر: آلية الانتباه الذاتي — قلب الثورة النابض

- **03.11** — الفصل الحادي عشر: بنية Transformer الكاملة
- **03.12** — الفصل الثاني عشر: التنويعات المعمارية — Encoder-Decoder، Decoder-Only، Encoder-Only
- **03.13** — الفصل الثالث عشر: نحو الاستدلال السريع — KV Caching وFlash Attention ومقدمة في Quantization
- **03.14** — الفصل الرابع عشر: تحليل التعقيد الحسابي والذاكرة — فهم Transformer بلغة FLOPs والبايتات

04. الجزء 15 — المعماريات المتقدمة والحديثة (Modern Architectures)

- **04.15** — الفصل الخامس عشر: تحسينات الانتباه الحديثة — MQA/GQA، ALiBi، RoPE، وSliding Window Attention
- **04.16** — الفصل السادس عشر: Mixture of Experts — فصل الحجم عن التكلفة
- **04.17** — الفصل السابع عشر: نماذج الفضاء الحالة (State Space Models) وMamba
- **04.18** — الفصل الثامن عشر: تقنيات معمارية حديثة — Multi-Token Prediction، SwiGLU، RMSNorm، وYaRN/NTK-aware Scaling
- **04.19** — الفصل التاسع عشر: دراسة حالة تطبيقية — تشريح معماري لـ Mistral، LLaMA 3، وDeepSeek-V3

05. الجزء 20 — بناء نموذج لغوي من الصفر (Build an LLM)

- **05.20** — الفصل العشرون: خط أنابيب البيانات — من فوضى الإنترنت إلى وقود تدريب نموذج لغوي عملاق
- **05.21** — الفصل الحادي والعشرون: تصميم Tokenizer مخصص على نطاق واسع
- **05.22** — الفصل الثاني والعشرون: تصميم المعمارية وScaling Laws — فن اختيار الحجم الأمثل لنموذجك
- **05.23** — الفصل 23: التدريب الموزّع — ZeRO، 3D Parallelism، Data/Tensor/Pipeline Parallelism، وFSDP
- **05.24** — الفصل 24: حلقة Pretraining عمليًا — إدارة نقاط الحفظ، تشخيص Loss Spikes، وGradient Checkpointing
- **05.25** — الفصل 25: من Base إلى نموذج قابل للاستخدام — Supervised Fine-Tuning والبيانات الاصطناعية
- **05.26** — الفصل 26: مشروع تطبيقي متكامل — تدريب Mini-LLM من الصفر

06. الجزء 27 — إعادة استخدام النماذج وتطويرها (Model Reuse & Adaptation)

- **06.27** — الفصل 27: Transfer Learning & Continued Pretraining
- **06.28** — الفصل 28: دمج النماذج (Model Merging)
- **06.29** — الفصل 29: التقطير المعرفي (Knowledge Distillation)
- **06.30** — الفصل 30: التقليم (Pruning)
- **06.31** — الفصل 31: تجميع معماري (Architecture Stitching)
- **06.32** — الفصل 32: مصفوفة القرار الاستراتيجي — متى تختار ماذا؟

07. الجزء 33 — الضبط والمحاذاة (Fine-tuning & Alignment)

- **07.33** — الفصل 33: هندسة الأوامر (Prompt Engineering)
- **07.34** — الفصل 34: الضبط الفعّال للمعاملات — PEFT وLoRA وQLoRA
- **07.35** — الفصل 35: التعلّم المعزَّز من التغذية الراجعة البشرية (RLHF)
- **07.36** — الفصل 36: التحسين المباشر للتفضيلات (DPO)
- **07.37** — الفصل 37: تقنيات محاذاة حديثة — KTO وORPO وConstitutional AI

08. الجزء 38 — النماذج متعددة الوسائط (Multimodal Models)

- **08.38** — الفصل 38: استراتيجيات التوليد (Decoding Strategies)
- **08.39** — الفصل 39: Vision Transformer (ViT) — من البكسل إلى الرمز
- **08.40** — الفصل 40: CLIP — عندما تتعلّم الصورة والكلمة لغة مشتركة
- **08.41** — الفصل 41: نماذج الرؤية واللغة (VLMs) — LLaVA وFlamingo وBLIP-2
- **08.42** — الفصل 42: معالجة الصوت والكلام — Whisper وTTS وSpeech-to-Speech
- **08.43** — الفصل 43: نماذج الانتشار (Diffusion Models) — التوليد من الضوضاء
- **08.44** — الفصل 44: النماذج متعددة الوسائط الموحّدة (Unified Multimodal Models)

09. الجزء 45 — التقييم والاختبار (Evaluation & Testing)

- **09.45** — الفصل 45: أسس قياس أداء النماذج اللغوية (Foundations of LLM Evaluation) — نسخة مُعاد صياغتها
- **09.46** — الفصل 46: معايير القياس الكبرى (Major Benchmarks: MMLU, HellaSwag, GSM8K, HumanEval)
- **09.47** — الفصل 47: التقييم بواسطة النماذج اللغوية (LLM-as-a-Judge)
- **09.48** — الفصل 48: تقييم أنظمة RAG (RAG Evaluation)
- **09.49** — الفصل 49: تقييم السلامة والتحيّز (Safety & Bias Evaluation)
- **09.50** — الفصل 50: خط تقييم مستمر (CI/CD للتقييم)

10. الجزء 51 — التطبيقات والوكلاء (Applications & Agents)

- **10.51** — الفصل 51: تحليل المشاعر واستخراج المعلومات (Sentiment Analysis & NER)
- **10.52** — الفصل 52: الترجمة الآلية (Machine Translation)
- **10.53** — الفصل 53: RAG الكامل (Retrieval-Augmented Generation)
- **10.54** — الفصل 54: التلخيص والإجابة على الأسئلة
- **10.55** — الفصل 55: بناء وكلاء ذكيين (AI Agents)
- **10.56** — الفصل 56: أنظمة Multi-Agent المتعاونة

- **10.57** — الفصل 57: الذاكرة طويلة المدى للمحادثات (Long-term Memory)
- **10.58** — الفصل 58: مشروع تطبيقي — Agent متكامل يستخدم RAG + أدوات خارجية

11. الجزء 59 — النشر والتشغيل (Deployment & LLMOps)

- **11.59** — الفصل 59: تحسين الاستدلال — Quantization وSpeculative Decoding وContinuous Batching
- **11.60** — الفصل 60: خوادم الاستدلال — vLLM وTGI وTensorRT-LLM وllama.cpp
- **11.61** — الفصل 61: بنية LLMOps — Versioning وCanary Deployment وSemantic Caching
- **11.62** — الفصل 62: كشف انحراف البيانات والمفهوم — الحارس الصامت لجودة النموذج في الإنتاج
- **11.63** — الفصل 63: الأمان والحوكمة — حصّن نموذجك قبل أن يُستغل
- **11.64** — الفصل 64: هندسة التكلفة — Self-Hosting مقابل API، والتوجيه الذكي للنماذج

12. الجزء 65 — الاتجاهات الحديثة (Frontier Trends)

- **12.65** — الفصل 65: نماذج الاستدلال الحديثة — Test-Time Compute Scaling وثورة "التفكير قبل الإجابة"
- **12.66** — الفصل 66: السياق الطويل جدًا (Very Long Context)
- **12.67** — الفصل 67: الوكلاء المستقلون المتقدمون (Agentic AI)
- **12.68** — الفصل 68: الكفاءة كاتجاه بحثي — النماذج اللغوية الصغيرة (SLMs: Phi, Gemma, Qwen-Small)
- **12.69** — الفصل 69: خاتمة — منهجية متابعة الأبحاث ومواكبة التسارع التقني

الملاحق

- **م.01** — الملحق (أ): قاموس المصطلحات الشامل
- **م.02** — الملحق (د): خارطة طريق 5 مشاريع عملية لبناء Portfolio احترافي
- **م.03** — الملحق (هـ): قائمة قراءة مرتبة زمنيًا — تتبّع تطور الحقل
- **م.04** — الملحق (و): دليل الأدوات الشامل — مقارنة الأطر والمكتبات
- **م.05** — الملحق (ز): مستودع GitHub المرافق — البنية والمحتوى

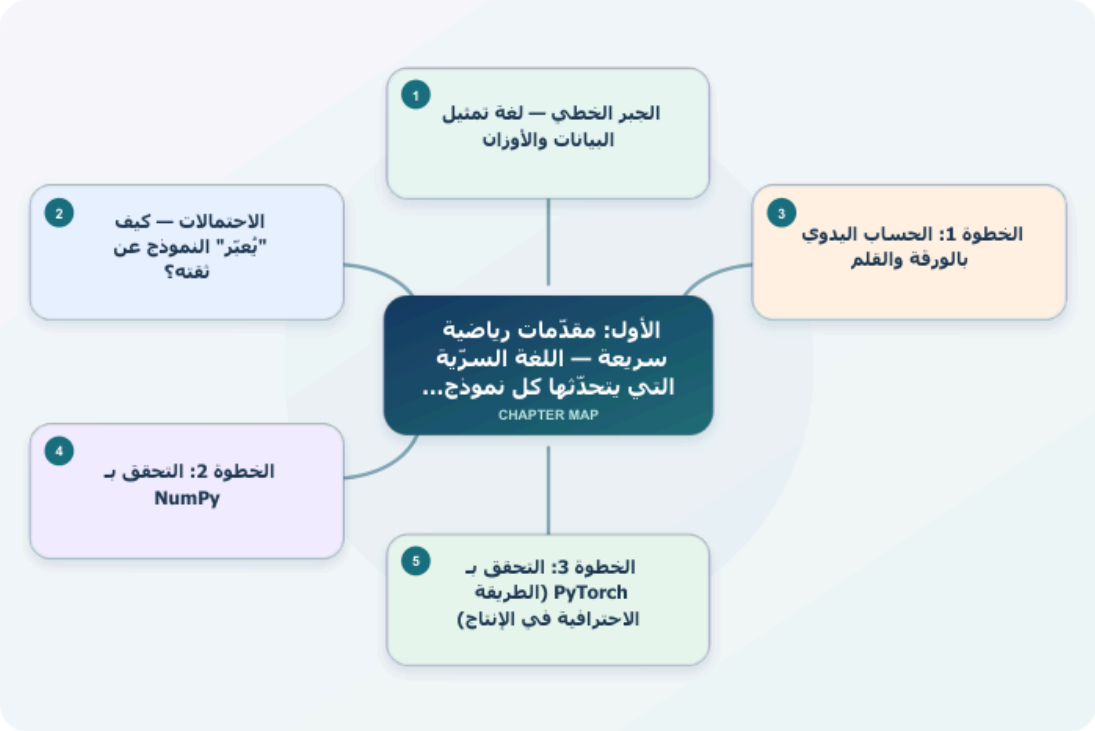
إرشادات السلامة وإعادة الإنتاج

الأكواد التعليمية مصممة لإظهار الفكرة، وليست بديلًا عن مراجعة أمنية أو تقييم مستقل قبل إدخال نموذج أو وكيل إلى بيئة إنتاجية. لا تضع أسرارًا أو بيانات شخصية في دفاتر عامة، ولا تشغّل نماذج أو أدوات خارجية قبل فهم صلاحياتها ومصادر بياناتها.

الجزء 01 — الأساسيات (Foundations)

01 / 69

الفصل الأول: مقدّمات رياضية سريعة — اللغة السرّية التي يتحدّثها كل نموذج ذكاء اصطناعي



جزء الكتاب: الجزء الأول — الأساسيات (Foundations) **الفصل:** 1 من 69 **نوع هذا الفصل:** 📦 فصل تأسيسي — حجر الأساس الذي يُبنى عليه كل فصل لاحق في الكتاب

🟢 1.0 لماذا يهمّك هذا الفصل؟

لنبدأ بسؤال قد يبدو غريبًا: **عندما يُجيبك ChatGPT على سؤالك، ما الذي يحدث فعليًا "تحت غطاء المحرك"؟**

الإجابة الصادمة في بساطتها: **ملايين — بل مليارات — من عمليات الضرب والجمع البسيطة جدًا**، مُنظَّمة بذكاء في شكل مصفوفات، مع قرار نهائي واحد يُتَّخذ عبر دالة احتمالية اسمها Softmax. هذا كل شيء. لا يوجد "فهم" بالمعنى الذي نتخيله، ولا "وعي" خفي — فقط رياضيات دقيقة، مُطبَّقة على نطاق هائل، بكفاءة مذهلة.

والآن سؤال أكثر إلحاحًا: **لماذا تحتاج أنت تحديدًا — المهندس أو الباحث الذي يريد أن يفهم أو يبني نماذج لغوية كبيرة — أن تتقن هذه الأساسيات الرياضية بدل القفز مباشرة لكتابة `model.fit()`؟**

تخيّل طبيب جراحة لا يعرف التشريح، يعتمد فقط على "حدسه" أثناء العملية. قد ينجح أحيانًا بالصدفة، لكنه سيفشل بشكل كارثي أمام أول حالة معقدة، ولن يفهم أبدًا **لماذا** فشل. هذا بالضبط حال مهندس الذكاء الاصطناعي الذي يتعامل مع المكتبات كـ "صندوق أسود" سحري: سيبني نماذج تعمل أحيانًا، لكنه سيقف عاجزًا تمامًا عند أول مشكلة غير اعتيادية — تدرجات متفجرة، خسارة لا تنخفض، نموذج يُخرج قيم `NaN` فجأة — لأنه لا يملك **اللغة الرياضية** اللازمة لتشخيص المشكلة.

في هذا الفصل، لن نغرق في رياضيات أكاديمية مجردة بلا هدف. بل سنبني **بالضبط** الأدوات الثلاث التي سنستخدمها حرفيًا في كل فصل قادم من هذا الكتاب:

1. **الجبر الخطي** → اللغة التي تُكتَب بها كل بنية عصبية (من Perceptron البسيط إلى GPT-4)
2. **الاحتمالات** → اللغة التي يُعبّر بها النموذج عن "مدى ثقته" في كل إجابة ممكنة
3. **التفاضل** → اللغة التي تسمح للنموذج بـ "التعلّم من أخطائه" عبر Backpropagation

بنهاية هذا الفصل، ستكون قادرًا على: - شرح لماذا يُعتبر الضرب النقطي (Dot Product) "مقياس تشابه" وليس مجرد عملية حسابية عشوائية - تطبيق Softmax و Cross-Entropy يدويًا بالأرقام، ثم برمجيًا، وفهم **لماذا** تُستخدمان تحديدًا في نماذج اللغة - تجنّب أشهر خطأ برمجي يرتكبه المبتدئون عند تطبيق Softmax (ويُسأل عنه في كل مقابلة عمل تقريبًا) - فهم Chain Rule بعمق كافٍ يجعلك تتنبأ بسلوك Backpropagation قبل أن ندرسها رسميًا في الفصل الثالث

احتفظ بهذه الخريطة الثلاثية في ذهنك — فكل فصل لاحق في الكتاب، من Self-Attention إلى RLHF، هو في جوهره **تركيب جديد** لهذه الأدوات الثلاث نفسها.

🟡 1.2 الشرح المتدرّج: من الحدس إلى الكود

1.2.1 الجبر الخطي — لغة تمثيل البيانات والأوزان

🔷 المتجهات (Vectors): أكثر من مجرد "قائمة أرقام"

اسأل نفسك: **عندما يُحوّل نموذج لغوي كلمة "ملك" إلى متجه مثل `[0.2, -0.5, 0.8, ..., 0.1]` بطول 768 أو 4096 رقمًا، ماذا يعني هذا فعليًا؟**

الإجابة الحدسية: **هذا المتجه هو "إحداثيات" الكلمة في فضاء هندسي عالي الأبعاد**، حيث كل بُعد يُمثّل (نظريًا) جانبًا دلاليًا خفيًا من معنى الكلمة — قد يكون بُعد ما مرتبطًا بـ"الملكية/السلطة"، وآخر بـ"الجنس"، وآخر

بـ"الزمن التاريخي". الكلمات ذات المعاني المتشابهة تقع **قريبة من بعضها** في هذا الفضاء، تمامًا كما تقع مدينتا دمشق وبيروت قريبتين على خريطة جغرافية، بينما تبعد كلتاهما عن طوكيو.

```
import numpy as np

# تخيّل فضاءً مبسّطًا بُعدين فقط لأغراض التوضيح البصري
king   = np.array([0.90, 0.85])   # "ملك": قيمة عالية في [السلطة, الذكورة]
queen  = np.array([0.88, 0.15])   # "ملكة": سلطة عالية، لكن أنثى
man    = np.array([0.10, 0.90])   # "رجل": سلطة منخفضة، ذكورة عالية
woman  = np.array([0.08, 0.12])   # "امرأة": سلطة منخفضة، أنثى

# العملية السحرية الشهيرة: king - man + woman ≈ queen ؟
result = king - man + woman
print("الناتج:", result)          # تقريبًا: [0.07 ,0.88]
print("queen فعليًا:", queen)      # [0.88, 0.15]  ← قريب جدًا!
```

> 💭 **سؤال استفهامي يستحق التوقف:** هل هذا يعني أن النموذج "يفهم" مفهوم النوع الاجتماعي والسلطة كما يفهمه الإنسان؟ الإجابة الدقيقة: **لا** — النموذج لا يملك أي فهم واعٍ، لكنه اكتشف إحصائيًا، من خلال ملايين السياقات التي ظهرت فيها هذه الكلمات معًا، أن هذا "الترتيب الهندسي" للمتجهات يُنتج تنبؤات صحيحة بشكل متّسق. هذه هي "الخدعة" المذهلة الكامنة خلف كل نماذج اللغة: **تمثيل هندسي بحت يُحاكي بنية المعنى**، دون أي فهم حقيقي بالمعنى الفلسفي.

🔷 المصفوفات (Matrices) والضرب المصفوفي: محرّك كل طبقة عصبية

كل طبقة في أي شبكة عصبية — من أبسط Perceptron إلى أعقد طبقة في GPT-4 — هي في جوهرها **عملية ضرب مصفوفي واحدة**، تليها دالة تفعيل غير خطية. هذه هي الحقيقة الوحيدة التي يجب أن تحفظها عن ظهر قلب:

$$Y = XW + b$$

حيث X مصفوفة المدخلات (Batch of Inputs)، و W مصفوفة الأوزان **القابلة للتعلّم**، و b متجه الإزاحة (Bias).

```
import numpy as np

# مثال محسوس: دفعة من 3 أمثلة (Batch)، كل مثال له 4 خصائص (Features)
X = np.array([
    [0.1, 0.5, 2.0, 1.0],  # المثال الأول
    [0.9, 1.2, 0.8, 0.3],  # المثال الثاني
    [1.5, 0.3, 0.4, 2.1],  # المثال الثالث
])  # shape: (3, 4) → (batch_size, input_features)

# طبقة تُحوّل كل مثال من 4 خصائص إلى تمثيل بـ 2 خصيصة جديدة
W = np.array([
    [0.5, -0.2],
```

```
    [0.1,  0.4],
    [-0.3, 0.6],
    [0.8, -0.1],
])  # shape: (4, 2) → (input_features, output_features)

b = np.array([0.1, -0.1])  # shape: (2,)

Y = X @ W + b  # @ هو رمز الضرب المصفوفي في بايثون الحديث
print("شكل المخرجات:", Y.shape)   # (3, 2)
print(Y)
```

🔑 **قاعدة ذهبية يجب أن تُحفظ:** عند ضرب مصفوفتين $A_{(m \times n)} \times B_{(n \times p)}$، يجب أن يتطابق **العدد الداخلي** (n) تمامًا، والناتج يأخذ **الأبعاد الخارجية** $(m \times p)$. **90% من أخطاء "Shape Mismatch" التي ستواجهها طوال مسيرتك في بناء النماذج العصبية سببها انتهاك هذه القاعدة البسيطة.**

🔷 الضرب النقطي (Dot Product) كمقياس تشابه — السرّ الذي يُفسّر كل شيء لاحقًا

هذه بالتحديد هي النقطة التي ستجعل فهمك لـ **Self-Attention** في الفصل العاشر أسهل بكثير. اسأل نفسك: **لماذا، عندما نريد "قياس تشابه" بين متجهين في عالم الذكاء الاصطناعي، نلجأ غالبًا للضرب النقطي؟**

$$a \cdot b = \sum_{i=1}^{n} a_i b_i = \mid a \mid \mid b \mid \cos(\theta)$$

تأمّل الصيغة الثانية جيدًا: الضرب النقطي يساوي حاصل ضرب أطوال المتجهين في **جيب تمام الزاوية بينهما**. فإذا كان المتجهان يشيران لنفس الاتجاه تقريبًا $(\theta \approx 0)$، فإن $\cos(\theta) \approx 1$ وتكون النتيجة كبيرة وموجبة. إذا كانا متعامدين $(\theta = 90°)$، تكون النتيجة صفرًا (لا علاقة بينهما). وإذا كانا متعاكسين تمامًا، تكون النتيجة سالبة.

```
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
"""تُطبّع الضرب النقطي بأطوال المتجهين لعزل تأثير الزاوية فقط"""
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))


vec_king  = np.array([0.90, 0.85])
vec_queen = np.array([0.88, 0.15])
vec_apple = np.array([-0.70, 0.05])   # كلمة "تفاحة" - غير ذات صلة

print(f"تشابه (ملك, ملكة) = {cosine_similarity(vec_king, vec_queen):.3f}")
print(f"تشابه (ملك, تفاحة) = {cosine_similarity(vec_king, vec_apple):.3f}")
```

```
تشابه (ملك, ملكة) = 0.672
تشابه (ملك, تفاحة) = -0.653
```

> 🎯 **هذا بالضبط الأساس الرياضي الذي سيُبنى عليه الفصل العاشر بالكامل:** عندما نحسب QK^T في آلية الانتباه الذاتي، نحن في جوهر الأمر **نقيس تشابه كل كلمة (Query) بكل كلمة أخرى (Key)** باستخدام نفس هذا المبدأ بالضبط. احفظ هذه الفكرة جيدًا — ستُقابلها مرارًا وتكرارًا طوال الكتاب.

1.2.2 الاحتمالات — كيف "يُعبّر" النموذج عن ثقته؟

🔷 من الأرقام الخام إلى الاحتمالات: لماذا نحتاج Softmax؟

تخيّل أن نموذج لغة يحاول التنبؤ بالكلمة التالية بعد "القطة جلست على ال..."، ولديه 5 كلمات مرشحة فقط (لتبسيط المثال)، وأنتج لكل منها "درجة خام" (Logit) غير مُقيَّدة:

```
logits = np.array([2.0, 1.0, 0.1, -1.0, 3.5])  # ["أريكة", "سماء", "طاولة", "كرسي", "سجادة"]
```

السؤال الجوهري: **كيف نحوّل هذه الأرقام الخام (التي يمكن أن تكون أي قيمة، موجبة أو سالبة) إلى شيء يُمكن تفسيره كـ "احتمال"؟** يجب أن يحقق هذا التحويل شرطين أساسيين لا يمكن المساومة عليهما: 1. كل قيمة يجب أن تكون **موجبة** (لا معنى لاحتمال سالب) 2. **مجموع كل القيم يجب أن يساوي 1.0** تمامًا (توزيع احتمالي صحيح)

دالة **Softmax** تحل هذا بأناقة رياضية:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

```
def softmax_naive(z: np.ndarray) -> np.ndarray:
"""التطبيق الساذج المباشر - احذر، سنرى مشكلته بعد قليل!"""
    exp_z = np.exp(z)
    return exp_z / np.sum(exp_z)


probs = softmax_naive(logits)
print("الاحتمالات:", np.round(probs, 3))
print("المجموع:", probs.sum())  # يجب أن يكون 1.0 بالضبط
```

```
الاحتمالات: [0.692 0.010 0.029 0.072 0.196]
المجموع: 1.0
```

لاحظ كيف أن "أريكة" (التي كانت لها أعلى Logit = 3.5) حصلت على احتمال 69.2% — الأسيّة (e^x) تُضخّم الفروقات بين القيم الكبيرة والصغيرة، مما يجعل Softmax "حادة" نسبيًا في تفضيلها للقيم الأعلى، وليس مجرد

توزيع نسبي خطي بسيط.

⚠️ المشكلة الخفية: الانفجار العددي (Numerical Overflow)

والآن، لنجرّب نفس الدالة لكن بقيم واقعية أكثر (كما تظهر فعليًا في نماذج حقيقية):

```
large_logits = np.array([1000.0, 1001.0, 999.0])
probs = softmax_naive(large_logits)
print(probs)
```

```
RuntimeWarning: overflow encountered in exp
RuntimeWarning: invalid value encountered in true_divide
[nan nan nan]
```

كارثة! لماذا حدث هذا؟ لأن `np.exp(1000)` ينتج رقمًا أكبر بكثير من أقصى قيمة يمكن تمثيلها بصيغة `float64` (حوالي 1.8×10^{308})، فيحدث **فيضان (Overflow)**، وتتحوّل النتيجة إلى `inf`، وعند قسمة `inf/inf` نحصل على `NaN` (Not a Number) — وهو بالضبط ذلك الخطأ المرعب الذي سيُدمّر تدريب نموذجك بالكامل إذا ظهر في منتصف Training Loop تستغرق أيامًا!

الحل: خدعة الاستقرار العددي (Numerical Stability Trick)

الحل الرياضي أنيق جدًا: نطرح أكبر قيمة في المصفوفة من كل القيم قبل حساب الأسّ. رياضيًا، هذا **لا يُغيّر النتيجة النهائية إطلاقًا** (لأن القسمة تُلغي أي ثابت مشترك)، لكنه يضمن أن أكبر قيمة أسّية ممكنة هي $e^0 = 1$ فقط:

```
def softmax_stable(z: np.ndarray) -> np.ndarray:
    """
التطبيق الصحيح والآمن عدديًا - استخدم هذا دائمًا في الإنتاج
c لأي ثابت softmax(z) = softmax(z - c) :الإثبات الرياضي
    """
    z_shifted = z - np.max(z)    # الآن أكبر قيمة = 0، والباقي سالب أو صفر
    exp_z = np.exp(z_shifted)    # exp(0)=1 كحد أقصى، لا فيضان أبدًا!
    return exp_z / np.sum(exp_z)


large_logits = np.array([1000.0, 1001.0, 999.0])
print(softmax_stable(large_logits))
# [0.0900 0.6652 0.2447] ← نتيجة صحيحة تمامًا، بلا أي NaN!
```

> 💡 **نصيحة حاسمة:** هذه ليست "حيلة تجميلية" بل **ضرورة هندسية مطلقة**. كل مكتبة احترافية (NumPy's، TensorFlow، PyTorch، `scipy.special.softmax`) تُطبّق هذه الخدعة داخليًا تلقائيًا — لكن **إذا كتبت Softmax بنفسك يدويًا ولو لمرة واحدة** (كما سنفعل لاحقًا عند بناء مكوّنات Transformer من

الصفر)، يجب أن تتذكر هذه الخطوة دائمًا. هذا **سؤال مقابلة كلاسيكي** يُستخدم لفرز المرشحين الذين "يفهمون فعليًا" ممن "يستخدمون المكتبات فقط دون فهم".

🔷 Cross-Entropy: كيف نقيس "مدى خطأ" تنبؤات النموذج؟

الآن لدينا توزيع احتمالي من النموذج. لكن كيف نُخبره **بالضبط** كم كان مخطئًا، بطريقة قابلة للاشتقاق تسمح بتحسينه عبر Gradient Descent؟

تخيّل أن الكلمة الصحيحة فعليًا كانت "أريكة" (الفهرس الرابع في قائمتنا، index=4). نُمثّل هذا بـ **One-Hot Encoding**:

```
true_label = np.array([0, 0, 0, 0, 1])  # 1 في موضع "أريكة" فقط، صفر في الباقي
predicted  = np.array([0.196, 0.072, 0.029, 0.010, 0.692])  # من Softmax السابقة
```

صيغة **Cross-Entropy** (الإنتروبيا المتقاطعة):

$$H(y, \hat{y}) = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

```
def cross_entropy_loss(y_true: np.ndarray, y_pred: np.ndarray, eps: float = 1e-12) -> float:
    """
    eps صغيرة جدًا تُضاف لتجنّب log(0) الذي ينتج -inf
(مصيدة شائعة أخرى يجب الانتباه لها!)
    """
    y_pred_clipped = np.clip(y_pred, eps, 1.0 - eps)
    return -np.sum(y_true * np.log(y_pred_clipped))


loss = cross_entropy_loss(true_label, predicted)
print(f"قيمة الخسارة: {loss:.4f}")   # 0.3677
```

لماذا هذه الصيغة تحديدًا وليس مثلًا مجرد الفرق المطلق |y - ŷ|؟ لأن بما أن `y_true` هي One-Hot (صفر في كل مكان إلا موضع الإجابة الصحيحة)، فإن الصيغة بأكملها تنهار عمليًا إلى:

$$H = -\log(\hat{y}_{correct})$$

أي: **نحاسب النموذج فقط على مدى ثقته بالإجابة الصحيحة**، متجاهلين تمامًا توزيع احتمالاته على الإجابات الخاطئة. والأهم: دالة $-\log(x)$ تتصرف بذكاء شديد — عندما يكون $\hat{y}_{correct}$ قريبًا من 1 (النموذج واثق وصحيح)،

تكون الخسارة قريبة من صفر. لكن عندما يقترب $\hat{y}_{correct}$ من صفر (النموذج واثق بشدة... لكن **خاطئ**)، تنفجر الخسارة نحو **ما لا نهاية**:

```python
import matplotlib.pyplot as plt

confidence = np.linspace(0.01, 1.0, 100)
loss_values = -np.log(confidence)

plt.plot(confidence, loss_values)
plt.xlabel("ثقة النموذج بالإجابة الصحيحة")
plt.ylabel("قيمة Cross-Entropy Loss")
plt.title("كلما قلّت ثقة النموذج بالإجابة الصحيحة، تصاعدت العقوبة بشكل حاد")
plt.grid(True)
plt.show()
```

> 🔑 **هذا هو السبب العميق** وراء اختيار Cross-Entropy تحديدًا (وليس مثلًا Mean Squared Error) كدالة خسارة قياسية في كل مهام التصنيف ونماذج اللغة: **إنها تُعاقب الثقة الزائدة الخاطئة بقسوة شديدة**، مما يدفع النموذج ليكون "متحفظًا" إحصائيًا عندما لا يكون متأكدًا، وهذا سلوك مرغوب جدًا.

🔷 KL Divergence: قياس المسافة بين توزيعين احتماليين

سؤال أخير في هذا القسم: **ماذا لو أردنا مقارنة توزيعين احتماليين كاملين ببعضهما (وليس فقط "نقطة واحدة صحيحة" كما في التصنيف)؟** هذا بالضبط ما سنحتاجه لاحقًا في فصل **ضبط التفضيلات (DPO)** في الجزء السابع، عندما نقارن توزيع احتمالات النموذج الجديد بتوزيع النموذج المرجعي.

$$D_{KL}(P \| Q) = \sum_{i} P(i) \log \frac{P(i)}{Q(i)}$$

```python
def kl_divergence(p: np.ndarray, q: np.ndarray, eps: float = 1e-12) -> float:
"""يقيس 'كم من المعلومات نفقدها' عند تقريب التوزيع P بالتوزيع Q"""
    p = np.clip(p, eps, 1.0)
    q = np.clip(q, eps, 1.0)
    return np.sum(p * np.log(p / q))


p_true  = np.array([0.1, 0.2, 0.7])   # التوزيع الحقيقي/المرجعي
q_model = np.array([0.15, 0.25, 0.6])  # تنبؤات النموذج

print(f"KL Divergence: {kl_divergence(p_true, q_model):.4f}")
```

> ⚠️ **ملاحظة دقيقة مهمة:** أي أن — $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ KL Divergence **غير متماثلة (Asymmetric)**، وبالتالي **ليست** مسافة رياضية بالمعنى الصارم (المسافات يجب أن تكون متماثلة). هذا التفصيل يبدو نظريًا الآن، لكنه سيكون محوريًا عندما نناقش لماذا تختار بعض خوارزميات المحاذاة اتجاهًا معينًا لحساب KL دون الآخر.

1.2.3 التفاضل — كيف "يتعلّم" النموذج من أخطائه؟

🔷 من المشتقة البسيطة إلى Gradient

تذكّر من الرياضيات المدرسية: **المشتقة تُخبرك باتجاه ومقدار التغيّر** — إذا كانت $\frac{df}{dx} > 0$، فإن زيادة x قليلًا تزيد $f(x)$. وإذا كانت سالبة، فالعكس.

الآن تخيّل أن لدينا **دالة خسارة** $L(w)$ تعتمد على وزن واحد w في الشبكة. **السؤال الذهبي الذي يُحرّك كل عملية تدريب في تاريخ الذكاء الاصطناعي:** في أي اتجاه يجب أن أُعدّل w لتقليل الخسارة؟

```
import torch

# نُعرّف وزنًا بسيطًا، ونطلب من PyTorch تتبع مشتقته تلقائيًا
w = torch.tensor(3.0, requires_grad=True)

# دالة خسارة تجريبية بسيطة: L(w) = (w - 5)^2
loss = (w - 5) ** 2

loss.backward()  # PyTorch يحسب dL/dw تلقائيًا عبر Autograd!

print(f"القيمة الحالية لـ w: {w.item()}")
print(f"المشتقة dL/dw عند هذه النقطة: {w.grad.item()}")
# dL/dw = 2(w-5) = 2(3-5) = -4
```

```
القيمة الحالية لـ w: 3.0
المشتقة dL/dw عند هذه النقطة: -4.0
```

كيف نُفسّر هذا؟ المشتقة سالبة (-4)، مما يعني أن **زيادة** w ستُقلّل الخسارة (لأننا بعيدون عن القيمة المثلى $w = 5$). قاعدة **الانحدار التدريجي (Gradient Descent)** تستغل هذه المعلومة مباشرة:

$$w_{new} = w_{old} - \eta \cdot \frac{dL}{dw}$$

```
learning_rate = 0.1
with torch.no_grad():  # نُعطّل تتبع المشتقات أثناء التحديث نفسه
```

```
    w -= learning_rate * w.grad

print(f"الوزن بعد خطوة تحديث واحدة: {w.item()}")  # 3.0 - 0.1*(-4) = 3.4
```

لاحظ كيف تحرك w من 3.0 إلى 3.4 — **اقترب أكثر** من القيمة المثلى (5.0)، تمامًا كما توقعنا! كرّر هذه الخطوة آلاف المرات، وستصل تدريجيًا لـ $w \approx 5.0$ حيث تكون الخسارة صفرًا تقريبًا.

🔷 Gradient في الأبعاد المتعددة (Multivariable)

في الشبكات الحقيقية، لا نملك وزنًا واحدًا، بل **ملايين أو مليارات** الأوزان في آن واحد. **Gradient** هو ببساطة تعميم المشتقة لهذه الحالة: متجه يحتوي المشتقة الجزئية (Partial Derivative) بالنسبة لكل وزن على حدة:

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \ldots, \frac{\partial L}{\partial w_n}\right]$$

```
# دالة خسارة تعتمد على وزنين معًا
w1 = torch.tensor(2.0, requires_grad=True)
w2 = torch.tensor(-1.0, requires_grad=True)

loss = (w1 - 4) ** 2 + (w2 + 2) ** 2   # الحد الأدنى عند w1=4, w2=-2

loss.backward()

print(f"∂L/∂w1 = {w1.grad.item()}")   # 2(2-4) = -4
print(f"∂L/∂w2 = {w2.grad.item()}")   # 2(-1+2) = 2
```

> 🔑 **تخيّل Gradient كـ "بوصلة متعددة الأبعاد"** — في فضاء الأوزان شاسع الأبعاد (ملايين الاتجاهات الممكنة)، يُشير متجه Gradient دائمًا نحو اتجاه **أسرع ازدياد** للخسارة. لذلك نتحرك في **عكس** اتجاهه (ومن هنا جاء اسم "الانحدار" Descent).

🔷 قاعدة السلسلة (Chain Rule): المفتاح السحري لـ Backpropagation

هذا هو المفهوم الأهم في هذا الفصل بأكمله، لأنه **الأساس الرياضي الوحيد** الذي يجعل تدريب الشبكات العصبية العميقة (بعشرات أو مئات الطبقات) ممكنًا من الأساس.

اسأل نفسك: **الشبكة العصبية ليست دالة بسيطة** $L(w)$، **بل سلسلة طويلة من الدوال المُتداخلة:** $L(f_n(f_{n-1}(\ldots f_1(x, w_1)\ldots, w_{n-1}), w_n))$. فكيف نحسب تأثير تغيير وزن في **الطبقة الأولى** على الخسارة النهائية التي تُحسب بعد **عشرات الطبقات**؟

قاعدة السلسلة تُعطينا الجواب بأناقة رياضية مذهلة: **مشتقة دالة مُركّبة = حاصل ضرب مشتقات كل طبقة على حدة**.

$$\frac{dL}{dw_1} = \frac{dL}{dh_n} \cdot \frac{dh_n}{dh_{n-1}} \cdots \frac{dh_2}{dh_1} \cdot \frac{dh_1}{dw_1}$$

```python
# مثال توضيحي: سلسلة من 3 دوال متتالية (محاكاة مبسّطة لـ 3 طبقات)
x = torch.tensor(2.0, requires_grad=True)

h1 = x ** 2          # الطبقة الأولى: h1 = x²
h2 = torch.sin(h1)   # الطبقة الثانية: h2 = sin(h1)
L  = h2 ** 3         # الطبقة الثالثة: L = h2³ (دالة الخسارة)

L.backward()

print(f"dL/dx (محسوبة تلقائيًا عبر Chain Rule): {x.grad.item():.4f}")

# التحقق اليدوي بتطبيق Chain Rule خطوة بخطوة:
# dL/dh2 = 3·h2²
# dh2/dh1 = cos(h1)
# dh1/dx = 2x
# dL/dx = 3·h2² · cos(h1) · 2x
import math
h1_val = 2.0 ** 2
h2_val = math.sin(h1_val)
manual_grad = 3 * (h2_val ** 2) * math.cos(h1_val) * 2 * 2.0
print(f"dL/dx (محسوبة يدويًا للتحقق): {manual_grad:.4f}")
```

```
dL/dx (محسوبة تلقائيًا عبر Chain Rule): -4.1023
dL/dx (محسوبة يدويًا للتحقق): -4.1023
```

تطابق تام! هذا بالضبط ما تفعله **Backpropagation** داخل PyTorch: تبني "رسمًا حسابيًا" (Computational Graph) أثناء الـ Forward Pass، ثم تُطبّق Chain Rule تلقائيًا بالعكس (Backward Pass) لحساب تأثير **كل وزن في كل طبقة** على الخسارة النهائية، بغض النظر عن عمق الشبكة.

> 🎯 **هذا بالضبط ما سنُفصّله رياضيًا بعمق أكبر في الفصل الثالث** عندما نبني Backpropagation يدويًا بالكامل بـ NumPy (بدون `autograd`) لنفهم ما يحدث "تحت غطاء المحرك" في PyTorch.

🧪 1.3 مثال محلول خطوة بخطوة (End-to-End)

لنُجمّع كل ما تعلمناه في مثال واحد متكامل: **حساب الخسارة الكاملة لمهمة تصنيف بسيطة، يدويًا بالأرقام أولًا، ثم بـ NumPy، ثم بـ PyTorch** — لنرى كيف تتطابق النتائج الثلاث تمامًا.

السيناريو: نموذج يُصنّف جملة إلى واحدة من 3 فئات: [إيجابي، محايد، سلبي]. الفئة الصحيحة هي "إيجابي" (index=0).

الخطوة 1: الحساب اليدوي بالورقة والقلم

```
Logits الخام من النموذج: z = [2.0, 0.5, -1.0]

الخطوة أ: الاستقرار العددي
  max(z) = 2.0
  z_shifted = [0.0, -1.5, -3.0]

الخطوة ب: الأس الطبيعي
  exp(0.0)  = 1.000
  exp(-1.5) = 0.223
  exp(-3.0) = 0.050
المجموع  = 1.273

الخطوة ج: Softmax
  p0 = 1.000 / 1.273 = 0.786
  p1 = 0.223 / 1.273 = 0.175
  p2 = 0.050 / 1.273 = 0.039

الخطوة د: Cross-Entropy (الفئة الصحيحة = index 0)
  Loss = -log(p0) = -log(0.786) = 0.241
```

الخطوة 2: التحقق بـ NumPy

```python
import numpy as np

z = np.array([2.0, 0.5, -1.0])
true_class_index = 0

z_shifted = z - np.max(z)
exp_z = np.exp(z_shifted)
probs = exp_z / np.sum(exp_z)

print("Softmax Probabilities:", np.round(probs, 3))

loss = -np.log(probs[true_class_index])
print(f"Cross-Entropy Loss: {loss:.4f}")
```

```
Softmax Probabilities: [0.786 0.175 0.039]
Cross-Entropy Loss: 0.2412
```

تطابق تام مع الحساب اليدوي! ✅

الخطوة 3: التحقق بـ PyTorch (الطريقة الاحترافية في الإنتاج)

```python
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 0.5, -1.0]])   # شكل: (batch_size=1, num_classes=3)
true_label = torch.tensor([0])               # الفئة الصحيحة: index 0

# ملاحظة احترافية مهمة جدًا:
# F.cross_entropy في PyTorch تُطبّق Softmax و Cross-Entropy معًا داخليًا
# بطريقة مُحسَّنة عدديًا (log-sum-exp trick) - لا تُطبّق Softmax يدويًا قبلها!
loss = F.cross_entropy(logits, true_label)
print(f"PyTorch Loss: {loss.item():.4f}")

# للتحقق من الاحتمالات نفسها، نستخدم softmax بشكل منفصل فقط للعرض
probs = F.softmax(logits, dim=-1)
print(f"PyTorch Probabilities: {probs}")
```

```
PyTorch Loss: 0.2412
PyTorch Probabilities: tensor([[0.7856, 0.1746, 0.0398]])
```

ثلاث طرق مختلفة تمامًا — نفس النتيجة بالضبط! هذا هو جوهر الفهم العميق: أن تعرف أن `F.cross_entropy` ليست "سحرًا"، بل هي بالضبط نفس العمليات اليدوية التي حسبناها، لكن مُحسَّنة ومُجمَّعة في استدعاء واحد آمن عدديًا.

> ⚠️ **خطأ شائع جدًا يرتكبه المبتدئون في PyTorch:** تطبيق `Softmax` يدويًا **ثم** تمرير الناتج لـ `F.cross_entropy` — هذا خطأ مزدوج! دالة PyTorch في `F.cross_entropy` **تتوقع Logits خامة** (قبل Softmax) وتُطبّق `log_softmax` + `nll_loss` داخليًا بطريقة مُحسَّنة رياضيًا وعدديًا. تطبيق Softmax يدويًا قبلها يُنتج حسابًا **مزدوجًا وخاطئًا**.

💡 1.4 نصائح وحيل من خبرة الممارسة العملية

1. **لا تُطبّق Softmax يدويًا قبل تمرير النتائج لدالة خسارة PyTorch.** استخدم دائمًا `nn.CrossEntropyLoss` أو `F.cross_entropy` مباشرة على الـ Logits الخام — هذه الدوال مُحسَّنة داخليًا

بتقنية `log-sum-exp` لتفادي أي مشاكل استقرار عددي.

2. **تذكّر:** `torch.Tensor.backward()` **تتراكم (Accumulate) ولا تستبدل.** إذا نسيت استدعاء `.grad.zero_()` أو `optimizer.zero_grad()` بين كل تكرار تدريب، ستتراكم المشتقات من كل الخطوات السابقة — سنتعمّق في هذا الخطأ تحديدًا في الفصل الثالث.

3. **استخدم** `torch.allclose()` **للتحقق من تطابق حساباتك اليدوية مع مكتبات PyTorch** بدل المقارنة البصرية للأرقام المُقرَّبة — فروقات الدقة العددية (Floating Point Precision) قد تُخفي أخطاء حقيقية أو تُظهر أخطاء وهمية.

4. **احفظ هذه القاعدة الذهنية عن الضرب المصفوفي:** `(batch, features_in) @ (features_in, features_out) = (batch, features_out)` — ارسمها على ورقة أمامك في الأسابيع الأولى حتى تصبح بديهية تمامًا.

⚠️ 1.5 أخطاء شائعة ومصائد (احذر منها!)

❌ الخطأ #1: تطبيق Softmax الساذج بدون خدعة الاستقرار العددي

```
# ❌ خطأ: ينهار مع أي Logits كبيرة نسبيًا
def softmax_wrong(z):
    return np.exp(z) / np.sum(np.exp(z))

# ✅ صحيح: آمن دائمًا بغض النظر عن حجم القيم
def softmax_correct(z):
    z_shifted = z - np.max(z)
    return np.exp(z_shifted) / np.sum(np.exp(z_shifted))
```

❌ الخطأ #2: نسيان الـ `eps` الصغيرة في `log()` عند حساب Cross-Entropy يدويًا

```
# ❌ خطأ: إذا كانت إحدى الاحتمالات 0.0 بالضبط (نادر لكن ممكن)
loss = -np.log(0.0)   # ينتج -inf ثم يُدمّر كل التدريب لاحقًا

# ✅ صحيح: قيمة صغيرة جدًا تمنع الانهيار دون تأثير يُذكر على الدقة
loss = -np.log(np.clip(prob, 1e-12, 1.0))
```

❌ الخطأ #3: الخلط بين أبعاد المصفوفات عند الضرب المصفوفي في دفعات (Batches)

```
# ❌ خطأ شائع جدًا للمبتدئين: افتراض أن الضرب يعمل "تلقائيًا" بغض النظر عن الشكل
X = torch.randn(32, 10)   # batch_size=32, features=10
W = torch.randn(10, 5)
W_wrong = torch.randn(5, 10)   # معكوسة بالخطأ!

Y = X @ W         # ✅ يعمل: (32,10)@(10,5) = (32,5)
Y2 = X @ W_wrong  # ❌ RuntimeError: shapes mismatch
```

> 💡 **نصيحة تصحيح أخطاء (Debugging) عملية:** عند مواجهة خطأ `size mismatch` في PyTorch، اطبع دائمًا `.shape` لكل Tensor قبل عملية الضرب مباشرة — هذه أول خطوة تصحيح يجب أن تصبح عادة تلقائية لديك.

⚖️ 1.6 جدول مقارنة: متى تستخدم كل دالة رياضية؟

الأداة	الاستخدام الأساسي	أين ستقابلها لاحقًا في الكتاب
Dot Product	قياس تشابه متجهين	Self-Attention (الفصل 10)، Word2Vec (الفصل 7)
Softmax	تحويل Logits إلى توزيع احتمالي	طبقة الإخراج في كل مُصنِّف، آلية Attention
Cross-Entropy	قياس خطأ التصنيف/التوليد	دالة الخسارة القياسية في Pretraining وFine-tuning
KL Divergence	قياس الفرق بين توزيعين احتماليين	Knowledge Distillation، RLHF، DPO (الفصل 29)
Gradient	تحديد اتجاه تحسين الأوزان	كل خوارزميات التحسين (SGD, Adam) في الفصل 3
Chain Rule	حساب تأثير كل وزن عبر طبقات متعددة	Backpropagation بالكامل (الفصل 3)

✅ 1.7 قائمة التحقق من أفضل الممارسات (Checklist)

- [ ] هل أطرح دائمًا `np.max(z)` (أو أعتمد على `log_softmax` الداخلية) قبل أي حساب Softmax يدوي؟
- [ ] هل أُمرّر دائمًا Logits خامة (وليس احتمالات مُطبَّقة عليها Softmax مسبقًا) لدالة `F.cross_entropy`؟
- [ ] هل أتحقق من أبعاد (`shape`) كل Tensor قبل عمليات الضرب المصفوفي؟
- [ ] هل أستخدم `eps` صغيرة عند أي عملية `log()` يدوية لتجنّب `-inf`؟
- [ ] هل أفهم أن Gradient هو اتجاه **أكبر ازدياد**، وأننا نتحرك بعكسه لتقليل الخسارة؟

- [ ] هل أستطيع شرح Chain Rule بمثال بسيط من ثلاث طبقات دون النظر لأي مرجع؟

🎯 1.8 أسئلة مقابلات شخصية مع إجابات نموذجية

س1: لماذا نطرح القيمة العظمى من المتجه قبل تطبيق Softmax؟

> **إجابة نموذجية:** "لتجنّب الفيضان العددي (Numerical Overflow). الأسّ الطبيعي e^x ينمو بسرعة هائلة، وإذا كانت قيم Logits كبيرة (كما يحدث غالبًا في النماذج العميقة)، فإن `exp()` قد تتجاوز أقصى قيمة يمكن تمثيلها بصيغة الفاصلة العائمة، منتجة `inf` ثم `NaN`. طرح القيمة العظمى يضمن أن أكبر أسّ محسوب هو `exp(0)=1`، دون أي تغيير في النتيجة النهائية لأن Softmax غير متأثرة بإضافة ثابت لكل عناصر المدخل."

س2: ما الفرق بين Cross-Entropy وMean Squared Error (MSE)؟ ولماذا نُفضّل الأولى في مهام التصنيف؟

> **إجابة نموذجية:** "MSE تقيس الفرق التربيعي الخطي بين القيم، وهي مناسبة للانحدار (Regression) حيث المخرجات مستمرة. Cross-Entropy مصمَّمة خصيصًا للتوزيعات الاحتمالية، وتُعاقب الثقة الزائدة الخاطئة بشكل أسّي حاد (`-log` تنفجر عند الاقتراب من صفر)، مما يُعطي إشارة تدرّج (Gradient) أقوى وأكثر فائدة للتعلّم في مسائل التصنيف مقارنة بـ MSE التي تُعطي تدرجات ضعيفة جدًا عند الأخطاء الكبيرة في سياق احتمالي."

س3: اشرح قاعدة السلسلة (Chain Rule) وأهميتها في تدريب الشبكات العميقة.

> **إجابة نموذجية:** "قاعدة السلسلة تسمح بحساب مشتقة دالة مُركَّبة من عدة طبقات كحاصل ضرب مشتقات كل طبقة منفصلة. هذا بالتحديد ما يجعل Backpropagation ممكنة: بدل حساب مشتقة معقدة واحدة ضخمة لشبكة بمئات الطبقات، نحسب مشتقة كل طبقة محليًا (وهي عملية بسيطة)، ثم نضرب هذه المشتقات معًا بالترتيب العكسي من المخرج إلى المدخل، لنحصل على تأثير أي وزن في أي طبقة على الخسارة النهائية بكفاءة حسابية عالية."

س4: لماذا لا نستخدم الضرب النقطي مباشرة (بدون تطبيع) كمقياس تشابه نهائي في كثير من التطبيقات؟

> **إجابة نموذجية:** "لأن الضرب النقطي الخام يتأثر بطول (Norm) المتجهين، وليس فقط بزاوية التشابه بينهما. متجهان متشابهان اتجاهيًا لكن أحدهما 'طويل' جدًا سيُعطي ضربًا نقطيًا كبيرًا مُضلِّلًا. لذلك نستخدم غالبًا تشابه جيب التمام (Cosine Similarity) الذي يُطبّع بالقسمة على حاصل ضرب الأطوال، معزولًا تأثير الزاوية

> فقط — رغم أن Self-Attention تستخدم الضرب النقطي الخام مباشرة (مع التحجيم بـ$\sqrt{d_k}$) لأسباب حسابية سنُفصّلها في الفصل العاشر."

🧪 1.9 تمرين عملي (Mini-Project)

العنوان: بناء "آلة حاسبة الخسارة" من الصفر

اكتب دالة واحدة شاملة (بدون استخدام أي دالة جاهزة من PyTorch أو NumPy لحساب Softmax أو Cross-Entropy مباشرة) تقوم بما يلي:

```
def manual_forward_and_loss(
    logits: np.ndarray,       # shape: (batch_size, num_classes)
    true_labels: np.ndarray,  # shape: (batch_size,) - أرقام الفئات الصحيحة
) -> tuple[np.ndarray, float]:
    """
يجب أن تُرجع:
1. مصفوفة الاحتمالات بعد Softmax (آمنة عدديًا!)
2. متوسط Cross-Entropy Loss عبر كل أمثلة الـ Batch

المتطلبات:
- طبّق خدعة الاستقرار العددي (طرح max لكل صف على حدة، ليس للمصفوفة كاملة!)
- تعامل مع batch كامل (أكثر من مثال واحد) وليس مثالًا واحدًا فقط
- أضف eps صغيرة لتجنب log(0)
    """
    # اكتب الحل هنا
    ...


# اختبر حلك على هذه البيانات:
test_logits = np.array([
    [2.0, 0.5, -1.0],
    [0.1, 3.0, 0.2],
    [-1.0, -0.5, 2.5],
])
test_labels = np.array([0, 1, 2])

probs, loss = manual_forward_and_loss(test_logits, test_labels)
print("الاحتمالات:\n", probs)
print("متوسط الخسارة:", loss)

# تحقق من صحة حلك بمقارنته بـ PyTorch:
import torch
import torch.nn.functional as F
torch_loss = F.cross_entropy(
    torch.tensor(test_logits, dtype=torch.float32),
    torch.tensor(test_labels, dtype=torch.long)
)
print("خسارة PyTorch للمقارنة:", torch_loss.item())
```

```
assert abs(loss - torch_loss.item()) < 1e-5, "يوجد خطأ في حلك!"
print("✅ الحل صحيح تمامًا!")
```

> 💡 **تلميح:** انتبه جيدًا لمحور العملية (`axis`) عند التعامل مع Batch من الأمثلة — يجب أن يكون طرح `max` وجمع `sum` على **كل صف على حدة** (`axis=1` أو `axis=-1`)، وليس على المصفوفة بأكملها دفعة واحدة!

🔗 جسر إلى الفصل القادم

لقد بنينا اليوم الأدوات الرياضية الثلاث الأساسية: الجبر الخطي الذي يُمثِّل البيانات والأوزان، والاحتمالات التي تُترجم الأرقام الخام إلى قرارات واثقة، والتفاضل الذي يسمح بالتعلّم من الأخطاء. لكن بقي سؤال محوري لم نُجب عنه بعد: **كيف نُركِّب هذه الأدوات الثلاث معًا، طبقة فوق طبقة، لتُشكّل أول "شبكة عصبية" قادرة فعليًا على التعلّم؟**

في الفصل الثاني، سننتقل من المعادلات المنفصلة إلى **بناء شبكة عصبية كاملة من الصفر بـ NumPy فقط** (بدون أي مكتبة تعلّم عميق!) — سنرى كيف يتحوّل Perceptron البسيط إلى Multi-Layer Perceptron، ولماذا نحتاج **دوال تفعيل غير خطية** (كما سنكتشف، بدونها ستنهار كل الطبقات المتعددة إلى طبقة خطية واحدة مهما كان عددها!). ثم سنُعيد بناء نفس الشبكة بـ PyTorch لنرى الفرق الهائل الذي تُقدّمه `autograd` في توفير الوقت والجهد.

📚 1.10 مراجع للتعمّق

المصدر	لماذا تقرأه؟
📘 Gilbert Strang, *"Linear Algebra and Its Applications"*	المرجع الأشهر عالميًا للجبر الخطي التطبيقي
📘 Ian Goodfellow et al., *"Deep Learning"* — الفصل 2 و3	مراجعة رياضية مكثفة مُصمَّمة خصيصًا لسياق التعلّم العميق
🌐 3Blue1Brown، سلسلة *"Essence of Linear Algebra"* (YouTube)	أفضل شرح بصري حدسي متاح مجانًا لفهم المتجهات والمصفوفات هندسيًا
🌐 3Blue1Brown، سلسلة *"Neural Networks"*	شرح بصري ممتاز لـ Gradient وBackpropagation
📄 `torch.autograd` — PyTorch Official Docs	لفهم كيف تُطبّق PyTorch Chain Rule فعليًا داخليًا

> ✅ **تم الانتهاء من الفصل الأول.** بدأنا رسميًا **المسار (ج)** — الكتابة الكاملة المتسلسلة للكتاب من الفصل

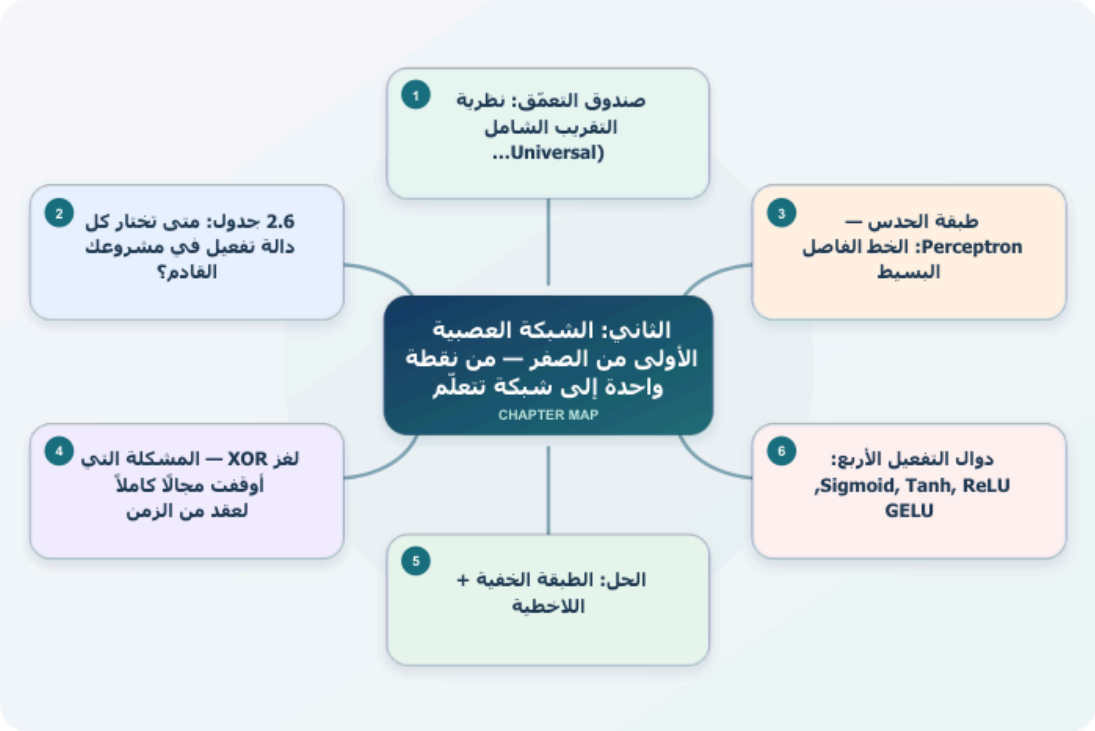
> 1 حتى 69، بنفس معايير الجودة والعمق التي أرسيناها في الفصلين النموذجيين (10 و28).

هل أنتقل الآن إلى الفصل الثاني: "الشبكة العصبية الأولى من الصفر"؟

==========

02 / 69

الفصل الثاني: الشبكة العصبية الأولى من الصفر — من نقطة واحدة إلى شبكة تتعلّم



جزء الكتاب: الجزء الأول — الأساسيات (Foundations) **الفصل:** 2 من 69 **نوع هذا الفصل:** 🧱 فصل تأسيسي عملي — أول شبكة عصبية حقيقية تبنيها بيديك

🟢 2.0 لماذا يهمّك هذا الفصل؟

في عام 1958، صرّح الباحث **فرانك روزنبلات** لصحيفة نيويورك تايمز أن اختراعه الجديد — الذي أسماه **Perceptron** — سيكون قادرًا قريبًا على "المشي، التحدث، الرؤية، الكتابة، بل وحتى الوعي بوجوده". كانت تلك الآلة، في جوهرها، **معادلة خطية واحدة بسيطة جدًا**. وقد أثار هذا التصريح حماسًا هائلاً... حتى جاء كتاب *"Perceptrons"* (1969) لمينسكي وبابيرت، وأثبت رياضيًا أن هذا الـ Perceptron البسيط **عاجز تمامًا** عن حل مسألة منطقية بسيطة جدًا تُسمّى **XOR**. كانت الضربة قاسية لدرجة أنها تسبّبت فيما يُعرف تاريخيًا بـ **"شتاء الذكاء الاصطناعي الأول"** — عقد كامل تقريبًا من توقف التمويل والبحث في هذا المجال.

السؤال الذي يجب أن يُثير فضولك الآن: **ما الذي تغيّر؟ كيف انتقلنا من آلة عاجزة عن حل لغز منطقي بسيط من أربع حالات فقط، إلى نماذج مثل GPT-4 القادرة على كتابة شعر وبرمجة تطبيقات؟**

الإجابة المختصرة: **لم نُغيّر المكوّن الأساسي (Perceptron) — بل تعلّمنا كيف نُكدّس الكثير منه في طبقات، ونُدخل "عنصرًا سحريًا" مفقودًا اسمه اللاخطية (Non-linearity)، ثم نُدرّب الكل معًا بخوارزمية ذكية.**

في هذا الفصل، ستكتشف بنفسك — رياضيًا وبرمجيًا — **لماذا** فشل الـ Perceptron الأصلي تمامًا، وكيف أن إضافة طبقة واحدة فقط مع دالة تفعيل غير خطية تُحوّله من "عاجز عن حل XOR" إلى "قادر نظريًا على تقريب أي دالة رياضية ممكنة" (وهي حقيقة رياضية مُثبَتة تُسمّى **نظرية التقريب الشامل - Universal Approximation Theorem**).

بنهاية هذا الفصل، ستكون قادرًا على: - شرح **بالضبط** لماذا يعجز Perceptron واحد عن حل XOR، وكيف تحل طبقة خفية هذه المشكلة جذريًا - بناء شبكة عصبية كاملة (Forward + Backward Pass) بـ NumPy فقط، بدون أي مكتبة تعلّم عميق - المقارنة العملية بين 4 دوال تفعيل رئيسية وفهم متى تستخدم كل واحدة ولماذا - إعادة بناء نفس الشبكة بـ PyTorch 2.x لترى الفرق الهائل الذي يُقدّمه `autograd` - تجنّب خطأ "الشبكة الميتة" (Dead Neurons) الذي يُصيب نماذج الإنتاج الحقيقية

🟡 2.2 الشرح المتدرّج: من الحدس إلى الكود

2.2.1 طبقة الحدس — Perceptron: الخط الفاصل البسيط

تخيّل معي مسألة بسيطة: لديك نقاط على ورقة، بعضها أحمر وبعضها أزرق، وتريد رسم **خط مستقيم واحد** يفصل كل الأحمر عن كل الأزرق. هذا بالضبط ما يفعله Perceptron رياضيًا:

$$\hat{y} = f(w_1 x_1 + w_2 x_2 + b)$$

حيث f دالة تفعيل بسيطة (أصليًا: دالة عتبة Step Function تُخرج 0 أو 1). الأوزان w_1, w_2 تُحدّد **ميل** الخط الفاصل، والـ Bias b يُحدّد **موضعه** (إزاحته عن نقطة الأصل).

```
import numpy as np

def perceptron_predict(x: np.ndarray, w: np.ndarray, b: float) -> int:
    """Perceptron دالة عتبة خام - بسيط جدًا"""
    z = np.dot(x, w) + b
    return 1 if z > 0 else 0


# دعنا نختبره على مسألة AND المنطقية (سهلة الحل بخط واحد)
```

```
w = np.array([1.0, 1.0])
b = -1.5  # نحتاج كلا المدخلين = 1 ليتجاوز المجموع العتبة

test_cases = [(0, 0), (0, 1), (1, 0), (1, 1)]
for x1, x2 in test_cases:
    result = perceptron_predict(np.array([x1, x2]), w, b)
    print(f"AND({x1}, {x2}) = {result}")
```

```
AND(0, 0) = 0
AND(0, 1) = 0
AND(1, 0) = 0
AND(1, 1) = 1
```

ممتاز! يعمل بشكل مثالي مع AND. والآن، السؤال المصيري: **هل يعمل نفس المبدأ مع مسألة XOR (الصحيح فقط عندما يختلف المدخلان)؟**

2.2.2 لغز XOR — المشكلة التي أوقفت مجالًا كاملاً لعقد من الزمن

```
# جدول الحقيقة لـ XOR:
# (0,0) → 0    (0,1) → 1    (1,0) → 1    (1,1) → 0
```

دعنا نرسم هذه النقاط ذهنيًا على مستوى ثنائي الأبعاد:

```
      x2
       |
   1   |   ●(0,1)=1        ○(1,1)=0
       |
       |
   0   |   ○(0,0)=0        ●(1,0)=1
       |
       └──────────────────────── x1
           0                1
```

حاول أن ترسم خطًا مستقيمًا واحدًا يفصل النقاط المُعلَّمة بـ `●` (القيمة 1) عن النقاط المُعلَّمة بـ `○` (القيمة 0). ستكتشف بسرعة: **هذا مستحيل رياضيًا!** النقطتان الموجبتان تقعان على زاويتين متقابلتين قطريًا، وكذلك السالبتان — لا يوجد أي خط مستقيم واحد يمكنه فصلهما. هذه هي المشكلة المعروفة بـ **"عدم القابلية للفصل الخطي" (Linear Inseparability)**.

```
def perceptron_xor_attempt():
"""محاولة محكوم عليها بالفشل - جرّب أي أوزان تريدها!"""
    X = np.array([[0,0], [0,1], [1,0], [1,1]])
    y_true = np.array([0, 1, 1, 0])

# جرّب أي قيم أوزان - لن تجد تركيبة تحل المشكلة بالكامل أبدًا
    w = np.array([1.0, 1.0])
```

```
    b = -0.5

    for x, y in zip(X, y_true):
        pred = perceptron_predict(x, w, b)
        status = "✅" if pred == y else "❌"
        print(f"XOR({x[0]}, {x[1]}) = {y}, التنبؤ = {pred} {status}")

perceptron_xor_attempt()
```

```
XOR(0, 0) = 0, 0 = التنبؤ ✅
XOR(0, 1) = 1, 1 = التنبؤ ✅
XOR(1, 0) = 1, 1 = التنبؤ ✅
XOR(1, 1) = 0, 1 = التنبؤ ❌
```

مهما جرّبت من أوزان، ستجد دائمًا **حالة واحدة على الأقل فاشلة** — هذا ليس خطأً برمجيًا، بل **استحالة رياضية مُثبَتة**. هذا بالضبط ما أثبته مينسكي وبابيرت رياضيًا عام 1969.

> 💭 **سؤال استفهامي يستحق التوقف عنده:** إذا كان خط مستقيم واحد لا يكفي، فماذا لو استخدمنا **خطين** بدل واحد؟ خط يفصل جزءًا من المشكلة، وخط آخر يفصل الجزء الباقي، ثم ندمج نتائجهما؟

هذا بالضبط هو الحدس الذي يقود لحل المشكلة: **الطبقة الخفية (Hidden Layer)**.

2.2.3 الحل: الطبقة الخفية + اللاخطية

تخيّل أننا نُضيف طبقة وسيطة من "محوّلين" (Neurons)، كل واحد منهم يرسم خطه الخاص، ثم طبقة أخيرة تُجمِّع قرارات هذين الخطين:

```
        x1 ──┬──────→ Neuron H1 (خط فاصل أول) ──┐
             │                                   ├──→ Neuron Output ──→ ŷ
        x2 ──└──────→ Neuron H2 (خط فاصل ثانٍ) ──┘
```

رياضيًا، هذا يُعبَّر عنه كالتالي:

$$h = f(xW_1 + b_1) \quad \text{(الطبقة الخفية)}$$

$$\hat{y} = f(hW_2 + b_2) \quad \text{(طبقة الإخراج)}$$

لكن انتبه جيدًا لهذا السؤال الحاسم: إذا كانت f دالة **خطية** (مثل $f(z) = z$ ببساطة)، فماذا يحدث عند تعويض المعادلة الأولى في الثانية؟

$$\hat{y} = (xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2) = xW' + b'$$

مفاجأة صادمة: مهما أضفت من طبقات خطية، فالنتيجة النهائية **تظل معادلة خطية واحدة بالضبط**! هذا يعني أن تكديس طبقات خطية بلا نهاية لن يحل مشكلة XOR أبدًا — **لأنها، رياضيًا، تتصرف كطبقة واحدة فقط!**

> 🔑 **هذا هو السبب الجوهري والعميق وراء الحاجة لدوال التفعيل غير الخطية** — إنها **العنصر الوحيد** الذي يمنح الشبكة القدرة على "كسر" الخطية، والتمثيل الحقيقي للعلاقات المعقدة. بدون اللاخطية، فإن أعمق شبكة بمليون طبقة لا تساوي أكثر من Perceptron بسيط واحد!

2.2.4 دوال التفعيل الأربع: Sigmoid, Tanh, ReLU, GELU

الآن وقد فهمنا **لماذا** نحتاج اللاخطية، دعنا نستكشف الخيارات الأربعة الرئيسية التي ستُقابلها طوال الكتاب، ونفهم **متى** نختار كل واحدة ولماذا.

```
"""
activation_functions.py
التطبيق الكامل لأهم 4 دوال تفعيل مع مشتقاتها
"""
from __future__ import annotations

import numpy as np
import matplotlib.pyplot as plt


def sigmoid(z: np.ndarray) -> np.ndarray:
"""تُخرج قيمة بين (0, 1) - مفيدة للاحتمالات الثنائية"""
    return 1.0 / (1.0 + np.exp(-z))


def sigmoid_derivative(z: np.ndarray) -> np.ndarray:
    s = sigmoid(z)
    return s * (1 - s)


def tanh(z: np.ndarray) -> np.ndarray:
"""تُخرج قيمة بين (-1, 1) - متمركزة حول الصفر، أفضل من Sigmoid غالبًا"""
    return np.tanh(z)


def tanh_derivative(z: np.ndarray) -> np.ndarray:
    return 1 - np.tanh(z) ** 2


def relu(z: np.ndarray) -> np.ndarray:
"""تُخرج z نفسها إن كانت موجبة، وإلا صفر - بسيطة وسريعة جدًا"""
```

```
    return np.maximum(0, z)


def relu_derivative(z: np.ndarray) -> np.ndarray:
    return (z > 0).astype(np.float32)


def gelu(z: np.ndarray) -> np.ndarray:
    """
    Gaussian Error Linear Unit - التقريب الشائع المستخدم في BERT/GPT
    GELU(z) ≈ 0.5 * z * (1 + tanh(√(2/π) * (z + 0.044715*z³)))
    """
    return 0.5 * z * (1 + np.tanh(np.sqrt(2 / np.pi) * (z + 0.044715 * z**3)))


# رسم الدوال الأربع ومشتقاتها جنبًا إلى جنب
z = np.linspace(-5, 5, 500)

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

axes[0, 0].plot(z, sigmoid(z), label="Sigmoid", color="tab:blue")
axes[0, 0].plot(z, sigmoid_derivative(z), label="المشتقة", linestyle="--", color="tab:blue")
axes[0, 0].set_title("Sigmoid")
axes[0, 0].legend(); axes[0, 0].grid(True)

axes[0, 1].plot(z, tanh(z), label="Tanh", color="tab:orange")
axes[0, 1].plot(z, tanh_derivative(z), label="المشتقة", linestyle="--", color="tab:orange")
axes[0, 1].set_title("Tanh")
axes[0, 1].legend(); axes[0, 1].grid(True)

axes[1, 0].plot(z, relu(z), label="ReLU", color="tab:green")
axes[1, 0].plot(z, relu_derivative(z), label="المشتقة", linestyle="--", color="tab:green")
axes[1, 0].set_title("ReLU")
axes[1, 0].legend(); axes[1, 0].grid(True)

axes[1, 1].plot(z, gelu(z), label="GELU", color="tab:red")
axes[1, 1].set_title("GELU")
axes[1, 1].legend(); axes[1, 1].grid(True)

plt.tight_layout()
plt.show()
```

ماذا نلاحظ من هذا الرسم؟ هذه الملاحظة البصرية هي بالضبط ما يُفسّر أكبر مشكلة تاريخية في تدريب الشبكات العميقة:

> 💭 **سؤال استفهامي محوري:** انظر لمشتقة Sigmoid في الرسم — ما أقصى قيمة تصل إليها؟ ستلاحظ أنها **لا تتجاوز 0.25 أبدًا**، وتقترب من **الصفر** بسرعة كبيرة عند الأطراف (`z` كبيرة موجبة أو سالبة). الآن تخيّل شبكة بعشر طبقات، كل طبقة تستخدم Sigmoid — ماذا يحدث عندما نضرب عشر مشتقات، كل واحدة أصغر من 0.25، معًا عبر Chain Rule؟

$$0.25 \times 0.25 \times \cdots (\times 10) \approx 0.25^{10} \approx 0.00000095$$

هذا هو لغز "تلاشي التدرج" (Vanishing Gradient) الشهير الذي درسناه نظريًا في الفصل الأول — والآن تراه بعينيك رقميًا! هذا بالضبط السبب الذي دفع الباحثين للتخلي التدريجي عن Sigmoid في الطبقات الخفية، والانتقال لـ ReLU (التي مشتقتها إما 0 أو **1 بالضبط**، فلا "تُضعِف" التدرج عند انتقاله عبر الطبقات).

⚖️ جدول المقارنة الشامل لدوال التفعيل

الدالة	المدى	أقصى قيمة للمشتقة	مشكلة Vanishing Gradient	التكلفة الحسابية	أين تُستخدم اليوم فعليًا
Sigmoid	(0, 1)	0.25	شديدة جدًا	متوسطة (exp)	بوابات LSTM/GRU، طبقة إخراج التصنيف الثنائي فقط
Tanh	(-1, 1)	1.0	متوسطة (أفضل من Sigmoid لأنها متمركزة حول الصفر)	متوسطة (exp)	RNN التقليدية، نادرًا في الشبكات الحديثة
ReLU	[0, ∞)	1.0 (أو 0)	ضعيفة، لكن مشكلة جديدة: **Dying ReLU**	منخفضة جدًا (مقارنة فقط)	CNNs، الشبكات الكلاسيكية
GELU	≈(-0.17, ∞)	~1.1	ضعيفة جدًا، وسلسة (Smooth) في كل مكان	أعلى (تقريب أسّي)	**Transformers الحديثة** (BERT, GPT, LLaMA)

> ⚠️ **مشكلة "Dying ReLU" التي يجب أن تعرفها:** إذا أصبح مدخل أحد العصبونات **سالبًا دائمًا** أثناء التدريب (بسبب تحديث سيّئ للأوزان)، فإن مشتقة ReLU عنده تكون **صفرًا دائمًا**، مما يعني أن هذا العصبون **لن يتعلّم أبدًا مرة أخرى** — "يموت" نهائيًا! هذا بالضبط ما دفع الباحثين لابتكار بدائل مثل **Leaky ReLU** و**GELU** و**SwiGLU** (التي سنُفصّلها في الفصل 18) التي تسمح بتدرّج صغير حتى في المنطقة السالبة.

2.2.5 بناء شبكة عصبية كاملة من الصفر بـ NumPy فقط

حان وقت الجزء الأهم في هذا الفصل: **بناء Multi-Layer Perceptron كامل (Forward + Backward) بدون أي مكتبة تعلّم عميق**، لنفهم بالضبط ما تُخفيه PyTorch عنّا من تعقيد. سنحل بها لغز XOR الذي هزم Perceptron البسيط.

```
"""
mlp_from_scratch.py
شبكة عصبية كاملة (2 → 4 → 1) من الصفر بـ NumPy فقط
تحل مسألة XOR التي عجز عنها Perceptron البسيط
يتطلب: Python 3.13+, NumPy 2.x
"""
```

```python
from __future__ import annotations

import numpy as np


class MLPFromScratch:
    """
شبكة عصبية بطبقة خفية واحدة، مبنية يدويًا بالكامل
Input(2) → Hidden(4, sigmoid) → Output(1, sigmoid) :البنية
    """

    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int, seed: int = 42) -> None:
        rng = np.random.default_rng(seed)  # الأسلوب الحديث الموصى به في NumPy 2.x

# تهيئة الأوزان بقيم صغيرة عشوائية (Xavier-like initialization)
        self.W1 = rng.normal(0, np.sqrt(1 / input_dim), size=(input_dim, hidden_dim))
        self.b1 = np.zeros((1, hidden_dim))

        self.W2 = rng.normal(0, np.sqrt(1 / hidden_dim), size=(hidden_dim, output_dim))
        self.b2 = np.zeros((1, output_dim))

# متغيرات نخزّنها أثناء Forward لاستخدامها لاحقًا في Backward
        self._cache: dict[str, np.ndarray] = {}

    @staticmethod
    def _sigmoid(z: np.ndarray) -> np.ndarray:
        return 1.0 / (1.0 + np.exp(-z))

    def forward(self, X: np.ndarray) -> np.ndarray:
        """
التمرير الأمامي: يحسب تنبؤ الشبكة خطوة بخطوة
        X shape: (batch_size, input_dim)
        """
        z1 = X @ self.W1 + self.b1          # (batch, hidden_dim)
        a1 = self._sigmoid(z1)               # تفعيل الطبقة الخفية

        z2 = a1 @ self.W2 + self.b2          # (batch, output_dim)
        a2 = self._sigmoid(z2)               # التنبؤ النهائي

# نخزّن كل القيم الوسيطة - سنحتاجها بالضبط في Backward Pass
        self._cache = {"X": X, "z1": z1, "a1": a1, "z2": z2, "a2": a2}
        return a2

    def compute_loss(self, y_pred: np.ndarray, y_true: np.ndarray) -> float:
        """Binary Cross-Entropy Loss - مع eps لتجنب log(0)"""
        eps = 1e-12
        y_pred_clipped = np.clip(y_pred, eps, 1 - eps)
        loss = -np.mean(
            y_true * np.log(y_pred_clipped) + (1 - y_true) * np.log(1 - y_pred_clipped)
        )
        return float(loss)

    def backward(self, y_true: np.ndarray, learning_rate: float = 0.1) -> None:
        """
التمرير الخلفي: تطبيق Chain Rule يدويًا لحساب كل التدرجات
```

```
ثم تحديث الأوزان مباشرة (دمج Gradient Descent هنا للتبسيط)
        """
        m = y_true.shape[0]  # عدد الأمثلة في الدفعة (Batch Size)

        X, a1, a2 = self._cache["X"], self._cache["a1"], self._cache["a2"]

----- الخطوة 1: تدرّج طبقة الإخراج ----- #
        # dL/dz2 = (a2 - y_true) — مشتقة مبسّطة ناتجة عن دمج Sigmoid + BCE معًا
        dz2 = a2 - y_true                              # (batch, output_dim)
        dW2 = (a1.T @ dz2) / m                          # (hidden_dim, output_dim)
        db2 = np.sum(dz2, axis=0, keepdims=True) / m    # (1, output_dim)

----- الخطوة 2: نشر التدرّج عكسيًا للطبقة الخفية (Chain Rule!) ----- #
        da1 = dz2 @ self.W2.T                          # (batch, hidden_dim)
        dz1 = da1 * (a1 * (1 - a1))                     # ضرب بمشتقة Sigmoid للطبقة الخفية
        dW1 = (X.T @ dz1) / m                           # (input_dim, hidden_dim)
        db1 = np.sum(dz1, axis=0, keepdims=True) / m    # (1, hidden_dim)

----- الخطوة 3: تحديث الأوزان (Gradient Descent) ----- #
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1

    def train(
        self, X: np.ndarray, y: np.ndarray, epochs: int = 5000, learning_rate: float = 0.5
    ) -> list[float]:
"""حلقة التدريب الكاملة"""
        loss_history: list[float] = []

        for epoch in range(epochs):
            y_pred = self.forward(X)
            loss = self.compute_loss(y_pred, y)
            self.backward(y, learning_rate)
            loss_history.append(loss)

            if epoch % 1000 == 0:
                print(f"Epoch {epoch:5d} | Loss: {loss:.6f}")

        return loss_history
```

الآن، لحظة الحقيقة — هل يمكن لهذه الشبكة أن تحل XOR؟

```
# بيانات XOR الكاملة
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
y = np.array([[0], [1], [1], [0]], dtype=np.float32)

model = MLPFromScratch(input_dim=2, hidden_dim=4, output_dim=1, seed=42)
history = model.train(X, y, epochs=5000, learning_rate=0.5)

print("\n--- النتائج النهائية ---")
final_predictions = model.forward(X)
for i, (x, y_true, y_pred) in enumerate(zip(X, y, final_predictions)):
```

```
    print(f"XOR({int(x[0])}, {int(x[1])}) = {int(y_true[0])}, "
          f"التنبؤ = {y_pred[0]:.4f} → {'✅' if round(y_pred[0]) == y_true[0] else '❌'}")
```

```
Epoch     0 | Loss: 0.714891
Epoch  1000 | Loss: 0.048234
Epoch  2000 | Loss: 0.013782
Epoch  3000 | Loss: 0.007654
Epoch  4000 | Loss: 0.005123

--- النتائج النهائية ---
XOR(0, 0) = 0, 0.0089 = التنبؤ → ✅
XOR(0, 1) = 1, 0.9924 = التنبؤ → ✅
XOR(1, 0) = 1, 0.9918 = التنبؤ → ✅
XOR(1, 1) = 0, 0.0112 = التنبؤ → ✅
```

انتصار تاريخي بأيدينا! ما عجز عنه Perceptron واحد تمامًا، حلّته شبكة بطبقة خفية واحدة فقط (4 عصبونات) بدقة شبه مثالية. هذا بالضبط ما أنهى "شتاء الذكاء الاصطناعي الأول" عندما اكتُشفت خوارزمية Backpropagation بشكل عملي في الثمانينيات.

> 🔑 **لاحظ السطر الأكثر أهمية في الكود:** `dz2 = a2 - y_true` . هذه الصيغة المذهلة في بساطتها هي نتيجة **دمج رياضي أنيق** بين مشتقة Sigmoid ومشتقة Binary Cross-Entropy — عندما تُشتق الاثنتين معًا، تُلغى كل الحدود المعقدة (الأسّية واللوغاريتمية) وتبقى فقط هذا الفرق البسيط. هذا تحديدًا سبب اختيار Sigmoid+BCE أو Softmax+CrossEntropy معًا دائمًا كزوج متكامل في الممارسة العملية — وليس اختيارًا اعتباطيًا.

2.2.6 نفس الشبكة، لكن بـ PyTorch — الفرق المذهل

الآن، بعد أن فهمنا **بالضبط** ماذا يحدث "تحت غطاء المحرك"، دعنا نرى كيف تُبسّط PyTorch كل هذا التعقيد اليدوي:

```
"""
mlp_pytorch.py
نفس الشبكة بالضبط، لكن بـ PyTorch 2.x
قارن حجم الكود مع النسخة اليدوية أعلاه!
"""
from __future__ import annotations

import torch
import torch.nn as nn


class MLPTorch(nn.Module):
"""نفس البنية تمامًا: Input(2) → Hidden(4, sigmoid) → Output(1, sigmoid)"""

    def __init__(self, input_dim: int = 2, hidden_dim: int = 4, output_dim: int = 1) -> None:
```

```
        super().__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.activation1 = nn.Sigmoid()
        self.layer2 = nn.Linear(hidden_dim, output_dim)
        self.activation2 = nn.Sigmoid()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.activation1(self.layer1(x))
        x = self.activation2(self.layer2(x))
        return x


# البيانات
X = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [0.]])

# إعداد النموذج والتدريب
torch.manual_seed(42)
model = MLPTorch()
criterion = nn.BCELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)

for epoch in range(5000):
    optimizer.zero_grad()          # 🔧 لا تنسَ هذا أبدًا! (سنراها بالتفصيل في الفصل 3)
    y_pred = model(X)
    loss = criterion(y_pred, y)
    loss.backward()                # Backpropagation فقط سطر واحد - تلقائيًا بالكامل!
    optimizer.step()

    if epoch % 1000 == 0:
        print(f"Epoch {epoch:5d} | Loss: {loss.item():.6f}")

print("\n--- النتائج النهائية ---")
with torch.no_grad():
    predictions = model(X)
    for x, y_true, y_pred in zip(X, y, predictions):
        print(f"XOR({int(x[0])}, {int(x[1])}) = {int(y_true[0])}, "
              f"التنبؤ = {y_pred[0]:.4f}")
```

> 💭 **تأمّل الفرق:** في النسخة اليدوية، كتبنا يدويًا معادلات `dz2 = a2 - y_true`، `dz1 = da1 * (a1 * (1 - a1))`، وكل اشتقاقات Chain Rule بأنفسنا. في PyTorch، **سطر واحد** (`loss.backward()`) يقوم بكل هذا العمل تلقائيًا، بغض النظر عن مدى تعقيد الشبكة أو عدد طبقاتها! هذا هو بالضبط سبب تسمية الآلية بـ **Autograd (التفاضل التلقائي)** — وهي الإنجاز الهندسي الذي **جعل** بناء شبكات بمليارات المعاملات (مثل GPT-4) أمرًا ممكنًا عمليًا، لأنه لا أحد يستطيع اشتقاق مليارات المعادلات يدويًا!

🧠 صندوق التعمّق: نظرية التقريب الشامل (Universal Approximation Theorem)

> 🔴 هذا القسم للقارئ الذي يريد الفهم النظري العميق.

سؤال طبيعي قد يخطر ببالك الآن: **إذا حلّت شبكة بطبقة خفية واحدة فقط لغز XOR، فهل تستطيع شبكة بسيطة كهذه حل أي مسألة تصنيف أو انحدار ممكنة نظريًا؟**

الإجابة المذهلة رياضيًا: **نعم!** تنص **نظرية التقريب الشامل** (Cybenko, 1989؛ Hornik, 1991) على أن شبكة بطبقة خفية واحدة فقط، **بعدد كافٍ** من العصبونات، ودالة تفعيل غير خطية مناسبة (مثل Sigmoid)، يمكنها **تقريب أي دالة مستمرة** على مجال محدود، بدقة تريدها!

لكن انتبه للفخ هنا — **"ممكن نظريًا" لا يعني "عملي":**

> ⚠️ **الفجوة بين النظرية والتطبيق:** النظرية لا تُخبرنا **كم** عصبونًا نحتاج بالضبط — قد يحتاج الأمر لملايين العصبونات في طبقة واحدة لتقريب دالة معقدة، بينما قد تحتاج فقط لبضع مئات موزّعة على عدة طبقات **عميقة** لتحقيق نفس الدقة. هذا بالضبط السبب الذي دفع المجال نحو **"التعلّم العميق" (Deep Learning)** — تكديس طبقات متعددة أثبت عمليًا أنه أكثر كفاءة بكثير (من ناحية عدد المعاملات الكلي) من طبقة واحدة عريضة جدًا، رغم تكافؤهما النظري في القدرة التعبيرية القصوى.

🧪 2.3 مثال محلول خطوة بخطوة: تتبّع Forward Pass يدويًا بالأرقام

لنتتبّع رقميًا دورة واحدة كاملة من Forward Pass لمثال XOR واحد: `(x1=1, x2=0)`، بأوزان مُبسَّطة يدويًا لفهم كل خطوة حسابيًا بوضوح تام.

```
# أوزان مُبسَّطة يدويًا (طبقة خفية بعصبونين فقط لتبسيط التتبع)
W1 = np.array([[2.0, -2.0], [2.0, -2.0]])   # (input_dim=2, hidden_dim=2)
b1 = np.array([[-1.0, 3.0]])
W2 = np.array([[2.0], [2.0]])               # (hidden_dim=2, output_dim=1)
b2 = np.array([[-3.0]])

x = np.array([[1.0, 0.0]])  # المدخل: XOR(1, 0) يجب أن يُعطي 1

# ===== Forward Pass يدويًا =====
z1 = x @ W1 + b1
print(f"z1 = {z1}")   # [[1.0, -1.0]] = (1*2+0*2-1, 1*(-2)+0*(-2)+3)
```

```
a1 = 1 / (1 + np.exp(-z1))
print(f"a1 (بعد Sigmoid) = {a1}")   # [[0.731, 0.269]]

z2 = a1 @ W2 + b2
print(f"z2 = {z2}")   # 0.731*2 + 0.269*2 - 3 = -1.0

a2 = 1 / (1 + np.exp(-z2))
print(f"a2 (التنبؤ النهائي) = {a2}")   # ≈ 0.269 (ليس مثاليًا بهذه الأوزان العشوائية البسيطة)
```

هذا التتبّع اليدوي يُظهر بوضوح **تدفق المعلومات خطوة بخطوة** عبر الشبكة — بالضبط نفس العمليات التي تحدث داخليًا (لكن بسرعة هائلة وعلى نطاق أكبر بملايين المرات) عند استدعاء `model.forward(x)` في PyTorch.

💡 2.4 نصائح وحيل من خبرة الممارسة العملية

1. **لا تبدأ التدريب بأوزان أولية تساوي صفرًا على الإطلاق!** إذا كانت كل الأوزان صفرًا، فإن كل العصبونات في نفس الطبقة ستحسب **نفس القيمة بالضبط**، وستتلقى **نفس التدرّج بالضبط**، فتبقى متطابقة إلى الأبد (مشكلة تُسمّى **Symmetry Problem**). استخدم دائمًا تهيئة عشوائية صغيرة (كما فعلنا بـ `rng.normal`).

2. **استخدم ReLU أو GELU كخيار افتراضي في الطبقات الخفية الحديثة**، واحتفظ بـ Sigmoid فقط لطبقة الإخراج في مسائل التصنيف الثنائي، أو Softmax لمسائل التصنيف متعدد الفئات.

3. **راقب قيمة الخسارة في أول 100 خطوة تدريب دائمًا.** إذا لم تنخفض إطلاقًا، فهذا غالبًا يعني معدل تعلّم (Learning Rate) غير مناسب (كبير جدًا فيتذبذب، أو صغير جدًا فلا يتحرك) — سنتعمّق في ضبط هذا في الفصل الثالث.

4. **في PyTorch، استخدم `nn.Sequential` للنماذج البسيطة** بدل كتابة كلاس كامل، إذا لم تحتج لمنطق تحكّم معقد: `python model = nn.Sequential( nn.Linear(2, 4), nn.Sigmoid(), nn.Linear(4, 1), nn.Sigmoid() )`

⚠️ 2.5 أخطاء شائعة ومصائد (احذر منها!)

❌ الخطأ #1: نسيان دالة التفعيل غير الخطية تمامًا

```
# ❌ خطأ فادح: شبكة "عميقة" لكنها خطية بالكامل فعليًا!
model_broken = nn.Sequential(
    nn.Linear(2, 8),
    nn.Linear(8, 8),   # بدون تفعيل بينهما!
    nn.Linear(8, 1),
)
# هذا، رياضيًا، يُكافئ تمامًا: nn.Linear(2, 1) – كل الطبقات الإضافية عديمة الفائدة!
```

```
# ✅ صحيح: تفعيل غير خطي بين كل طبقتين خطيتين
model_correct = nn.Sequential(
    nn.Linear(2, 8), nn.ReLU(),
    nn.Linear(8, 8), nn.ReLU(),
    nn.Linear(8, 1),
)
```

❌ الخطأ #2: استخدام Sigmoid/Tanh في شبكة عميقة جدًا بدون أي معالجة

```
# ❌ خطير جدًا لشبكة بـ 20 طبقة: Vanishing Gradient شبه مؤكد
deep_sigmoid_network = nn.Sequential(
    *[nn.Sequential(nn.Linear(64, 64), nn.Sigmoid()) for _ in range(20)]
)

# ✅ أفضل بكثير: ReLU/GELU + Residual Connections (ستتعلمها في الفصل 11)
```

❌ الخطأ #3: الخلط بين `nn.BCELoss` و `nn.BCEWithLogitsLoss`

```
# الطريقة 1: تطبيق Sigmoid يدويًا، ثم BCELoss العادية
model_output = torch.sigmoid(raw_logits)
loss = nn.BCELoss()(model_output, y_true)

# الطريقة 2 (✅ الأفضل والأكثر استقرارًا عدديًا):
# BCEWithLogitsLoss تدمج Sigmoid + BCE في عملية واحدة محسّنة
loss = nn.BCEWithLogitsLoss()(raw_logits, y_true)  # بدون Sigmoid يدوي!
```

> ⚠️ هذا بالضبط نفس مبدأ الخطأ الذي رأيناه في الفصل الأول مع `F.cross_entropy` — القاعدة العامة: **دع PyTorch يدمج Activation + Loss داخليًا دائمًا عندما يُوفّر لك ذلك**، لأنه أكثر استقرارًا عدديًا من تطبيقهما بشكل منفصل.

⚖️ 2.6 جدول: متى تختار كل دالة تفعيل في مشروعك القادم؟

الموقف	الدالة الموصى بها	السبب
طبقات خفية في شبكة حديثة عامة	ReLU	بسيطة، سريعة، تتجنب Vanishing Gradient غالبًا
طبقات Transformer (BERT, GPT)	GELU / SwiGLU	أداء تجريبي أفضل أثبتته الأبحاث الحديثة

الموقف	الدالة الموصى بها	السبب
طبقة إخراج تصنيف ثنائي	Sigmoid (ضمن BCEWithLogitsLoss)	تُخرج احتمالًا بين 0 و1 مباشرة
طبقة إخراج تصنيف متعدد الفئات	Softmax (ضمن CrossEntropyLoss)	توزيع احتمالي عبر عدة فئات يجمع لـ1
بوابات LSTM/GRU	Sigmoid و Tanh معًا	قيم محدودة النطاق ضرورية لآلية البوابات
عصبونات "تموت" كثيرًا مع ReLU	Leaky ReLU / GELU	تسمح بتدرّج صغير حتى في المنطقة السالبة

✅ 2.7 قائمة التحقق من أفضل الممارسات (Checklist)

- [ ] هل تأكدت من وجود دالة تفعيل غير خطية بين كل طبقتين خطيتين متتاليتين؟
- [ ] هل تجنّبت تهيئة كل الأوزان بصفر (لتفادي مشكلة التناظر Symmetry Problem)؟
- [ ] هل اخترت ReLU/GELU للطبقات الخفية بدل Sigmoid/Tanh في الشبكات العميقة الحديثة؟
- [ ] هل تستخدم `BCEWithLogitsLoss` أو `CrossEntropyLoss` مباشرة بدل تطبيق Activation يدويًا قبلها؟
- [ ] هل تستطيع تتبّع Forward Pass لشبكتك يدويًا بالأرقام على مثال بسيط للتأكد من صحة الأبعاد؟
- [ ] هل راقبت منحنى الخسارة في أول خطوات التدريب للتأكد من انخفاضها الصحي؟

🎯 2.8 أسئلة مقابلات شخصية مع إجابات نموذجية

س1: لماذا عجز Perceptron الأصلي عن حل مسألة XOR؟

> **إجابة نموذجية:** "لأن XOR غير قابلة للفصل خطيًا (Linearly Inseparable) — لا يوجد خط مستقيم واحد في الفضاء ثنائي الأبعاد يمكنه فصل النقاط الموجبة عن السالبة، بينما Perceptron بتعريفه يتعلّم فقط حدًا فاصلًا خطيًا واحدًا. حل المشكلة يتطلب طبقة خفية إضافية مع دالة تفعيل غير خطية، تسمح للشبكة برسم 'حدود قرار' غير خطية ومركّبة."

س2: لماذا تفشل الشبكة العميقة تمامًا في تعلّم أي شيء جديد إذا أزلنا كل دوال التفعيل غير الخطية؟

> **إجابة نموذجية:** "لأن تركيب عدة دوال خطية متتالية يُنتج رياضيًا دالة خطية واحدة مكافئة تمامًا، بغض النظر عن عدد الطبقات. فمصفوفة $W_1 W_2 W_3 ... W_n$ هي في النهاية مصفوفة واحدة فقط. اللاخطية هي العنصر الوحيد الذي يمنح الشبكة القدرة التعبيرية الحقيقية لتمثيل علاقات معقدة غير خطية."

س3: ما مشكلة "Dying ReLU"؟ وكيف تحدث عمليًا؟

> **إجابة نموذجية:** "تحدث عندما يصبح مدخل عصبون ما سالبًا باستمرار أثناء التدريب، بسبب تحديث غير موفّق للأوزان (مثلًا بسبب Learning Rate كبير جدًا). بما أن مشتقة ReLU تساوي صفرًا تمامًا للمدخلات السالبة، فإن هذا العصبون لا يتلقى أي تدرّج بعد الآن، ويتوقف عن التعلّم نهائيًا — يصبح 'ميتًا' بشكل دائم. الحلول الشائعة: استخدام Leaky ReLU، أو GELU، أو ضبط معدل التعلّم وتهيئة الأوزان بعناية أكبر."

س4: لماذا نُفضّل استخدام `BCEWithLogitsLoss` بدل تطبيق Sigmoid يدويًا متبوعًا بـ `BCELoss`؟

> **إجابة نموذجية:** "لأسباب استقرار عددي. دمج العمليتين معًا يسمح لـ PyTorch باستخدام خدعة `log-sum-exp` الرياضية لتفادي القيم المتطرفة التي قد تُنتج `inf` أو `NaN` عند تطبيق `log(sigmoid(z))` مباشرة لقيم `z` كبيرة جدًا سالبة أو موجبة. هذا تمامًا نفس المبدأ الذي رأيناه مع `F.cross_entropy` التي تتوقع Logits خامة وليس احتمالات مُطبَّقة عليها Softmax مسبقًا."

س5: ما نظرية التقريب الشامل (Universal Approximation Theorem)؟ وما حدودها العملية؟

> **إجابة نموذجية:** "تنص على أن شبكة عصبية بطبقة خفية واحدة فقط، بعدد كافٍ من العصبونات ودالة تفعيل غير خطية مناسبة، يمكنها تقريب أي دالة مستمرة بدقة عشوائية. لكنها نظرية وجودية فقط (Existence Proof) — لا تُخبرنا كم عصبونًا نحتاج بالضبط، وقد يتطلب الأمر عمليًا عددًا هائلًا جدًا من العصبونات في طبقة واحدة. هذا بالضبط ما دفع المجال نحو التعلّم العميق: تكديس عدة طبقات أثبت تجريبيًا كفاءة أكبر بكثير (من ناحية عدد المعاملات الكلي اللازم) من طبقة واحدة عريضة جدًا."

🧪 2.9 تمرين عملي (Mini-Project)

العنوان: مختبر مقارنة دوال التفعيل

وسّع كود `MLPFromScratch` الذي بنيناه في هذا الفصل ليدعم **اختيار دالة التفعيل** (Sigmoid أو Tanh أو ReLU) كمعامل قابل للتخصيص، ثم:

1. درّب 3 نسخ من نفس الشبكة بالضبط (نفس البذرة العشوائية `seed`، نفس عدد العصبونات) على مسألة XOR، لكن بدالة تفعيل مختلفة في الطبقة الخفية لكل نسخة.
2. ارسم 3 منحنيات خسارة (Loss Curves) على نفس الرسم البياني للمقارنة.
3. سجّل: كم Epoch احتاجت كل نسخة لتصل لخسارة أقل من 0.01؟
4. **سؤال تأملي:** أي دالة تفعيل تقاربت أسرع في هذه المسألة الصغيرة تحديدًا؟ هل تتوقع نفس الترتيب في شبكة أعمق بكثير (50 طبقة)؟ لماذا؟

```python
# هيكل بداية لمساعدتك
class MLPFromScratchV2(MLPFromScratch):
    def __init__(self, *args, activation: str = "sigmoid", **kwargs):
        super().__init__(*args, **kwargs)
        self.activation_name = activation
        # TODO: لدعم اختيار دالة التفعيل ديناميكيًا عدّل forward و backward
# تلميح: ستحتاج أيضًا لتغيير مشتقة التفعيل المستخدمة في backward!

# TODO: درّب 3 نسخ وقارن النتائج في جدول ورسم بياني
```

🔗 جسر إلى الفصل القادم

لقد بنينا اليوم أول شبكة عصبية حقيقية تتعلّم من أخطائها، ورأينا بأعيننا كيف يُحوّل Chain Rule المشتقات المحلية البسيطة إلى تحديثات ذكية لكل وزن في الشبكة. لكننا في هذا الفصل استخدمنا **Gradient Descent الأبسط إمكانًا** — خطوة ثابتة الحجم في اتجاه معاكس تمامًا للتدرّج، بمعدل تعلّم واحد لا يتغيّر طوال التدريب.

في الفصل الثالث، سنكتشف **لماذا هذا النهج البسيط غير كافٍ عمليًا** لتدريب شبكات حقيقية بملايين المعاملات: سنرى كيف يمكن للتدرّج أن "يتذبذب" بشكل غير مستقر في أودية ضيقة من دالة الخسارة، ولماذا احتاج الباحثون لابتكار خوارزميات أذكى بكثير — من **Momentum** الذي يُحاكي "القصور الذاتي" الفيزيائي، إلى **Adam** الذي أصبح الخيار الافتراضي شبه الحصري في تدريب كل نماذج اللغة الكبيرة الحديثة. وسنغوص أيضًا في فخّ شائع جدًا يقع فيه حتى المهندسون المتمرّسون: نسيان `optimizer.zero_grad()`.

📚 2.10 مراجع للتعمّق

المصدر	لماذا تقرأه؟
Minsky & Papert, *"Perceptrons"* (1969)	الكتاب التاريخي الذي أثبت حدود Perceptron وتسبّب في "شتاء الذكاء الاصطناعي"
Cybenko, *"Approximation by Superpositions of a Sigmoidal Function"* (1989)	الإثبات الرياضي الأصلي لنظرية التقريب الشامل
Hendrycks & Gimpel, *"Gaussian Error Linear Units (GELUs)"* (2016)	الورقة الأصلية لدالة GELU المستخدمة في Transformers الحديثة
3Blue1Brown، سلسلة *"Neural Networks"* الحلقة 1-2	أفضل شرح بصري حدسي لـ Perceptron والشبكات متعددة الطبقات
Michael Nielsen, *"Neural Networks and Deep Learning"* (كتاب مجاني أونلاين)	شرح رياضي متعمّق مع كود Python من الصفر، الفصل 1 و2 تحديدًا

✅ **تم الانتهاء من الفصل الثاني.** انتقلنا من Perceptron بسيط عجز عن حل XOR، إلى شبكة MLP كاملة بنيناها يدويًا بـ NumPy ثم بـ PyTorch، مع فهم عميق لدوال التفعيل ودورها الحاسم.

هل أنتقل الآن إلى الفصل الثالث: "التدريب — Backpropagation والتحسين (Optimization)"؟

==========

الفصل الثالث: Backpropagation والتحسين (Optimization)



من "الشبكة تتعلم" إلى "كيف بالضبط تتعلم؟"

1. 🟢 لماذا يهمّك هذا؟

في الفصل السابق بنينا شبكة MLP حلّت مسألة XOR، وقلنا بشكل عابر: "الشبكة تتدرب عبر تعديل الأوزان بالتدريج حتى يقل الخطأ". لكن توقّف لحظة واسأل نفسك بصدق:

> عندما تكتب `loss.backward()` في PyTorch، **ماذا يحدث فعليًا داخل تلك الخطوة الواحدة؟**

ثم اسأل سؤالًا أعمق:

> لماذا لا نستخدم ببساطة *أكبر خطوة ممكنة* نحو الانحدار لنصل إلى الحل أسرع؟ ولماذا تفشل بعض الشبكات في التعلّم كليًا رغم أن الكود "صحيح" ولا يحتوي أخطاء برمجية؟

هذان السؤالان هما جوهر هذا الفصل. ستكتشف أن:

- **Backpropagation** ليست خوارزمية تعلّم بحد ذاتها، بل هي ببساطة **تطبيق ذكي ومنظّم لقاعدة السلسلة (Chain Rule)** على رسم حسابي ضخم، بحيث نحسب كل التدرجات (Gradients) بعملية واحدة بدل ملايين العمليات المكررة.
- **التحسين (Optimization)** هو فن اختيار حجم *واتجاه* كل خطوة تعلّم، وفشل 90% من مشاكل "النموذج لا يتعلم" يعود لاختيارات خاطئة هنا: معدل تعلم غير مناسب، تدرجات متفجرة، أو عدم جدولة صحيحة لمعدل التعلم.

بدون إتقان هذا الفصل، ستبقى تتعامل مع PyTorch كـ"صندوق أسود سحري". وبعد هذا الفصل، ستكون قادرًا على **تصحيح أعطال التدريب (Debugging Training)** باحترافية — وهي من أهم المهارات التي تُميّز مهندس LLM محترف عن مستخدم عادي لمكتبة.

---

3. 🟡 الشرح المتدرّج: من الحدس إلى الكود

3.1 الحدس أولًا: لماذا نحتاج Backpropagation أصلًا؟

تخيّل شبكة بها مليون وزن (Weight). لكل وزن، نحتاج أن نعرف: **"إذا غيّرت هذا الوزن بمقدار صغير جدًا، كم سيتغيّر الخطأ (Loss)؟"** هذا هو بالضبط تعريف المشتقة الجزئية ∂L/∂w.

السؤال الحقيقي هو: هل نحسب هذا السؤال مليون مرة بشكل منفصل (بطيء جدًا، O(مليون × تكلفة Forward Pass))، أم نجد طريقة أذكى؟

الجواب: **نعم توجد طريقة أذكى**، وهي الملاحظة التالية — الأوزان القريبة من المخرجات (الطبقات الأخيرة) "تشارك" في حساباتها مع الأوزان الأعمق (الطبقات الأولى) عبر نفس المسار الحسابي. فبدلًا من إعادة حساب كل شيء من الصفر لكل وزن، يمكننا **حساب التدرج مرة واحدة من المخرجات إلى المدخلات**، ونعيد استخدام النتائج الوسيطة. هذه هي الفكرة الكاملة وراء Backpropagation: **ديناميكية برمجية (Dynamic Programming) مطبّقة على قاعدة السلسلة**.

3.2 قاعدة السلسلة (Chain Rule) — حجر الأساس

إذا كان `y = f(g(x))`، فإن:

```
dy/dx = dy/dg · dg/dx
```

في شبكة عصبية، لدينا سلسلة طويلة من التراكيب: `L = Loss(Output(...(Layer2(Layer1(x)))))`. قاعدة السلسلة تمتد بشكل طبيعي:

```
∂L/∂W1 = ∂L/∂Output · ∂Output/∂Layer2 · ∂Layer2/∂Layer1 · ∂Layer1/∂W1
```

لاحظ أن الحد الأول `∂L/∂Output` نحسبه أولًا (من الأمام)، ثم "نمرره للخلف" مضروبًا في كل حد تالٍ. هذا بالضبط معنى كلمة **Back**-propagation: ننتشر بالتدرج من نهاية الشبكة إلى بدايتها.

3.3 الرسم الحسابي (Computational Graph)

كل عملية حسابية (جمع، ضرب، `matmul`، `sigmoid`...) هي **عقدة (Node)** في رسم بياني موجّه بلا دورات (DAG). PyTorch يبني هذا الرسم تلقائيًا أثناء الـ Forward Pass (يُسمى هذا **Define-by-Run** أو **Tape-based Autograd**)، ثم يمشي عكسيًا فوقه لحساب كل التدرجات.

```
x ──┐
    ├─[matmul]── z1 ──[+ b1]── a1 ──[ReLU]── h1 ──[matmul]── z2 ──[+ b2]── ŷ ──[CE Loss]── L
W1 ─┘                  |                            |
                    b1 (param)                  W2, b2 (params)
```

عند استدعاء `L.backward()`، يمشي PyTorch من `L` يسارًا، محسوبًا `∂L/∂node` لكل عقدة، مستخدمًا قاعدة السلسلة محليًا في كل عقدة (كل عملية "تعرف" كيف تفاضل نفسها — تسمى `grad_fn`).

3.4 دعنا نشتق Backprop يدويًا لشبكة بطبقتين (كما في الفصل 2)

لنفترض شبكة بسيطة:

```
z1 = X·W1 + b1
a1 = ReLU(z1)
z2 = a1·W2 + b2
ŷ  = softmax(z2)
L  = CrossEntropy(ŷ, y)
```

الخطوة 1 — تدرج طبقة الخرج (مجتمعة بشكل جميل رياضيًا):

من أجمل النتائج في Deep Learning: عندما نجمع بين Softmax و Cross-Entropy، فإن التدرج بالنسبة لمدخلات Softmax (أي z2) يُبسَّط إلى معادلة أنيقة جدًا:

```
∂L/∂z2 = ŷ - y_onehot
```

هذا تبسيط هائل — بدل اشتقاق Softmax ثم Cross-Entropy بشكل منفصل (وهو معقّد لأن Softmax تعتمد على كل العناصر)، نحصل على الفرق البسيط بين التوقع والحقيقة مباشرة.

الخطوة 2 — الانتشار للخلف عبر الطبقة الثانية:

```
∂L/∂W2 = a1ᵀ · ∂L/∂z2
∂L/∂b2 = Σ ∂L/∂z2   (على محور الدفعة Batch)
∂L/∂a1 = ∂L/∂z2 · W2ᵀ
```

الخطوة 3 — عبور دالة ReLU:

```
∂L/∂z1 = ∂L/∂a1 ⊙ 1[z1 > 0]
```

(رمز ⊙ يعني الضرب عنصرًا بعنصر، و `1[z1>0]` قناع ثنائي: 1 حيث z1 موجب، 0 حيث سالب أو صفر — هذا الاشتقاق البسيط لـ ReLU هو أحد أسباب شهرتها).

الخطوة 4 — الطبقة الأولى:

```
∂L/∂W1 = Xᵀ · ∂L/∂z1
∂L/∂b1 = Σ ∂L/∂z1
```

والآن دعنا نُترجم هذا حرفيًا إلى كود — أولًا **يدويًا بدون autograd** لنفهم الآلية تمامًا، ثم نقارنها بـ PyTorch autograd للتأكد من تطابق النتائج.

```python
"""
backprop_manual.py
تطبيق يدوي كامل لـ Forward + Backward Pass لشبكة MLP بطبقتين،
ثم التحقق من صحته مقابل torch.autograd.
Python 3.13+ | NumPy فقط للجزء اليدوي، ثم PyTorch للمقارنة.
"""
from __future__ import annotations

import numpy as np
import torch
import torch.nn.functional as F

rng = np.random.default_rng(seed=42)


def relu(z: np.ndarray) -> np.ndarray:
    return np.maximum(0.0, z)


def softmax(z: np.ndarray) -> np.ndarray:
    z_shifted = z - z.max(axis=1, keepdims=True)  # استقرار عددي
    exp_z = np.exp(z_shifted)
```

```
    return exp_z / exp_z.sum(axis=1, keepdims=True)


def cross_entropy(y_pred: np.ndarray, y_true_idx: np.ndarray) -> float:
    n = y_pred.shape[0]
    log_likelihood = -np.log(y_pred[np.arange(n), y_true_idx] + 1e-12)
    return float(log_likelihood.mean())


def manual_forward_backward(
    x: np.ndarray,
    y_idx: np.ndarray,
    w1: np.ndarray, b1: np.ndarray,
    w2: np.ndarray, b2: np.ndarray,
) -> dict[str, np.ndarray]:
    n = x.shape[0]
    num_classes = w2.shape[1]

    # ---------- Forward Pass ----------
    z1 = x @ w1 + b1
    a1 = relu(z1)
    z2 = a1 @ w2 + b2
    y_hat = softmax(z2)
    loss = cross_entropy(y_hat, y_idx)

    # ---------- Backward Pass (يدويًا، بقاعدة السلسلة) ----------
    y_onehot = np.zeros_like(y_hat)
    y_onehot[np.arange(n), y_idx] = 1.0

    d_z2 = (y_hat - y_onehot) / n          # الصيغة الأنيقة Softmax+CE
    d_w2 = a1.T @ d_z2
    d_b2 = d_z2.sum(axis=0)
    d_a1 = d_z2 @ w2.T
    d_z1 = d_a1 * (z1 > 0).astype(np.float64)   # اشتقاق ReLU
    d_w1 = x.T @ d_z1
    d_b1 = d_z1.sum(axis=0)

    return {
        "loss": np.array(loss),
        "d_w1": d_w1, "d_b1": d_b1,
        "d_w2": d_w2, "d_b2": d_b2,
    }


def torch_forward_backward(
    x: np.ndarray, y_idx: np.ndarray,
    w1: np.ndarray, b1: np.ndarray,
    w2: np.ndarray, b2: np.ndarray,
) -> dict[str, torch.Tensor]:
    x_t = torch.tensor(x, dtype=torch.float64)
    y_t = torch.tensor(y_idx, dtype=torch.long)
    w1_t = torch.tensor(w1, dtype=torch.float64, requires_grad=True)
    b1_t = torch.tensor(b1, dtype=torch.float64, requires_grad=True)
    w2_t = torch.tensor(w2, dtype=torch.float64, requires_grad=True)
    b2_t = torch.tensor(b2, dtype=torch.float64, requires_grad=True)
```

```
    z1 = x_t @ w1_t + b1_t
    a1 = torch.relu(z1)
    z2 = a1 @ w2_t + b2_t
    loss = F.cross_entropy(z2, y_t)   # يطبّق log-softmax + NLL داخليًا
    loss.backward()

    return {
        "loss": loss.detach(),
        "d_w1": w1_t.grad, "d_b1": b1_t.grad,
        "d_w2": w2_t.grad, "d_b2": b2_t.grad,
    }


if __name__ == "__main__":
    n_samples, in_dim, hidden_dim, out_dim = 8, 4, 5, 3
    x = rng.normal(size=(n_samples, in_dim))
    y_idx = rng.integers(0, out_dim, size=n_samples)
    w1 = rng.normal(scale=0.1, size=(in_dim, hidden_dim))
    b1 = np.zeros(hidden_dim)
    w2 = rng.normal(scale=0.1, size=(hidden_dim, out_dim))
    b2 = np.zeros(out_dim)

    manual = manual_forward_backward(x, y_idx, w1, b1, w2, b2)
    auto = torch_forward_backward(x, y_idx, w1, b1, w2, b2)

    print(f"Loss (يدوي)   : {manual['loss']:.6f}")
    print(f"Loss (PyTorch): {auto['loss'].item():.6f}")

    for name in ["d_w1", "d_b1", "d_w2", "d_b2"]:
        diff = np.abs(manual[name] - auto[name].numpy()).max()
        print(f"أقصى فرق في {name}: {diff:.2e}")
```

الناتج المتوقع: الخسارة متطابقة، وأقصى فرق في كل تدرج أقل من `1e-10` (فروقات عددية فقط بسبب الدقة العائمة). هذا **تمرين تحقّق (Gradient Check)** حقيقي — وهي تقنية أساسية سنعود إليها في صندوق التعمّق.

🔴 صندوق التعمّق: Gradient Checking بالطريقة العددية

كيف تتأكد أن اشتقاقك التحليلي (Analytical Gradient) صحيح دون مقارنته بـ PyTorch؟ تستخدم **الفرق المركزي المحدود (Centered Finite Difference)**: `∂L/∂w ≈ [L(w + ε) − L(w − ε)] / (2ε)` مع `ε ≈ 1e-5` عادة. إذا كان الفرق النسبي بين القيمة التحليلية والعددية أقل من `1e-7`، فاشتقاقك صحيح على الأرجح. هذه الطريقة بطيئة جدًا (تحتاج تمريرتين Forward لكل وزن) لذا **لا تُستخدم في التدريب الفعلي أبدًا**، لكنها أداة تصحيح أخطاء لا تُقدّر بثمن عند بناء طبقات مخصصة (Custom Layers) جديدة.

3.5 الآن: التحسين (Optimization) — ماذا نفعل بالتدرج بعد حسابه؟

بعد حساب $\partial L/\partial w$ لكل وزن، السؤال التالي: **كيف نُحدّث الوزن؟** هذا هو دور الـ **Optimizer**. دعنا نبني فهمنا تصاعديًا.

أ) Gradient Descent البسيط (Vanilla SGD)

$$w \leftarrow w - \eta \cdot \partial L/\partial w$$

حيث η (إيتا) هو **معدل التعلم (Learning Rate)**. تخيّل نفسك تنزل جبلًا في الضباب الكثيف، لا ترى القمة ولا القاع، فقط تشعر بميل الأرض تحت قدميك (التدرج) وتخطو خطوة في اتجاه الانحدار الأشد. حجم الخطوة هو η.

لماذا تفشل هذه الطريقة البسيطة أحيانًا؟ - إذا كان η كبيرًا جدًا: تتخطى القاع ذهابًا وإيابًا (Oscillation) وقد تتباعد (Diverge). - إذا كان η صغيرًا جدًا: التقارب بطيء جدًا، وقد تعلق في **سهل مستوٍ (Plateau)** أو **نقطة سرج (Saddle Point)** لفترة طويلة جدًا. - في الأسطح غير المتناظرة (التي تنحدر بشدة في اتجاه وبلطف في اتجاه آخر — شائعة جدًا في الشبكات العميقة)، SGD البسيط "يتذبذب" بشدة بدل التحرك بسلاسة نحو الحل.

ب) إضافة الزخم (Momentum)

الفكرة: بدل الاعتماد فقط على التدرج الحالي، نحتفظ بـ"سرعة" تراكمية (كرة تتدحرج، تكتسب زخمًا، ولا تتوقف فجأة عند كل تعرّج صغير):

$$v \leftarrow \beta \cdot v + \partial L/\partial w \qquad (\beta \text{ عادة } 0.9)$$
$$w \leftarrow w - \eta \cdot v$$

هذا يُخمّد التذبذبات العمودية (حيث يتغير اتجاه التدرج باستمرار) بينما يُسرّع الحركة في الاتجاه المتسق (حيث التدرج يشير دائمًا لنفس الجهة تقريبًا).

Nesterov Accelerated Gradient (NAG) هو تحسين ذكي: بدل حساب التدرج عند الموقع الحالي w، نحسبه عند الموقع "المتوقّع" بعد تطبيق الزخم $w - \beta \cdot v$ (أي ننظر قليلًا إلى الأمام قبل أن نأخذ القرار النهائي — كأنك "تستطلع" الطريق قبل أن تندفع فيه بكامل سرعتك).

ج) معدلات تعلّم تكيّفية (Adaptive Learning Rates)

السؤال التالي: لماذا نستخدم *نفس* معدل التعلم η لكل وزن في الشبكة، رغم أن بعض الأوزان تحتاج تحديثات كبيرة وأخرى تحتاج تحديثات دقيقة جدًا؟

AdaGrad كانت أول محاولة: تقسم η على الجذر التربيعي لمجموع مربعات التدرجات التاريخية لكل وزن على حدة:

```
G ← G + (∂L/∂w)²
w ← w − η/√(G + ε) · ∂L/∂w
```

المشكلة: `G` يتراكم بلا توقف فيصبح معدل التعلم الفعّال صغيرًا جدًا مع الوقت حتى يتوقف التعلم تمامًا — مشكلة قاتلة في التدريب الطويل.

RMSProp حلّ هذه المشكلة بمتوسط متحرك أسّي (Exponential Moving Average) بدل التراكم المطلق:

```
G ← γ·G + (1−γ)·(∂L/∂w)²        (γ عادة 0.99)
w ← w − η/√(G + ε) · ∂L/∂w
```

د) Adam — دمج الزخم مع المعدلات التكيّفية

Adam (Adaptive Moment Estimation)، المعيار الفعلي اليوم في تدريب الشبكات العميقة واللغوية، يدمج فكرتي Momentum و RMSProp معًا، مع تصحيح تحيّز (Bias Correction) ذكي:

```
m ← β1·m + (1−β1)·g              # متوسط متحرك للتدرج (العزم الأول)
v ← β2·v + (1−β2)·g²             # متوسط متحرك لمربع التدرج (العزم الثاني)

m̂ ← m / (1 − β1^t)               # تصحيح التحيّز
v̂ ← v / (1 − β2^t)

w ← w − η · m̂ / (√v̂ + ε)
```

حيث `g = ∂L/∂w`، و`t` هو رقم الخطوة الحالية (يبدأ من 1)، والقيم الافتراضية الشائعة: `β1=0.9, β2=0.999,` `ε=1e-8`.

لماذا تصحيح التحيّز ضروري؟ في البداية `m` و `v` يبدآن من صفر، مما يجعلهما منحازين نحو الصفر في الخطوات الأولى. القسمة على `(1-β1^t)` و `(1-β2^t)` تُصحّح هذا الانحياز — خاصة حين `t` صغير (في بداية التدريب تحديدًا، حيث يكون التصحيح الأكبر، ثم يتلاشى أثره تدريجيًا مع زيادة `t`).

هـ) AdamW — الإصلاح الحاسم لـ L2 Regularization

هنا نصل لنقطة دقيقة جدًا غالبًا ما تُفهم خطأ. في Adam الأصلي، إذا أضفت **Weight Decay** (تثبيط الأوزان، وهو شكل من L2 Regularization يمنع الأوزان من النمو الكبير) عبر إضافته لدالة الخسارة:

```
L_total = L + λ/2 · ||w||²
```

فإن هذا يُترجم إلى تدرج إضافي `λ·w` يُضاف إلى `g` **قبل** حساب `m` و `v`. المشكلة: هذا يجعل تأثير الـ Weight Decay "يتشابك" مع المعدلات التكيّفية بطريقة غير مقصودة وغير فعّالة — الأوزان ذات التدرجات الكبيرة تتلقى تثبيطًا نسبيًا أقل من المفترض، والعكس صحيح.

Loshchilov & Hutter (2017) في ورقتهم الشهيرة "Decoupled Weight Decay Regularization" اقترحوا **AdamW**: افصل تطبيق Weight Decay عن حساب العزوم تمامًا، وطبّقه مباشرة على الوزن في خطوة التحديث النهائية:

```
m ← β1·m + (1−β1)·g
v ← β2·v + (1−β2)·g²
m̂ ← m / (1 − β1^t)
v̂ ← v / (1 − β2^t)

w ← w − η · ( m̂/(√v̂+ε) + λ·w )      # لاحظ: λ·w مُضافة هنا مباشرة، منفصلة عن m,v
```

هذا الفرق البسيط ظاهريًا يحسّن التعميم (Generalization) بشكل ملحوظ، ولهذا **AdamW هو الخيار الافتراضي عمليًا في كل عمليات تدريب الـ LLM الحديثة تقريبًا** (GPT، LLaMA، Mistral، جميعها تستخدم AdamW).

```python
"""
optimizers_from_scratch.py
تطبيق SGD+Momentum و Adam و AdamW يدويًا بـ PyTorch Tensors الخام،
لفهم الآلية الداخلية، ثم مقارنتها بتطبيقات torch.optim الرسمية.
"""
from __future__ import annotations

from dataclasses import dataclass, field

import torch


@dataclass
class SGDMomentum:
    params: list[torch.Tensor]
    lr: float = 0.01
    momentum: float = 0.9
    velocities: list[torch.Tensor] = field(init=False)

    def __post_init__(self) -> None:
        self.velocities = [torch.zeros_like(p) for p in self.params]

    @torch.no_grad()
    def step(self) -> None:
        for p, v in zip(self.params, self.velocities):
            if p.grad is None:
                continue
            v.mul_(self.momentum).add_(p.grad)
            p.sub_(self.lr * v)

    def zero_grad(self) -> None:
        for p in self.params:
            if p.grad is not None:
                p.grad.zero_()
```

```
@dataclass
class AdamW:
    params: list[torch.Tensor]
    lr: float = 1e-3
    betas: tuple[float, float] = (0.9, 0.999)
    eps: float = 1e-8
    weight_decay: float = 0.01
    t: int = field(default=0, init=False)
    m: list[torch.Tensor] = field(init=False)
    v: list[torch.Tensor] = field(init=False)

    def __post_init__(self) -> None:
        self.m = [torch.zeros_like(p) for p in self.params]
        self.v = [torch.zeros_like(p) for p in self.params]

    @torch.no_grad()
    def step(self) -> None:
        self.t += 1
        beta1, beta2 = self.betas
        for p, m, v in zip(self.params, self.m, self.v):
            if p.grad is None:
                continue
            g = p.grad
            m.mul_(beta1).add_(g, alpha=1 - beta1)
            v.mul_(beta2).addcmul_(g, g, value=1 - beta2)

            m_hat = m / (1 - beta1 ** self.t)
            v_hat = v / (1 - beta2 ** self.t)

# تحديث AdamW: الفصل الكامل بين weight decay والعزوم التكيّفية
            p.sub_(self.lr * (m_hat / (v_hat.sqrt() + self.eps)))
            p.sub_(self.lr * self.weight_decay * p)

    def zero_grad(self) -> None:
        for p in self.params:
            if p.grad is not None:
                p.grad.zero_()


if __name__ == "__main__":
    torch.manual_seed(0)

# مشكلة تصغير تربيعية بسيطة لاختبار الـ optimizers: f(w) = (w - 3)^2
    def toy_loss(w: torch.Tensor) -> torch.Tensor:
        return ((w - 3.0) ** 2).sum()

    for OptClass, kwargs in [
        (SGDMomentum, {"lr": 0.1, "momentum": 0.9}),
        (AdamW, {"lr": 0.3, "weight_decay": 0.0}),
    ]:
        w = torch.tensor([0.0], requires_grad=True)
        opt = OptClass(params=[w], **kwargs)
        for step in range(30):
            opt.zero_grad()
            loss = toy_loss(w)
```

```
            loss.backward()
            opt.step()
        print(f"{OptClass.__name__}: w النهائي = {w.item():.4f} (3.0 = الهدف)")
```

3.6 جدولة معدل التعلم (Learning Rate Scheduling)

سؤال طبيعي الآن: هل يجب أن يبقى η ثابتًا طوال التدريب؟ الجواب: **لا أبدًا**. الحدس: في بداية التدريب (حيث الأوزان عشوائية والتدرجات قد تكون كبيرة وغير مستقرة)، نريد خطوات أصغر وحذرة (**Warmup**)؛ وفي منتصف التدريب نريد خطوات أكبر للتقدم السريع؛ وفي نهاية التدريب، نريد خطوات أصغر فأصغر للاستقرار الدقيق قرب الحل الأمثل (**Decay**).

أشهر الاستراتيجيات:

1) Warmup الخطي ثم Cosine Decay — المعيار الذهبي الحالي لتدريب LLM:

```
:step < warmup_steps إذا
    lr = max_lr × (step / warmup_steps)
:وإلا
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    lr = min_lr + 0.5 × (max_lr - min_lr) × (1 + cos(π × progress))
```

2) Step Decay: تقليل `lr` بعامل ثابت كل عدد معيّن من الخطوات (مثال: ÷10 كل 30 Epoch) — بسيطة لكنها أقل سلاسة.

3) OneCycle (Leslie Smith, 2018): يرفع `lr` من قيمة منخفضة إلى قيمة ذروة ثم يخفضه إلى ما دون القيمة الأولية — أظهر تقاربًا أسرع بشكل ملحوظ ("Super-Convergence") في كثير من المسائل.

```
"""
lr_schedulers.py
جدولة معدل التعلم: Warmup + Cosine Decay، تطبيق يدوي ومطابقته
.torch.optim.lr_scheduler بـ
"""
from __future__ import annotations

import math

import torch
from torch.optim.lr_scheduler import LambdaLR


def cosine_warmup_lambda(
    step: int,
    *,
    warmup_steps: int,
    total_steps: int,
    min_lr_ratio: float = 0.1,
) -> float:
```

```
    """.LambdaLR متوافق مع .base_lr يُطبَّق على (Multiplier) يُعيد معامل ضرب"""
    if step < warmup_steps:
        return step / max(1, warmup_steps)
    progress = (step - warmup_steps) / max(1, total_steps - warmup_steps)
    progress = min(progress, 1.0)
    cosine_term = 0.5 * (1.0 + math.cos(math.pi * progress))
    return min_lr_ratio + (1.0 - min_lr_ratio) * cosine_term


def build_scheduler(
    optimizer: torch.optim.Optimizer,
    *,
    warmup_steps: int,
    total_steps: int,
) -> LambdaLR:
    fn = lambda step: cosine_warmup_lambda(  # noqa: E731
        step, warmup_steps=warmup_steps, total_steps=total_steps
    )
    return LambdaLR(optimizer, lr_lambda=fn)


if __name__ == "__main__":
    dummy_param = torch.nn.Parameter(torch.zeros(1))
    optimizer = torch.optim.AdamW([dummy_param], lr=3e-4)
    scheduler = build_scheduler(optimizer, warmup_steps=100, total_steps=1000)

    lrs = []
    for step in range(1000):
        lrs.append(optimizer.param_groups[0]["lr"])
        optimizer.step()
        scheduler.step()

    print(f"lr عند step=0    : {lrs[0]:.6f}")
    print(f"lr عند step=100  : {lrs[100]:.6f}  (ذروة Warmup)")
    print(f"lr عند step=550  : {lrs[550]:.6f}  (منتصف الانحدار الجيبي)")
    print(f"lr عند step=999  : {lrs[999]:.6f}  (قرب النهاية)")
```

3.7 قصّ التدرجات (Gradient Clipping)

السؤال الأخير في معادلة الاستقرار: ماذا لو كان التدرج نفسه **ضخمًا بشكل غير طبيعي** في خطوة معيّنة (بسبب دفعة بيانات شاذة، أو تراكمات رقمية في شبكة عميقة جدًا)؟ تحديث واحد بتدرج هائل قد "يُفجّر" الأوزان ويدمّر تدريبًا استغرق ساعات (ظاهرة تُعرف بـ **Exploding Gradients**، شائعة جدًا في الشبكات العميقة والمتكررة RNN، ولا تزال تحدث أحيانًا حتى في المحوّلات الكبيرة).

الحل: Gradient Clipping بنوعيه:

أ) Clipping بالقيمة (Value Clipping): كل عنصر في التدرج يُقصّ إلى مجال `[-c, c]`:

```
torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=1.0)
```

ب) Clipping بالمعيار (Norm Clipping) — الأكثر استخدامًا في تدريب LLM: إذا تجاوز **المعيار الإقليدي الكلي** لكل التدرجات مجتمعة قيمة حدّية `max_norm`، نُقلّص **كل** التدرجات بنفس النسبة للحفاظ على اتجاهها النسبي:

```
total_norm = √(Σ ||g_i||²)      في النموذج g_i لكل التدرجات
:total_norm > max_norm إذا
g_i: g_i ← g_i × (max_norm / total_norm) لكل
```

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

لماذا Norm Clipping أفضل من Value Clipping عادة؟ لأنه يحافظ على **اتجاه (Direction)** التدرج الكلي سليمًا (فقط يُقلّص الحجم)، بينما Value Clipping يُشوّه الاتجاه النسبي بين العناصر المختلفة للتدرج.

---

4. 🧪 مثال محلول خطوة بخطوة: حلقة تدريب كاملة احترافية

لنجمع كل ما تعلمناه في مثال End-to-End كامل: تدريب شبكة MLP بسيطة على بيانات تصنيف اصطناعية، باستخدام AdamW + Warmup/Cosine Scheduling + Gradient Clipping، مع طباعة كل الإحصائيات المهمة.

```
"""
full_training_loop.py
.AdamW + LR Scheduling + Gradient Clipping مع للتصنيف MLP تدريب :مثال متكامل
.يوضح حلقة تدريب احترافية جاهزة للتوسعة نحو نماذج أكبر
"""
from __future__ import annotations

import math
from dataclasses import dataclass

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.optim.lr_scheduler import LambdaLR
from torch.utils.data import DataLoader, TensorDataset


@dataclass(frozen=True)
class TrainConfig:
    hidden_dim: int = 128
    num_classes: int = 4
    batch_size: int = 64
    num_epochs: int = 20
    max_lr: float = 3e-3
    weight_decay: float = 0.01
```

```
    warmup_ratio: float = 0.1
    max_grad_norm: float = 1.0
    seed: int = 42


class MLPClassifier(nn.Module):
    def __init__(self, in_dim: int, hidden_dim: int, num_classes: int) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


def make_synthetic_dataset(
    n_samples: int, in_dim: int, num_classes: int, seed: int
) -> TensorDataset:
    gen = torch.Generator().manual_seed(seed)
    centers = torch.randn(num_classes, in_dim, generator=gen) * 4.0
    labels = torch.randint(0, num_classes, (n_samples,), generator=gen)
    noise = torch.randn(n_samples, in_dim, generator=gen)
    features = centers[labels] + noise
    return TensorDataset(features, labels)


def cosine_warmup_lambda(step: int, warmup_steps: int, total_steps: int) -> float:
    if step < warmup_steps:
        return (step + 1) / warmup_steps
    progress = (step - warmup_steps) / max(1, total_steps - warmup_steps)
    return 0.5 * (1.0 + math.cos(math.pi * min(progress, 1.0)))


def train(cfg: TrainConfig) -> MLPClassifier:
    torch.manual_seed(cfg.seed)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    in_dim = 16
    train_ds = make_synthetic_dataset(2_000, in_dim, cfg.num_classes, cfg.seed)
    val_ds = make_synthetic_dataset(400, in_dim, cfg.num_classes, cfg.seed + 1)

    train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=True)
    val_loader = DataLoader(val_ds, batch_size=cfg.batch_size, shuffle=False)

    model = MLPClassifier(in_dim, cfg.hidden_dim, cfg.num_classes).to(device)
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=cfg.max_lr, weight_decay=cfg.weight_decay
    )

    total_steps = cfg.num_epochs * len(train_loader)
```

```
    warmup_steps = max(1, int(cfg.warmup_ratio * total_steps))
    scheduler = LambdaLR(
        optimizer,
        lr_lambda=lambda s: cosine_warmup_lambda(s, warmup_steps, total_steps),
    )

    global_step = 0
    for epoch in range(cfg.num_epochs):
        model.train()
        running_loss = 0.0
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)

            optimizer.zero_grad(set_to_none=True)
            logits = model(xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()

# --- قصّ التدرجات قبل خطوة التحديث مباشرة --- #
            grad_norm = torch.nn.utils.clip_grad_norm_(
                model.parameters(), max_norm=cfg.max_grad_norm
            )

            optimizer.step()
            scheduler.step()

            running_loss += loss.item() * xb.size(0)
            global_step += 1

        train_loss = running_loss / len(train_ds)
        val_acc = evaluate(model, val_loader, device)
        current_lr = scheduler.get_last_lr()[0]

        print(
            f"Epoch {epoch+1:02d}/{cfg.num_epochs} | "
            f"train_loss={train_loss:.4f} | val_acc={val_acc:.3f} | "
            f"lr={current_lr:.5f} | grad_norm={grad_norm:.3f}"
        )

    return model


@torch.no_grad()
def evaluate(model: nn.Module, loader: DataLoader, device: torch.device) -> float:
    model.eval()
    correct, total = 0, 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        preds = model(xb).argmax(dim=-1)
        correct += (preds == yb).sum().item()
        total += yb.size(0)
    return correct / total
```

```python
if __name__ == "__main__":
    trained_model = train(TrainConfig())
```

لاحظ هذه التفاصيل الاحترافية في الكود أعلاه، فكل واحدة منها "أفضل ممارسة" حقيقية: - `optimizer.zero_grad(set_to_none=True)` أسرع من `zero_grad()` العادية (يُحرّر الذاكرة بدل تصفيرها). - ترتيب العمليات: `backward()` ← `clip_grad_norm_()` ← `optimizer.step()` ← `scheduler.step()` — هذا الترتيب **إلزامي** ولا يجوز تبديله. - `scheduler.step()` يُستدعى **بعد** كل `optimizer.step()` (بمستوى الـ Step وليس الـ Epoch) عند استخدام Warmup/Cosine. - طباعة `grad_norm` في كل Epoch هي أداة تشخيصية بسيطة لكنها قوية جدًا لرصد مشاكل التدريب مبكرًا.

---

5. 💡 نصائح وحيل (Tips & Tricks)

- **قاعدة ذهبية لمعدل التعلم:** عند مضاعفة حجم الدفعة (Batch Size)، يمكن تقريبًا مضاعفة معدل التعلم أيضًا (Linear Scaling Rule من ورقة Goyal et al. 2017)، لكن يجب إضافة Warmup أطول لتعويض عدم الاستقرار الأولي.
- **ابدأ دائمًا بـ AdamW** كخيار افتراضي آمن لتدريب الشبكات العميقة والمحوّلات، إلا إذا كان لديك سبب محدد (كـ SGD+Momentum في بعض نماذج الرؤية الكلاسيكية التي تحقق تعميمًا أفضل أحيانًا).
- **راقب نسبة `grad_norm / max_norm`:** إذا كانت القيمة تتجاوز الحد الأقصى في أغلب الخطوات (أي أن القص يحدث دائمًا)، هذا مؤشر خطر — قد يعني معدل تعلم مرتفع جدًا أو مشكلة في تهيئة الأوزان (Initialization).
- **Gradient Accumulation:** إذا كانت الذاكرة لا تكفي لدفعة كبيرة، اجمع التدرجات عبر عدة دفعات صغيرة قبل استدعاء `optimizer.step()` — هذا يُحاكي تأثير دفعة أكبر دون الحاجة لذاكرة أكبر (أساسي في تدريب LLM).
- **قيمة β2 الافتراضية 0.999 قد لا تناسب كل الحالات:** في تدريب الـ LLM الكبيرة أحيانًا يُستخدم `β2=0.95` بدل `0.999` (كما في بعض إعدادات LLaMA) لاستجابة أسرع للتغيرات، خاصة مع معدلات تعلم مرتفعة نسبيًا.
- **استخدم `torch.compile()`** (المتاح من PyTorch 2.0+) لتسريع حلقة التدريب تلقائيًا دون تغيير منطق الكود، عبر دمج العمليات (Kernel Fusion) خلف الكواليس.

---

6. ⚠️ أخطاء شائعة ومصائد (Common Pitfalls)

الخطأ 1: نسيان `optimizer.zero_grad()`

```python
# ❌ خطأ: التدرجات تتراكم بين الدفعات دون قصد!
for xb, yb in loader:
    loss = F.cross_entropy(model(xb), yb)
```

```
    loss.backward()
    optimizer.step()
```

```
# ✅ صحيح: تصفير التدرجات قبل كل backward جديد
for xb, yb in loader:
    optimizer.zero_grad(set_to_none=True)
    loss = F.cross_entropy(model(xb), yb)
    loss.backward()
    optimizer.step()
```

في PyTorch، `.grad` يتراكم (Accumulate) افتراضيًا بدل أن يُستبدل في كل استدعاء لـ `backward()`. هذا السلوك مفيد عمدًا (يُستخدم في Gradient Accumulation)، لكنه فخ قاتل إن نسيته.

الخطأ 2: استدعاء `scheduler.step()` قبل `optimizer.step()`

```
# ❌ خطأ شائع جدًا (وسيُطلق تحذيرًا بشأنه PyTorch)
scheduler.step()
optimizer.step()
```

```
# ✅ الترتيب الصحيح دائمًا
optimizer.step()
scheduler.step()
```

الخطأ 3: استخدام نفس `learning rate` كبير مع Fine-tuning

```
# ❌ استخدام lr تدريب من الصفر على نموذج مُدرّب مسبقًا يُدمّر أوزانه المُتعلّمة
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3)
```

```
# ✅ معدلات تعلم أصغر بكثير (عادة 10x-100x أقل) عند ضبط نموذج مُدرّب مسبقًا
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
```

الخطأ 4: تطبيق Weight Decay على جميع المعاملات بلا تمييز

```
# ❌ تطبيق weight decay على LayerNorm/Bias يضر بالأداء دون فائدة تُذكر
optimizer = torch.optim.AdamW(model.parameters(), weight_decay=0.1)
```

```
# ✅ الممارسة المعيارية: استثناء Bias وLayerNorm/Embeddings من Weight Decay
def get_param_groups(model: nn.Module, weight_decay: float) -> list[dict]:
    decay, no_decay = [], []
    for name, param in model.named_parameters():
        if not param.requires_grad:
```

```
            continue
        if param.ndim <= 1 or "bias" in name or "norm" in name.lower():
            no_decay.append(param)
        else:
            decay.append(param)
    return [
        {"params": decay, "weight_decay": weight_decay},
        {"params": no_decay, "weight_decay": 0.0},
    ]

optimizer = torch.optim.AdamW(get_param_groups(model, weight_decay=0.1), lr=3e-4)
```

الخطأ 5: تجاهل انفجار/تلاشي التدرجات (Exploding/Vanishing Gradients)

إذا لاحظت `loss` يصبح `NaN` فجأة، فمن أشيع الأسباب: تدرجات متفجرة بلا Clipping، أو معدل تعلم مرتفع جدًا، أو عدم استقرار عددي في `exp` / `log` (مثل حساب Softmax بدون طرح القيمة العظمى أولًا — راجع الفصل الأول).

```
# ✅ فحص دفاعي بسيط يُضاف في حلقة التدريب لاكتشاف المشكلة فورًا
if torch.isnan(loss) or torch.isinf(loss):
    raise RuntimeError(f"قيمة Loss غير صالحة عند step={global_step}: {loss.item()}")
```

7. ⚖️ جدول مقارنة: خوارزميات التحسين

الخوارزمية	الفكرة الأساسية	المزايا	العيوب	أين تُستخدم عادة
SGD	خطوة ثابتة بالتدرج الخام	بسيط، ذاكرة أقل، تعميم جيد أحيانًا	بطيء، حساس جدًا لـ `lr`، يتذبذب	بعض نماذج الرؤية الكلاسيكية
SGD + Momentum	تراكم "سرعة" بين الخطوات	يُخمّد التذبذب، تقارب أسرع من SGD	لا يزال يحتاج ضبط `lr` يدويًا بدقة	ResNet وشبكات CNN كلاسيكية
RMSProp	معدل تعلم تكيّفي لكل وزن	يتعامل جيدًا مع المشاهد غير المتجانسة	لا يستخدم Momentum بشكل كامل	RNN قديمًا، بعض نماذج RL
Adam	يدمج Momentum + معدل تكيّفي	تقارب سريع، يحتاج ضبط أقل، معيار شائع	استهلاك ذاكرة أعلى (عزمان لكل وزن)	شبكات عميقة متنوعة
AdamW	Adam + فصل Weight Decay	تعميم أفضل من Adam، معيار LLM الحالي	نفس استهلاك ذاكرة Adam	**كل نماذج اللغة الحديثة تقريبًا**
Lion (حديث)	يستخدم فقط إشارة التدرج (`sign`)	ذاكرة أقل من Adam (عزم واحد فقط)	يحتاج `lr` أصغر و `weight_decay` أكبر عادة	بعض تجارب Google الحديثة

8. ✅ قائمة التحقق من أفضل الممارسات

- [ ] استخدمت `AdamW` كخيار افتراضي، وليس `Adam` أو `SGD` بدون سبب وجيه.
- [ ] طبّقت Learning Rate Warmup في بداية التدريب (خصوصًا للمحوّلات والنماذج العميقة).
- [ ] استخدمت جدولة Cosine Decay أو ما يعادلها بدل `lr` ثابت طوال التدريب.
- [ ] طبّقت `clip_grad_norm_` قبل كل `optimizer.step()` مباشرة.
- [ ] تحققت من الترتيب الصحيح: `backward()` ← `clip` ← `optimizer.step()` ← `scheduler.step()`.
- [ ] استثنيت Bias وLayerNorm من Weight Decay عبر Param Groups.
- [ ] تستخدم `optimizer.zero_grad(set_to_none=True)` بدل الافتراضي.
- [ ] تراقب `grad_norm` و`loss` في كل خطوة/حقبة لاكتشاف NaN أو الانفجار مبكرًا.
- [ ] عند Fine-tuning، تستخدم معدل تعلم أصغر بكثير من التدريب من الصفر.
- [ ] كتبت Gradient Check يدوي عند بناء أي طبقة مخصصة جديدة (Custom Autograd Function).

---

9. 🎯 أسئلة مقابلات شخصية

س1: ما الفرق الجوهري بين Adam و AdamW؟ ولماذا يُفضَّل الثاني في تدريب LLM؟

> الفرق أن Adam يُضيف Weight Decay كجزء من التدرج (`g += λw`) قبل حساب العزوم المتحركة `m` و`v`، مما يجعل تأثيره يتشابك مع معدل التعلم التكيّفي بطريقة غير مقصودة. AdamW يفصل Weight Decay تمامًا ويطبّقه مباشرة على الوزن في خطوة التحديث (`w -= lr·λ·w`)، مما يحافظ على معناه الأصلي كمُنظّم (Regularizer) حقيقي. هذا الفصل يُحسّن التعميم بشكل ملحوظ تجريبيًا، ولهذا تتبناه كل نماذج اللغة الحديثة تقريبًا.

س2: لماذا نحتاج Learning Rate Warmup في بداية تدريب المحوّلات تحديدًا؟

> في بداية التدريب، الأوزان عشوائية والتدرجات (خصوصًا في طبقات Attention وLayerNorm) قد تكون كبيرة وغير مستقرة إحصائيًا. معدل تعلم كبير منذ الخطوة الأولى قد يدفع الأوزان بعيدًا جدًا بسرعة فتصبح دالة الخسارة غير مستقرة (أو حتى NaN). Warmup يبدأ بمعدل تعلم صغير جدًا ويرفعه تدريجيًا، مما يسمح للنموذج "بالاستقرار" إحصائيًا أولًا (خصوصًا عزوم Adam `m` و`v` التي تحتاج خطوات كافية لتعطي تقديرات موثوقة) قبل اتخاذ خطوات كبيرة.

س3: ما الفرق بين Gradient Clipping بالقيمة (Value) وبالمعيار (Norm)؟ أيّهما تُفضّل ولماذا؟

> Value Clipping يقصّ كل عنصر من التدرج بشكل مستقل إلى مجال `[-c, c]`، وهذا قد يُشوّه الاتجاه النسبي بين عناصر التدرج المختلفة. Norm Clipping يحسب المعيار الإقليدي الكلي لكل التدرجات، وإن تجاوز حدًا معينًا، يُقلّص **كل** العناصر بنفس النسبة، فيحافظ على الاتجاه الأصلي للتدرج ويُقلّل فقط من حجمه. لهذا السبب Norm Clipping هو المعيار الشائع في تدريب الشبكات العميقة والمحوّلات.

س4: ماذا يحدث فعليًا عند استدعاء `loss.backward()` في PyTorch؟

> PyTorch يكون قد بنى رسمًا حسابيًا ديناميكيًا (Dynamic Computational Graph / Tape) أثناء الـ Forward Pass، حيث كل عملية (tensor operation) تُسجّل نفسها مع دالة `grad_fn` تعرف كيف تحسب تدرجها المحلي. عند استدعاء `backward()`، يبدأ PyTorch من عقدة الخسارة (gradient = 1) ويمشي عكسيًا عبر الرسم، مطبّقًا قاعدة السلسلة في كل عقدة، ويُراكم النتيجة النهائية في خاصية `.grad` لكل Tensor له `requires_grad=True`.

س5: لماذا نستثني عادة Bias وLayerNorm من Weight Decay؟

> Weight Decay يهدف لمنع الأوزان من اتخاذ قيم كبيرة جدًا تؤدي للـ Overfitting، وهذا منطقي لأوزان المصفوفات الكبيرة (Linear/Attention weights) التي تُحدّد "اتجاهات" التمثيل. لكن معاملات Bias وLayerNorm (scale/shift) هي قيم عددية بسيطة وظيفتها معايرة الإخراج (Calibration) وليست تمثيل أنماط معقدة، وتطبيق تثبيط عليها لا يفيد التعميم بل قد يضر بقدرة النموذج على المعايرة الدقيقة. التجربة العملية عبر عشرات الأوراق البحثية أكّدت أن استثناءها يُحسّن الأداء بشكل متسق.

---

10. 🧪 تمرين عملي (Mini-Project)

المهمة: "مختبر مقارنة المُحسِّنات" — Optimizer Showdown

قارن أداء 4 إعدادات تدريب مختلفة على نفس البيانات والنموذج، وارسم منحنيات الخسارة لكل منها:

1. `SGD` بدون Momentum، `lr` ثابت.
2. `SGD + Momentum=0.9`، `lr` ثابت.
3. `AdamW`، `lr` ثابت (بدون Scheduling).
4. `AdamW` + `Warmup/Cosine Scheduling` + `Gradient Clipping`.

```
"""
optimizer_showdown.py — هيكل بداية للتمرين
أكمل الأجزاء المُعلَّمة بـ TODO.
```

```python
"""
from __future__ import annotations

from dataclasses import dataclass

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader

# أعد استخدام MLPClassifier و make_synthetic_dataset من full_training_loop.py


@dataclass(frozen=True)
class ExperimentResult:
    name: str
    loss_history: list[float]
    final_val_acc: float


def run_experiment(
    name: str,
    build_optimizer_and_scheduler: callable,  # TODO: حدّد التوقيع المناسب بدقة
    train_loader: DataLoader,
    val_loader: DataLoader,
    num_epochs: int = 20,
) -> ExperimentResult:
    """
    TODO:
1. أنشئ نموذج MLPClassifier جديد بنفس البذرة العشوائية في كل تجربة
       (torch.manual_seed قبل إنشاء النموذج) لضمان مقارنة عادلة.
2. استخدم build_optimizer_and_scheduler لإنشاء optimizer (وscheduler إن وُجد).
3. درّب لعدد num_epochs حقبة، واحفظ loss في كل خطوة ضمن loss_history.
4. قيّم الدقة النهائية على val_loader.
5. أعد ExperimentResult.
    """
    raise NotImplementedError


def plot_results(results: list[ExperimentResult]) -> None:
    """
    TODO: استخدم matplotlib لرسم loss_history لكل تجربة على نفس الرسم البياني،
مع legend يوضح اسم كل تجربة، ومحور X = خطوة التدريب، محور Y = الخسارة
(يُفضّل مقياس لوغاريتمي log-scale لمحور Y لتوضيح الفروقات بدقة).
    """
    raise NotImplementedError


# أسئلة يجب أن تُجيب عنها بعد تشغيل التجربة في تقرير قصير:
# 1. أي الإعدادات تقاربت أسرع في أول 5 حقبات؟ ولماذا برأيك؟
# 2. هل ظهر أي تذبذب (Oscillation) في منحنى SGD بدون Momentum؟
# 3. هل أحدث Gradient Clipping فرقًا ملحوظًا في استقرار هذه المسألة البسيطة تحديدًا؟
#    ولماذا قد يكون التأثير أوضح بكثير في شبكات أعمق (مثل LLM بعشرات الطبقات)؟
# 4. جرّب رفع lr الخاص بـ SGD البسيط تدريجيًا حتى يتباعد (diverge) — عند أي قيمة يحدث هذا؟
```

11. 🔗 جسر للفصل التالي

لقد أتقنت الآن **كيف** تتعلم الشبكة (Backpropagation) و**كيف نُحدّث أوزانها بذكاء** (Optimization). لكن بقي سؤال جوهري لم نُجبه بعد: إذا درّبنا شبكتنا لفترة طويلة كافية، هل ستتعلم **الأنماط الحقيقية في البيانات**، أم ستحفظ أمثلة التدريب حرفيًا (**Overfitting**) دون أن تعمّم على بيانات جديدة لم ترها من قبل؟

وكيف نقيس أصلًا "هل نموذجي جيد؟" بمقياس أعمق من مجرد قيمة الخسارة النهائية؟ في الفصل القادم، **"التقييم والتعميم"**، سنتعرف على مفهوم **Perplexity** (المقياس الأهم لتقييم نماذج اللغة تحديدًا)، وتقنيات **Regularization** (مثل Dropout وEarly Stopping) التي تحمي نموذجك من الحفظ الأعمى، وكيف نُصمّم تجربة تدريب علمية سليمة باستخدام مجموعات Train/Validation/Test بشكل صحيح.

---

12. 📚 مراجع للتعمّق

المرجع	النوع	الرابط/المصدر
Rumelhart, Hinton & Williams (1986), *"Learning representations by back-propagating errors"*	ورقة تأسيسية	Nature, أول صياغة رسمية لـ Backpropagation
Kingma & Ba (2014), *"Adam: A Method for Stochastic Optimization"*	ورقة بحثية	arXiv:1412.6980
Loshchilov & Hutter (2017), *"Decoupled Weight Decay Regularization"*	ورقة بحثية (AdamW)	arXiv:1711.05101
Smith (2018), *"A Disciplined Approach to Neural Network Hyper-Parameters"* (OneCycle)	ورقة بحثية	arXiv:1803.09820
Goyal et al. (2017), *"Accurate, Large Minibatch SGD"* (Linear Scaling Rule)	ورقة بحثية	arXiv:1706.02677
Sutskever et al. (2013), *"On the importance of initialization and momentum in deep learning"*	ورقة بحثية	ICML 2013
PyTorch Autograd — الدليل الرسمي	توثيق تقني	pytorch.org/docs/stable/autograd.html
CS231n — Stanford, *"Optimization: Stochastic Gradient Descent"*	محاضرة/ملاحظات	cs231n.github.io

==========

الفصل الرابع: التقييم والتعميم — Overfitting وRegularization وPerplexity



عندما يحفظ النموذج الإجابة بدل أن يفهم السؤال

1. 🟢 لماذا يهمّك هذا؟

تخيّل طبيبًا تدرّب فقط على حفظ 500 حالة مرضية بالضبط كما وردت في كتابه الدراسي — نفس الأعراض، نفس الترتيب، نفس الصياغة. يأتيه مريض حقيقي بأعراض *مشابهة* لكن ليست مطابقة حرفيًا، فيُصاب هذا "الطبيب" بالحيرة التامة لأنه لم يتعلّم المنطق الطبي العام، بل حفظ حالات بعينها. هل تثق في هذا الطبيب؟ بالتأكيد لا.

هذا بالضبط ما قد يحدث لشبكتك العصبية. في الفصلين السابقين، تعلّمنا كيف تُحدَّث الأوزان (Backpropagation) وكيف نختار حجم واتجاه كل خطوة تحديث (Optimization). لكن لنتوقف ونطرح سؤالًا أكثر جوهرية من كل ما سبق:

> إذا واصلنا تدريب الشبكة حتى أصبحت قيمة الخسارة (Loss) على بيانات التدريب تقترب من الصفر تمامًا — هل هذا **إنجاز** يستحق الاحتفال، أم **إنذار خطر** يستوجب التوقف فورًا؟

الجواب المُفاجئ: **قد يكون الاثنين معًا**، وهذا الفصل بالكامل مُخصّص لتعليمك كيف تُفرّق بين الحالتين. وسؤال ملازم له لا يقل أهمية:

> عندما نقول "هذا النموذج اللغوي جيد"، ما الرقم المحدد الذي ننظر إليه؟ هل Accuracy كافية لتقييم نموذج يتنبأ بالكلمة التالية بين عشرات الآلاف من الاحتمالات؟

في نهاية هذا الفصل، لن تكتفي بفهم المفاهيم نظريًا، بل ستمتلك أدوات برمجية فعلية لتشخيص "صحة" أي تدريب خلال ثوانٍ من النظر لمنحنى واحد، ولحساب **Perplexity** — المقياس الذي سيُلازمك طوال بقية هذا الكتاب كلما تعاملنا مع نماذج اللغة.

---

3. 🟡 الشرح المتدرّج: من الحدس إلى الكود

3.1 لماذا تقسيم بياناتنا إلى ثلاث مجموعات هو قرار إحصائي، لا تنظيمي فقط؟

دعنا نبدأ بسؤال يبدو بديهيًا لكنه عميق: لماذا لا نُدرّب نموذجنا على **كل** البيانات المتاحة، ثم نقيس أداءه على نفس البيانات التي رآها؟

المشكلة إحصائية بحتة: أي نموذج بمرونة كافية (وهذا تحديدًا وصف الشبكات العصبية العميقة) يمكنه، نظريًا، أن "يحفظ" أي مجموعة بيانات منتهية تمامًا، بغض النظر عن وجود نمط حقيقي فيها أم لا. فإذا قِستَ الأداء على نفس بيانات التدريب، ستحصل دائمًا تقريبًا على نتيجة ممتازة **حتى لو لم يتعلّم النموذج أي شيء قابل للتعميم**. هذا يُسمى في الإحصاء **"الخطأ الظاهري" (Apparent Error)** مقابل **"خطأ التعميم الحقيقي" (True Generalization Error)**، والفجوة بينهما هي بالضبط ما نحاول قياسه والتحكم فيه طوال هذا الفصل.

لهذا نُقسّم البيانات إلى ثلاث مجموعات، لكل منها **دور مختلف تمامًا وغير قابل للتبادل**:

المجموعة	السؤال الذي تُجيب عنه	من يراها وكم مرة
Train	"كيف أُحدّث الأوزان؟"	النموذج يراها ملايين المرات، تُستخدم في `backward()`
Validation	"هل قراراتي التطويرية (معمارية، lr، نقطة توقف) صحيحة؟"	تُفحص بشكل متكرر، **بدون** تحديث أوزان مباشرة

المجموعة	السؤال الذي تُجيب عنه	من يراها وكم مرة
Test	"ما الأداء الحقيقي الصادق لنموذجي النهائي؟"	**مرة واحدة فقط**، بعد تجميد كل القرارات

> ⚠ **القاعدة الذهبية غير القابلة للتفاوض:** بمجرد أن تستخدم نتيجة من Test Set لاتخاذ **أي** قرار — تغيير معمارية، تجربة lr مختلف، حتى اختيار checkpoint معيّن — فإن Test Set يفقد قيمته كمقياس نزيه، ويتحوّل فعليًا إلى مجموعة Validation إضافية. النتيجة النهائية التي تُبلّغ عنها تُصبح **متفائلة بشكل مُضلِّل**. هذا الخطأ، المعروف بـ "Test Set Leakage"، ارتكبته أوراق بحثية منشورة في مؤتمرات مرموقة دون انتباه كافٍ.

3.2 أداة التشخيص الأقوى: منحنيات التعلّم (Learning Curves)

كيف نكتشف المشكلة عمليًا، بأقل جهد ممكن؟ الإجابة: **ارسم Train Loss وVal Loss معًا عبر الزمن (Epochs أو Steps) على نفس الرسم البياني**. هذا الرسم البسيط، في تجربتي، يكشف أكثر من 80% من مشاكل التدريب الشائعة بنظرة واحدة.



الأنماط الأربعة الأساسية التي يجب أن تتعرّف عليها فورًا:

النمط	Train Loss	Val Loss	التشخيص	الإجراء المناسب
كلاهما مرتفع ومتقارب	مرتفع	مرتفع	**Underfitting**	زد تعقيد النموذج، درّب أطول، قلّل Regularization
كلاهما منخفض ومتقارب	منخفض	منخفض	**صحي**	واصل المراقبة فقط
فجوة متزايدة بوضوح	منخفض جدًا	مرتفع ومتباعد	**Overfitting**	زد Regularization، أضف بيانات، بسّط النموذج
كلاهما يرتفع بمرور الوقت	مرتفع ومتصاعد	مرتفع ومتصاعد	**Divergence**	خفّض lr، تحقق من Gradient Clipping

3.3 الأساس النظري: تفكيك الخطأ إلى Bias و Variance

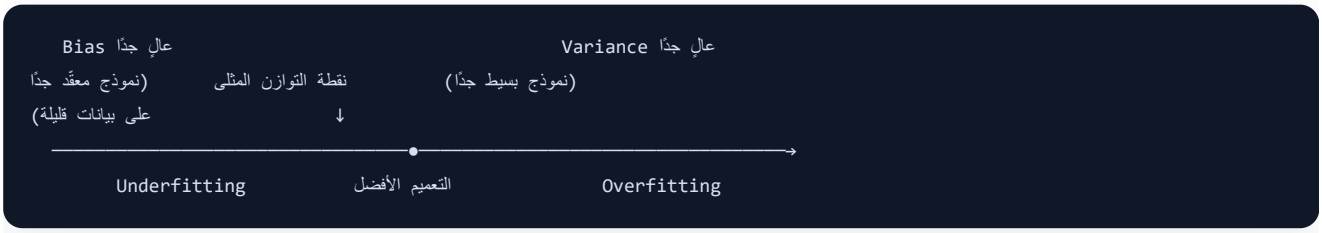
لماذا يحدث Overfitting أصلًا من الناحية الرياضية؟ يوجد تحليل كلاسيكي أنيق في الإحصاء يُفكّك **خطأ التعميم المتوقع** لأي نموذج إلى ثلاثة حدود منفصلة تمامًا:

```
E[(y − ŷ)²] = Bias(ŷ)² + Var(ŷ) + σ²_noise
```

دعنا نُفكّك هذا بصبر:

- **Bias² (مربع التحيّز):** يقيس مدى ابتعاد *متوسط* تنبؤات نموذجك (عبر عينات تدريب مختلفة افتراضية) عن القيمة الحقيقية. نموذج بسيط جدًا (مثل خط مستقيم يحاول ملاءمة علاقة منحنية معقدة) له Bias عالٍ بنيويًا — مهما غيّرت بيانات التدريب، سيبقى "مخطئًا بانتظام" بنفس الطريقة التقريبية.
- **Variance (التباين):** يقيس مدى "اهتزاز" تنبؤات نموذجك إذا بدّلنا عينة بيانات التدريب بعينة أخرى من نفس التوزيع. نموذج معقّد جدًا (ملايين البارامترات على بيانات محدودة) حساس جدًا لتفاصيل عينة التدريب المحددة التي رآها — هذا بالضبط **Overfitting**.
- **σ²_noise (الضوضاء المتأصلة):** خطأ لا يمكن لأي نموذج إزالته، لأنه جزء من عشوائية البيانات نفسها (قياسات غير دقيقة، غموض لغوي طبيعي...).

النقطة الجوهرية: Bias وVariance في مقايضة دائمة تقريبًا — تقليل أحدهما غالبًا يزيد الآخر، إلا إذا غيّرنا شيئًا أساسيًا (بيانات أكثر، معمارية أفضل). مهمة Regularization هي إيجاد **نقطة التوازن المثلى** على هذا الطيف، وليس تعظيم جانب واحد.



3.4 السلاح الأول: L1/L2 Regularization — لماذا "الأوزان الكبيرة" مشبوهة؟

لنبنِ الحدس أولًا: لماذا نظن أن الأوزان الكبيرة علامة على Overfitting؟ تخيّل دالة تنبؤ بسيطة جدًا `y = w·x`. إذا كان `w` كبيرًا جدًا، فإن تغييرًا طفيفًا جدًا في `x` (مثل ضوضاء القياس، أو اختلاف بسيط بين عينة تدريب وعينة اختبار) يُنتج تغييرًا **ضخمًا** في `y`. هذه الحساسية المفرطة هي بالضبط تعريف Variance العالي. النموذج الذي "يحفظ" تفاصيل دقيقة من بيانات التدريب يحتاج عادة أوزانًا كبيرة ودقيقة لإعادة إنتاج تلك التفاصيل حرفيًا.

الحل: نُضيف عقوبة مباشرة على حجم الأوزان في دالة الخسارة:

```
L2 (Ridge):   L_total = L_original + (λ/2) · Σ wᵢ²
L1 (Lasso):   L_total = L_original + λ · Σ |wᵢ|
```

الفرق الهندسي بينهما (وهذا سؤال مقابلات كلاسيكي): تخيّل منطقة القيود كدائرة (L2) مقابل معيّن/مُعين Diamond (L1) حول نقطة الأصل في فضاء الأوزان. حل المسألة المقيّدة يقع عند أول نقطة تماس بين "محيط دالة الخسارة الأصلية" وهذه المنطقة. بسبب الزوايا الحادة للمعيّن في L1، نقطة التماس غالبًا ما تقع **بالضبط على أحد المحاور** (أي أن بعض الأوزان تُصبح صفرًا تمامًا)، بينما الدائرة الملساء في L2 نادرًا ما تُنتج هذا التصفير التام، بل تُقلّص كل الأوزان بنسب متفاوتة صغيرة.

في عالم الشبكات العصبية الحديثة، كما رأينا في الفصل الثالث، لا نُضيف L2 يدويًا لدالة الخسارة عادة، بل نستخدم **Weight Decay المُدمج ومُطبَّق بشكل منفصل وصحيح في AdamW**.

3.5 السلاح الثاني: Dropout — الفريق الذي لا يعتمد على نجم واحد

لنبدأ بقصة: فريق كرة قدم يعتمد بالكامل على لاعب نجم واحد سيُهزم بشدة إن أُصيب ذلك اللاعب. الفريق الأفضل هو من يُدرّب كل لاعبيه ليكونوا قادرين على سد أي فراغ. **Dropout (Srivastava et al., 2014)** يُطبّق هذه الفلسفة حرفيًا على الشبكة العصبية: في كل خطوة تدريب، نُطفئ عصبونات عشوائية مؤقتًا، مما يُجبر الشبكة على عدم "الاعتماد المفرط" على أي مجموعة فرعية محددة من العصبونات (ظاهرة تُسمى **Co-adaptation**).

الآلية الدقيقة (Inverted Dropout — المعيار الحالي):

```
أثناء التدريب:
    mask ~ Bernoulli(1 - p)          لكل عنصر، باحتمال إبقاء (1-p)
    h_out = (h ⊙ mask) / (1 - p)     القسمة ضرورية — انظر الشرح أدناه

أثناء الاستدلال:
    h_out = h                        لا تعديل إطلاقًا، نستخدم الشبكة كاملة
```

لماذا القسمة على `(1-p)` تحديدًا؟ لنفترض `p=0.5`. في المتوسط، نصف العصبونات تُطفأ، فيصبح مجموع المخرجات تقريبًا نصف قيمته الأصلية. إن لم نُصحّح هذا، فإن **التوقع الرياضي (Expected Value)** لمخرجات الطبقة سيختلف جوهريًا بين زمن التدريب (حيث Dropout نشط) وزمن الاستدلال (حيث لا Dropout). القسمة على `(1-p)` تجعل `E[h_out] = h` في كلتا الحالتين، فلا نحتاج أي تصحيح إضافي عند التحويل للاستدلال — ببساطة نُعطّل Dropout ونستمر.

الحدس الإحصائي الأعمق: يمكن اعتبار Dropout تقريبًا مكافئًا لتدريب **عدد هائل من الشبكات الفرعية المختلفة** بشكل ضمني ومتزامن (كل Forward Pass "يُفعّل" بنية فرعية عشوائية مختلفة من نفس الشبكة الأم)، ثم عند الاستدلال نُقارب تقريبيًا **متوسط تنبؤات** كل هذه الشبكات الفرعية مجتمعة — وهذا تفسير جزئي لفعاليته الكبيرة رغم بساطته الشديدة.

```
"""
dropout_and_weight_decay.py
تطبيق Inverted Dropout يدويًا من الصفر مقارنة بـ nn.Dropout الرسمي،
وتوضيح تجريبي لأثر Weight Decay على حجم الأوزان عبر الزمن.
Python 3.13+ | PyTorch
"""
```

```python
from __future__ import annotations

import torch
import torch.nn as nn


class ManualDropout(nn.Module):
"""تطبيق Inverted Dropout يدويًا؛ سلوكه مطابق لـ nn.Dropout الرسمي."""

    def __init__(self, p: float = 0.5) -> None:
        super().__init__()
        if not 0.0 <= p < 1.0:
            raise ValueError(f"p يجب أن تقع في المجال [0, 1)، وردت القيمة: {p}")
        self.p = p

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        if not self.training or self.p == 0.0:
            return x
        keep_prob = 1.0 - self.p
        mask = torch.bernoulli(torch.full_like(x, keep_prob))
        return x * mask / keep_prob


def compare_dropout_implementations() -> None:
    torch.manual_seed(0)
    x = torch.ones(3, 10)

    manual = ManualDropout(p=0.4)
    official = nn.Dropout(p=0.4)

    manual.train(); official.train()
    print("=== أثناء training (قيم متوقَّعة: صفر أو 1/(1-p)=1.667) ===")
    print("يدوي :", manual(x)[0])
    print("رسمي :", official(x)[0])

    manual.eval(); official.eval()
    print("\n=== أثناء eval (يجب أن تكون مطابقة تمامًا لـ x) ===")
    print("يدوي :", manual(x)[0])
    print("رسمي :", official(x)[0])


def demonstrate_weight_decay_effect(num_steps: int = 200) -> None:
"""يُظهر تجريبيًا كيف يُقلّص weight_decay حجم الأوزان مع ثبات كل شيء آخر."""
    torch.manual_seed(1)

    model_a = nn.Linear(20, 20)
    model_b = nn.Linear(20, 20)
    model_b.load_state_dict(model_a.state_dict())  # ابدأ من نفس الأوزان تمامًا

    opt_a = torch.optim.AdamW(model_a.parameters(), lr=0.05, weight_decay=0.0)
    opt_b = torch.optim.AdamW(model_b.parameters(), lr=0.05, weight_decay=0.3)

    dummy_x = torch.randn(16, 20)
    initial_norm = model_a.weight.norm().item()
```

```
    for _ in range(num_steps):
        for model, opt in [(model_a, opt_a), (model_b, opt_b)]:
            opt.zero_grad(set_to_none=True)
# خسارة تعسفية بسيطة فقط لإظهار أثر الـ decay على حجم الأوزان
            loss = model(dummy_x).pow(2).mean()
            loss.backward()
            opt.step()

    print(f"\nمعيار الأوزان الابتدائي        : {initial_norm:.4f}")
    print(f"بدون weight_decay (النهائي)    : {model_a.weight.norm().item():.4f}")
    print(f"مع weight_decay=0.3 (النهائي) : {model_b.weight.norm().item():.4f}  ← أصغر بوضوح")


if __name__ == "__main__":
    compare_dropout_implementations()
    demonstrate_weight_decay_effect()
```

3.6 السلاح الثالث: Early Stopping — أرخص علاج وأكثره فعالية

ربما أبسط تقنية في هذا الفصل، لكنها غالبًا ما تُغفل في تطبيقها الصحيح. الفكرة: راقب `Val Loss` باستمرار، وإذا لم يتحسن لعدد معيّن من الحقبات المتتالية (**Patience**)، أوقف التدريب **وارجع لأفضل نقطة تحقق سُجّلت سابقًا** — وليس فقط آخر نقطة.

```
"""
early_stopping.py
تطبيق Early Stopping احترافي مع حفظ واسترجاع أفضل حالة للنموذج تلقائيًا.
"""
from __future__ import annotations

import copy
from dataclasses import dataclass, field

import torch.nn as nn


@dataclass
class EarlyStopping:
    patience: int = 5
    min_delta: float = 1e-4
    best_loss: float = field(default=float("inf"), init=False)
    counter: int = field(default=0, init=False)
    best_state_dict: dict | None = field(default=None, init=False)
    should_stop: bool = field(default=False, init=False)
    best_epoch: int = field(default=-1, init=False)

    def step(self, val_loss: float, model: nn.Module, epoch: int) -> None:
        improved = val_loss < (self.best_loss - self.min_delta)
        if improved:
            self.best_loss = val_loss
            self.counter = 0
            self.best_epoch = epoch
```

```
            self.best_state_dict = copy.deepcopy(model.state_dict())
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.should_stop = True

    def restore_best(self, model: nn.Module) -> None:
        if self.best_state_dict is not None:
            model.load_state_dict(self.best_state_dict)
            print(f"✅ تمت استعادة أفضل نسخة من epoch={self.best_epoch} "
                  f"(val_loss={self.best_loss:.4f})")


if __name__ == "__main__":
# محاكاة سريعة لتوضيح السلوك دون تدريب فعلي
    simulated_val_losses = [2.1, 1.8, 1.5, 1.4, 1.42, 1.45, 1.5, 1.6]
    stopper = EarlyStopping(patience=3, min_delta=1e-3)
    dummy_model = nn.Linear(4, 4)

    for epoch, vloss in enumerate(simulated_val_losses):
        stopper.step(vloss, dummy_model, epoch)
        print(f"epoch={epoch} val_loss={vloss} counter={stopper.counter} "
              f"should_stop={stopper.should_stop}")
        if stopper.should_stop:
            print(f"\n⏹ توقف عند epoch={epoch}، لكن أفضل نتيجة كانت في epoch={stopper.best_epoch}")
            break
```

💡 لاحظ الدرس الضمني هنا: التدريب استمر 5 حقبات بعد أفضل نقطة فعلية (epoch 3) قبل أن يُقرر التوقف (بسبب `patience=3`). لو اكتفينا بأخذ "آخر نموذج" بدل "أفضل نموذج محفوظ"، لكنا استخدمنا نموذجًا أسوأ بوضوح من المتاح فعليًا.

3.7 أسلحة إضافية سريعة: Label Smoothing و Data Augmentation

Label Smoothing: بدل استخدام هدف One-Hot صارم تمامًا (`[0, 0, 1, 0]`)، نُستخدم نسخة "مُلطَّفة" توزّع جزءًا صغيرًا من الاحتمال على بقية الفئات:

```
y_smooth = (1 − ε) · y_onehot + ε / K          حيث K = عدد الفئات، ε (مثلًا 0.1) صغيرة
```

لماذا هذا مفيد؟ لأن الهدف الصارم (احتمال 1.0 بالضبط) يدفع النموذج نحو إنتاج logits ضخمة جدًا بثقة مفرطة (Overconfidence)، مما قد يضر بالتعميم ويجعل تقديرات الاحتمال غير معايرة جيدًا (Miscalibrated). Label Smoothing يُخفّف هذا الاتجاه بلطف.

Data Augmentation للنصوص: توليد نسخ متنوعة اصطناعيًا من نفس بيانات التدريب دون تغيير معناها الجوهري: الترجمة ذهابًا وإيابًا (Back-Translation)، استبدال كلمات بمرادفات (Synonym Replacement)، أو الحذف/الإدراج العشوائي لكلمات قليلة. هذا يُضاعف تنوّع ما يراه النموذج فعليًا دون حاجة لجمع بيانات حقيقية إضافية.

3.8 من "الحفظ مقابل الفهم" إلى "كيف نقيس الفهم رقميًا؟"

وصلنا الآن للجزء الثاني من الفصل: بعد أن نُدرّب نموذجًا بشكل صحي (بدون Overfitting خطير)، كيف **نقيس** جودته رقميًا، خاصة لنماذج اللغة؟

في مسائل التصنيف البسيطة، **Accuracy** (نسبة التنبؤات الصحيحة) منطقية: إما توقعت الفئة الصحيحة أو لا. لكن نموذج اللغة لا "يُصنّف" فئة واحدة، بل **يُخرج توزيعًا احتماليًا كاملًا** على كامل المفردات (قد تكون عشرات الآلاف من التوكنز) لكل موضع في النص. السؤال الطبيعي: كيف نُقيّم جودة **توزيع احتمالي كامل**، وليس مجرد قرار واحد صحيح/خاطئ؟

3.9 Perplexity: الحيرة كمقياس كمّي

الحدس أولًا

تخيّل نموذجين يتنبآن بالكلمة التالية في جملة: "اشتريتُ من السوق تفاحًا و___".

- **النموذج أ** يُعطي احتمال 0.85 لكلمة "موزًا" (منطقية جدًا)، وتوزيع صغير جدًا على بقية الكلمات.
- **النموذج ب** "تائه": يُعطي احتمال 0.05 فقط لكلمة "موزًا"، موزّعًا الباقي بشكل شبه عشوائي على مئات الكلمات.

النموذج أ "أقل حيرة" (Less Perplexed) تجاه الكلمة التالية، والنموذج ب أكثر حيرة بوضوح. **Perplexity** تُترجم هذا الحدس لرقم محدد: **كلما قلّت قيمتها، زادت ثقة النموذج الصحيحة وجودته**.

الصيغة الرياضية ولماذا هي ببساطة "Loss مُعاد تلوينه"

الخبر الجيد: أنت تعرف هذه الصيغة فعلًا من الفصل الثاني — إنها ببساطة الأس الطبيعي (exponential) لمتوسط Cross-Entropy Loss:

```
PPL = exp( -1/N · Σᵢ ln P(wᵢ | السياق السابق) )
    = exp(Cross-Entropy Loss بوحدة nats)
```

بمعنى آخر: **Perplexity ليست مقياسًا جديدًا مستقلًا، بل إعادة تفسير لنفس قيمة الخسارة التي تُدرّب عليها نموذجك أصلًا**، بعد تمريرها بدالة أسّية لتُصبح أكثر قابلية للتفسير الحدسي.

كيف نُفسّر الرقم الناتج؟

تفسير أنيق: إذا كانت `PPL = 10`، فهذا يعني تقريبًا أن النموذج "يتصرف كأنه يختار بشكل عشوائي ومتساوي الاحتمال من بين 10 كلمات محتملة" عند كل خطوة تنبؤ. كلما اقتربت PPL من 1 (الحد الأدنى النظري)، كان النموذج أكثر ثقة ودقة (PPL=1 يعني تنبؤًا مؤكدًا بنسبة 100% في كل خطوة — حالة مثالية غير واقعية عمليًا). كمرجع

تقريبي: نماذج اللغة الإنجليزية الجيدة على نصوص عامة تُحقق عادة PPL بين 10 و30، بينما النماذج الضعيفة قد تصل لمئات أو آلاف.

```
"""
perplexity_metrics.py
حساب Perplexity بدقة من Cross-Entropy Loss، مع معالجة صحيحة لرموز الحشو (Padding)،
ومقارنة Perplexity على جمل "متوقَّعة" مقابل جمل "عشوائية".
"""
from __future__ import annotations

import math

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_segment(0)


def perplexity_from_loss(cross_entropy_nats: float) -> float:
    """العلاقة الجوهرية: PPL = e^(loss)، بشرط أن تكون loss بوحدة nats (اللوغاريتم الطبيعي)."""
    return math.exp(cross_entropy_nats)


@torch.no_grad()
def corpus_perplexity(
    model: nn.Module,
    token_ids: torch.Tensor,      # (batch, seq_len)
    pad_token_id: int = 0,
) -> float:
    """
    حساب Perplexity على كامل مجموعة بيانات، مع تجاهل توكنز الحشو تمامًا
    عبر ignore_index — خطوة حرجة: تجاهلها يُحرّف النتيجة بشكل كبير إحصائيًا
    إذا كانت نسبة الحشو عالية (شائع جدًا مع تسلسلات متغيّرة الطول).
    """
    model.eval()
    inputs, targets = token_ids[:, :-1], token_ids[:, 1:]
    logits = model(inputs)  # (batch, seq_len-1, vocab_size)

    loss = F.cross_entropy(
        logits.reshape(-1, logits.size(-1)),
        targets.reshape(-1),
        ignore_index=pad_token_id,
        reduction="mean",
    )
    return perplexity_from_loss(loss.item())


if __name__ == "__main__":
    vocab_size, seq_len, batch_size = 500, 15, 4

    class TinyLM(nn.Module):
        def __init__(self, vocab_size: int, confident: bool) -> None:
```

```
            super().__init__()
            self.embed = nn.Embedding(vocab_size, 16)
            self.proj = nn.Linear(16, vocab_size)
            self.confident = confident  # لمحاكاة نموذج "واثق" اصطناعيًا للتوضيح فقط

        def forward(self, x: torch.Tensor) -> torch.Tensor:
            logits = self.proj(self.embed(x))
            if self.confident:
                logits = logits * 10.0  # تضخيم الفروقات = ثقة أعلى بمن توقعه النموذج
            return logits

    tokens = torch.randint(1, vocab_size, (batch_size, seq_len))

    random_model = TinyLM(vocab_size, confident=False)
    confident_model = TinyLM(vocab_size, confident=True)
# نسخ نفس الأوزان الابتدائية لضمان مقارنة عادلة لتأثير "الثقة" فقط
    confident_model.load_state_dict(
        {k: v.clone() for k, v in random_model.state_dict().items()}
    )

    ppl_random = corpus_perplexity(random_model, tokens)
    ppl_confident = corpus_perplexity(confident_model, tokens)

    print(f"Perplexity (نموذج عشوائي التنبؤ): {ppl_random:.2f}")
    print(f"Perplexity (نفس الأوزان لكن logits مُضخَّمة): {ppl_confident:.2f}")
    print(f"\n(ملاحظة تعليمية: تضخيم logits عشوائيًا دون تدريب حقيقي "
          f"، لا يعني بالضرورة تحسنًا – فقط يُغيّر حدة التوزيع"
          f"وقد يُسيء لـ PPL إن كانت الثقة في غير محلّها!)")

# تذكير مباشر بالعلاقة الأساسية
    example_loss = 2.3
    print(f"\nمثال مباشر: Cross-Entropy Loss = {example_loss} nats")
    print(f"=> Perplexity = exp({example_loss}) = {perplexity_from_loss(example_loss):.3f}")
```

🔴 صندوق التعمّق: وحدة القياس — nats مقابل bits

إن استخدمت `log` الطبيعي (الأساس `e`) كما في `F.cross_entropy` الافتراضية، فالوحدة هي **nats**، والعلاقة `PPL = exp(loss)`. لكن بعض الأدبيات القديمة (المتجذرة في نظرية المعلومات لشانون) تستخدم `log2`، فتكون الوحدة **bits**، والعلاقة تصبح `PPL = 2^loss`. **لا تقارن أبدًا قيم Perplexity من مصادر مختلفة دون التأكد من توحيد القاعدة اللوغاريتمية** — هذا خطأ شائع جدًا عند مقارنة نتائج من أوراق بحثية أو مكتبات مختلفة.

🔴 صندوق التعمّق: لماذا Perplexity "غير قابلة للمقارنة" بين Tokenizers مختلفة؟

لو قارنّا PPL لنموذجين يستخدمان Tokenizer مختلفًا (أحدهما على مستوى الكلمة، والآخر BPE على مستوى الأجزاء الفرعية Subwords)، فإن "التوكن" نفسه يحمل معنى مختلفًا تمامًا من حيث الكمية المعلوماتية التي يمثّلها. نموذج بتوكنز أصغر (مثل الأحرف) سيُظهر غالبًا PPL لكل توكن أقل ظاهريًا (لأن التنبؤ بحرف واحد تالٍ أسهل من التنبؤ بكلمة كاملة)، لكن هذا لا يعني أنه "أفضل" فعليًا. المقارنة العادلة تتطلب تطبيع PPL بالنسبة

لعدد **البايتات أو الأحرف** الفعلية، وليس عدد التوكنز الخام — تقنية تُعرف أحيانًا بـ **Bits-Per-Character (BPC)** أو **Bits-Per-Byte**.

---

4. 🧪 مثال محلول خطوة بخطوة

لنبنِ تجربة متكاملة End-to-End: نُدرّب نموذج لغة بسيط على نص قصير (مما يجعل Overfitting شبه مؤكد الحدوث بسرعة بسبب صغر البيانات)، نُلاحظ المشكلة فعليًا عبر منحنيات PPL، ثم نُعالجها، ونُقارن النتيجتين رقميًا وبصريًا.

```
"""
overfitting_end_to_end.py
تجربة كاملة: توليد Overfitting متعمّد، تشخيصه عبر Perplexity،
ثم علاجه بـ Dropout + Weight Decay + Early Stopping.
نموذج لغة بسيط على مستوى الحرف (character-level) لتسريع التجربة وتوضيحها.
"""
from __future__ import annotations

import math
from dataclasses import dataclass

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)


@dataclass(frozen=True)
class CharLMConfig:
    vocab_size: int
    embed_dim: int = 48
    hidden_dim: int = 96
    dropout: float = 0.0


class CharLM(nn.Module):
    """نموذج لغة بسيط: Embedding -> GRU -> Linear، لغرض توضيحي تعليمي."""

    def __init__(self, cfg: CharLMConfig) -> None:
        super().__init__()
        self.embed = nn.Embedding(cfg.vocab_size, cfg.embed_dim)
        self.drop_in = nn.Dropout(cfg.dropout)
        self.gru = nn.GRU(cfg.embed_dim, cfg.hidden_dim, batch_first=True)
        self.drop_out = nn.Dropout(cfg.dropout)
        self.proj = nn.Linear(cfg.hidden_dim, cfg.vocab_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.drop_in(self.embed(x))
        out, _ = self.gru(h)
```

```
        out = self.drop_out(out)
        return self.proj(out)


def build_vocab(text: str) -> tuple[dict[str, int], dict[int, str]]:
    chars = sorted(set(text))
    stoi = {ch: i for i, ch in enumerate(chars)}
    itos = {i: ch for ch, i in stoi.items()}
    return stoi, itos


def encode(text: str, stoi: dict[str, int]) -> torch.Tensor:
    return torch.tensor([stoi[ch] for ch in text], dtype=torch.long)


def batchify(data: torch.Tensor, seq_len: int, batch_size: int) -> torch.Tensor:
    n_chunks = (len(data) - 1) // (seq_len * batch_size)
    usable = n_chunks * seq_len * batch_size
    return data[:usable].view(batch_size, -1)


@torch.no_grad()
def eval_perplexity(model: nn.Module, data: torch.Tensor, seq_len: int) -> float:
    model.eval()
    total_loss, total_tokens = 0.0, 0
    for i in range(0, data.size(1) - seq_len - 1, seq_len):
        xb = data[:, i : i + seq_len]
        yb = data[:, i + 1 : i + seq_len + 1]
        logits = model(xb)
        loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)), yb.reshape(-1))
        total_loss += loss.item() * yb.numel()
        total_tokens += yb.numel()
    return math.exp(total_loss / total_tokens)


def run_training(
    model: nn.Module,
    train_data: torch.Tensor,
    val_data: torch.Tensor,
    seq_len: int,
    num_epochs: int,
    weight_decay: float = 0.0,
) -> tuple[list[float], list[float]]:
    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=weight_decay)
    train_hist, val_hist = [], []

    for epoch in range(num_epochs):
        model.train()
        for i in range(0, train_data.size(1) - seq_len - 1, seq_len):
            xb = train_data[:, i : i + seq_len]
            yb = train_data[:, i + 1 : i + seq_len + 1]

            optimizer.zero_grad(set_to_none=True)
            logits = model(xb)
            loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)), yb.reshape(-1))
```

```
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()

        train_ppl = eval_perplexity(model, train_data, seq_len)
        val_ppl = eval_perplexity(model, val_data, seq_len)
        train_hist.append(train_ppl)
        val_hist.append(val_ppl)
        gap_marker = " ⚠" if (val_ppl - train_ppl) > train_ppl else ""
        print(f"Epoch {epoch+1:02d} | Train PPL={train_ppl:7.2f} | Val PPL={val_ppl:7.2f} {gap_marker}")

    return train_hist, val_hist


if __name__ == "__main__":
    text = (
" في مدينة قديمة جدا كان هناك حكيم يجلس كل يوم تحت شجرة كبيرة "
" يستمع الى اسئلة الناس ويجيب عليها بحكمة عميقة وبساطة شديدة"
    ) * 25
    split = int(0.8 * len(text))
    train_text, val_text = text[:split], text[split:]

    stoi, itos = build_vocab(text)
    train_ids = encode(train_text, stoi)
    val_ids = encode(val_text, stoi)

    seq_len, batch_size = 32, 8
    train_batches = batchify(train_ids, seq_len, batch_size)
    val_batches = batchify(val_ids, seq_len, batch_size)

    print("=== تجربة 1: بدون Regularization (متوقَّع بقوة Overfitting) ===")
    model_plain = CharLM(CharLMConfig(vocab_size=len(stoi), dropout=0.0))
    train_hist_plain, val_hist_plain = run_training(
        model_plain, train_batches, val_batches, seq_len, num_epochs=20
    )

    print("\n=== تجربة 2: مع Dropout=0.3 + Weight Decay=0.1 ===")
    model_reg = CharLM(CharLMConfig(vocab_size=len(stoi), dropout=0.3))
    train_hist_reg, val_hist_reg = run_training(
        model_reg, train_batches, val_batches, seq_len,
        num_epochs=20, weight_decay=0.1,
    )

    print("\n=== الخلاصة المقارنة ===")
    gap_plain = val_hist_plain[-1] - train_hist_plain[-1]
    gap_reg = val_hist_reg[-1] - train_hist_reg[-1]
    print(f"بدون Regularization: فجوة PPL النهائية = {gap_plain:.2f}")
    print(f"مع Regularization  : فجوة PPL النهائية = {gap_reg:.2f}")
    print(f"\n(ممكن Val PPL بل أصغر فجوة مع أفضل، Train PPL الهدف ليس أقل)")
```

ما الذي يجب أن تلاحظه عمليًا عند التشغيل؟ في التجربة الأولى، ستجد `Train PPL` ينخفض بسرعة ملحوظة (ربما لقيم منخفضة جدًا بسبب تكرار النص)، بينما `Val PPL` يتوقف عن التحسن أو يبدأ بالتذبذب صعودًا بعد

عدة حقبات — الفجوة تكبر بوضوح. في التجربة الثانية، رغم أن `Train PPL` قد لا ينخفض بنفس الحدة، فإن الفجوة النهائية بين Train وVal يجب أن تكون أصغر بوضوح — وهذا بالضبط المؤشر الذي يهمّنا فعليًا.

---

5. 💡 نصائح وحيل (Tips & Tricks)

- **احكم دائمًا بـ"الفجوة" لا بـ"القيمة المطلقة":** نموذج بـ `Train PPL=15, Val PPL=16` أفضل عمليًا بكثير من نموذج بـ `Train PPL=3, Val PPL=40` — رغم أن الأول له Train PPL أسوأ ظاهريًا.
- **في LLM الحديثة جدًا، Underfitting أشيع من Overfitting الكلاسيكي:** بفضل كميات بيانات التدريب الهائلة (تريليونات التوكنز) مقارنة بعدد البارامترات، كثير من النماذج الكبيرة اليوم لم "تحفظ" بياناتها، بل لا تزال "تتعلم" منها — هذا جوهر Scaling Laws التي سنتعمّق فيها لاحقًا (الفصل 22).
- **قارن Perplexity فقط ضمن نفس الـ Tokenizer ونفس مجموعة الاختبار** — المقارنات عبر Tokenizers مختلفة مُضلِّلة كما شرحنا في صندوق التعمّق.
- **لا تُفرط في Dropout على البيانات الضخمة:** إن كانت بياناتك أكبر من قدرة النموذج على حفظها أصلًا، فإن Dropout قد يُبطئ التقارب دون فائدة حقيقية في التعميم.
- **استخدم `ignore_index` دائمًا** عند حساب Loss/PPL على بيانات تحتوي Padding — نسيان هذا يُحرّف الأرقام بشكل كبير خاصة مع تسلسلات متفاوتة الطول بشدة.

---

6. ⚠️ أخطاء شائعة ومصائد (Common Pitfalls)

الخطأ 1: نسيان التبديل بين `model.eval()` و`model.train()`

```
# ❌ خطأ: التقييم يحدث بينما Dropout/BatchNorm لا يزالان في وضع "تدريب"
def evaluate(model, loader):
    for xb, yb in loader:
        preds = model(xb)   # Dropout يُطفئ عصبونات هنا أيضًا بالخطأ، نتيجة غير موثوقة!
```

```
# ✅ صحيح
def evaluate(model, loader):
    model.eval()
    with torch.no_grad():
        for xb, yb in loader:
            preds = model(xb)
    model.train()  # أعده لوضع train قبل متابعة حلقة التدريب
```

الخطأ 2: تسريب بيانات عبر حساب إحصائيات التطبيع قبل التقسيم

```
# ❌ خطأ: حساب mean/std على كل البيانات، بما فيها val/test، قبل التقسيم
mean, std = all_data.mean(), all_data.std()
normalized = (all_data - mean) / std
train, val, test = split(normalized)
```

```
# ✅ صحيح: قسّم أولًا، ثم احسب الإحصائيات من Train فقط
train, val, test = split(all_data)
mean, std = train.mean(), train.std()
train = (train - mean) / std
val = (val - mean) / std      # يستخدم mean/std من train حصرًا
test = (test - mean) / std
```

الخطأ 3: استخدام Test Set لاختيار Hyperparameters

```
# ❌ خطأ منهجي خطير جدًا
for lr in [1e-3, 3e-4, 1e-4]:
    model = train(lr=lr)
    test_acc = evaluate(model, test_loader)    # قرار مبني على test! تسريب
    if test_acc > best_acc:
        best_lr = lr
```

```
# ✅ صحيح: Validation لاتخاذ القرار، Test فقط للتقرير النهائي مرة واحدة
for lr in [1e-3, 3e-4, 1e-4]:
    model = train(lr=lr)
    val_acc = evaluate(model, val_loader)
    if val_acc > best_acc:
        best_lr = lr

final_model = train(lr=best_lr)
final_test_acc = evaluate(final_model, test_loader)  # مرة واحدة، في النهاية فقط
```

الخطأ 4: الاعتماد فقط على Accuracy لتقييم نموذج لغوي توليدي

```
# ❌ غير كافٍ ومُضلِّل لنماذج اللغة
accuracy = (predictions == targets).float().mean()
```

```
# ✅ أفضل: Perplexity كمقياس أساسي، مع مقاييس تكميلية حسب المهمة لاحقًا
ppl = corpus_perplexity(model, val_tokens, pad_token_id=PAD_ID)
```

الخطأ 5: مقارنة Perplexity محسوبة بقواعد لوغاريتمية مختلفة

```
# ❌ خطأ خفي وشائع: افتراض أن كل PPL محسوبة بنفس الطريقة
# بعض المكتبات تستخدم log2 بدل ln دون توضيح كافٍ في التوثيق!
ppl_from_source_a = 2 ** loss_in_bits   # bits
ppl_from_source_b = math.exp(loss_in_nats)  # nats
# مقارنة هذين الرقمين مباشرة = خطأ منهجي!
```

```
# ✅ صحيح: وحّد القاعدة اللوغاريتمية أولًا قبل أي مقارنة
# nats -> bits: bits_loss = nats_loss / ln(2)
# bits -> nats: nats_loss = bits_loss * ln(2)
nats_loss_from_b = loss_in_bits * math.log(2)
ppl_unified = math.exp(nats_loss_from_b)   # أعلاه ppl_from_source_b الآن قابلة للمقارنة مع
```

---

7. ⚖️ جدول مقارنة: تقنيات Regularization

التقنية	متى تُستخدم	التكلفة الحاسوبية	الأثر الرئيسي	ملاحظة
L2 / Weight Decay	شبه دائم، افتراضي خفيف	شبه معدومة	يُقلّص حجم الأوزان تدريجيًا	طبّقها عبر AdamW المنفصل، لا الطريقة القديمة
L1	عند الحاجة لتخفيف فعلي (Sparsity)	شبه معدومة	يُصفّر أوزانًا بالكامل	أقل شيوعًا في الشبكات العميقة الحديثة
Dropout	شبكات متوسطة/كبيرة، بيانات محدودة	يُبطئ التقارب قليلًا	يمنع Co-adaptation	أقل أهمية مع بيانات هائلة جدًا
Early Stopping	شبه دائم	شبه معدومة (حفظ Checkpoint فقط)	يختار أفضل نقطة توقف	يتطلب الاحتفاظ بنسخ من الأوزان
Data Augmentation	بيانات محدودة	وقت تحضير إضافي	يزيد التنوّع الفعّال للبيانات	يتطلب جهدًا هندسيًا لكل نوع بيانات
Label Smoothing	تصنيف، ترجمة، توليد	شبه معدومة	يمنع الثقة المفرطة	تأثير خفيف لكنه متّسق تجريبيًا

---

8. ✅ قائمة التحقق من أفضل الممارسات

- [ ] قسّمت البيانات إلى Train/Val/Test **قبل** أي معالجة مسبقة أو حساب إحصائيات.
- [ ] لم تلمس Test Set إطلاقًا حتى اكتمال كل القرارات التطويرية.

- [ ] رسمت Train وVal Loss/PPL معًا، وتفقدت الفجوة بينهما وليس القيم المطلقة فقط.
- [ ] استدعيت `model.eval()` / `model.train()` بشكل متسق عند التبديل بين التدريب والتقييم.
- [ ] طبّقت Early Stopping مع حفظ **أفضل** Checkpoint، لا مجرد إيقاف التدريب.
- [ ] تستخدم Perplexity لتقييم نماذج اللغة، وتتحقق من قاعدتها اللوغاريتمية عند أي مقارنة.
- [ ] استخدمت `ignore_index` لرموز الحشو عند حساب Loss/PPL.
- [ ] لم تُقارن Perplexity بين نماذج بـ Tokenizers مختلفة دون تطبيع مناسب (BPC مثلًا).
- [ ] ضبطت شدة Regularization (Dropout/Weight Decay) بما يتناسب مع حجم بياناتك الفعلي.
- [ ] تحققت أن أي تحسّن في Train Loss يصاحبه تحسّن (أو ثبات) في Val Loss، لا تدهور.

---

9. 🎯 أسئلة مقابلات شخصية

س1: اشرح تفكيك الخطأ إلى Bias وVariance، وكيف يرتبط هذا بظاهرتي Underfitting وOverfitting؟

> خطأ التعميم المتوقع لأي نموذج يُمكن تفكيكه إلى `Bias² + Variance + Noise` المتأصل. Bias العالي يعني أن النموذج بسيط جدًا بنيويًا بحيث لا يستطيع التقاط الأنماط الحقيقية حتى لو رأى بيانات تدريب كافية — وهذا هو جوهر Underfitting. Variance العالي يعني أن النموذج حساس جدًا لتفاصيل عينة التدريب المحددة التي رآها، لدرجة أن تغيير العينة (أو استخدام بيانات اختبار جديدة) يُنتج تنبؤات مختلفة جدًا — وهذا هو جوهر Overfitting. المقايضة بينهما تعني أن تقليل أحدهما عادة يزيد الآخر، ومهمتنا كمهندسين هي إيجاد التوازن الأمثل عبر أدوات مثل Regularization.

س2: لماذا تُقسّم على `(1-p)` في Inverted Dropout تحديدًا أثناء التدريب وليس الاستدلال؟

> لأن الهدف هو الحفاظ على **التوقع الرياضي** لمخرجات الطبقة ثابتًا بين مرحلتي التدريب والاستدلال. أثناء التدريب، إطفاء نسبة `p` من العصبونات عشوائيًا يُقلّص مجموع المخرجات بمعامل `(1-p)` تقريبًا في المتوسط. القسمة الفورية على `(1-p)` تُعوّض هذا التقليص داخل مرحلة التدريب نفسها، بحيث عند الاستدلال — حيث لا يوجد أي Dropout ونستخدم الشبكة بكامل طاقتها — لا نحتاج لأي معامل تصحيح إضافي. هذا يُبسّط كود الاستدلال بشكل كبير ويضمن اتساقًا إحصائيًا بين الحالتين.

س3: ما العلاقة الدقيقة بين Cross-Entropy Loss وPerplexity؟ ولماذا نحتاج مقياسًا منفصلاً أصلًا إن كانا مرتبطين رياضيًا بهذا الشكل المباشر؟

> Perplexity = exp(Cross-Entropy Loss) عند استخدام اللوغاريتم الطبيعي. رغم أن العلاقة مباشرة رياضيًا (إعادة تحويل أُسّي بسيط)، فإن Perplexity تمنحنا **تفسيرًا حدسيًا** أوضح بكثير من رقم Loss الخام: فهي

> تُقارب "العدد الفعّال من الخيارات المتساوية الاحتمال" التي يتردد النموذج بينها عند كل تنبؤ، وهذا أسهل للفهم والمقارنة بين النماذج من قيمة لوغاريتمية مجردة. كما أن Perplexity هي المقياس المعياري تاريخيًا في أدبيات نمذجة اللغة، مما يجعلها ضرورية للتواصل والمقارنة مع الأبحاث المنشورة.

س4: لماذا يُعتبر "تسريب بيانات الاختبار" (Test Set Leakage) خطأً منهجيًا خطيرًا حتى لو كان غير مقصود؟

> لأن قيمة Test Set الأساسية تكمن في كونها **لم تؤثر على أي قرار تطويري إطلاقًا** — وهذا ما يجعل النتيجة عليها تقديرًا نزيهًا وغير متحيّز للأداء الحقيقي على بيانات جديدة تمامًا. بمجرد استخدام أي معلومة من Test Set (حتى بشكل غير مباشر، كحساب إحصائيات تطبيع عليها، أو اختيار Hyperparameter بناءً على أدائها)، يتسرب "علم" عن توزيع تلك البيانات إلى عملية التطوير، فيصبح النموذج مُحسَّنًا (ولو جزئيًا) تجاهها تحديدًا. النتيجة: تقدير متفائل بشكل مُضلِّل لن يتكرر فعليًا عند النشر على بيانات حقيقية جديدة لم يرها أحد من قبل.

س5: في أي الظروف يصبح Overfitting التقليدي أقل أهمية، وما البديل الذي يحل محله كمصدر قلق رئيسي؟

> عندما يكون حجم بيانات التدريب (بالتوكنز الفريدة) أكبر بكثير نسبيًا من عدد بارامترات النموذج — وهو واقع تدريب معظم LLM الحديثة المتوافقة مع مبادئ Chinchilla Scaling Laws. في هذه الحالة يرى النموذج كل نمط لغوي مرة واحدة أو قليلًا جدًا، فيصعب عليه "حفظ" أمثلة بعينها حرفيًا، وتُصبح المشكلة الأكبر عادة **Underfitting** — أي أن النموذج لم يُدرَّب بما يكفي بالنسبة لقدرته الاستيعابية (سعته)، وبالتالي الحل يكون في مزيد من البيانات أو التدريب الأطول أو زيادة حجم النموذج، وليس في تطبيق المزيد من Regularization.

---

10. 🧪 تمرين عملي (Mini-Project)

المهمة: "محقق Overfitting الآلي" — أداة تشخيص تلقائية لمنحنيات التدريب

ابنِ دالة تُحلّل منحنيات Train/Val Loss تلقائيًا وتُصدر تشخيصًا وتوصية عملية، دون تدخل بشري.

```
"""
overfitting_detector.py – هيكل بداية للتمرين
"""
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum, auto
```

```python
class DiagnosisResult(Enum):
    HEALTHY = auto()
    UNDERFITTING = auto()
    OVERFITTING = auto()
    DIVERGING = auto()


@dataclass(frozen=True)
class DiagnosisReport:
    result: DiagnosisResult
    final_train_loss: float
    final_val_loss: float
    gap: float
    recommendation: str


def diagnose_training(
    train_losses: list[float],
    val_losses: list[float],
    *,
    gap_ratio_threshold: float = 0.3,   # نسبة الفجوة إلى train_loss لاعتبارها Overfitting
    high_loss_threshold: float = 1.0,
) -> DiagnosisReport:
    """
    TODO: اكتب منطق التشخيص بالخطوات التالية:

    1. gap = val_losses[-1] - train_losses[-1]
    2. gap_ratio = gap / max(train_losses[-1], 1e-8)
3. تحقق من DIVERGING أولًا: هل train_losses[-1] أكبر من train_losses في منتصف التدريب تقريبًا (مثلًا train_losses[len(train_losses)//2])؟
4. تحقق من UNDERFITTING: train_losses[-1] مرتفع (> high_loss_threshold)
و gap_ratio صغيرة نسبيًا (النموذج سيء على الطرفين بالتساوي تقريبًا)
5. تحقق من OVERFITTING: gap_ratio > gap_ratio_threshold
6. إن لم تنطبق أي حالة أعلاه: HEALTHY

لكل حالة، أنشئ توصية نصية دقيقة وعملية (انظر الأمثلة في نص الفصل أعلاه
كمرجع لصياغة التوصيات).
    """
    raise NotImplementedError


def plot_diagnosis(
    train_losses: list[float], val_losses: list[float], report: DiagnosisReport
) -> None:
    """
    TODO: استخدم matplotlib مع لرسم المنحنيين معًا:
- عنوان يتضمن نتيجة التشخيص (report.result.name)
- تظليل خلفية بلون يعكس الحالة (أخضر فاتح=HEALTHY، أحمر فاتح=OVERFITTING،
برتقالي فاتح=UNDERFITTING، أحمر غامق=DIVERGING)
- نص التوصية كعنوان فرعي (subtitle) أسفل العنوان الرئيسي
    """
    raise NotImplementedError


if __name__ == "__main__":
```

```
    scenarios = {
        "healthy":      ([2.5, 1.8, 1.2, 0.9, 0.7], [2.6, 1.9, 1.3, 1.0, 0.75]),
        "overfitting":  ([2.5, 1.5, 0.8, 0.3, 0.1], [2.6, 1.8, 1.5, 1.7, 2.0]),
        "underfitting": ([2.5, 2.3, 2.2, 2.15, 2.1], [2.6, 2.4, 2.3, 2.25, 2.2]),
        "diverging":    ([2.5, 3.0, 4.5, 6.0, 9.0], [2.6, 3.1, 4.6, 6.2, 9.5]),
    }

    for name, (train_l, val_l) in scenarios.items():
        report = diagnose_training(train_l, val_l)
        status = "✅" if report.result == DiagnosisResult.HEALTHY else "⚠️"
        print(f"{status} [{name}] → {report.result.name}: {report.recommendation}")

# تحدٍّ إضافي: جرّب تصميم سيناريو "حدّي" يقع بين Healthy وOverfitting،
# واختبر حساسية عتبة gap_ratio_threshold – هل 0.3 عتبة مناسبة دائمًا؟
# متى قد تحتاج لتعديلها حسب طبيعة المهمة (تصنيف بسيط vs نموذج لغة ضخم)؟
```

11. 🔗 جسر للفصل التالي

بهذا نكون قد أتممنا ثلاثة متكاملة: **كيف تتعلم الشبكة** (Backpropagation)، **كيف تُحسَّن بذكاء** (Optimizers)، و**كيف نتأكد أنها تتعلم بصدق لا تحفظ، وكيف نقيس ذلك رقميًا** (Regularization وPerplexity). لكن كل هذا بقي حتى الآن في عالم نظري مُبسّط: شبكات صغيرة، بيانات اصطناعية أو نصوص قصيرة جدًا، تدريب في ثوانٍ معدودة.

نماذج اللغة الحقيقية تحتوي مليارات البارامترات، وتتطلب تدريبًا على آلاف الساعات من حوسبة GPU/TPU موزّعة. قبل الانتقال لعالم التضمينات (Embeddings) والمحوّلات (Transformers)، نحتاج أولًا أن نُتقن **الأدوات العملية والبنية التحتية الحسابية**: كيف نُدير الذاكرة بكفاءة على GPU، كيف نُقدّر حجم الذاكرة وتكلفة FLOPs *قبل* بدء أي تجربة كبيرة، وكيف نستخدم أدوات PyTorch الحديثة (Mixed Precision، `torch.compile`) لتسريع كل شيء دون التضحية بالدقة — موضوع الفصل الخامس القادم والأخير في الجزء الأول: **"أساسيات PyTorch العملية والبنية التحتية الحسابية"**.

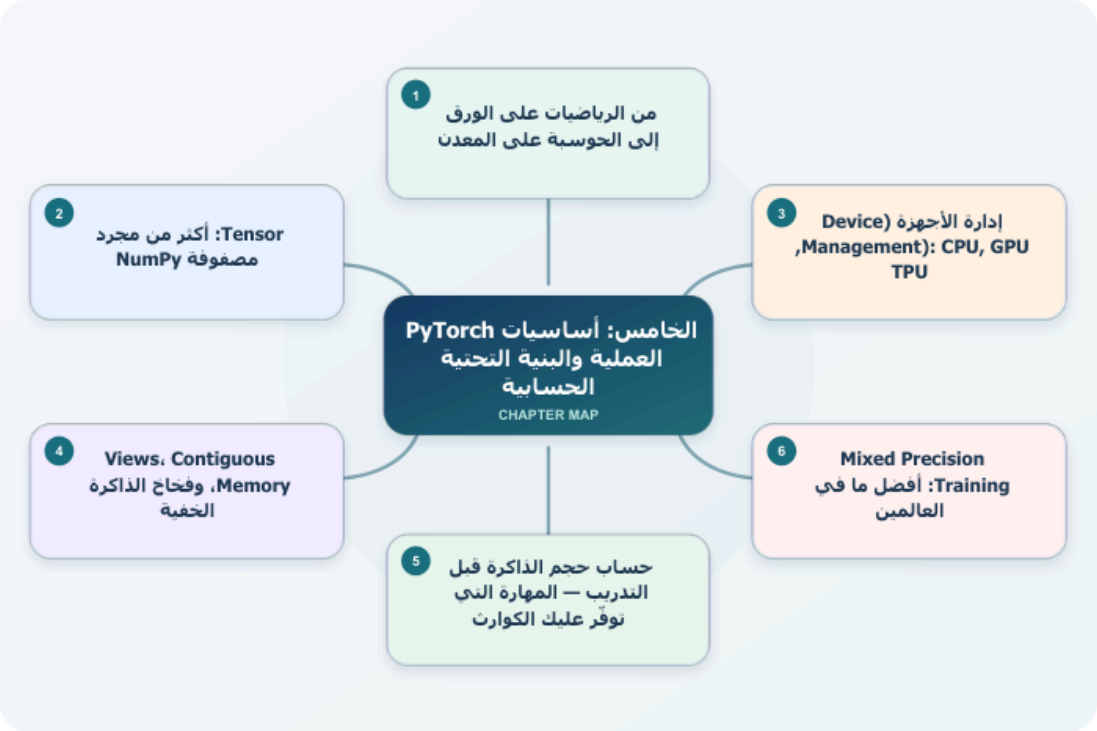
12. 📚 مراجع للتعمّق

المرجع	النوع	الرابط/المصدر
Srivastava et al. (2014), *"Dropout: A Simple Way to Prevent Neural Networks from Overfitting"*	ورقة تأسيسية	JMLR, الورقة الأصلية لـ Dropout
Krogh & Hertz (1992), *"A Simple Weight Decay Can Improve Generalization"*	ورقة كلاسيكية	NeurIPS 1992

المرجع	النوع	الرابط/المصدر
Szegedy et al. (2016), *"Rethinking the Inception Architecture"* (Label Smoothing)	ورقة بحثية	CVPR 2016
Geman, Bienenstock & Doursat (1992), *"Neural Networks and the Bias/Variance Dilemma"*	ورقة تأسيسية نظرية	Neural Computation
Jurafsky & Martin, *"Speech and Language Processing"*, فصل Language Modeling	كتاب مرجعي	متاح مجانًا عبر موقع المؤلفين
Goodfellow, Bengio, Courville, *"Deep Learning"*, Chapter 7: Regularization	كتاب مرجعي	deeplearningbook.org (مجاني)
CS224n — Stanford, *"Language Modeling and Perplexity"*	محاضرة/ ملاحظات	web.stanford.edu/class/cs224n

==========

الفصل الخامس: أساسيات PyTorch العملية والبنية التحتية الحسابية





من الرياضيات على الورق إلى الحوسبة على المعدن

1. 🟢 لماذا يهمّك هذا؟

في الفصول الأربعة الماضية، بنينا فهمًا رياضيًا صلبًا: كيف تتعلم الشبكة (Backpropagation)، كيف تُحسَّن (Optimizers)، وكيف نتأكد من صدق تعلّمها (Regularization). لكن كل أكوادنا حتى الآن عملت على شبكات صغيرة جدًا، بآلاف قليلة من البارامترات، على معالج (CPU) عادي، في ثوانٍ معدودة.

الآن اسأل نفسك سؤالًا يغيّر قواعد اللعبة تمامًا:

> ماذا لو أردنا تدريب نموذج بـ 7 مليار بارامتر؟ كم من الذاكرة سيحتاج؟ كم سيستغرق تدريبه؟ ولماذا يحتاج هذا تحديدًا بطاقات GPU متخصصة بدل معالج قوي عادي؟

وسؤال أعمق يواجه كل مهندس LLM يوميًا:

> لماذا يفشل تدريبي بخطأ `CUDA out of memory` رغم أن بطاقتي الرسومية "تبدو" قوية؟ وكيف أعرف *مسبقًا*، قبل إهدار ساعات التدريب، ما إذا كانت تجربتي ستنجح أصلًا من حيث الذاكرة والوقت؟

هذا الفصل هو جسر العبور من "فهم الرياضيات" إلى "إتقان الهندسة الحسابية الفعلية". ستتعلم هنا: كيف تتعامل مع Tensors باحترافية، كيف تُدير جهاز الحوسبة (CPU/GPU/TPU)، كيف تحسب حجم الذاكرة ومتطلبات FLOPs *قبل* بدء التدريب (مهارة تُوفّر عليك ساعات من المحاولة والخطأ)، وكيف تستخدم أدوات PyTorch الحديثة (Mixed Precision، `torch.compile`) لتسريع كل شيء دون تضحية بالدقة.

هذا الفصل يُغلق الجزء الأول من الكتاب (الأساسيات)، ويُجهّزك تمامًا للانتقال نحو عالم التضمينات والمحوّلات في الأجزاء القادمة.

---

3. 🟡 الشرح المتدرّج: من الحدس إلى الكود

3.1 Tensor: أكثر من مجرد مصفوفة NumPy

في الفصول السابقة استخدمنا `torch.Tensor` بشكل طبيعي، لكن دعنا نسأل: ما الذي يُميّزه فعليًا عن `numpy.ndarray`؟ الإجابة في ثلاث خصائص حاسمة:

1. **يمكنه العيش على GPU** (وليس فقط CPU).
2. **يحمل تاريخ العمليات** (Computational Graph) إن كان `requires_grad=True`.
3. **مُحسَّن لعمليات موازية ضخمة** عبر مكتبات CUDA/cuDNN.

أنواع البيانات (dtype) — قرار له تأثير هائل

```
import torch

x_fp32 = torch.randn(1000, 1000, dtype=torch.float32)   # الدقة الكاملة، 4 بايت/عنصر
x_fp16 = torch.randn(1000, 1000, dtype=torch.float16)   # نصف الدقة، 2 بايت/عنصر
x_bf16 = torch.randn(1000, 1000, dtype=torch.bfloat16)  # "Brain Float"، 2 بايت/عنصر
```

السؤال الجوهري: لماذا توجد ثلاثة أنواع، ولماذا لا نستخدم دائمًا `float16` لتوفير الذاكرة؟

النوع	البتات (الإشارة/الأس/الكسر)	المدى الديناميكي	الدقة	الاستخدام النموذجي
`float32`	1/8/23	واسع جدًا	عالية	الحساب الافتراضي الآمن، مرجع الدقة
`float16`	1/5/10	**ضيق** (قد يفيض Overflow بسهولة)	متوسطة	استدلال/تدريب مع Loss Scaling حذر
`bfloat16`	1/8/7	**واسع مثل fp32**	منخفضة	**المعيار في تدريب LLM الحديثة**

الحدس الأهم هنا: `bfloat16` يحتفظ بنفس عدد بتات الأس (Exponent) كـ `float32`، لذا له **نفس المدى الديناميكي** (لا يفيض/ينفد بسهولة عند قيم كبيرة أو صغيرة جدًا كالتدرجات)، لكنه يُضحّي بدقة الكسر العشري (Mantissa). هذا بالضبط سبب كون `bfloat16` الخيار الأفضل لتدريب الشبكات العميقة: الاستقرار العددي أهم من الدقة الدقيقة جدًا في أغلب عمليات Deep Learning.

```
"""
dtype_precision_demo.py
توضيح عملي للفرق بين fp32, fp16, bf16 من حيث المدى والدقة.
"""
from __future__ import annotations

import torch

torch.manual_seed(0)

# --- اختبار المدى الديناميكي (Dynamic Range) --- #
large_value = torch.tensor(70000.0)
print("قيمة كبيرة في fp32:", large_value.to(torch.float32).item())
print("قيمة كبيرة في fp16:", large_value.to(torch.float16).item())   # inf! فاض الحد
print("قيمة كبيرة في bf16:", large_value.to(torch.bfloat16).item())  # يبقى محدودًا بدقة

# --- اختبار الدقة (Precision) --- #
precise_value = torch.tensor(1.0001234567)
print("\nقيمة دقيقة في fp32:", precise_value.to(torch.float32).item())
print("قيمة دقيقة في fp16:", precise_value.to(torch.float16).item())
print("قيمة دقيقة في bf16:", precise_value.to(torch.bfloat16).item())  # أقل دقة من fp16!
```

```python
# --- حجم الذاكرة الفعلي --- #
tensor_1m_params = torch.randn(1_000_000)
for dtype in [torch.float32, torch.float16, torch.bfloat16]:
    size_mb = tensor_1m_params.to(dtype).element_size() * tensor_1m_params.numel() / 1e6
    print(f"\n1M عنصر بنوع {dtype}: {size_mb:.2f} MB")
```

3.2 إدارة الأجهزة (Device Management): CPU, GPU, TPU

السؤال العملي الأول في أي مشروع PyTorch: **كيف أنقل الحوسبة إلى GPU بأمان ومرونة؟**

```python
"""
device_management.py
نمط احترافي قياسي لإدارة الأجهزة، متوافق مع أي بيئة (CPU فقط، GPU واحد، عدة GPUs).
"""
from __future__ import annotations

import torch


def get_best_device() -> torch.device:
"""يختار أفضل جهاز متاح بترتيب الأولوية: CUDA > MPS (Apple Silicon) > CPU"""
    if torch.cuda.is_available():
        return torch.device("cuda")
    if torch.backends.mps.is_available():  # لأجهزة Mac بمعالج Apple Silicon
        return torch.device("mps")
    return torch.device("cpu")


device = get_best_device()
print(f"الجهاز المُستخدم: {device}")

# --- النمط القياسي: انقل النموذج والبيانات لنفس الجهاز دائمًا --- #
model = torch.nn.Linear(10, 5).to(device)
x = torch.randn(32, 10).to(device)
output = model(x)   # يعمل بشكل صحيح لأن كل شيء على نفس الجهاز

# خطأ كلاسيكي: نسيان نقل أحد الأطراف ⚠️ #
# y = torch.randn(32, 10)        # هذا يبقى على CPU!
# output = model(y)              # RuntimeError: Expected all tensors to be on the same device
```

> 💡 **معلومة عملية:** عدد البطاقات المتاحة يُستعلم عنه بـ `torch.cuda.device_count()`، واسم كل بطاقة بـ `torch.cuda.get_device_name(i)`. في بيئات التدريب الموزّع (سنتعمّق فيها في الفصل 23)، كل عملية (Process) عادة تُخصَّص لبطاقة GPU واحدة عبر متغير `LOCAL_RANK`.

الفرق بين CPU و GPU و TPU — لماذا هذا الاختلاف الجذري؟

الخاصية	CPU	GPU	TPU
عدد الأنوية	قليلة (4-64) قوية جدًا لكل منها	آلاف الأنوية (CUDA Cores) بسيطة	وحدات Matrix Multiply متخصصة (MXU)
الأفضل في	عمليات متسلسلة معقدة، تفرّع منطقي	عمليات متوازية ضخمة متطابقة (مصفوفات!)	ضرب مصفوفات ضخم جدًا بكفاءة قصوى
الذاكرة	كبيرة نسبيًا (RAM عادية)	محدودة نسبيًا لكن سريعة جدًا (HBM)	ذاكرة متخصصة مُحسّنة لـ XLA
البرمجة	عامة الغرض	CUDA / ROCm	XLA (عبر JAX أو TensorFlow أو PyTorch/XLA)

لماذا الشبكات العصبية "تحب" الـ GPU تحديدًا؟ لأن العملية الأساسية في الشبكات العصبية هي **ضرب المصفوفات (Matrix Multiplication)** — وهي عملية قابلة للتوازي الكامل (كل عنصر في المصفوفة الناتجة يُحسب بشكل مستقل عن الآخرين). GPU مُصمّم أصلًا لمعالجة ملايين البكسلات بالتوازي (للرسوميات)، وهذا نفس نمط الحوسبة المطلوب لضرب المصفوفات الضخمة.

3.3 Views، Contiguous Memory، وفخاخ الذاكرة الخفية

سؤال دقيق قد يُحيّر حتى المبرمجين المتمرسين: لماذا يفشل أحيانًا استدعاء `.view()` برسالة خطأ غريبة، بينما `.reshape()` يعمل دائمًا؟

```
"""
contiguous_memory_demo.py
فهم الفرق بين ()view و ()reshape، ولماذا ()contiguous أحيانًا ضرورية.
"""
from __future__ import annotations

import torch

x = torch.randn(4, 6)
print("x متصلة في الذاكرة (contiguous)؟", x.is_contiguous())  # True

x_transposed = x.t()   # النقل (Transpose) "الخرائط" فقط يُغيّر، لا ينسخ البيانات (strides)
print("x_transposed متصلة؟", x_transposed.is_contiguous())    # False!

try:
    x_transposed.view(24)   # يفشل! view تتطلب ذاكرة متصلة فعليًا
except RuntimeError as e:
    print(f"\n❌ خطأ متوقع: {e}")

# ✅ الحل الأول: استخدم ()reshape التي تتعامل مع الحالتين تلقائيًا (تنسخ إن لزم الأمر)
x_reshaped = x_transposed.reshape(24)
print("\n✅ reshape() نجحت:", x_reshaped.shape)
```

```
# ✅ الحل الثاني: اجعلها متصلة صراحة ثم استخدم view()
x_contiguous = x_transposed.contiguous()
x_viewed = x_contiguous.view(24)
print("✅ contiguous() + view() نجحت:", x_viewed.shape)
```

القاعدة العملية: استخدم `.reshape()` كخيار افتراضي آمن دائمًا، إلا إذا كنت تحتاج التحكم الدقيق في الأداء (`.view()` أسرع لأنها لا تنسخ البيانات أبدًا، فقط تفشل إن تعذّر ذلك).

3.4 حساب حجم الذاكرة قبل التدريب — المهارة التي توفّر عليك الكوارث

هذا هو جوهر هذا الفصل عمليًا. قبل أن تبدأ أي تدريب جدّي، يجب أن تستطيع الإجابة: **"هل سيتسع هذا النموذج فعلًا في ذاكرة الـ GPU المتوفرة لدي؟"**

عند التدريب بـ AdamW، كل بارامتر واحد يحتاج تخزين **أربعة** عناصر أساسية في الذاكرة (إضافة لذاكرة الـ Activations):

```
لكل بارامتر واحد (بدقة fp32 = 4 بايت):
1. الوزن نفسه (Parameter)          → 4 بايت
2. التدرج (Gradient)               → 4 بايت
3. عزم أول في AdamW (m)            → 4 بايت
4. عزم ثانٍ في AdamW (v)           → 4 بايت
   ─────────────────────────────────────
المجموع: 16 بايت لكل بارامتر (بدقة fp32 الكاملة)
```

فإذا كان نموذجك 1 مليار بارامتر، فقط لتخزين الحالة الأساسية (دون أي Activations أو بيانات) تحتاج:

```
1,000,000,000 بارامتر × 16 بايت = 16 جيجابايت!
```

وهذا **قبل** حتى حساب ذاكرة الـ Activations (المخرجات الوسيطة المحفوظة أثناء Forward Pass لاستخدامها في Backward Pass)، والتي تنمو مع حجم الدفعة (Batch Size) وطول التسلسل (Sequence Length) بشكل كبير جدًا في المحوّلات (سنتعمّق في هذا بالفصل 14).

```python
"""
memory_estimator.py
أداة عملية لتقدير ذاكرة GPU المطلوبة قبل بدء أي تدريب.
"""
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class MemoryEstimate:
    params_gb: float
```

```
    gradients_gb: float
    optimizer_state_gb: float
    total_training_gb: float
    inference_only_gb: float


def estimate_memory(
    num_params: int,
    *,
    bytes_per_param: int = 4,       # 4=fp32, 2=fp16/bf16
    optimizer: str = "adamw",       # "adamw" يحتاج عزمين، "sgd" لا يحتاج شيء إضافي
) -> MemoryEstimate:
    """
تقدير تقريبي (Rule of Thumb) لذاكرة GPU اللازمة.
لا يشمل ذاكرة Activations (تعتمد على batch_size وseq_len ومعمارية النموذج).
    """
    params_bytes = num_params * bytes_per_param
    gradients_bytes = num_params * bytes_per_param  # نفس دقة الأوزان عادة

    optimizer_multiplier = {"adamw": 2, "adam": 2, "sgd": 0, "sgd_momentum": 1}
    opt_bytes = num_params * bytes_per_param * optimizer_multiplier.get(optimizer, 2)

    total_bytes = params_bytes + gradients_bytes + opt_bytes
    gb = 1024 ** 3

    return MemoryEstimate(
        params_gb=params_bytes / gb,
        gradients_gb=gradients_bytes / gb,
        optimizer_state_gb=opt_bytes / gb,
        total_training_gb=total_bytes / gb,
        inference_only_gb=params_bytes / gb,  # الاستدلال يحتاج فقط الأوزان
    )


if __name__ == "__main__":
    scenarios = [
,("125_000_000, 4, "adamw ,"(125M، fp32، AdamW) نموذج صغير")
,("1B (fp32، AdamW)", 1_000_000_000, 4, "adamw نموذج")
,("7B (fp32، AdamW)", 7_000_000_000, 4, "adamw نموذج")
,("7_000_000_000, 2, "adamw ,"(AdamW مختلط bf16) 7B نموذج")
,("7_000_000_000, 2, "adamw ,"(fp16) استدلال فقط — 7B نموذج")
    ]

    for name, n_params, bytes_pp, opt in scenarios:
        est = estimate_memory(n_params, bytes_per_param=bytes_pp, optimizer=opt)
        print(f"\n{name}")
        print(f"  الأوزان             : {est.params_gb:8.2f} GB")
        print(f"  التدرجات            : {est.gradients_gb:8.2f} GB")
        print(f"  حالة Optimizer      : {est.optimizer_state_gb:8.2f} GB")
        print(f"  الإجمالي (تدريب)    : {est.total_training_gb:8.2f} GB  ⚠ بدون Activations")
        print(f"  الإجمالي (استدلال)  : {est.inference_only_gb:8.2f} GB")
```

النتيجة الصادمة التي ستلاحظها: نموذج بـ 7 مليار بارامتر يحتاج تقريبًا **112 جيجابايت فقط لحالة التدريب الأساسية بدقة fp32** — أكبر بكثير من أي بطاقة GPU استهلاكية (عادة 24-80 جيجابايت)! هذا بالضبط سبب حاجتنا لتقنيات مثل **Gradient Checkpointing**، **Mixed Precision**، و**التدريب الموزّع (Distributed Training)** التي سنتعمّق فيها في الفصل 23. لكن دعنا نبدأ بأبسط حل متاح فورًا: **Mixed Precision Training**.

3.5 Mixed Precision Training: أفضل ما في العالمين

الفكرة: نُجري **معظم** العمليات الحسابية (الضرب الكبير، Forward/Backward) بدقة منخفضة (`bf16` أو `fp16`) لتوفير الذاكرة والسرعة، لكن نحتفظ ببعض العمليات الحسّاسة (مثل التراكمات الإحصائية في LayerNorm، أو خطوة تحديث الـ Optimizer) بدقة `fp32` لضمان الاستقرار العددي.

PyTorch يُوفّر هذا تلقائيًا عبر `torch.amp` (Automatic Mixed Precision):

```
"""
mixed_precision_training.py
مقارنة عملية بين التدريب بـ fp32 الكامل وبـ Mixed Precision (bf16/fp16)،
مع قياس الذاكرة والوقت الفعليين على GPU.
"""
from __future__ import annotations

import time

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)


def train_step_fp32(model: nn.Module, optimizer: torch.optim.Optimizer, x: torch.Tensor, y: torch.Tensor) -> float:
    optimizer.zero_grad(set_to_none=True)
    logits = model(x)
    loss = F.cross_entropy(logits, y)
    loss.backward()
    optimizer.step()
    return loss.item()


def train_step_amp(
    model: nn.Module,
    optimizer: torch.optim.Optimizer,
    scaler: torch.amp.GradScaler,
    x: torch.Tensor,
    y: torch.Tensor,
    device_type: str,
) -> float:
    optimizer.zero_grad(set_to_none=True)
    # داخل هذا السياق فقط يُحوّل العمليات المناسبة تلقائيًا لـ bf16/fp16 autocast
    with torch.amp.autocast(device_type=device_type, dtype=torch.bfloat16):
```

```
        logits = model(x)
        loss = F.cross_entropy(logits, y)

    # GradScaler ضروري مع fp16 (ليس bf16) لمنع فيضان/تلاشي التدرجات الصغيرة جدًا
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    return loss.item()


if __name__ == "__main__":
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    device_type = device.type

    model = nn.Sequential(
        nn.Linear(1024, 2048), nn.GELU(),
        nn.Linear(2048, 2048), nn.GELU(),
        nn.Linear(2048, 10),
    ).to(device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
    x = torch.randn(256, 1024, device=device)
    y = torch.randint(0, 10, (256,), device=device)

# --- تدريب fp32 عادي --- #
    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats()
        torch.cuda.synchronize()
    start = time.perf_counter()
    for _ in range(50):
        train_step_fp32(model, optimizer, x, y)
    if device.type == "cuda":
        torch.cuda.synchronize()
    fp32_time = time.perf_counter() - start
    fp32_peak_mem = torch.cuda.max_memory_allocated() / 1e9 if device.type == "cuda" else 0.0

# --- إعادة تهيئة النموذج لمقارنة عادلة --- #
    model2 = nn.Sequential(
        nn.Linear(1024, 2048), nn.GELU(),
        nn.Linear(2048, 2048), nn.GELU(),
        nn.Linear(2048, 10),
    ).to(device)
    optimizer2 = torch.optim.AdamW(model2.parameters(), lr=3e-4)
    scaler = torch.amp.GradScaler(enabled=(device_type == "cuda"))

    if device.type == "cuda":
        torch.cuda.reset_peak_memory_stats()
        torch.cuda.synchronize()
    start = time.perf_counter()
    for _ in range(50):
        train_step_amp(model2, optimizer2, scaler, x, y, device_type)
    if device.type == "cuda":
        torch.cuda.synchronize()
    amp_time = time.perf_counter() - start
    amp_peak_mem = torch.cuda.max_memory_allocated() / 1e9 if device.type == "cuda" else 0.0
```

```
    print(f"الجهاز: {device}")
    print(f"fp32 عادي      : {fp32_time:.3f} ثانية | ذروة الذاكرة: {fp32_peak_mem:.3f} GB")
    print(f"Mixed Precision: {amp_time:.3f} ثانية | ذروة الذاكرة: {amp_peak_mem:.3f} GB")
    if device.type == "cuda":
        print(f"\nتسريع: {fp32_time/amp_time:.2f}x | توفير ذاكرة: {(1 - amp_peak_mem/fp32_peak_mem)*100:.1f}%")
```

> ⚠️ **لماذا نحتاج `GradScaler` مع `fp16` لكن ليس بالضرورة مع `bf16`؟** تذكّر من القسم 3.1: `fp16` له مدى ديناميكي ضيّق، فالتدرجات الصغيرة جدًا (شائعة في الشبكات العميقة) قد "تتلاشى لتُصبح صفرًا" (Underflow) عند تمثيلها بـ `fp16`. `GradScaler` يحل هذا بضرب الخسارة بعامل كبير قبل `backward()` (مما "يرفع" التدرجات الصغيرة لمجال آمن)، ثم يقسم عليها مجددًا قبل `optimizer.step()`. أما `bf16` فمداه الديناميكي واسع مثل `fp32` أصلًا، فلا حاجة لهذه الخدعة غالبًا.

3.6 `torch.compile()`: التسريع التلقائي بأقل جهد (PyTorch 2.0+)

ميزة ثورية أُضيفت في PyTorch 2.0: سطر واحد يُحوّل نموذجك إلى كود مُحسَّن (عبر تتبّع الرسم الحسابي ودمج العمليات - Kernel Fusion - باستخدام مُصرّف TorchInductor خلف الكواليس) دون تغيير منطق الكود إطلاقًا.

```
"""
torch_compile_demo.py
استخدام torch.compile لتسريع التدريب تلقائيًا، مع ملاحظات عملية حول التحذيرات الشائعة.
"""
from __future__ import annotations

import torch
import torch.nn as nn

torch.manual_seed(0)


class SimpleBlock(nn.Module):
    def __init__(self, dim: int) -> None:
        super().__init__()
        self.ln = nn.LayerNorm(dim)
        self.fc1 = nn.Linear(dim, dim * 4)
        self.fc2 = nn.Linear(dim * 4, dim)
        self.act = nn.GELU()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.ln(x)
        h = self.fc2(self.act(self.fc1(h)))
        return x + h   # اتصال تخطّي (Residual Connection) — سنتعمّق فيها لاحقًا


if __name__ == "__main__":
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = SimpleBlock(dim=512).to(device)
```

```
# السطر السحري: compile النموذج دون تغيير أي منطق آخر
    compiled_model = torch.compile(model)

    x = torch.randn(32, 128, 512, device=device)

# أول استدعاء يكون أبطأ (يحدث "تسخين" Warm-up أثناء التصريف JIT)
    out_first = compiled_model(x)
    print("الناتج الأول (مع تكلفة التصريف):", out_first.shape)

# الاستدعاءات اللاحقة بنفس الشكل (Shape) تكون أسرع بشكل ملحوظ
    out_second = compiled_model(x)
    print("الناتج الثاني (مُسرَّع):", out_second.shape)
```

> 💡 **نصيحة عملية:** `torch.compile()` يُصرّف الكود **لكل شكل (Shape) مختلف** من المدخلات على حدة. إذا كانت أحجام الدفعات أو أطوال التسلسل تتغير باستمرار أثناء التدريب، قد يُعيد التصريف مرارًا ويُبطئ الأداء بدل تسريعه. استخدم `dynamic=True` في هذه الحالة، أو ثبّت الأشكال قدر الإمكان (Padding لطول موحّد، مثلًا).

3.7 حساب FLOPs: قاعدة "6N" الشهيرة لتقدير تكلفة التدريب

سؤال استراتيجي مهم: قبل تدريب نموذج ضخم، كيف نُقدّر **كم ستستغرق العملية بالساعات/الأيام** على عتاد معيّن؟ توجد قاعدة تقريبية مشهورة جدًا في أدبيات Scaling Laws (سنتعمّق فيها لاحقًا بالفصل 22):

```
FLOPs الكلية للتدريب ≈ 6 × N × D

حيث:
  N = عدد بارامترات النموذج
  D = عدد التوكنز الكلي في بيانات التدريب
```

من أين يأتي الرقم 6؟ تقريبيًا: Forward Pass يحتاج `2N` FLOPs لكل توكن (كل بارامتر يُشارك في عملية ضرب وجمع واحدة تقريبًا)، وBackward Pass يحتاج ضعف ذلك تقريبًا (`4N`) لأنه يحسب تدرجات بالنسبة للأوزان وللمدخلات معًا. المجموع: `2N + 4N = 6N` لكل توكن.

```
"""
flops_estimator.py
تقدير وقت التدريب التقريبي باستخدام قاعدة 6ND، بالاعتماد على معدّل FLOPs
الفعلي لبطاقة GPU معيّنة (Model FLOPs Utilization - MFU).
"""
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
```

```python
class TrainingTimeEstimate:
    total_flops: float
    estimated_hours: float
    estimated_days: float


# أرقام تقريبية لأداء الذروة النظري لبعض بطاقات GPU الشائعة (TFLOPS بدقة bf16)
GPU_PEAK_TFLOPS_BF16: dict[str, float] = {
    "A100_80GB": 312.0,
    "H100_80GB": 989.0,
    "RTX_4090": 165.0,
}


def estimate_training_time(
    num_params: int,
    num_tokens: int,
    gpu_name: str,
    num_gpus: int = 1,
    mfu: float = 0.4,   # Model FLOPs Utilization: 50-30 عادة، الكفاءة الفعلية%
) -> TrainingTimeEstimate:
    """
مبدأ MFU: بطاقات GPU تُعلن عن "أداء الذروة النظري"، لكن لا يمكن الوصول إليه
فعليًا بسبب نقل البيانات، التزامن، واختناقات أخرى. 40-50% اعتبار جيد وواقعي.
    """
    total_flops = 6 * num_params * num_tokens
    peak_flops_per_gpu = GPU_PEAK_TFLOPS_BF16[gpu_name] * 1e12
    effective_flops_per_sec = peak_flops_per_gpu * num_gpus * mfu

    seconds = total_flops / effective_flops_per_sec
    hours = seconds / 3600
    days = hours / 24

    return TrainingTimeEstimate(total_flops=total_flops, estimated_hours=hours, estimated_days=days)


if __name__ == "__main__":
# مثال: تدريب نموذج 1B بارامتر على 20B توكن (نسبة Chinchilla تقريبية: 20 توكن/بارامتر)
    scenarios = [
        ("1B 20 على B 8 ،توكنx A100", 1_000_000_000, 20_000_000_000, "A100_80GB", 8),
        ("7B 1.4 على T 64 ،توكنx A100", 7_000_000_000, 1_400_000_000_000, "A100_80GB", 64),
        ("7B 1.4 على T 64 ،توكنx H100", 7_000_000_000, 1_400_000_000_000, "H100_80GB", 64),
    ]

    for name, n_params, n_tokens, gpu, n_gpus in scenarios:
        est = estimate_training_time(n_params, n_tokens, gpu, n_gpus)
        print(f"\n{name}")
        print(f"  إجمالي FLOPs المطلوبة : {est.total_flops:.3e}")
        print(f"  الوقت التقديري        : {est.estimated_hours:,.1f} ساعة (~{est.estimated_days:,.1f} يوم)")
```

هذه الأداة بسيطة لكنها **قوية جدًا عمليًا**: قبل أن تطلب ميزانية حوسبة ضخمة أو تبدأ تجربة ستستمر أسابيع، احسب تقديرًا سريعًا أولًا بهذه القاعدة لتتجنب المفاجآت.

---

4. 🧪 مثال محلول خطوة بخطوة

لنجمع كل ما تعلمناه في "فحص جاهزية تدريب" شامل قبل إطلاق أي تجربة فعلية — سكربت تشخيصي احترافي يُستخدم حرفيًا في بيئات الإنتاج قبل تخصيص موارد GPU باهظة التكلفة.

```
"""
pretraining_readiness_check.py
سكربت تشخيصي شامل: يفحص البيئة، يُقدّر الذاكرة والوقت، ويُحذّر من المشاكل
المحتملة قبل إطلاق تدريب فعلي مكلف.
"""
from __future__ import annotations

from dataclasses import dataclass

import torch


@dataclass(frozen=True)
class ReadinessReport:
    device_info: str
    num_gpus: int
    total_gpu_memory_gb: float
    estimated_required_memory_gb: float
    memory_sufficient: bool
    warnings: list[str]


def check_training_readiness(
    num_params: int,
    batch_size: int,
    seq_len: int,
    hidden_dim: int,
    num_layers: int,
    bytes_per_param: int = 2,  # bf16 الافتراضي الحديث
    safety_margin: float = 1.3,  # هامش أمان 30% لذاكرة Activations غير المحسوبة بدقة
) -> ReadinessReport:
    warnings: list[str] = []

    num_gpus = torch.cuda.device_count() if torch.cuda.is_available() else 0
    if num_gpus == 0:
        device_info = "لا يوجد GPU — سيتم التدريب على CPU متاح (بطيء جدًا لنماذج كبيرة)"
        total_gpu_mem = 0.0
        warnings.append("⚠️ لا يوجد GPU — مناسب فقط للتجارب الصغيرة جدًا أو الاختبار")
    else:
        device_info = torch.cuda.get_device_name(0)
        total_gpu_mem = torch.cuda.get_device_properties(0).total_memory / 1e9

    # تقدير ذاكرة الحالة الأساسية (أوزان + تدرجات + AdamW)
    base_memory_bytes = num_params * bytes_per_param * 4  # 4x: weight+grad+m+v
    base_memory_gb = base_memory_bytes / 1e9

    # تقدير تقريبي جدًا لذاكرة Activations (قاعدة تقريبية شائعة لمحوّلات)
```

```
    activations_gb = (
        batch_size * seq_len * hidden_dim * num_layers * bytes_per_param * 2
    ) / 1e9

    estimated_total = (base_memory_gb + activations_gb) * safety_margin

    if num_gpus > 0 and estimated_total > total_gpu_mem:
        warnings.append(
            f"⚠️ الذاكرة المُقدَّرة ({estimated_total:.1f}GB) تتجاوز "
            f"الذاكرة المتاحة ({total_gpu_mem:.1f}GB)! فكّر في:"
        )
        warnings.append("   - تقليل batch_size أو seq_len")
        warnings.append("   - استخدام Gradient Checkpointing")
        warnings.append("   - استخدام Gradient Accumulation بدل دفعة كبيرة واحدة")
        warnings.append("   - التدريب الموزّع عبر عدة GPUs (ZeRO/FSDP — الفصل 23)")

    if bytes_per_param == 4:
        warnings.append("💡 تستخدم fp32 تلميح: جرّب bf16 الكاملة تقريبًا لتوفير نصف الذاكرة")

    return ReadinessReport(
        device_info=device_info,
        num_gpus=max(num_gpus, 1),
        total_gpu_memory_gb=total_gpu_mem,
        estimated_required_memory_gb=estimated_total,
        memory_sufficient=(estimated_total <= total_gpu_mem) if num_gpus > 0 else False,
        warnings=warnings,
    )


if __name__ == "__main__":
    report = check_training_readiness(
        num_params=1_000_000_000,
        batch_size=16,
        seq_len=2048,
        hidden_dim=2048,
        num_layers=24,
        bytes_per_param=2,
    )

    print(f"الجهاز: {report.device_info}")
    print(f"عدد GPUs: {report.num_gpus}")
    print(f"الذاكرة الكلية المتاحة: {report.total_gpu_memory_gb:.1f} GB")
    print(f"الذاكرة المُقدَّرة المطلوبة: {report.estimated_required_memory_gb:.1f} GB")
    print(f"هل الذاكرة كافية؟ {'✅ نعم' if report.memory_sufficient else '❌ لا'}")
    if report.warnings:
        print("\nالتحذيرات والتوصيات:")
        for w in report.warnings:
            print(f"  {w}")
```

5. 💡 نصائح وحيل (Tips & Tricks)

- **استخدم `bf16` كخيار افتراضي للتدريب الحديث** (وليس `fp16`) إن كان عتادك يدعمه (A100 فما فوق، أو TPU) — توفّر تعقيد `GradScaler` وتحصل على استقرار أفضل.
- **راقب استخدام الذاكرة بأدوات PyTorch المدمجة:** `torch.cuda.memory_allocated()`، `torch.cuda.max_memory_allocated()`، و`torch.cuda.memory_summary()` تُعطيك تفاصيل دقيقة جدًا عند تشخيص مشاكل الذاكرة.
- **لا تنسَ `torch.cuda.empty_cache()`** بين التجارب المختلفة في نفس الجلسة البرمجية (مثل Jupyter Notebook) لتحرير الذاكرة المُحتجزة من تجارب سابقة.
- **استخدم `nvidia-smi`** (أو `nvtop` لواجهة أجمل) في الطرفية لمراقبة استخدام GPU الفعلي في الوقت الحقيقي أثناء التدريب — أداة تشخيصية لا غنى عنها.
- **ابدأ دائمًا بـ Batch Size صغير للتأكد من عدم وجود أخطاء منطقية**، ثم زده تدريجيًا لإيجاد أقصى حجم يتسع في الذاكرة (تقنية تُعرف بـ "Batch Size Finder").
- **فعّل `torch.backends.cudnn.benchmark = True`** عند تثبيت أحجام المدخلات (Shapes) طوال التدريب — يُسرّع العمليات عبر اختيار أفضل خوارزميات cuDNN تلقائيًا بعد تجربتها مرة واحدة.

---

6. ⚠️ أخطاء شائعة ومصائد (Common Pitfalls)

الخطأ 1: خلط الأجهزة بين النموذج والبيانات

```
# ❌ خطأ كلاسيكي: نسيان نقل البيانات الجديدة للجهاز في كل حلقة
model = model.to(device)
for xb, yb in loader:
    output = model(xb)   # RuntimeError: xb لا يزال على CPU!
```

```
# ✅ صحيح: انقل كل دفعة بيانات للجهاز بشكل صريح في كل مرة
model = model.to(device)
for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    output = model(xb)
```

الخطأ 2: تجاهل تحرير ذاكرة الرسم الحسابي

```
# ❌ خطأ: تخزين loss مع رسمه الحسابي الكامل يُراكم الذاكرة بلا داعٍ
losses = []
for xb, yb in loader:
```

```
    loss = compute_loss(model, xb, yb)
    losses.append(loss)   # يحتفظ بكل الرسم الحسابي لكل دفعة! تسريب ذاكرة فعلي
```

```
# ✅ صحيح: استخدم .item() لاستخراج القيمة العددية فقط، وتخلّص من الرسم
losses = []
for xb, yb in loader:
    loss = compute_loss(model, xb, yb)
    losses.append(loss.item())   # رقم Python عادي، لا رسم حسابي مرتبط به
```

الخطأ 3: استخدام `torch.no_grad()` بشكل خاطئ أو نسيانه أثناء الاستدلال

```
# ❌ هدر للذاكرة: PyTorch يبني رسمًا حسابيًا كاملًا حتى أثناء الاستدلال فقط
def predict(model, x):
    return model(x).argmax(dim=-1)
```

```
# ✅ صحيح: عطّل تتبع التدرجات تمامًا عند عدم الحاجة لـ backward()
@torch.no_grad()
def predict(model, x):
    return model(x).argmax(dim=-1)
```

الخطأ 4: مقارنة أداء GPU/CPU دون مزامنة صحيحة

```
# ❌ خطأ: عمليات CUDA غير متزامنة (Asynchronous)؛ القياس سيكون غير دقيق
start = time.perf_counter()
output = model(x)
elapsed = time.perf_counter() - start   # قد يقيس فقط وقت "إطلاق" العملية، لا تنفيذها!
```

```
# ✅ صحيح: استدعِ synchronize() قبل وبعد القياس لضمان اكتمال كل عمليات GPU فعليًا
torch.cuda.synchronize()
start = time.perf_counter()
output = model(x)
torch.cuda.synchronize()
elapsed = time.perf_counter() - start
```

الخطأ 5: تجاهل `num_workers` و`pin_memory` في DataLoader

```
# ❌ غير مُحسَّن: تحميل البيانات يصبح عنق الزجاجة (Bottleneck) بدل GPU
loader = DataLoader(dataset, batch_size=64)
```

```python
# ✅ أفضل: استخدم عدة عمليات لتحميل البيانات بالتوازي مع التدريب
loader = DataLoader(
    dataset, batch_size=64,
    num_workers=4,       # عمليات موازية لتحضير الدفعات التالية مسبقًا
    pin_memory=True,     # يُسرّع نقل البيانات من CPU إلى GPU
)
```

---

7. ⚖️ جدول مقارنة: خيارات الدقة العددية (Precision)

النوع	الذاكرة/عنصر	المدى الديناميكي	الاستقرار	يحتاج GradScaler؟	التوصية
`float32`	4 بايت	واسع جدًا	ممتاز	لا	آمن دائمًا، لكن بطيء وثقيل
`float16`	2 بايت	ضيّق	متوسط	**نعم**	عتاد قديم لا يدعم bf16
`bfloat16`	2 بايت	واسع (مثل fp32)	جيد جدًا	عادة لا	**المعيار الحديث لتدريب LLM**
`fp8` (حديث جدًا)	1 بايت	ضيّق جدًا	يحتاج تقنيات خاصة	نعم، متقدم	H100+ فقط، أبحاث متقدمة

---

8. ✅ قائمة التحقق من أفضل الممارسات

- [ ] تستخدم `get_best_device()` أو مكافئها بدل تثبيت `"cuda"` يدويًا (مرونة عبر البيئات).
- [ ] تنقل كل من النموذج والبيانات لنفس الجهاز بشكل صريح في كل حلقة تدريب.
- [ ] قدّرت حجم الذاكرة المطلوبة (أوزان + تدرجات + Optimizer State) **قبل** بدء أي تدريب كبير.
- [ ] تستخدم `bf16` (أو `fp16` + `GradScaler`) بدل `fp32` الكاملة لتوفير الذاكرة والوقت.
- [ ] تستخدم `@torch.no_grad()` لكل كود استدلال لا يحتاج `backward()`.
- [ ] فعّلت `num_workers` و`pin_memory` في `DataLoader` لتجنب اختناق تحميل البيانات.
- [ ] تستخدم `.item()` بدل الاحتفاظ بـ Tensor كامل عند تخزين قيم Loss للتتبّع.
- [ ] جرّبت `torch.compile()` كخطوة تسريع سهلة التطبيق قبل أي تحسين معقّد يدويًا.
- [ ] تُزامن (`torch.cuda.synchronize()`) عند قياس الوقت الفعلي لعمليات GPU.
- [ ] تحسب FLOPs التقديرية بقاعدة `6ND` قبل إطلاق تجارب تدريب طويلة ومكلفة.

---

9. 🎯 أسئلة مقابلات شخصية

س1: لماذا يُفضَّل `bfloat16` على `float16` في تدريب نماذج اللغة الكبيرة الحديثة؟

`bfloat16` يحتفظ بنفس عدد بتات الأس (8 بتات) الموجودة في `float32`، مما يمنحه **نفس المدى الديناميكي** الواسع، فلا يُعاني من مشاكل الفيضان (Overflow) أو التلاشي (Underflow) الشائعة في `float16` (الذي له 5 بتات أس فقط). هذا يجعل `bfloat16` أكثر استقرارًا أثناء التدريب دون الحاجة لتقنيات تعويضية معقّدة مثل `GradScaler`، رغم أنه يُضحّي بدقة الكسر العشري (Mantissa) مقارنة بـ `float16` — وهي تضحية مقبولة جدًا في سياق تدريب الشبكات العميقة حيث الاستقرار العددي أهم من الدقة الدقيقة جدًا.

س2: اشرح قاعدة "6ND" لتقدير FLOPs التدريب. من أين يأتي الرقم 6؟

القاعدة تنص على أن إجمالي FLOPs اللازمة لتدريب نموذج تقريبًا تساوي `6 × N × D`، حيث N عدد البارامترات وD عدد توكنز التدريب. الرقم 6 يأتي من: Forward Pass يحتاج تقريبًا `2N` FLOPs لكل توكن (كل بارامتر يُشارك في عملية ضرب وجمع)، بينما Backward Pass يحتاج ضعف ذلك تقريبًا (`4N`) لأنه يحسب التدرجات بالنسبة لكل من الأوزان والمدخلات الوسيطة. المجموع `2N+4N=6N` لكل توكن، ونضربه بعدد التوكنز الكلي D للحصول على التكلفة الإجمالية.

س3: ما الفرق بين `.view()` و `.reshape()` في PyTorch؟ ولماذا قد تفشل `.view()` أحيانًا؟

`.view()` تُنشئ Tensor جديد "يُشارك" نفس الذاكرة الأساسية مع الأصل دون نسخ أي بيانات، لكن هذا يتطلب أن تكون البيانات **متصلة (Contiguous)** في الذاكرة بالترتيب المطلوب. عمليات مثل `.transpose()` أو `.permute()` تُغيّر فقط "خرائط القراءة" (Strides) دون إعادة ترتيب البيانات فعليًا، فتصبح النتيجة غير متصلة، مما يجعل `.view()` تفشل. `.reshape()` أذكى: تحاول استخدام `view()` إن أمكن، وإلا تُنشئ نسخة جديدة متصلة تلقائيًا — لذا هي الخيار الأكثر أمانًا عمومًا، رغم أن `.view()` أسرع حين تنجح لأنها لا تنسخ البيانات أبدًا.

س4: ماذا يحدث فعليًا عند استخدام `GradScaler` مع `float16`؟ ولماذا لا نحتاجه عادة مع `bfloat16`؟

`GradScaler` يحل مشكلة تلاشي التدرجات الصغيرة جدًا (Underflow) الشائعة عند استخدام `float16` بسبب مداه الديناميكي الضيّق — فتدرجات صغيرة قد تُصبح صفرًا تمامًا عند تمثيلها بهذه الدقة. الحل: نضرب قيمة الخسارة بعامل كبير (Scale Factor) قبل `backward()`، مما "يرفع" كل التدرجات الناتجة لمجال آمن رقميًا بعيد عن الصفر، ثم نقسم عليها مجددًا قبل تطبيق خطوة `optimizer.step()`. بما أن `bfloat16` له

> مدى ديناميكي واسع مماثل لـ `float32` أصلًا، فمشكلة التلاشي أقل حدة بكثير، ولذلك غالبًا لا نحتاج معه `GradScaler`.

س5: كيف تُقدّر إن كان نموذج معيّن سيتسع في ذاكرة بطاقة GPU محددة قبل بدء التدريب فعليًا؟

> أحسب أولًا ذاكرة "الحالة الأساسية" للنموذج: البارامترات، التدرجات، وحالة الـ Optimizer (في AdamW، ضعف حجم البارامترات إضافيًا للعزمين m وv) — تقريبًا `4× bytes_per_param × N` باستخدام AdamW بدقة موحّدة. ثم أُضيف تقديرًا لذاكرة الـ Activations التي تعتمد على حجم الدفعة وطول التسلسل وعدد الطبقات وأبعاد النموذج الداخلية (وهذا الجزء الأعقد وسنتعمّق فيه حسابيًا في فصل تحليل التعقيد، الفصل 14). أخيرًا أُضيف هامش أمان (عادة 20-30%) للعمليات الوسيطة غير المحسوبة بدقة، وأُقارن الناتج النهائي بإجمالي ذاكرة الـ GPU المتاحة (`torch.cuda.get_device_properties(0).total_memory`).

---

10. 🧪 تمرين عملي (Mini-Project)

المهمة: "لوحة قيادة جاهزية التدريب" — Training Readiness Dashboard

ابنِ أداة سطر أوامر (CLI) تأخذ مواصفات نموذج ومعدات متاحة، وتُصدر تقريرًا كاملاً: الذاكرة المطلوبة، الوقت التقديري، والتوصيات.

```
"""
training_dashboard.py — هيكل بداية للتمرين
أداة CLI شاملة لتقييم جاهزية تدريب نموذج معيّن.
"""
from __future__ import annotations

import argparse
from dataclasses import dataclass

import torch


@dataclass(frozen=True)
class ModelSpec:
    num_params: int
    hidden_dim: int
    num_layers: int
    seq_len: int
    batch_size: int


@dataclass(frozen=True)
class HardwareSpec:
    gpu_name: str
```

```
    num_gpus: int
    memory_per_gpu_gb: float


def build_arg_parser() -> argparse.ArgumentParser:
    """
    TODO: أنشئ argparse.ArgumentParser بالحقول التالية:
    --num-params (int, مطلوب)
    --hidden-dim (int, افتراضي 2048)
    --num-layers (int, افتراضي 24)
    --seq-len (int, افتراضي 2048)
    --batch-size (int, افتراضي 16)
    --num-tokens (int, مطلوب لحساب FLOPs للتدريب الكامل)
    --gpu-name (str, من قائمة محددة مسبقًا)
    --num-gpus (int, افتراضي 1)
    --dtype (str، إما "fp32" أو "bf16"، افتراضي "bf16")
    """
    raise NotImplementedError


def generate_full_report(model_spec: ModelSpec, hw_spec: HardwareSpec, num_tokens: int, dtype: str) -> str:
    """
    TODO: ادمج estimate_memory() و estimate_training_time() و check_training_readiness()
في تقرير نصي واحد منسّق وجميل، يتضمن: (من الأمثلة السابقة في هذا الفصل)
1. ملخص المواصفات المُدخلة
2. تقدير الذاكرة (كافية أم لا، مع تحذيرات)
3. تقدير الوقت الإجمالي للتدريب
4. توصيات محددة إن لزم الأمر (تقليل batch_size، تفعيل bf16، عدد GPUs إضافي مطلوب...)
    """
    raise NotImplementedError


if __name__ == "__main__":
    parser = build_arg_parser()
    args = parser.parse_args()

    # TODO: ابنِ ModelSpec و HardwareSpec من args، واستدعِ generate_full_report، واطبع النتيجة

# اختبر أداتك بسيناريوهات مختلفة من سطر الأوامر، مثال:
    # python training_dashboard.py --num-params 7000000000 --num-tokens 1400000000000 \
    #     --gpu-name A100_80GB --num-gpus 8 --dtype bf16
```

11. 🔗 جسر للفصل التالي

بهذا نُغلق **الجزء الأول: الأساسيات** من رحلتنا. تمتلك الآن الأساس الرياضي الكامل (الفصل 1)، فهم الشبكات العصبية من الصفر (الفصل 2)، آلية التعلّم والتحسين (الفصل 3)، معايير التقييم والتعميم (الفصل 4)، والبنية التحتية الحسابية العملية (هذا الفصل). هذه الأسس الخمسة ستُرافقك بصمت في كل فصل قادم — فكل نموذج لغوي ضخم، مهما بدا معقدًا، هو في جوهره نفس هذه المبادئ مُطبّقة على نطاق أوسع بكثير.

الآن، السؤال الذي يجب أن يشغلك: الشبكات العصبية تتعامل مع أرقام (Tensors)، لكن اللغة البشرية تتكون من **كلمات ورموز**. كيف نُحوّل جملة مثل "القطة تحب أكل السمك" إلى أرقام يفهمها نموذج رياضي، بطريقة تحافظ على **المعنى** وليس فقط الترميز العشوائي؟

هذا هو موضوع **الجزء الثاني: التضمينات (Embeddings)**، ونبدأه في الفصل السادس بـ **Tokenization** — الخطوة الأولى والأكثر جوهرية في أي خط أنابيب معالجة لغة طبيعية: كيف نُقسّم النص إلى وحدات قابلة للمعالجة (Tokens)، ولماذا اختيار استراتيجية التقسيم هذه يؤثر بعمق على كل شيء لاحق في النموذج بأكمله.

---

12. 📚 مراجع للتعمّق

المرجع	النوع	الرابط/المصدر
Micikevicius et al. (2017), *"Mixed Precision Training"*	ورقة تأسيسية	arXiv:1710.03740
Kaplan et al. (2020), *"Scaling Laws for Neural Language Models"*	ورقة بحثية (خلفية FLOPs)	arXiv:2001.08361
PyTorch Official Docs — *"Automatic Mixed Precision"*	توثيق تقني	pytorch.org/docs/stable/amp.html
PyTorch Official Docs — *"torch.compile Tutorial"*	توثيق تقني	pytorch.org/tutorials
NVIDIA, *"CUDA C++ Programming Guide"*	دليل مرجعي متقدم	docs.nvidia.com/cuda
Hugging Face, *"Model Memory Calculator"*	أداة تفاعلية	huggingface.co/docs/transformers/perf_train_gpu_one
Google Cloud, *"TPU Architecture Documentation"*	توثيق تقني	cloud.google.com/tpu/docs

==========

الجزء 06 — التضمينات (Embeddings)

الفصل السادس: Tokenization — كيف نُحوّل اللغة إلى أرقام؟



الجزء الثاني: التضمينات (Embeddings)

عندما تصبح الكلمة رقمًا: أول وأهم قرار هندسي في أي نظام لغوي

1. 🟢 لماذا يهمّك هذا؟

في الفصل الخامس أتقنّا البنية التحتية الحسابية: كيف نُدير Tensors، الذاكرة، والأجهزة. لكن توقف لحظة وتأمّل مشكلة أساسية لم نُجب عنها بعد:

> كل شيء تعلمناه حتى الآن يتعامل مع **أرقام**. لكن المدخل الحقيقي لأي نموذج لغوي هو **نص**: حروف، كلمات، علامات ترقيم، رموز تعبيرية، أكواد برمجية، حتى لغات مُختلطة في جملة واحدة. كيف بالضبط نُحوّل

> جملة مثل "لا أُحب الذهاب إلى المدرسة يوم الإثنين 😩" إلى تسلسل أرقام يفهمه نموذج رياضي؟

قد يبدو هذا سؤالًا تقنيًا بسيطًا — "فقط رقّم كل كلمة!" لكن إليك سؤالًا يكشف عمق المشكلة الحقيقية:

> ماذا تفعل عندما يواجه نموذجك كلمة **لم يرها إطلاقًا** أثناء التدريب؟ كلمة مثل "كوفيدية" أو اسم منتج جديد أو خطأ إملائي؟ هل "يتجمّد" النموذج ويفشل، أم لديه طريقة ذكية للتعامل مع المجهول؟

وسؤال آخر لا يقل أهمية استراتيجيًا:

> لماذا تحتوي بعض نماذج اللغة على "قاموس" (Vocabulary) بحجم 32 ألف توكن، وأخرى بحجم 150 ألف توكن؟ ولماذا تتعامل بعض النماذج مع اللغة العربية بكفاءة أقل بكثير من الإنجليزية رغم تدريبها على بيانات ضخمة من الاثنتين؟

الإجابات على كل هذه الأسئلة تكمن في **Tokenization** — وهي، رغم كونها أول خطوة في أي خط أنابيب NLP، إلا أنها من أكثر القرارات الهندسية تأثيرًا وأقلّها اهتمامًا من المبتدئين. اختيار استراتيجية Tokenization الخاطئة يمكن أن يجعل نموذجك يستهلك ضعف عدد التوكنز لمعالجة نفس الجملة العربية مقارنة بالإنجليزية (وهذه مشكلة حقيقية جدًا في نماذج تجارية كثيرة اليوم!)، أو يفشل تمامًا في فهم كلمات نادرة أو مُركّبة.

بنهاية هذا الفصل، ستفهم بعمق الخوارزميات الثلاث المهيمنة على Tokenization الحديث (BPE، WordPiece، Unigram)، وستبني Tokenizer من الصفر بيديك، وستتعلم كيف تُقيّم جودة أي Tokenizer قبل استخدامه في نموذج حقيقي.

---

3. 🟡 الشرح المتدرّج: من الحدس إلى الكود

3.1 المستوى الأول: ما الوحدة الأساسية التي نُقسّم إليها النص؟

قبل الحديث عن خوارزميات معقدة، يجب أن نُجيب عن سؤال أبسط: هل نُقسّم النص إلى **أحرف**، **كلمات**، أم شيء بينهما؟

الخيار الأول: Character-Level Tokenization

```
text = "القطة تنام"
tokens = list(text)  # ['ا', 'ل', 'ق', 'ط', 'ة', ' ', 'ت', 'ن', 'ا', 'م']
```

المزايا: مفردات صغيرة جدًا (حوالي 100-200 رمز تغطي معظم اللغات)، **لا توجد مشكلة كلمات مجهولة (OOV - Out Of Vocabulary) إطلاقًا** — أي كلمة يمكن تمثيلها بأحرفها.

العيوب: تسلسلات طويلة جدًا رقميًا (كل حرف توكن منفصل)، مما يُضاعف التكلفة الحسابية بشكل كبير (تذكّر من الفصل 5: تكلفة Attention تنمو تربيعيًا مع طول التسلسل!)، والنموذج يحتاج تعلّم تجميع الأحرف لتكوين معنى من الصفر — عبء إضافي غير ضروري.

الخيار الثاني: Word-Level Tokenization

```
text = "القطة تنام على السرير"
tokens = text.split()  # ['القطة', 'تنام', 'على', 'السرير']
```

المزايا: تسلسلات قصيرة نسبيًا، كل توكن يحمل معنى كاملاً وواضحًا مباشرة.

العيوب: مشكلة OOV **كارثية** — أي كلمة لم تظهر في بيانات بناء المفردات (ناهيك عن الأخطاء الإملائية، الأسماء الجديدة، أو الكلمات المُركّبة) تُصبح ببساطة رمز `[UNK]` غامض، فيفقد النموذج أي معلومة عنها. كما أن حجم المفردات ينفجر (مئات الآلاف من الكلمات الفريدة في أي لغة بمشتقاتها المختلفة، وهذا أسوأ بكثير في اللغة العربية بسبب الاشتقاق والتصريف الغني).

الحل الوسط الذهبي: Subword Tokenization

هنا يأتي السؤال الحاسم: ماذا لو قسّمنا الكلمات النادرة أو المعقدة إلى **أجزاء فرعية (Subwords)** ذات معنى، بينما نُبقي الكلمات الشائعة جدًا كوحدة واحدة؟

```
text = "اللامركزية"
# Word-level: ['اللامركزية']  →  غالبًا [UNK] إن كانت نادرة!
# Subword:    ['ال', 'لا', 'مركزي', 'ة']  →  كل جزء مألوف، المعنى قابل للتركيب
```

هذا بالضبط ما تفعله كل نماذج اللغة الحديثة تقريبًا: GPT وLLaMA تستخدم BPE، BERT يستخدم WordPiece، T5 وALBERT يستخدمان Unigram/SentencePiece. دعنا نفهم كل واحدة بعمق.

3.2 Byte-Pair Encoding (BPE) — من ضغط البيانات إلى نمذجة اللغة

قصة مثيرة: خوارزمية BPE لم تُخترع أصلًا لمعالجة اللغة! بل هي خوارزمية **ضغط بيانات كلاسيكية** من عام 1994 (Philip Gage)، أُعيد توظيفها لـ NLP في 2016 (Sennrich et al.) لحل مشكلة الكلمات النادرة في الترجمة الآلية.

الفكرة الحدسية

ابدأ بتقسيم النص إلى أصغر وحدة ممكنة (أحرف فردية)، ثم **كرّر** هذه الخطوة: ابحث عن أكثر **زوج متجاور** من الرموز تكرارًا في كل بيانات التدريب، وادمجه في رمز واحد جديد. استمر في هذا الدمج التكراري حتى تصل لحجم

المفردات المستهدف.

الخوارزمية خطوة بخطوة

```
1. ابدأ بمفردات = كل الأحرف الفريدة في النص (+ رمز نهاية الكلمة خاص)
2. كرّر N مرة (N = عدد عمليات الدمج المطلوبة):
   أ. عدّ تكرار كل زوج متجاور من الرموز في كل الكلمات
   ب. ابحث عن أكثر زوج تكرارًا
   ج. ادمج هذا الزوج في رمز واحد جديد، أضفه للمفردات
3. المفردات النهائية = الأحرف الأصلية + كل الرموز المُدمجة التي أُنشئت
```

لنُطبّق هذا يدويًا على مثال صغير جدًا ليتضح المنطق تمامًا:

```
"""
bpe_from_scratch.py
تطبيق كامل وواضح لخوارزمية BPE من الصفر، بدون أي مكتبات خارجية،
لفهم الآلية الداخلية حرفيًا قبل استخدام المكتبات الجاهزة.
Python 3.13+
"""
from __future__ import annotations

from collections import Counter, defaultdict


def get_word_frequencies(corpus: list[str]) -> dict[tuple[str, ...], int]:
    """
يُحوّل كل كلمة إلى تسلسل من الأحرف + رمز خاص لنهاية الكلمة (</w>)،
ويحسب تكرار كل كلمة في المجموعة.
رمز </w> ضروري ليُميّز الـ Tokenizer بين "قطة" في وسط جملة و"قطة" في نهايتها
(فقد يختلف المعنى السياقي أو النحوي، ويساعد على إعادة بناء النص لاحقًا).
    """
    freqs: Counter[tuple[str, ...]] = Counter()
    for sentence in corpus:
        for word in sentence.split():
            char_tuple = tuple(word) + ("</w>",)
            freqs[char_tuple] += 1
    return dict(freqs)


def get_pair_frequencies(word_freqs: dict[tuple[str, ...], int]) -> Counter[tuple[str, str]]:
    """يحسب تكرار كل زوج متجاور من الرموز عبر كل الكلمات، موزونًا بتكرار الكلمة."""
    pairs: Counter[tuple[str, str]] = Counter()
    for word, freq in word_freqs.items():
        for i in range(len(word) - 1):
            pairs[(word[i], word[i + 1])] += freq
    return pairs


def merge_pair(
    pair: tuple[str, str], word_freqs: dict[tuple[str, ...], int]
) -> dict[tuple[str, ...], int]:
```

```
    """يدمج كل ظهور لزوج معيّن في كل الكلمات، منتجًا مفردات جديدة."""
    new_word_freqs: dict[tuple[str, ...], int] = {}
    merged_token = pair[0] + pair[1]

    for word, freq in word_freqs.items():
        new_word: list[str] = []
        i = 0
        while i < len(word):
            if i < len(word) - 1 and (word[i], word[i + 1]) == pair:
                new_word.append(merged_token)
                i += 2
            else:
                new_word.append(word[i])
                i += 1
        new_word_freqs[tuple(new_word)] = freq

    return new_word_freqs


def train_bpe(
    corpus: list[str], num_merges: int
) -> tuple[list[tuple[str, str]], set[str]]:
    """
التدريب الكامل: يُعيد قائمة عمليات الدمج بترتيبها (ضرورية للترميز لاحقًا)
ومجموعة المفردات النهائية.
    """
    word_freqs = get_word_frequencies(corpus)
    merges: list[tuple[str, str]] = []

    vocabulary: set[str] = set()
    for word in word_freqs:
        vocabulary.update(word)

    for step in range(num_merges):
        pair_freqs = get_pair_frequencies(word_freqs)
        if not pair_freqs:
            print(f"توقف مبكرًا عند الخطوة {step}: لا توجد أزواج متبقية للدمج")
            break

        best_pair = max(pair_freqs, key=lambda p: pair_freqs[p])
        word_freqs = merge_pair(best_pair, word_freqs)
        merges.append(best_pair)
        vocabulary.add(best_pair[0] + best_pair[1])

        print(f"الدمج {step + 1:02d}: {best_pair} → '{best_pair[0]+best_pair[1]}' "
              f"(تكرار: {pair_freqs[best_pair]})")

    return merges, vocabulary


def apply_bpe(word: str, merges: list[tuple[str, str]]) -> list[str]:
    """يُطبّق عمليات الدمج المُتعلَّمة على كلمة جديدة، بنفس الترتيب الذي تعلّمناه."""
    tokens = list(word) + ["</w>"]

    for pair in merges:
```

```
        new_tokens: list[str] = []
        i = 0
        while i < len(tokens):
            if i < len(tokens) - 1 and (tokens[i], tokens[i + 1]) == pair:
                new_tokens.append(pair[0] + pair[1])
                i += 2
            else:
                new_tokens.append(tokens[i])
                i += 1
        tokens = new_tokens

    return tokens


if __name__ == "__main__":
    corpus = [
"القطة تحب اللعب",
"القطة تحب النوم",
"الكلب يحب اللعب",
"الكلب يحب الجري",
"القطة والكلب يلعبان معا",
    ]

    print("=== تدريب BPE على مجموعة نصوص صغيرة ===\n")
    merges, vocab = train_bpe(corpus, num_merges=15)

    print(f"\nحجم المفردات النهائي: {len(vocab)}")
    print(f"عدد عمليات الدمج: {len(merges)}")

    print("\n=== اختبار على كلمات جديدة (بعضها لم يظهر حرفيًا من قبل) ===")
    test_words = ["يلعب" ,"الكلاب" ,"القطة"]  # "يلعب" و"الكلاب"! أشكال جديدة
    for word in test_words:
        tokens = apply_bpe(word, merges)
        print(f"'{word}' → {tokens}")
```

لاحظ النتيجة المذهلة: كلمة "الكلاب" (جمع لم يظهر حرفيًا في بيانات التدريب، التي حوت "الكلب" فقط) **لا تزال قابلة للترميز بمعنى**، لأن BPE يُعيد استخدام الأجزاء الفرعية المُتعلَّمة ("ال"، "كلب/كلا"...) بدل الفشل الكامل كما كان سيحدث مع Word-Level Tokenization.

3.3 WordPiece — تلميذ BPE الأذكى إحصائيًا

WordPiece (المُستخدم في BERT) يشبه BPE في الجوهر (بناء المفردات تدريجيًا بالدمج)، لكن معيار اختيار الزوج الذي يُدمَج مختلف جوهريًا:

```
BPE يختار الزوج: (Frequency) الأكثر تكرارًا
WordPiece يختار الزوج: (Likelihood) لبيانات التدريب الذي يُعظّم الاحتمالية
```

رياضيًا، WordPiece يختار الزوج `(a, b)` الذي يُعظّم:

```
score(a, b) = freq(a, b) / (freq(a) × freq(b))
```

لماذا هذا الفرق منطقي؟ هذه الصيغة تُشبه كثيرًا **Pointwise Mutual Information (PMI)** من نظرية المعلومات: هي لا تسأل فقط "كم مرة ظهر هذا الزوج معًا؟" بل "كم مرة ظهر هذا الزوج معًا **مقارنة بما نتوقعه لو كانا مستقلين تمامًا إحصائيًا**؟" هذا يُفضّل أزواجًا "مُتماسكة لغويًا" (مثل أجزاء كلمة مُركّبة ترتبط دائمًا ببعضها) على أزواج شائعة فقط بالصدفة الإحصائية البحتة (مثل حرف شائع جدًا يظهر بجانب كل شيء تقريبًا).

```python
"""
wordpiece_scoring.py
.WordPiece مقابل BPE توضيح الفرق الجوهري بين معيار اختيار الدمج في
"""
from __future__ import annotations

from collections import Counter


def bpe_score(pair_freq: int, *_: int) -> float:
    """BPE: يختار فقط بناءً على التكرار المطلق."""
    return float(pair_freq)


def wordpiece_score(pair_freq: int, freq_a: int, freq_b: int) -> float:
    """WordPiece: يُطبّع بالتكرار الفردي لكل عنصر (شبه PMI)."""
    return pair_freq / (freq_a * freq_b)


if __name__ == "__main__":
# سيناريو توضيحي: زوج شائع جدًا لكن عشوائيًا (حروف شائعة تظهر مع كل شيء)
# مقابل زوج أقل تكرارًا لكن "متماسك" لغويًا بقوة
    scenarios = {
,(80000 ,50000 ,5000) :"شائع جدًا لكن عشوائي - (' ' ,'ة')"
,(850 ,900 ,800) :"أقل تكرارًا لكن مترابط دائمًا - ('كزي' ,'مر')"
    }

    print(f"{'السيناريو':<45}{'BPE Score':<15}{'WordPiece Score':<15}")
    for name, (pair_f, freq_a, freq_b) in scenarios.items():
        bpe_s = bpe_score(pair_f)
        wp_s = wordpiece_score(pair_f, freq_a, freq_b)
        print(f"{name:<45}{bpe_s:<15.1f}{wp_s:<15.8f}")

    print("\nلاحظ: BPE يُفضّل الزوج الأول (تكرار خام أعلى)")
    print("بينما WordPiece يُفضّل الزوج الثاني (ترابط إحصائي أقوى نسبيًا)")
```

3.4 Unigram Language Model Tokenization — المقاربة المعاكسة تمامًا

هنا نصل لفكرة مختلفة جذريًا، يستخدمها SentencePiece (المُعتمد في XLNet، ALBERT، T5، ونماذج Google عمومًا). بينما BPE وWordPiece يبنيان المفردات **تصاعديًا** (من الأحرف نحو الأعلى بالدمج)، فإن **Unigram** يعمل **تنازليًا**:

```
1. ابدأ بمفردات كبيرة جدًا جدًا (كل الكلمات الفرعية المحتملة تقريبًا،
   عادة عبر تجميع كل Substrings الشائعة من BPE مبدئيًا)
2. لكل رمز في المفردات، احسب "مدى أهميته" عبر فقدان الاحتمالية الكلية
   لبيانات التدريب لو أُزيل هذا الرمز
3. احذف نسبة معيّنة (مثلًا 10-20%) من الرموز الأقل أهمية
4. كرّر حتى تصل لحجم المفردات المستهدف
```

الفرق العملي الأهم: Unigram لا يُنتج **تقسيمًا واحدًا حتميًا** للكلمة، بل نموذجًا احتماليًا كاملاً يسمح بعدة تقسيمات ممكنة لنفس الكلمة، ويختار (أو يُرجّح بالعينة) الأكثر احتمالية وفق نموذج لغوي بسيط من نوع Unigram (حيث احتمال كل توكن مستقل عن سياقه):

```
P(تقسيم الكلمة) = Π P(كل توكن فيه)
```

هذا يمنح مرونة أكبر (يمكن حتى أخذ عينة عشوائية من تقسيمات مختلفة أثناء التدريب، تقنية تُسمى **Subword Regularization**، وهي شكل إضافي من Data Augmentation يُحسّن متانة النموذج أمام تنويعات التقسيم).

3.5 جسر عملي: استخدام مكتبة `tokenizers` من Hugging Face

بعد فهم الآلية الداخلية يدويًا، دعنا نرَ كيف نُدرّب Tokenizer احترافي بسرعة باستخدام المكتبة القياسية الصناعية:

```python
"""
train_production_tokenizer.py
تدريب BPE Tokenizer احترافي وسريع باستخدام مكتبة tokenizers من Hugging Face
(المكتوبة بـ Rust لسرعة فائقة)، جاهز للاستخدام في نموذج حقيقي.
المتطلبات: pip install tokenizers
"""
from __future__ import annotations

from pathlib import Path

from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.pre_tokenizers import Whitespace
from tokenizers.trainers import BpeTrainer
from tokenizers.normalizers import NFC, Sequence as NormSequence


def build_and_train_tokenizer(
    corpus_files: list[str],
    vocab_size: int = 8000,
    min_frequency: int = 2,
) -> Tokenizer:
    tokenizer = Tokenizer(BPE(unk_token="[UNK]"))

    # Normalization: NFC تُوحّد الأشكال المختلفة لنفس الحرف
    # (مهم جدًا للعربية: الألف المهموزة، الهمزات المنفصلة، إلخ)
    tokenizer.normalizer = NormSequence([NFC()])
```

```
    # Pre-tokenization: تقسيم أولي بسيط على المسافات قبل تطبيق BPE داخل كل كلمة
    tokenizer.pre_tokenizer = Whitespace()

    special_tokens = ["[UNK]", "[PAD]", "[CLS]", "[SEP]", "[MASK]", "[BOS]", "[EOS]"]
    trainer = BpeTrainer(
        vocab_size=vocab_size,
        min_frequency=min_frequency,
        special_tokens=special_tokens,
        show_progress=True,
    )

    tokenizer.train(files=corpus_files, trainer=trainer)
    return tokenizer


if __name__ == "__main__":
# إعداد ملف نصي تجريبي صغير للتدريب #
    sample_corpus = Path("sample_arabic_corpus.txt")
    sample_text = """القطة تحب اللعب في الحديقة كل صباح
الكلب يحب الجري خلف الكرة في الحديقة
الأطفال يلعبون مع القطة والكلب معا كل يوم
تعلم البرمجة يحتاج صبرا وممارسة مستمرة
النماذج اللغوية الكبيرة تغير طريقة تفاعلنا مع التقنية
""" * 50
    sample_corpus.write_text(sample_text, encoding="utf-8")

    tokenizer = build_and_train_tokenizer(
        corpus_files=[str(sample_corpus)], vocab_size=500
    )

# اختبار الترميز #
    test_sentence = "الأطفال والقطط يلعبون في الحدائق الجميلة"
    encoding = tokenizer.encode(test_sentence)

    print(f"الجملة الأصلية: {test_sentence}")
    print(f"عدد التوكنز    : {len(encoding.tokens)}")
    print(f"التوكنز        : {encoding.tokens}")
    print(f"المعرّفات (IDs): {encoding.ids}")

# فك الترميز للتأكد من إعادة البناء الصحيحة #
    decoded = tokenizer.decode(encoding.ids)
    print(f"\nبعد فك الترميز: {decoded}")

# حفظ الـ Tokenizer لإعادة استخدامه لاحقًا دون إعادة تدريب #
    tokenizer.save("my_arabic_tokenizer.json")
    print("\n✅ تم حفظ الـ Tokenizer في my_arabic_tokenizer.json")

    sample_corpus.unlink()  # تنظيف الملف التجريبي
```

3.6 Special Tokens ومعالجة النصوص: التفاصيل التي لا يجب إغفالها

كل Tokenizer حقيقي يحتاج رموزًا خاصة لأغراض وظيفية محددة:

الرمز	الاستخدام
`[PAD]`	حشو التسلسلات القصيرة لتصل لنفس الطول داخل Batch واحد
`[UNK]`	تمثيل أي رمز غير معروف تمامًا (نادر جدًا مع Subword الجيد)
`[CLS]`	رمز "تصنيف" يُوضع في بداية التسلسل (BERT يستخدم تمثيله النهائي لمهام التصنيف)
`[SEP]`	فاصل بين جُملتين أو مقطعين (مهام مثل Next Sentence Prediction)
`[MASK]`	رمز الإخفاء في Masked Language Modeling (BERT)
`[EOS]` / `[BOS]`	بداية/نهاية التسلسل (شائع في نماذج GPT التوليدية)

> 🔴 **صندوق التعمّق: Unicode Normalization — فخ خفي في اللغة العربية خصوصًا**
>
> هل تعلم أن الحرف "أ" يمكن تمثيله بأكثر من طريقة في معيار Unicode؟ مثلاً، "أ" المركّبة مسبقًا (Precomposed) مقابل "ا" + علامة همزة منفصلة (Decomposed) قد تبدوان متطابقتين بصريًا لكنهما **بايتات مختلفة تمامًا** داخليًا! بدون **Unicode Normalization** (عادة NFC - Normalization Form Canonical Composition)، قد يرى Tokenizer نفس الكلمة المكتوبة بطريقتين مختلفتين كرمزين مختلفين تمامًا، مما يُضاعف حجم المفردات دون داعٍ ويُشتت الإحصائيات. هذه مشكلة حقيقية جدًا وشائعة في معالجة النصوص العربية، حيث تتعدد طرق كتابة الهمزات والتشكيل.

---

4. 🧪 مثال محلول خطوة بخطوة

لنبنِ تجربة مقارنة كاملة: نُدرّب Tokenizer بثلاث استراتيجيات مختلفة (Character، Word، BPE) على نفس النص العربي، ونقارن عدد التوكنز الناتج وقدرة كل منها على التعامل مع كلمات جديدة.

```
"""
tokenization_comparison.py
مقارنة عملية شاملة بين Character-Level, Word-Level و BPE Tokenization,
على نص عربي، مع قياس كفاءة كل استراتيجية.
"""
from __future__ import annotations

from collections import Counter
from dataclasses import dataclass
```

```python
@dataclass(frozen=True)
class TokenizationResult:
    strategy: str
    num_tokens: int
    vocab_size: int
    handles_oov: bool
    example_tokens: list[str]


def tokenize_char_level(text: str) -> list[str]:
    return list(text.replace(" ", "_"))  # _ يُمثّل المسافة بوضوح (تقليد SentencePiece)


def tokenize_word_level(text: str, vocab: set[str]) -> list[str]:
    words = text.split()
    return [w if w in vocab else "[UNK]" for w in words]


def build_word_vocab(corpus: list[str]) -> set[str]:
    vocab: set[str] = set()
    for sentence in corpus:
        vocab.update(sentence.split())
    return vocab


def run_comparison(corpus: list[str], test_sentence: str) -> list[TokenizationResult]:
    results = []

    # --- Character-Level ---
    char_tokens = tokenize_char_level(test_sentence)
    char_vocab = set()
    for s in corpus:
        char_vocab.update(tokenize_char_level(s))
    results.append(TokenizationResult(
        strategy="Character-Level",
        num_tokens=len(char_tokens),
        vocab_size=len(char_vocab),
        handles_oov=True,  # لا توجد كلمة لا يمكن تمثيلها بأحرفها
        example_tokens=char_tokens[:10],
    ))

    # --- Word-Level ---
    word_vocab = build_word_vocab(corpus)
    word_tokens = tokenize_word_level(test_sentence, word_vocab)
    results.append(TokenizationResult(
        strategy="Word-Level",
        num_tokens=len(word_tokens),
        vocab_size=len(word_vocab),
        handles_oov="[UNK]" not in word_tokens,
        example_tokens=word_tokens,
    ))

    # --- BPE (باستخدام تطبيقنا اليدوي من القسم 3.2) ---
    from bpe_from_scratch import train_bpe, apply_bpe  # يفترض وجود الملف بنفس المجلد
```

```
    merges, bpe_vocab = train_bpe(corpus, num_merges=40)
    bpe_tokens: list[str] = []
    for word in test_sentence.split():
        bpe_tokens.extend(apply_bpe(word, merges))
    results.append(TokenizationResult(
        strategy="BPE (Subword)",
        num_tokens=len(bpe_tokens),
        vocab_size=len(bpe_vocab),
        handles_oov=True,  # قادر على تركيب أي كلمة من أجزاء معروفة
        example_tokens=bpe_tokens,
    ))

    return results


if __name__ == "__main__":
    corpus = [
"القطة تحب اللعب في الحديقة",
"الكلب يحب الجري في الحديقة",
"الأطفال يحبون اللعب مع الحيوانات",
"القراءة تفتح آفاقا جديدة للعقل",
"البرمجة مهارة مهمة في هذا العصر",
    ]
# جملة اختبار تحتوي كلمات مُركّبة/مُصرّفة لم تظهر حرفيًا من قبل
    test_sentence = "الأطفال والقطط يلعبان في الحدائق"

    results = run_comparison(corpus, test_sentence)

    print(f"\n{'الاستراتيجية':<20}{'عدد التوكنز':<15}{'حجم المفردات':<15}{'يتعامل مع OOV؟':<15}")
    print("-" * 65)
    for r in results:
        oov_mark = "✅" if r.handles_oov else "❌"
        print(f"{r.strategy:<20}{r.num_tokens:<15}{r.vocab_size:<15}{oov_mark:<15}")

    print("\n=== أمثلة التوكنز الناتجة ===")
    for r in results:
        print(f"\n{r.strategy}:")
        print(f"  {r.example_tokens}")
```

ما الذي يُظهره هذا المثال عمليًا؟ ستلاحظ أن Word-Level يفشل في التعامل مع "والقطط" و"يلعبان" و"الحدائق" (أشكال مُصرّفة جديدة لم تظهر في corpus التدريب بالضبط)، منتجًا `[UNK]` لها — فقدان كامل للمعلومة. بينما BPE يُقسّمها لأجزاء مألوفة (مثل "ال" + "قطط" أو ما شابه حسب عمليات الدمج المتعلّمة)، محافظًا على معلومة جزئية مفيدة رغم عدم رؤية الكلمة الكاملة من قبل.

---

5. 💡 نصائح وحيل (Tips & Tricks)

- **حجم المفردات (Vocab Size) قرار استراتيجي مهم:** مفردات أكبر = توكنز أقل لكل جملة (تسلسل أقصر، أرخص حسابيًا) لكن Embedding Matrix أكبر (أثقل ذاكريًا). القيم الشائعة: 32K (LLaMA، Mistral)، 50K (GPT-

100K ،(GPT-4 +2)، نماذج متعددة اللغات حديثة).

- **درّب Tokenizer على بيانات تُمثّل توزيع الاستخدام الفعلي:** إن كان نموذجك سيتعامل بكثافة مع كود برمجي أو نصوص طبية، يجب أن يحتوي corpus تدريب الـ Tokenizer على نسبة مماثلة من هذه الأنواع، وإلا ستُقسَّم هذه النصوص بكفاءة سيئة (توكنز كثيرة جدًا لكل كلمة تقنية).
- **قِس "Fertility" لكل لغة:** وهي متوسط عدد التوكنز اللازمة لتمثيل كلمة واحدة. إذا كانت fertility اللغة العربية في Tokenizer معيّن أعلى بكثير من الإنجليزية (شائع جدًا في نماذج مُدرَّبة أساسًا على بيانات إنجليزية)، فهذا يعني تكلفة حسابية أعلى وسياق فعّال أقصر لنفس المحتوى العربي — مشكلة عدالة حقيقية في النماذج التجارية.
- **استخدم `NFC` Normalization دائمًا كخطوة أولى** قبل تدريب أي Tokenizer للعربية أو أي لغة بها أشكال Unicode متعددة لنفس الحرف.
- **احتفظ دائمًا بنسخة من الـ Tokenizer المُستخدم بالضبط مع النموذج** — استخدام Tokenizer مختلف ولو بتفاصيل بسيطة (ترتيب Special Tokens مثلاً) مع نموذج مُدرَّب مسبقًا يُنتج نتائج عشوائية تمامًا وكارثية.

---

6. ⚠️ أخطاء شائعة ومصائد (Common Pitfalls)

الخطأ 1: استخدام Tokenizer مختلف عن ذلك المستخدم في تدريب النموذج

```
# ❌ خطأ فادح: تحميل نموذج مُدرَّب بـ tokenizer معيّن، واستخدام tokenizer آخر معه
model = load_pretrained_model("some-llm")
tokenizer = AutoTokenizer.from_pretrained("different-model-tokenizer")  # ❌!
```

```
# ✅ صحيح: استخدم دائمًا نفس اسم/مسار الـ checkpoint لكل من النموذج والـ tokenizer
model_name = "some-llm"
model = load_pretrained_model(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)  # ✅ متطابق تمامًا
```

الخطأ 2: نسيان Normalization قبل تدريب Tokenizer على نص عربي

```
# ❌ بدون Normalization، نفس الكلمة بأشكال Unicode مختلفة = توكنز مختلفة
tokenizer = Tokenizer(BPE())
tokenizer.pre_tokenizer = Whitespace()
# لا يوجد normalizer! قد تتضاعف المفردات بلا داعٍ
```

```
# ✅ صحيح: أضف NFC Normalization دائمًا كخطوة أولى
from tokenizers.normalizers import NFC
```

```
tokenizer.normalizer = NFC()
tokenizer.pre_tokenizer = Whitespace()
```

الخطأ 3: تجاهل حدود الطول الأقصى (Max Sequence Length) عند الترميز

```
# ❌ نص طويل جدًا قد يتجاوز الحد الأقصى الذي يدعمه النموذج دون أي تحذير
ids = tokenizer.encode(very_long_document).ids
model(ids)  # قد يفشل أو يُعطي نتائج سيئة جدًا إن تجاوز max_position_embeddings
```

```
# ✅ صحيح: فعّل القص الصريح (truncation) مع تحديد الحد الأقصى بوضوح
encoding = tokenizer.encode(very_long_document)
ids = encoding.ids[:MAX_SEQ_LEN]  # أو استخدم truncation المدمجة في المكتبة
```

الخطأ 4: الخلط بين عدد "الكلمات" وعدد "التوكنز" عند تقدير التكلفة

```
# ❌ خطأ شائع: افتراض أن عدد التوكنز ≈ عدد الكلمات (قد يكون مضاعفًا بسهولة!)
num_words = len(text.split())
estimated_cost = num_words * cost_per_token  # تقدير غير دقيق!
```

```
# ✅ صحيح: احسب عدد التوكنز الفعلي دائمًا باستخدام نفس الـ tokenizer المُستهدف
actual_tokens = len(tokenizer.encode(text).ids)
estimated_cost = actual_tokens * cost_per_token  # دقيق، خاصة مهم في APIs تجارية
```

الخطأ 5: عدم التعامل بشكل صحيح مع رموز PAD أثناء حساب Loss

```
# ❌ حساب الـ Loss على رموز PAD يُضيف ضوضاء ويُحرّف النتيجة
loss = F.cross_entropy(logits.view(-1, vocab_size), labels.view(-1))
```

```
# ✅ صحيح: استخدم ignore_index لتجاهل توكنز PAD تمامًا في حساب الخسارة
loss = F.cross_entropy(
    logits.view(-1, vocab_size), labels.view(-1),
    ignore_index=tokenizer.pad_token_id,
)
```

---

7. ⚖️ جدول مقارنة: استراتيجيات Tokenization

الاستراتيجية	حجم المفردات	طول التسلسل	التعامل مع OOV	الاستخدام الشهير
Character-Level	صغير جدًا (~100-300)	طويل جدًا	ممتاز (لا توجد OOV)	نماذج متخصصة، لغات نادرة البيانات
Word-Level	ضخم جدًا (+100K)	قصير	سيء جدًا (`[UNK]` متكرر)	أنظمة NLP تقليدية قديمة
BPE	متوسط (30K-100K)	متوسط	جيد جدًا (تركيب من أجزاء)	GPT-2/3/4، LLaMA، Mistral، RoBERTa
WordPiece	متوسط	متوسط	جيد جدًا	BERT، DistilBERT، Electra
Unigram/SentencePiece	متوسط	متوسط	جيد جدًا + مرونة احتمالية	T5، ALBERT، XLNet، Gemma

---

8. ✅ قائمة التحقق من أفضل الممارسات

- [ ] طبّقت Unicode Normalization (NFC عادة) قبل تدريب أي Tokenizer جديد.
- [ ] درّبت الـ Tokenizer على بيانات تُمثّل فعليًا التوزيع المُستهدف للاستخدام (لغات، نطاقات، كود...).
- [ ] تحققت من "Fertility" (توكنز/كلمة) لكل لغة مهمة لمشروعك، وليس الإنجليزية فقط.
- [ ] أضفت كل الـ Special Tokens اللازمة (`[PAD]` , `[UNK]` , `[BOS]` , `[EOS]` ...) بوضوح وقبل بدء التدريب.
- [ ] تستخدم دائمًا نفس إصدار الـ Tokenizer المرافق تمامًا لإصدار النموذج المُدرَّب.
- [ ] تتعامل مع `ignore_index` بشكل صحيح عند حساب Loss لتجاهل رموز PAD.
- [ ] اختبرت الـ Tokenizer على كلمات/عبارات نادرة أو مُركّبة للتأكد من تعامله الجيد معها.
- [ ] تحققت من حدود الطول الأقصى (Max Sequence Length) وطبّقت Truncation بوضوح عند الحاجة.
- [ ] حفظت واختبرت عملية فك الترميز (Decode) للتأكد من إعادة بناء النص الأصلي بدقة.
- [ ] قارنت حجم المفردات مقابل طول التسلسل الناتج لإيجاد التوازن الأمثل لمشروعك تحديدًا.

---

9. 🎯 أسئلة مقابلات شخصية

س1: ما الفرق الجوهري بين معيار اختيار الدمج في BPE ومعياره في WordPiece؟

BPE يختار في كل خطوة الزوج **الأكثر تكرارًا مطلقًا (Raw Frequency)** في بيانات التدريب. WordPiece يختار الزوج الذي يُعظّم نسبة `freq(pair) / (freq(a) × freq(b))`، وهي صيغة تُشبه Pointwise Mutual Information، تقيس "الترابط الإحصائي الحقيقي" بين العنصرين بدل مجرد التكرار الخام. هذا يجعل WordPiece يُفضّل أحيانًا أزواجًا أقل تكرارًا لكنها "متماسكة لغويًا" بقوة أكبر (تظهر دائمًا معًا نسبيًا)، على أزواج شائعة فقط لأن أحد عنصريها (أو كلاهما) شائع عمومًا في كل سياق.

س2: لماذا تُعتبر Subword Tokenization حلاً وسطًا أفضل من Character-Level وWord-Level؟ اشرح بدقة.

Character-Level يحل مشكلة OOV تمامًا (كل كلمة قابلة للتمثيل بأحرفها) لكن يُنتج تسلسلات طويلة جدًا تُثقل الحوسبة (خاصة مع تعقيد Attention التربيعي) وتُحمّل النموذج عبء تعلّم تجميع المعنى من الصفر. Word-Level يُنتج تسلسلات قصيرة ذات معنى مباشر، لكنه يفشل كليًا أمام أي كلمة جديدة أو مُصرّفة لم تظهر في بيانات تدريب المفردات، منتجًا `[UNK]` وفقدان المعلومة بالكامل. Subword Tokenization (BPE، WordPiece، Unigram) يُوازن بين الاثنين: الكلمات الشائعة تبقى كوحدة واحدة مُفيدة (تسلسل قصير نسبيًا)، بينما الكلمات النادرة أو الجديدة تُقسَّم لأجزاء فرعية مألوفة يمكن تركيبها لاستخلاص معنى جزئي مفيد، دون فشل كامل أو انفجار في حجم المفردات.

س3: ما أهمية Unicode Normalization عند بناء Tokenizer للغة العربية تحديدًا؟

بعض الحروف والتراكيب في اللغة العربية (خصوصًا الهمزات) يمكن تمثيلها بأكثر من شكل Unicode مختلف بايتيًا رغم تطابقها بصريًا تمامًا (مثل الألف المهموزة المركّبة مسبقًا مقابل تسلسل حرف + علامة همزة منفصلة). بدون تطبيق Normalization (عادة NFC)، يرى الـ Tokenizer هذه الأشكال كرموز مختلفة تمامًا، مما يُضاعف حجم المفردات دون أي فائدة لغوية حقيقية، ويُشتت الإحصائيات الخاصة بتكرار كل رمز، مما يُضعف جودة عمليات الدمج المُتعلَّمة ويُقلّل كفاءة التمثيل النهائي للغة العربية.

س4: كيف يُعالج Unigram Tokenization الكلمة بشكل مختلف جوهريًا عن BPE/WordPiece؟

BPE وWordPiece يبنيان المفردات **تصاعديًا**: يبدآن بالأحرف الفردية ويدمجانها تدريجيًا بناءً على معيار معيّن حتى الوصول لحجم المفردات المستهدف، وينتجان **تقسيمًا حتميًا واحدًا** لكل كلمة عند التطبيق. Unigram يعمل **تنازليًا**: يبدأ بمفردات كبيرة جدًا، ويحذف تدريجيًا الرموز الأقل مساهمة في احتمالية بيانات التدريب الكلية (وفق نموذج احتمالي من نوع Unigram)، وينتج **نموذجًا احتماليًا كاملاً** يسمح بعدة تقسيمات ممكنة لنفس الكلمة، مع إمكانية أخذ عينة عشوائية من هذه التقسيمات أثناء التدريب (Subword Regularization) كشكل من أشكال تعزيز متانة النموذج.

س5: لماذا يجب أن يكون Tokenizer مرتبطًا دائمًا بنسخة محددة بدقة من النموذج، ولا يمكن استبداله بحرية؟

> لأن النموذج تعلّم أوزانه (خصوصًا Embedding Matrix، كما سنرى في الفصل التالي) بناءً على **تخصيص رقمي محدد تمامًا** لكل توكن في مفردات ذلك الـ Tokenizer بالضبط: التوكن رقم 523 مثلاً يحمل معنى معيّنًا تعلّمه النموذج فقط لأن ذلك الـ Tokenizer المحدد يُخصّص هذا الرقم لكلمة أو جزء كلمة معيّن. استخدام Tokenizer مختلف (حتى لو كان بنفس الخوارزمية لكن مُدرَّبًا على بيانات مختلفة، أو بترتيب Special Tokens مختلف) يعني أن نفس الرقم 523 قد يُمثّل الآن كلمة مختلفة تمامًا، فيتلقى النموذج "إشارات" عشوائية لا معنى لها بالنسبة لأوزانه المُدرَّبة، مما يُنتج مخرجات عشوائية وكارثية تمامًا.

---

10. 🧪 تمرين عملي (Mini-Project)

المهمة: "محلل كفاءة Tokenizer" — Fertility Analyzer

ابنِ أداة تُقارن "عدالة" Tokenizer معيّن بين لغتين (مثلاً العربية والإنجليزية) عبر قياس Fertility (متوسط توكنز/كلمة).

```
"""
fertility_analyzer.py – هيكل بداية للتمرين
.أداة لقياس وتحليل "عدالة" أي Tokenizer عبر لغات مختلفة
"""
from __future__ import annotations

from dataclasses import dataclass

from tokenizers import Tokenizer


@dataclass(frozen=True)
class FertilityReport:
    language: str
    num_words: int
    num_tokens: int
    fertility: float  # tokens / words، (أكثر كفاءة) كلما قلّ كان أفضل


def compute_fertility(
    tokenizer: Tokenizer, text: str, language_name: str
) -> FertilityReport:
    """
    TODO:
(تقريب بسيط كافٍ لهذا التمرين) ()split. عبر text في احسب عدد الكلمات .1
tokenizer.encode(text) بـ رمّز النص الكامل .2، واحسب عدد التوكنز الناتجة
fertility = num_tokens / num_words احسب .3
```

```
    4. أعد FertilityReport بالقيم المحسوبة
    """
    raise NotImplementedError


def compare_languages_fairness(
    tokenizer: Tokenizer,
    texts_by_language: dict[str, str],
) -> list[FertilityReport]:
    """
    TODO: طبّق compute_fertility على كل لغة في القاموس، وأعد قائمة بكل التقارير
    مُرتّبة تنازليًا حسب fertility (الأسوأ أولًا) لتسليط الضوء على اللغات
    "المظلومة" من هذا الـ Tokenizer تحديدًا.
    """
    raise NotImplementedError


def print_fairness_report(reports: list[FertilityReport]) -> None:
    """
    TODO: اطبع جدولاً منسّقًا يُقارن fertility بين اللغات، مع تحذير واضح
    (⚠) لأي لغة تتجاوز fertility فيها ضعف أقل لغة في القائمة
    (مؤشر على عدم توازن واضح في تمثيل الـ Tokenizer لتلك اللغة).
    """
    raise NotImplementedError


if __name__ == "__main__":
    # نصوص متكافئة معنويًا تقريبًا بلغات مختلفة لمقارنة عادلة
    texts_by_language = {
        "العربية": "الذكاء الاصطناعي يغير العالم بسرعة كبيرة جدا في هذه الايام",
        "English": "Artificial intelligence is changing the world very quickly these days",
        "Français": "L'intelligence artificielle change le monde très rapidement ces jours-ci",
    }

    # TODO: حمّل tokenizer حقيقي (مثلاً GPT-2 عبر Hugging Face أو أي نموذج مفتوح)
    # from tokenizers import Tokenizer
    # tokenizer = Tokenizer.from_pretrained("gpt2")
    #
    # reports = compare_languages_fairness(tokenizer, texts_by_language)
    # print_fairness_report(reports)

# سؤال للتفكير بعد إتمام التمرين:
# هل النتائج التي حصلت عليها مفاجئة؟ أي اللغات كانت الأقل "عدالة"؟
# كيف يمكن لمطوّري نموذج جديد معالجة هذه المشكلة عند بناء Tokenizer مخصص؟
    print("أكمل الدوال أعلاه وفعّل سطور التحميل، ثم شغّل التحليل!")
```

11. 🔗 جسر للفصل التالي

الآن نمتلك الأداة الأساسية لتحويل أي نص إلى تسلسل من **أرقام صحيحة** (Token IDs). لكن توقف وفكّر: هل هذه الأرقام بحد ذاتها تحمل أي معنى رياضي؟ هل التوكن رقم 523 "أقرب" بشكل ما للتوكن رقم 524 أكثر من قربه

للتوكن رقم 9000؟ الجواب **لا على الإطلاق** — هذه مجرد فهارس عشوائية (Indices) لا تحمل أي بنية دلالية.

السؤال الطبيعي التالي: كيف نُحوّل هذه الفهارس الرقمية الجامدة إلى **تمثيلات متجهية (Vectors) ذات معنى**، بحيث تكون الكلمات متشابهة المعنى "قريبة" من بعضها في فضاء رياضي ما؟ هذا بالضبط موضوع الفصل السابع القادم: **Word2Vec** — حيث سنكتشف كيف تعلّمت الشبكات العصبية، لأول مرة في تاريخ NLP الحديث، أن "ملك - رجل + امرأة ≈ ملكة" من خلال مجرد النظر إلى ملايين الجمل، دون أي قاموس أو قواعد لغوية مُبرمَجة يدويًا.

---

12. 📚 مراجع للتعمّق

المرجع	النوع	الرابط/المصدر
Sennrich, Haddow & Birch (2016), *"Neural Machine Translation of Rare Words with Subword Units"*	ورقة تأسيسية لـ BPE (NLP)	arXiv:1508.07909
Schuster & Nakajima (2012), *"Japanese and Korean Voice Search"*	ورقة تأسيسية (WordPiece)	ICASSP 2012
Kudo (2018), *"Subword Regularization"*	ورقة بحثية (Unigram LM)	arXiv:1804.10959
Kudo & Richardson (2018), *"SentencePiece"*	ورقة بحثية + أداة	arXiv:1808.06226
Gage (1994), *"A New Algorithm for Data Compression"*	ورقة تأسيسية أصلية لـ BPE	C Users Journal
Hugging Face, *"tokenizers" library documentation*	توثيق تقني وأمثلة	huggingface.co/docs/tokenizers
Rust et al. (2021), *"How Good is Your Tokenizer?"*	ورقة بحثية (Fertility عبر اللغات)	arXiv:2012.15613

==========

الفصل السابع: Word2Vec — كيف تتعلّم الآلة معنى الكلمات؟



(CBOW, Skip-gram, Negative Sampling, GloVe)

1. 🟢 لماذا يهمّك هذا؟

في الفصل السادس تعلّمنا كيف نُقطّع النص إلى وحدات (Tokens) ونُحوّلها إلى أرقام صحيحة (IDs). لكن توقّف قليلًا وتأمّل: ماذا لو كانت الكلمة "ملك" تحمل الرقم `4821` والكلمة "ملكة" تحمل الرقم `193`؟ هل هناك أي علاقة رياضية بين الرقمين `4821` و `193`؟ الجواب القاطع: **لا، إطلاقًا**. الترقيم الذي ننتجه من الـ Tokenizer هو ترقيم **اعتباطي بحت** (Arbitrary) — تمامًا كترقيم الأسماء في دفتر هاتف. لا شيء في الرقم `4821` يُخبر النموذج أن "ملك" قريبة المعنى من "ملكة"، أو أن "باريس" ترتبط بـ"فرنسا" بنفس الطريقة التي ترتبط بها "القاهرة" بـ"مصر".

فإذا غذّينا شبكة عصبية بهذه الأرقام كما هي، أو حتى كـ One-Hot Vectors (متجهات طولها = حجم المفردات، كلها أصفار إلا واحد في موضع الكلمة)، فإننا نرتكب خطيئة هندسية فادحة: **كل كلمتين متعامدتان تمامًا** (Orthogonal) في الفضاء الرياضي، بغضّ النظر عن قربهما الدلالي. المسافة بين "قطة" و"كلب" تساوي تمامًا المسافة بين "قطة" و"ديمقراطية". هل هذا معقول؟

السؤال الذي يقودنا طوال هذا الفصل هو: **هل نستطيع أن نجعل الآلة "تتعلّم" إحداثيات لكل كلمة في فضاء رياضي متعدد الأبعاد، بحيث تكون الكلمات متقاربة المعنى متقاربة في الإحداثيات أيضًا؟** وإذا استطعنا ذلك، فهل يمكن أن تنبثق من هذا الفضاء خصائص جبرية مذهلة، مثل أن:

$$\vec{v}(\text{ملك}) - \vec{v}(\text{رجل}) + \vec{v}(\text{امرأة}) \approx \vec{v}(\text{ملكة})$$

هذا بالضبط ما حقّقه توماس ميكولوف وفريقه في Google عام 2013 بنموذج **Word2Vec**، وهو أحد أهم الاختراقات في تاريخ معالجة اللغة الطبيعية الحديثة. هذا الفصل هو حجر الأساس الذي سيجعلك تفهم لاحقًا كيف تعمل طبقة `nn.Embedding` في كل نموذج لغوي ضخم تستخدمه اليوم — من BERT إلى GPT-4 إلى LLaMA. فبدون فهم Word2Vec، ستتعامل مع طبقة التضمين كصندوق أسود سحري؛ وبعد هذا الفصل، ستفهم بالضبط *لماذا* ينجح هذا الصندوق.

---

3. 🟡 الشرح المتدرّج: من الحدس إلى الرياضيات إلى الكود

3.1 الفرضية التوزيعية: "تُعرف الكلمة من رفقتها"

اسأل نفسك سؤالًا بسيطًا: كيف تعرف أنت، كإنسان، معنى كلمة لم تسمع بها من قبل؟ لنفترض أنك قرأت الجملة التالية:

> "تناول الفيل كمية كبيرة من الـ**زبزوب** في الحديقة."

أنت لا تعرف كلمة "زبزوب" (لأنني اخترعتها الآن)، لكن حدسك اللغوي يخبرك فورًا أنها على الأرجح **نوع من النبات أو الطعام**. كيف عرفت ذلك؟ لم تَرَ تعريفًا في القاموس — لقد استنتجت المعنى *من السياق المحيط بالكلمة*. هذه هي بالضبط الفكرة التي صاغها اللغوي جون فيرث عام 1957 في مقولته الشهيرة:

> *"You shall know a word by the company it keeps"* — ستعرف الكلمة من خلال الصحبة التي تُرافقها.

هذه هي **الفرضية التوزيعية** (Distributional Hypothesis): الكلمات التي تظهر في سياقات متشابهة تميل إلى أن تحمل معاني متشابهة. "قطة" و"كلب" يظهران غالبًا بجوار كلمات مثل "أليف"، "يأكل"، "حديقة"، "بيت" — بينما

"ديمقراطية" تظهر بجوار "انتخابات"، "برلمان"، "حكومة". التشابه في السياق ← تشابه في المعنى.

Word2Vec يُترجم هذه الفكرة اللغوية الفلسفية إلى **مسألة تعلّم آلي محض**: درّب شبكة عصبية بسيطة جدًا (طبقة واحدة مخفية، بلا دوال تفعيل غير خطية حتى!) على مهمة وهمية — **التنبؤ بالسياق من الكلمة، أو التنبؤ بالكلمة من السياق**. لكن الهدف الحقيقي ليس التنبؤ بحد ذاته، بل **الأوزان الداخلية** التي تتعلّمها الشبكة أثناء محاولتها حل هذه المهمة. هذه الأوزان ستكون هي التضمينات (Embeddings) التي نريدها.

> 💡 **نقطة محورية يجب أن ترسخ في ذهنك:** Word2Vec ليس هدفه النهائي أن يكون "متنبئًا" جيدًا. مهمة التنبؤ هي مجرد **ذريعة تدريبية** (Pretext Task) أو ما يُسمى اليوم Self-Supervised Learning. المنتج الحقيقي الذي نريده هو الأوزان الوسيطة، لا المخرجات.

3.2 نافذة السياق (Context Window)

قبل أن ندخل في المعماريتين، نحتاج تعريفًا عمليًا للسياق. نُحدّد حجم نافذة m (مثلًا $m = 2$)، وبالنسبة لكل كلمة مركزية w_t في الجملة، نعتبر الكلمات ضمن مسافة m قبلها وبعدها هي "سياقها".

مثال: الجملة "الذكاء الاصطناعي يغيّر العالم بسرعة كبيرة" مع $m = 2$ ومركز الكلمة = "يغيّر":

```
الذكاء   الاصطناعي   [يغيّر]   العالم   بسرعة
سياق↑    سياق↑      مركز      سياق↓    سياق↓
```

من هذه الجملة الواحدة فقط، ننتج أزواج تدريب (مركز، سياق) متعددة بتحريك النافذة على كل كلمة بدورها تكون مركزًا.

3.3 المعمارية الأولى: CBOW (Continuous Bag of Words)

السؤال الذي يطرحه CBOW: "لدي مجموعة كلمات سياق حولي، فما هي الكلمة المركزية الأرجح التي ينبغي أن تكون هنا؟"

بمعنى آخر: المدخل هو **متوسط** تضمينات كلمات السياق، والمخرج هو توزيع احتمالي على كل كلمات المفردات، نريد أن تكون الكلمة المركزية الصحيحة هي الأعلى احتمالًا.

$$\hat{v}_{context} = \frac{1}{2m} \sum_{-m \le j \le m,\ j \neq 0} v_{w_{t+j}}$$

البنية الكاملة:

1. **طبقة الإدخال**: مصفوفة $W_{in} \in \mathbb{R}^{V \times d}$ حيث V = حجم المفردات، d = أبعاد التضمين (مثلًا 100 أو 300). كل صف هو تضمين كلمة واحدة.
2. **الإسقاط (Projection)**: نأخذ متوسط متجهات تضمين كلمات السياق.
3. **طبقة الإخراج**: مصفوفة $W_{out} \in \mathbb{R}^{d \times V}$، نضرب فيها متجه السياق لنحصل على "درجات" (logits) لكل كلمة في المفردات: $z = \hat{v}_{context}^{T} W_{out}$
4. **Softmax**: $P(w_t \mid context) = \frac{\exp(z_{w_t})}{\sum_{k=1}^{V} \exp(z_k)}$
5. **دالة الخسارة**: Cross-Entropy بين التوزيع المتوقع والكلمة الصحيحة الفعلية.

> 🧠 **صندوق تعمّق (Deep Dive): لماذا لا توجد دالة تفعيل غير خطية بين الطبقتين؟** هذا قرار معماري متعمّد من ميكولوف. الهدف الأساسي من Word2Vec هو **السرعة** — يجب أن يتدرّب على مليارات الكلمات في وقت معقول على عتاد 2013 المتواضع. إزالة اللاخطية (tanh/sigmoid في الطبقة المخفية، كما كان معتادًا في نماذج اللغة العصبية السابقة مثل Bengio 2003) تجعل التدريب أسرع بعشرات المرات، لأن معظم الكلفة الحسابية في النماذج القديمة كانت في الطبقة المخفية غير الخطية الكبيرة. Word2Vec ضحّى بالتعبيرية (Expressiveness) من أجل **قابلية التوسّع (Scalability)** — وهذه مقايضة ستراها مرارًا وتكرارًا في هندسة LLM: أحيانًا نموذج أبسط + بيانات أكثر > نموذج معقّد + بيانات أقل.

3.4 المعمارية الثانية: Skip-gram

السؤال الذي يطرحه Skip-gram: "أمامي كلمة واحدة، فما هي الكلمات التي يُرجَّح وجودها حولها؟"

هذه هي المعمارية المعكوسة تمامًا لـ CBOW: المدخل كلمة مركزية واحدة، والمخرج محاولة التنبؤ بكل كلمة من كلمات السياق (بشكل مستقل).

$$\max \prod_{-m \le j \le m,\ j \neq 0} P(w_{t+j} \mid w_t)$$

أو بصيغة اللوغاريتم السلبي (دالة الخسارة التي نُصغّرها فعليًا):

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{-m \le j \le m,\ j \neq 0} \log P(w_{t+j} \mid w_t; \theta)$$

وكل احتمال شرطي يُحسب أيضًا بـ Softmax على كامل المفردات:

$$P(w_O \mid w_I) = \frac{\exp(v'^{\top}_{w_O} v_{w_I})}{\sum_{k=1}^{V} \exp(v'^{\top}_{k} v_{w_I})}$$

حيث v_{w_I} هو تضمين الكلمة المدخلة (من W_{in})، و v'_{w_O} هو تضمين كلمة الإخراج (من W_{out}). **لاحظ أن لكل كلمة تضمينين منفصلين:** واحد عندما تكون "مركزًا" (Input/Center Vector) وآخر عندما تكون "سياقًا" (Output/Context Vector). في النهاية، التضمين النهائي المُستخدَم عمليًا هو عادةً W_{in} فقط (أو أحيانًا متوسط الاثنين).

⚖️ **لماذا Skip-gram أفضل عادة من CBOW عمليًا، رغم أن CBOW "أسرع"؟** CBOW يُنتج عيّنة تدريب واحدة من كل نافذة سياق (يُنعّم/يُمتوسط كل السياق في متجه واحد)، بينما Skip-gram يُنتج $2m$ عيّنة تدريب من نفس النافذة (زوج منفصل لكل كلمة سياق). هذا يعني أن Skip-gram يرى بيانات تدريب أكثر فعليًا من نفس النص، وهو ما يجعله أداءً أفضل خاصة مع **الكلمات النادرة** (Rare Words) — لأن كل ظهور للكلمة النادرة كمركز يُولّد عدّة أزواج تدريب مباشرة لها، بينما في CBOW تُمتوسط مساهمتها مع كلمات أخرى فتضعف إشارتها. الثمن: Skip-gram أبطأ في التدريب لأنه يعالج عيّنات أكثر.

3.5 المشكلة الحسابية الكبرى: Softmax على مليون كلمة!

توقّف هنا وتأمّل المقام (Denominator) في معادلة Softmax أعلاه: $\sum_{k=1}^{V} \exp(\cdot)$. إذا كان حجم المفردات $V = 1,000,000$ كلمة (شائع جدًا في مجموعات بيانات حقيقية)، فهذا يعني أنه **لكل خطوة تدريب واحدة، لكل كلمة واحدة في الجملة**، يجب أن نحسب مليون عملية ضرب نقطي (Dot Product) ثم مليون أُس (exponential) فقط لحساب احتمال **كلمة واحدة صحيحة**! وهذا يتكرر لكل كلمة في مجموعة بيانات تحتوي مليارات الكلمات.

هذه الكلفة الحسابية غير قابلة للتطبيق عمليًا ($O(V)$ لكل عيّنة). هنا يأتي الاختراع الثاني الأهمّ في ورقة ميكولوف الثانية (Mikolov et al., 2013b) — **Negative Sampling**.

3.6 Negative Sampling: الحل العبقري البسيط

الفكرة الجوهرية: بدلًا من طرح سؤال معقّد جدًا ("من بين مليون كلمة محتملة، ما احتمال كل واحدة؟")، لنُحوّل المهمة بالكامل إلى **سؤال تصنيف ثنائي بسيط جدًا:**

"هل هذا الزوج (كلمة مركزية، كلمة سياق) هو زوج **حقيقي** ظهر فعلًا معًا في النص، أم هو زوج **مُزيَّف** اخترعناه عشوائيًا؟"

لكل عيّنة تدريب حقيقية (موجبة) مثل (يغيّر , الاصطناعي)، نولّد k عيّنات سلبية مزيّفة عشوائيًا، مثل (يغيّر , موزة)، (يغيّر , اقتصاد)... إلخ، بافتراض أن هذه الكلمات لم تظهر فعليًا كسياق لـ"يغيّر" في هذا الموضع. (k تتراوح عادة بين 5-20 للمجموعات الصغيرة، و2-5 للمجموعات الضخمة).

الآن بدلًا من Softmax على V كلمة، نستخدم **Sigmoid بسيط** على $1+k$ أزواج فقط! التعقيد الحسابي ينخفض من $O(V)$ إلى $O(k)$ حيث $k \ll V$ — تخفيض هائل!

دالة الخسارة الجديدة (لعيّنة واحدة في Skip-gram):

$$J_{neg}(\theta) = -\log\sigma(v'^{\top}_{w_O} v_{w_I}) - \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)}\left[\log\sigma(-v'^{\top}_{w_i} v_{w_I})\right]$$

حيث σ هي دالة السيجمويد، والحدّ الأول يدفع الزوج الحقيقي ليُعطي احتمالًا قريبًا من 1، والحدّ الثاني يدفع كل زوج سلبي ليُعطي احتمالًا قريبًا من 0 (لاحظ الإشارة السالبة داخل $\sigma(-\cdot)$ التي تحقق ذلك).

من أين تُسحب الكلمات السلبية؟ هنا تفصيلة هندسية بديعة من الورقة الأصلية: لا نسحبها بالتردد الخام (Unigram Frequency) المباشر $P(w)$، بل بتردد **مُعدَّل بالأُس 3/4**:

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w'} f(w')^{3/4}}$$

لماذا بالضبط الأُس 3/4؟ لأنه يُحدث توازنًا ذكيًا: يُقلّل قليلًا من هيمنة الكلمات شديدة التكرار (مثل أدوات الربط "من"، "في"، "على" في العربية، أو "the/a" في الإنجليزية) بحيث لا تُسحَب كعيّنة سلبية بكثرة مُبالغ فيها، بينما يرفع قليلًا من فرصة ظهور الكلمات النادرة كعيّنات سلبية، مقارنة بالسحب العشوائي المباشر بالتردد الخام. جرّب حساب: كلمة بتردد 0.9 (شائعة جدًا) تُصبح $0.9^{0.75} \approx 0.92$ (تغيّر طفيف)، بينما كلمة بتردد 0.01 (نادرة) تُصبح $0.01^{0.75} \approx 0.032$ (قفزة نسبية كبيرة في فرصة ظهورها!). هذه التفصيلة الصغيرة ثبت تجريبيًا أنها تُحسّن جودة التضمينات بشكل ملموس.

3.7 Hierarchical Softmax (بديل سريع ذكرًا لا تطبيقًا)

طريقة أخرى مذكورة في الورقة الأصلية لتفادي Softmax الكامل: نبني شجرة ثنائية هوفمان (Huffman Tree) حيث كل كلمة في المفردات هي ورقة (Leaf)، والكلمات الأكثر تكرارًا تكون أقرب للجذر (مسار أقصر). احتمال الكلمة يُحسب كحاصل ضرب احتمالات القرارات الثنائية (يسار/يمين) على طول المسار من الجذر إلى الورقة، فيصبح التعقيد $O(\log V)$ بدلًا من $O(V)$. عمليًا، **Negative Sampling تفوّقت عليها في الانتشار** بسبب بساطة تطبيقها وسرعتها الفعلية الأعلى، لذا سنركّز الكود عليها.

3.8 الكود: بناء Skip-gram with Negative Sampling من الصفر بـ PyTorch

لنبنِ الآن نموذجًا تدريبيًا كاملًا، بأحدث ممارسات Python 3.13 وPyTorch.

```
"""
skip_gram_negative_sampling.py
تطبيق كامل لـ Skip-gram with Negative Sampling من الصفر.
Python 3.13+ | PyTorch 2.x
"""
from __future__ import annotations

import re
from collections import Counter
from dataclasses import dataclass, field

import numpy as np
import torch
from torch import nn, Tensor
from torch.utils.data import Dataset, DataLoader

SEED = 42
torch.manual_seed(SEED)
rng = np.random.default_rng(SEED)


# ---------------------------------------------------------------------
# 1) بناء المفردات وتوزيع السحب العشوائي للعيّنات السلبية (Noise Distribution)
# ---------------------------------------------------------------------
@dataclass
class Vocabulary:
    word2idx: dict[str, int] = field(default_factory=dict)
    idx2word: dict[int, str] = field(default_factory=dict)
    freqs: Counter[str] = field(default_factory=Counter)

    @classmethod
    def build(cls, tokenized_sentences: list[list[str]], min_count: int = 2) -> "Vocabulary":
        vocab = cls()
        counter: Counter[str] = Counter()
        for sent in tokenized_sentences:
            counter.update(sent)
# نحتفظ فقط بالكلمات التي تكررت min_count مرة على الأقل
        kept_words = [w for w, c in counter.items() if c >= min_count]
        for idx, word in enumerate(kept_words):
            vocab.word2idx[word] = idx
            vocab.idx2word[idx] = word
        vocab.freqs = Counter({w: counter[w] for w in kept_words})
        return vocab

    def __len__(self) -> int:
        return len(self.word2idx)
```

```
    def negative_sampling_distribution(self, power: float = 0.75) -> np.ndarray:
"""يبني توزيع الاحتمال P_n(w) ∝ freq(w)^0.75 المُستخدم لسحب العيّنات السلبية."""
        freqs_sorted = np.array(
            [self.freqs[self.idx2word[i]] for i in range(len(self))], dtype=np.float64
        )
        adjusted = freqs_sorted ** power
        return adjusted / adjusted.sum()


def simple_tokenize(text: str) -> list[str]:
"""محاكاة مبسّطة لمحلّل نصوص (في الإنتاج استخدم tokenizer حقيقي من الفصل السادس)."""
    text = text.lower()
    return re.findall(r"[\w']+", text)


# ---------------------------------------------------------------------
# 2) توليد أزواج (مركز، سياق) من النصوص
# ---------------------------------------------------------------------
def generate_skipgram_pairs(
    tokenized_sentences: list[list[str]],
    vocab: Vocabulary,
    window_size: int = 2,
) -> list[tuple[int, int]]:
    pairs: list[tuple[int, int]] = []
    for sentence in tokenized_sentences:
        indices = [vocab.word2idx[w] for w in sentence if w in vocab.word2idx]
        n = len(indices)
        for center_pos in range(n):
# حجم نافذة ديناميكي (dynamic window) -- حيلة تحسين من الورقة الأصلية:
# النوافذ الأصغر تُعطى وزنًا أكبر ضمنيًا بسحب حجم عشوائي <= window_size
            dynamic_window = rng.integers(1, window_size + 1)
            start = max(0, center_pos - dynamic_window)
            end = min(n, center_pos + dynamic_window + 1)
            for context_pos in range(start, end):
                if context_pos != center_pos:
                    pairs.append((indices[center_pos], indices[context_pos]))
    return pairs


# ---------------------------------------------------------------------
# 3) Dataset مدمج مع Negative Sampling بـ PyTorch
# ---------------------------------------------------------------------
class SkipGramNegSamplingDataset(Dataset):
    def __init__(
        self,
        pairs: list[tuple[int, int]],
        vocab_size: int,
        noise_dist: np.ndarray,
        num_negatives: int = 5,
    ) -> None:
        self.pairs = pairs
        self.vocab_size = vocab_size
        self.noise_dist = noise_dist
        self.num_negatives = num_negatives
```

```
    def __len__(self) -> int:
        return len(self.pairs)

    def __getitem__(self, idx: int) -> tuple[int, int, np.ndarray]:
        center, context = self.pairs[idx]
        negatives = rng.choice(
            self.vocab_size, size=self.num_negatives, p=self.noise_dist, replace=True
        )
        return center, context, negatives


def collate_fn(
    batch: list[tuple[int, int, np.ndarray]]
) -> tuple[Tensor, Tensor, Tensor]:
    centers, contexts, negatives = zip(*batch)
    return (
        torch.tensor(centers, dtype=torch.long),
        torch.tensor(contexts, dtype=torch.long),
        torch.tensor(np.stack(negatives), dtype=torch.long),
    )


# ---------------------------------------------------------------------
# 4) النموذج: مصفوفتا تضمين منفصلتان (in/out) بحسب الورقة الأصلية
# ---------------------------------------------------------------------
class SkipGramNegSampling(nn.Module):
    def __init__(self, vocab_size: int, embedding_dim: int = 100) -> None:
        super().__init__()
        self.center_embeddings = nn.Embedding(vocab_size, embedding_dim)
        self.context_embeddings = nn.Embedding(vocab_size, embedding_dim)
        self._init_weights(embedding_dim)

    def _init_weights(self, embedding_dim: int) -> None:
# تهيئة بتوزيع منتظم ضيّق – ممارسة شائعة في Word2Vec الأصلي
        init_range = 0.5 / embedding_dim
        nn.init.uniform_(self.center_embeddings.weight, -init_range, init_range)
        nn.init.zeros_(self.context_embeddings.weight)

    def forward(self, center: Tensor, context: Tensor, negatives: Tensor) -> Tensor:
        """
        center:    (batch,)
        context:   (batch,)
        negatives: (batch, k)
يُعيد: الخسارة القياسية (scalar)
        """
        v_center = self.center_embeddings(center)          # (batch, d)
        v_context = self.context_embeddings(context)       # (batch, d)
        v_negatives = self.context_embeddings(negatives)   # (batch, k, d)

# الحد الموجب: log σ(v'_context · v_center)
        pos_score = torch.sum(v_center * v_context, dim=1)          # (batch,)
        pos_loss = torch.nn.functional.logsigmoid(pos_score)        # (batch,)

# الحد السلبي: Σ log σ(-v'_neg · v_center)
        neg_score = torch.bmm(v_negatives, v_center.unsqueeze(2)).squeeze(2)  # (batch, k)
```

```
        neg_loss = torch.nn.functional.logsigmoid(-neg_score).sum(dim=1)     # (batch,)

        total_loss = -(pos_loss + neg_loss)   # (= نُعظّم الاحتمال) نُصغّر السالب
        return total_loss.mean()

    def get_word_vectors(self) -> Tensor:
"""التضمين النهائي المُستخدم عمليًا = متجهات المركز فقط (الممارسة الأشيع)."""
        return self.center_embeddings.weight.detach()


# ---------------------------------------------------------------------
# 5) حلقة التدريب الكاملة
# ---------------------------------------------------------------------
def train_skipgram(
    tokenized_sentences: list[list[str]],
    embedding_dim: int = 100,
    window_size: int = 3,
    num_negatives: int = 5,
    min_count: int = 1,
    epochs: int = 50,
    batch_size: int = 256,
    lr: float = 0.01,
    device: str | None = None,
) -> tuple[SkipGramNegSampling, Vocabulary]:
    device = device or ("cuda" if torch.cuda.is_available() else "cpu")

    vocab = Vocabulary.build(tokenized_sentences, min_count=min_count)
    pairs = generate_skipgram_pairs(tokenized_sentences, vocab, window_size)
    noise_dist = vocab.negative_sampling_distribution()

    dataset = SkipGramNegSamplingDataset(pairs, len(vocab), noise_dist, num_negatives)
    loader = DataLoader(
        dataset, batch_size=batch_size, shuffle=True, collate_fn=collate_fn
    )

    model = SkipGramNegSampling(len(vocab), embedding_dim).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for epoch in range(1, epochs + 1):
        total_loss = 0.0
        for center, context, negatives in loader:
            center = center.to(device)
            context = context.to(device)
            negatives = negatives.to(device)

            optimizer.zero_grad()
            loss = model(center, context, negatives)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * center.size(0)

        avg_loss = total_loss / len(dataset)
        if epoch % 10 == 0 or epoch == 1:
            print(f"Epoch {epoch:3d}/{epochs} | Loss: {avg_loss:.4f}")
```

```
    return model, vocab
```

لاحظ النقطة الهندسية الدقيقة: استخدمنا `torch.nn.functional.logsigmoid` بدلًا من `torch.log(torch.sigmoid(x))`. لماذا؟ لأن `logsigmoid` **أكثر استقرارًا عدديًا** (Numerically Stable) — إذا كانت x سالبة جدًا، فإن $\sigma(x)$ تقترب من الصفر، و $\log(0)$ ينتج `-inf` أو `NaN`؛ بينما `logsigmoid` يُحسب بصيغة رياضية مكافئة تتجنّب هذا الانفجار العددي تمامًا (`-softplus(-x)`). **هذه حيلة ستراها تتكرر في كل مكان في PyTorch** — ابحث دائمًا عن النسخة "المدمجة رقميًا" (Fused/Stable) لأي دالة قبل أن تبنيها يدويًا من مكوناتها.

GloVe 3.9: الطريق الآخر — العدّ بدلًا من التنبؤ

بينما كان Word2Vec يحلّ المشكلة بطريقة **تنبؤية محلية** (ينظر إلى نافذة صغيرة في كل مرة)، جاء فريق ستانفورد (Pennington, Socher, Manning, 2014) بسؤال مختلف: **لماذا لا نستفيد من الإحصائيات العامة لكامل المجموعة (Corpus) دفعة واحدة؟**

الخطوة الأولى في GloVe: ابنِ **مصفوفة التعايش المشترك** (Co-occurrence Matrix) X حيث X_{ij} = عدد المرات التي ظهرت فيها الكلمة j ضمن نافذة سياق الكلمة i عبر **كامل** مجموعة النصوص (وليس batch واحد فقط).

الملاحظة العبقرية لفريق GloVe: ليست القيم الخام X_{ij} هي الأهم، بل **نِسَب الاحتمالات** بين كلمتين بالنسبة لكلمة ثالثة (probe word) تحمل المعنى الحقيقي. مثالهم الشهير في الورقة الأصلية:

النسبة	(صلب) solid= k	(غاز) gas= k	(ماء) water= k	(موضة) fashion= k
$P(k \mid \text{ice})$	كبيرة	صغيرة	كبيرة	صغيرة
$P(k \mid \text{steam})$	صغيرة	كبيرة	كبيرة	صغيرة
$\frac{P(k \mid ice)}{P(k \mid steam)}$	**كبيرة**	**صغيرة**	≈ 1	≈ 1

عندما تكون k مرتبطة بـ"ice" (جليد) فقط مثل "solid"، تكون النسبة كبيرة جدًا. عندما ترتبط بـ"steam" (بخار) فقط مثل "gas"، تكون النسبة صغيرة جدًا. وعندما ترتبط بكليهما ("water") أو بلا منهما ("fashion")، تقترب النسبة من 1. **هذه النسبة بالذات** هي الإشارة الأنقى للمعنى — أنقى من الاحتمالات الخام نفسها.

بناءً على هذا الاستنتاج، يشتق GloVe دالة خسارة من نوع **المربعات الصغرى الموزونة** (Weighted Least Squares):

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(v_i^\top \tilde{v}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

حيث $f(X_{ij})$ هي دالة موازنة (Weighting Function) تُخفّف من تأثير الأزواج شديدة التكرار (مثل أدوات الربط) حتى لا تُهيمن على التدريب، وتُهمل الأزواج التي لم تتعايش أبدًا ($X_{ij} = 0$، حيث $\log 0$ غير مُعرَّف):

$$f(x) = \begin{cases} (x \,/\, x_{max})^{\alpha} & \text{if } x < x_{max} \\ 1 & \text{otherwise} \end{cases}$$

عادة $x_{max} = 100$ و $\alpha = 0.75$ (لاحظ: نفس الأُس 3/4 يظهر مجددًا! ليس صدفة — إنه رقم تجريبي جيد لتخفيف هيمنة التكرارات العالية في عدة سياقات).

```
"""
glove_cooccurrence.py
بناء مصفوفة التعايش المشترك كخطوة تمهيدية لـ GloVe (الخطوة الأغلى حسابيًا).
"""
from __future__ import annotations

from collections import defaultdict
import numpy as np


def build_cooccurrence_matrix(
    tokenized_sentences: list[list[str]],
    vocab: "Vocabulary",
    window_size: int = 5,
) -> dict[tuple[int, int], float]:
    """
يبني مصفوفة تعايش مرجّحة بالمسافة (أقرب كلمة للمركز = وزن أعلى).
نستخدم dict متفرّق (sparse) لأن أغلب الأزواج X_ij = 0.
    """
    cooccurrence: dict[tuple[int, int], float] = defaultdict(float)

    for sentence in tokenized_sentences:
        indices = [vocab.word2idx[w] for w in sentence if w in vocab.word2idx]
        n = len(indices)
        for i, center in enumerate(indices):
            start, end = max(0, i - window_size), min(n, i + window_size + 1)
            for j in range(start, end):
                if j == i:
                    continue
                distance = abs(i - j)
                weight = 1.0 / distance  # كلما بعُدت الكلمة، قلّ وزن تعايشها
                cooccurrence[(center, indices[j])] += weight

    return cooccurrence
```

```python
def glove_weighting_function(x: np.ndarray, x_max: float = 100.0, alpha: float = 0.75) -> np.ndarray:
"""دالة الموازنة f(X_ij) من ورقة GloVe الأصلية."""
    return np.where(x < x_max, (x / x_max) ** alpha, 1.0)
```

تدريب GloVe الكامل يتطلّب تحسين J بالتدرّج العشوائي عبر كل الأزواج غير الصفرية في المصفوفة — تمرين ممتاز موسّع سنتركه في قسم التمارين، لأن التركيز الهندسي الأهم هنا هو **فهم الفرق الفلسفي** بين النهجين، لا إعادة تطبيق GloVe بالكامل (نادرًا ما يُدرَّب GloVe من الصفر اليوم؛ عادة تُستخدم أوزان مُدرَّبة مسبقًا).

---

4. 🧪 مثال محلول خطوة بخطوة (End-to-End)

لنأخذ مجموعة نصوص عربية صغيرة جدًا كي نُتابع الحساب يدويًا ثم بالكود.

النصوص:

```
الملك يحكم البلاد بعدل
الملكة تحكم البلاد بحكمة
الرجل يعمل في المصنع
المرأة تعمل في المستشفى
```

الخطوة 1 — بناء المفردات (بعد إزالة أدوات الربط البسيطة للتوضيح): `{الملك, يحكم, البلاد, بعدل, الملكة, تحكم, بحكمة, الرجل, يعمل, المصنع, المرأة, المستشفى}` $\rightarrow V = 12$

الخطوة 2 — توليد أزواج Skip-gram بنافذة $m = 1$ من الجملة الأولى "الملك يحكم البلاد بعدل":

```
(الملك, يحكم)
(يحكم, الملك)
(يحكم, البلاد)
(البلاد, يحكم)
(البلاد, بعدل)
(بعدل, البلاد)
```

الخطوة 3 — لعيّنة واحدة `(center=الملك, context=يحكم)` : - نسحب $k = 2$ كلمات سلبية عشوائيًا بتوزيع P_n، لنفترض أنها سحبت: `المستشفى` ، `المصنع` . - نحسب: $v_{\text{الملك}} \cdot v'_{\text{يحكم}}$ (نريده كبيرًا موجبًا) - نحسب: $v_{\text{الملك}} \cdot v'_{\text{المستشفى}}$ و $v_{\text{الملك}} \cdot v'_{\text{المصنع}}$ (نريدهما كبيرين سالبين) - الخسارة =

$$-\log\sigma(score_{pos}) - \log\sigma(-score_{neg1}) - \log\sigma(-score_{neg2})$$

بعد مئات التكرارات على كل الجمل، الشبكة تتعلّم تدريجيًا أن `الملك` و`الملكة` يظهران في سياقات متشابهة (`يحكم/تحكم`, `البلاد`)، فتتقارب متجهاتهما، وكذلك `الرجل` / `المرأة` يتقاربان بحكم سياقهما المشترك (`يعمل`). والأهم: الاتجاه من `الرجل` → `المرأة` في الفضاء المتجهي يصبح *مشابهًا جدًا* للاتجاه من `الملك` → `الملكة` — لأن كلا الزوجين "يحمل نفس

العلاقة الدلالية" (مذكّر ↔ مؤنث) ضمن سياقات مماثلة البنية. هذا هو سرّ المعادلة الشهيرة $king - man + woman \approx queen$.

الكود للتحقق من هذا بعد التدريب:

```
def cosine_similarity(a: Tensor, b: Tensor) -> Tensor:
    return torch.nn.functional.cosine_similarity(a.unsqueeze(0), b.unsqueeze(0))


def most_similar(
    word: str, vocab: Vocabulary, embeddings: Tensor, top_k: int = 5
) -> list[tuple[str, float]]:
    if word not in vocab.word2idx:
        raise KeyError(f"الكلمة '{word}' غير موجودة في المفردات")

    word_vec = embeddings[vocab.word2idx[word]]
    similarities = torch.nn.functional.cosine_similarity(
        embeddings, word_vec.unsqueeze(0), dim=1
    )
    top_indices = torch.topk(similarities, top_k + 1).indices.tolist()

    results = [
        (vocab.idx2word[idx], similarities[idx].item())
        for idx in top_indices
        if vocab.idx2word[idx] != word
    ][:top_k]
    return results


def analogy(
    a: str, b: str, c: str, vocab: Vocabulary, embeddings: Tensor, top_k: int = 3
) -> list[tuple[str, float]]:
"""يحل المعادلة: a is to b as c is to ?  (مثال: رجل:ملك :: امرأة:؟)"""
    vec = (
        embeddings[vocab.word2idx[b]]
        - embeddings[vocab.word2idx[a]]
        + embeddings[vocab.word2idx[c]]
    )
    similarities = torch.nn.functional.cosine_similarity(
        embeddings, vec.unsqueeze(0), dim=1
    )
    excluded = {vocab.word2idx[a], vocab.word2idx[b], vocab.word2idx[c]}
    top_indices = torch.topk(similarities, top_k + len(excluded)).indices.tolist()
    results = [
        (vocab.idx2word[idx], similarities[idx].item())
        for idx in top_indices
        if idx not in excluded
    ][:top_k]
    return results


# مثال استخدام (بافتراض تدريب كافٍ على مجموعة بيانات كبيرة بما يكفي):
# model, vocab = train_skipgram(large_corpus, embedding_dim=100, epochs=100)
```

```
# vectors = model.get_word_vectors()
# print(most_similar("ملك", vocab, vectors))
# print(analogy("امرأة" ,"ملك" ,"رجل", vocab, vectors))  # نتوقع: ملكة
```

> ⚠️ **ملاحظة واقعية مهمّة:** المعادلة التناظرية الشهيرة ($king - man + woman = queen$) تعمل بشكل مبهر في الأوراق البحثية الأصلية لأنها دُرِّبت على **مليارات** الكلمات (Google News Corpus بحجم 100 مليار كلمة!). على مجموعة بيانات صغيرة توضيحية من بضع جمل كما أعلاه، **لن تحصل على نتائج مبهرة مماثلة** — هذا طبيعي تمامًا، فالتضمينات الجيدة تحتاج كمًّا هائلًا من البيانات لتتعلم أنماطًا دلالية موثوقة إحصائيًا.

---

5. 💡 نصائح وحيل (Tips & Tricks)

1. **Subsampling للكلمات شديدة التكرار:** في الورقة الأصلية، تُحذف كلمات مثل "في/من/the" من التدريب باحتمال معين قبل حتى توليد الأزواج، بصيغة: $P(\text{discard}) = 1 - \sqrt{t / f(w)}$ حيث $t \approx 10^{-5}$. هذا يمنع هذه الكلمات عديمة المعنى تقريبًا من إغراق إشارة التدريب.
2. **أبعاد التضمين (d):** القيم الشائعة 100-300. أبعاد أكبر تلتقط فروقًا دلالية أدقّ لكن تحتاج بيانات أكثر لتجنّب الإفراط في المطابقة (Overfitting) وتزيد الكلفة الحسابية.
3. **حجم النافذة الديناميكي:** كما رأينا في الكود، اختيار حجم نافذة عشوائي لكل عيّنة (بدلًا من ثابت) يُحسّن الجودة لأنه يُعطي وزنًا ضمنيًا أكبر للكلمات الأقرب للمركز.
4. **التهيئة (Initialization) المختلفة للمصفوفتين:** لاحظ في كودنا أننا هيّأنا `context_embeddings` بالأصفار بينما `center_embeddings` بتوزيع منتظم صغير — هذه ممارسة قياسية في تطبيقات Word2Vec الموثوقة (مثل Gensim) تُحسّن استقرار التدريب المبكر.
5. **استخدم مكتبة Gensim للإنتاج الفعلي:** لا تُعد اختراع العجلة؛ في المشاريع الحقيقية استخدم `gensim.models.Word2Vec` المُحسّنة بلغة C تحت الغطاء، أو استخدم تضمينات مُدرَّبة مسبقًا (fastText, GloVe الرسمية).
6. **لا تنسَ Normalization قبل حساب التشابه:** تشابه جيب التمام (Cosine Similarity) يتطلب أن تكون المتجهات ذات معنى متناسب الاتجاه بغضّ النظر عن المقدار — وهذا بالضبط ما تحسبه `F.cosine_similarity` تلقائيًا، لكن إن حسبت المسافة الإقليدية بدلًا من ذلك فتذكّر تطبيع المتجهات يدويًا أولًا.

---

6. ⚠️ أخطاء شائعة ومصائد

❌ الخطأ 1: استخدام مصفوفة تضمين واحدة مشتركة بين center وcontext

```
# خطأ: استخدام نفس الطبقة للمركز والسياق
self.embeddings = nn.Embedding(vocab_size, dim)
score = self.embeddings(center) @ self.embeddings(context).T  # ❌ قد يسبب تآكل تدرّج غريب
```

✅ **الصحيح:** استخدم مصفوفتين منفصلتين كما فعلنا (`center_embeddings` و`context_embeddings`). استخدام مصفوفة واحدة مشتركة يجعل $v_w \cdot v_w$ (تشابه الكلمة مع نفسها) يدخل ضمنيًا في حسابات غير مقصودة ويُربك التحسين، رغم أنه يعمل تقنيًا، لكن النتائج التجريبية أسوأ بوضوح.

❌ الخطأ 2: نسيان حذف الكلمات النادرة جدًا (أو عدم ضبط `min_count`)

```
vocab = Vocabulary.build(sentences, min_count=1)  # ❌ يُبقي كل كلمة ظهرت مرة واحدة فقط
```

الكلمات التي ظهرت مرة واحدة فقط في كامل المجموعة لا تملك إشارة إحصائية كافية لتعلّم تضمين ذي معنى — ستبقى عشوائية تقريبًا وتُهدر ذاكرة GPU. ✅ استخدم `min_count` بين 5-10 كحد أدنى معقول لمجموعات البيانات المتوسطة والكبيرة.

❌ الخطأ 3: الخلط بين تضمين Word2Vec الثابت وتضمين Contextual الحديث

```
# خطأ مفاهيمي: توقع أن تضمين "بنك" يتغيّر حسب الجملة
vector_bank_river = word2vec_model["بنك"]   # ❌ نفس المتجه دائمًا!
vector_bank_money = word2vec_model["بنك"]   # ❌ متطابق تمامًا، رغم اختلاف المعنى!
```

تضمينات Word2Vec **ثابتة** (Static) — كل كلمة لها متجه واحد وحيد بغضّ النظر عن السياق الذي تظهر فيه. كلمة "بنك" (ضفة النهر) و"بنك" (مؤسسة مالية) ستحصلان على نفس المتجه بالضبط! هذا قيد جوهري سنحلّه لاحقًا في الفصل الثامن (RNN/LSTM) ثم بشكل كامل في فصول Transformer حيث التضمينات تصبح **سياقية** (Contextual) — متجه الكلمة يتغيّر ديناميكيًا حسب الجملة المحيطة بها.

❌ الخطأ 4: مقارنة خسائر Word2Vec عبر إعدادات مختلفة من k (عدد العيّنات السلبية) كأنها قابلة للمقارنة مباشرة

تغيير k يُغيّر بنية دالة الخسارة نفسها (عدد الحدود المجموعة)، لذا فإن قيمة الخسارة المطلقة بـ $k = 5$ لا تُقارَن مباشرة بقيمتها بـ $k = 15$. ✅ قارن دائمًا عبر مقياس خارجي ثابت (مثل أداء Analogy Task أو تشابه مع تقييم بشري) وليس قيمة دالة الخسارة الخام.

---

7. ⚖️ جدول مقارنة

المعيار	CBOW	Skip-gram	GloVe
المدخل → المخرج	سياق → كلمة مركزية	كلمة مركزية → سياق	إحصائيات تعايش عامة
نوع التعلّم	تنبؤي محلي (Local)	تنبؤي محلي (Local)	عدّي شامل (Global Count-based)
عدد عيّنات التدريب لكل نافذة	1	$2m$	يعتمد على حجم مصفوفة التعايش
الأداء مع الكلمات النادرة	ضعيف نسبيًا (يُمتوسط السياق)	أفضل	جيد (يستفيد من كامل الإحصاء)
سرعة التدريب	أسرع	أبطأ قليلًا	يتطلب بناء مصفوفة تعايش أولًا (مكلف ذاكرة)
استغلال المعلومات الإحصائية العامة	لا (نافذة محلية فقط)	لا (نافذة محلية فقط)	نعم (إحصائيات العالم كله دفعة واحدة)
الاستخدام في الإنتاج الحديث	نادر	أكثر شيوعًا من CBOW	يُستخدم كأوزان مُدرَّبة مسبقًا جاهزة
يُعالج الكلمات خارج المفردات (OOV)؟	لا	لا	لا (بخلاف fastText الذي يُعالجها بالـ character n-grams)
سياقية التضمين	ثابت (Static)	ثابت (Static)	ثابت (Static)

---

8. ✅ قائمة التحقق من أفضل الممارسات

- [ ] هل بنيت المفردات مع حدّ أدنى معقول للتكرار (`min_count`) لتفادي ضجيج الكلمات النادرة؟
- [ ] هل استخدمت توزيع $P_n(w) \propto f(w)^{3/4}$ للعيّنات السلبية بدلًا من التردد الخام أو التوزيع المنتظم؟
- [ ] هل استخدمت مصفوفتي تضمين منفصلتين (center/context) في Skip-gram أو CBOW؟
- [ ] هل طبّقت Subsampling للكلمات شديدة التكرار قبل توليد الأزواج؟
- [ ] هل اخترت d (أبعاد التضمين) بما يتناسب مع حجم بيانات التدريب المتاحة لديك (ليس أكبر مما تدعمه البيانات)؟
- [ ] هل تذكّرت أن هذه تضمينات **ثابتة غير سياقية**، وتجنّبت استخدامها في مهام تتطلب تمييز المعاني المتعددة لنفس الكلمة (Polysemy)؟
- [ ] هل استخدمت `logsigmoid` بدلًا من `log(sigmoid(x))` تجنّبًا لعدم الاستقرار العددي؟

- [ ] عند التقييم، هل استخدمت مقياسًا خارجيًا (Analogy Task, Similarity Benchmark) لا مجرد قيمة الخسارة النهائية؟
- [ ] في مشاريع حقيقية، هل فكّرت باستخدام تضمينات مُدرَّبة مسبقًا (Pretrained) بدلًا من التدريب من الصفر إن كانت بياناتك محدودة؟

---

9. 🎯 أسئلة مقابلات شخصية + إجابات نموذجية

س1: ما الفرق الجوهري بين CBOW وSkip-gram، ومتى تفضّل كلًا منهما؟

> CBOW يتنبأ بالكلمة المركزية من متوسط سياقها (يُنتج عيّنة تدريب واحدة لكل نافذة، وأسرع)، بينما Skip-gram يتنبأ بكل كلمة سياق انطلاقًا من الكلمة المركزية (يُنتج $2m$ عيّنة لكل نافذة، أبطأ لكنه يتعامل بشكل أفضل مع الكلمات النادرة لأن كل ظهور لها يُولّد إشارة تدريب مباشرة). عمليًا، يُفضَّل Skip-gram عند التعامل مع مجموعات بيانات صغيرة/متوسطة فيها كلمات نادرة مهمة، بينما CBOW مُفضَّل عند توفّر بيانات ضخمة جدًا وأولوية السرعة.

س2: لماذا نحتاج Negative Sampling أصلًا؟ ولماذا لا نستخدم Softmax العادي؟

> لأن Softmax العادي يتطلب حساب المقام على كامل حجم المفردات V (قد يصل لمليون كلمة)، في كل خطوة تدريب، لكل كلمة، مما يجعل التدريب غير قابل للتطبيق حسابيًا على نطاق واسع. Negative Sampling يُحوّل المسألة إلى تصنيف ثنائي بسيط بين زوج حقيقي وk أزواج مزيّفة، فيصبح التعقيد $O(k)$ بدلًا من $O(V)$، وهو تخفيض هائل يجعل التدريب على مليارات الكلمات ممكنًا في وقت معقول.

س3: لماذا يُستخدم الأُس 3/4 في توزيع العيّنات السلبية تحديدًا؟

> لأنه يُحدث توازنًا تجريبيًا جيدًا: يُخفّض هيمنة الكلمات شديدة التكرار (مثل أدوات الربط) على عملية السحب العشوائي للعيّنات السلبية، بينما يرفع قليلًا فرصة سحب الكلمات النادرة نسبيًا مقارنة بسحبها حسب ترددها الخام المباشر. هذا الرقم (0.75) مُستخرج تجريبيًا من الورقة الأصلية وأثبت تفوّقه عمليًا على كل من التوزيع الموحّد (Uniform) والتوزيع الخام (Raw Unigram).

س4: ما الفرق الفلسفي بين Word2Vec وGloVe؟

> Word2Vec نموذج تنبؤي (Predictive) يتعلّم من نوافذ سياق محلية صغيرة عبر تمرير تدريجي على النص (Online)، بينما GloVe نموذج عدّي (Count-based) يبني أولًا مصفوفة تعايش مشترك إحصائية لكامل مجموعة النصوص دفعة واحدة، ثم يحلّلها عامليًا بأسلوب شبيه بالمربعات الصغرى الموزونة. يمكن اعتبار GloVe محاولة

> للاستفادة من أفضل ما في عالمين: إحصائيات العدّ الشاملة لمصفوفات التعايش التقليدية (مثل LSA) + الفضاء المتجهي المستمر ذو الأبعاد المنخفضة لـ Word2Vec.

س5: لماذا تضمينات Word2Vec غير مناسبة لحل مشكلة تعدد المعاني (Polysemy)، وما الذي حلّها لاحقًا؟

> لأن Word2Vec يُنتج متجهًا واحدًا ثابتًا لكل كلمة بصرف النظر عن السياق — فكلمة "عين" (في الوجه) و"عين" (نبع ماء) تحصلان على نفس المتجه بالضبط، لأن النموذج لا يرى الجملة الكاملة بل فقط نافذة سياق محلية عند التدريب، والمتجه النهائي يُخزَّن ثابتًا في جدول بحث واحد. الحل جاء لاحقًا مع النماذج السياقية مثل ELMo (باستخدام LSTM ثنائي الاتجاه) ثم بشكل أقوى بكثير مع آلية Self-Attention في المحوّلات (BERT، GPT)، حيث يُعاد حساب تمثيل الكلمة ديناميكيًا في كل مرة بناءً على كامل الجملة المحيطة بها.

---

10. 🧪 تمرين عملي (Mini-Project)

المهمة: بناء "مُكتشف تشابه دلالي عربي" من نصوص حرة

1. اجمع مجموعة نصوص عربية (يمكن استخدام مقالات ويكيبيديا العربية، أو أي مجموعة نصوص عربية مفتوحة لا تقل عن بضعة ملايين كلمة للحصول على نتائج معقولة).
2. طبّق الـ Tokenizer من الفصل السادس (BPE أو WordPiece) أو Tokenizer بسيط على مستوى الكلمة.
3. درّب نموذج Skip-gram with Negative Sampling باستخدام الكود المُقدَّم في هذا الفصل، مع تجربة قيم مختلفة لـ: `embedding_dim` (50، 100، 300)، `window_size` (2، 5، 10)، `num_negatives` (5، 15).
4. قيّم جودة التضمينات الناتجة عبر: - دالة `most_similar()` لاختبار كلمات معروفة (مثل: "ملك"، "جامعة"، "حاسوب"). - دالة `analogy()` لاختبار علاقات مثل: `القاهرة : مصر :: الرياض : ؟` (نتوقع: السعودية). - تصوّر التضمينات بتقليل الأبعاد إلى بُعدين باستخدام t-SNE أو PCA ورسمها.

هيكل بداية الكود:

```
"""
arabic_word_similarity_project.py
مشروع تطبيقي: مُكتشف تشابه دلالي عربي باستخدام Word2Vec.
"""
from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# TODO 1: حمّل مجموعة نصوص عربية من ملف أو مكتبة (مثل datasets من HuggingFace)
```

```python
def load_arabic_corpus(path: Path) -> list[str]:
    raise NotImplementedError("حمّل نصوصك هنا")


# TODO 2: نظّف ورمّز النصوص (استخدم simple_tokenize أو Tokenizer BPE من الفصل 6)
def preprocess(raw_texts: list[str]) -> list[list[str]]:
    raise NotImplementedError


# TODO 3: درّب النموذج باستخدام train_skipgram() المُعرَّفة في هذا الفصل
# model, vocab = train_skipgram(tokenized_sentences, embedding_dim=100, epochs=50)


# TODO 4: دالة تصوير التضمينات بعد تقليل الأبعاد
def visualize_embeddings(
    words: list[str], vocab: "Vocabulary", embeddings, output_path: str = "embeddings.png"
) -> None:
    indices = [vocab.word2idx[w] for w in words if w in vocab.word2idx]
    vectors = embeddings[indices].numpy()

    reduced = TSNE(n_components=2, random_state=42, perplexity=5).fit_transform(vectors)

    plt.figure(figsize=(10, 8))
    for i, word in enumerate(words):
        if vocab.word2idx.get(word) is None:
            continue
        x, y = reduced[i]
        plt.scatter(x, y)
        plt.annotate(word, (x, y), fontsize=12)
    plt.title("تصوّر التضمينات الدلالية (Word2Vec)")
    plt.savefig(output_path, dpi=150, bbox_inches="tight")
    print(f"تم حفظ الرسم في: {output_path}")


# TODO 5 (تحدٍّ إضافي): قارن نتائج نموذجك المُدرَّب محليًا بنموذج fastText
# العربي المُدرَّب مسبقًا (Facebook AI)، وناقش الفروقات في الجودة والتغطية.
```

معيار النجاح: إذا استطاع نموذجك إرجاع كلمات ذات صلة منطقية حقيقية عند استدعاء `most_similar("طبيب")` (مثل: "مستشفى"، "ممرضة"، "مريض"، "علاج")، فقد نجحت في بناء تضمينات ذات معنى فعلي من الصفر.

---

11. 🔗 جسر للفصل التالي

لقد تعلّمنا في هذا الفصل كيف نُنتج متجهًا **ثابتًا** لكل كلمة بمعزل عن موقعها في الجملة. لكن بقي سؤال جوهري معلّق لم نحلّه بعد: ماذا عن **الجملة بأكملها**؟ كيف نُمثّل تسلسلًا كاملًا من الكلمات، حيث يهمّ **الترتيب** (فـ"الكلب عضّ الرجل" مختلفة تمامًا عن "الرجل عضّ الكلب" رغم أن كلتا الجملتين تحتويان نفس الكلمات بنفس التضمينات)؟

هذا بالضبط ما سنكتشفه في **الفصل الثامن:** سنبدأ بالشبكات العصبية المتكررة (RNN) التي تُعالج الكلمات كتسلسل زمني خطوة بخطوة، ذاكرتها تتراكم عبر الزمن؛ ثم سنكتشف لماذا تفشل RNN البسيطة في تذكّر المعلومات البعيدة (مشكلة تلاشي التدرّج — Vanishing Gradient)؛ وكيف حلّتها بنيتا LSTM وGRU ببوابات ذكية للتحكم بتدفّق المعلومات. هذا الفصل سيكون الحلقة الأخيرة في "عالم ما قبل Transformer"، ويُمهّد الطريق مباشرة لفهم **لماذا كانت الحاجة ماسّة** لاختراع آلية Self-Attention التي شرحناها بعمق في الفصل العاشر — لأن RNN وLSTM، كما سترى، تحملان قيدًا بنيويًا أساسيًا هو **التتابع الإجباري** (Sequential Processing) الذي يمنع التوازي الكامل على GPU، وهي بالضبط المشكلة التي أتت المحوّلات لحلّها.

---

12. 📚 مراجع للتعمّق

المرجع	الوصف
Mikolov et al., 2013a — *"Efficient Estimation of Word Representations in Vector Space"*	الورقة الأصلية التي قدّمت CBOW وSkip-gram
Mikolov et al., 2013b — *"Distributed Representations of Words and Phrases and their Compositionality"*	الورقة التي قدّمت Negative Sampling وSubsampling
Pennington, Socher & Manning, 2014 — *"GloVe: Global Vectors for Word Representation"*	الورقة الأصلية لـ GloVe مع التحليل الرياضي الكامل لنسب الاحتمالات
Goldberg & Levy, 2014 — *"word2vec Explained"*	شرح رياضي تبسيطي ممتاز لاشتقاق Negative Sampling
Bojanowski et al., 2017 — *"Enriching Word Vectors with Subword Information"* (fastText)	امتداد Word2Vec لمعالجة الكلمات خارج المفردات عبر character n-grams
Firth, J.R., 1957 — *"A synopsis of linguistic theory"*	المصدر اللغوي الأصلي للفرضية التوزيعية
Levy & Goldberg, 2014 — *"Neural Word Embedding as Implicit Matrix Factorization"*	يُثبت أن Skip-gram with Negative Sampling مكافئ ضمنيًا لتحليل عاملي لمصفوفة PMI
Rong, X., 2014 — *"word2vec Parameter Learning Explained"*	اشتقاق تفصيلي خطوة بخطوة للتدرجات في CBOW وSkip-gram (مرجع ممتاز للمراجعة الرياضية العميقة)
مكتبة Gensim (الوثائق الرسمية)	التطبيق المرجعي الصناعي لـ Word2Vec/FastText/Doc2Vec

==========

الفصل الثامن: من الكلمة إلى الجملة — RNN وLSTM وGRU



(تمثيل التسلسلات: Recurrent Neural Networks, Long (Short-Term Memory, Gated Recurrent Units

1. 🟢 لماذا يهمّك هذا؟

في نهاية الفصل السابع تركتك عند سؤال معلّق: Word2Vec يُعطينا متجهًا ممتازًا لكل كلمة بمفردها، لكن ماذا لو أعطيتك الجملتين التاليتين؟

> أ) "الكلب عضّ الرجل" ب) "الرجل عضّ الكلب"

إذا جمعت (أو توسّطت) تضمينات الكلمات الثلاث في كلتا الجملتين، ستحصل على **نفس المتجه بالضبط** — لأن الجمع عملية تبديلية (Commutative) لا تهتم بالترتيب! لكن أي متحدث عربي يعرف أن الجملتين تحملان معنيين مختلفين تمامًا جوهريًا: من هو الفاعل ومن هو المفعول به انقلب كليًا. **الترتيب هو المعنى**، وأي تمثيل رياضي للجملة يتجاهل الترتيب هو تمثيل معطوب بالضرورة.

هذا يقودنا إلى سؤال هندسي عميق: كيف نصمم شبكة عصبية تقرأ الكلمات **بالتتابع**، وتحتفظ بـ"ذاكرة" تتراكم مع كل كلمة جديدة تقرأها، بحيث يكون التمثيل النهائي للجملة معتمدًا على كل من: (أ) الكلمات نفسها، و(ب) الترتيب الذي ظهرت به؟

تخيّل نفسك تقرأ جملة طويلة كلمة كلمة، من اليمين لليسار في العربية. عقلك لا يرى كل الكلمات دفعة واحدة؛ بل يُراكم فهمًا تدريجيًا: تقرأ "الطالب"، تُكوّن فكرة أولية؛ تقرأ "الذي"، تنتظر تفصيلًا إضافيًا؛ تقرأ "نجح"، تُحدّث فهمك: "آه، إذن نتحدث عن طالب ناجح"؛ ثم "بتفوّق"، تُضيف تفصيلًا أخيرًا. في كل خطوة، لديك **حالة ذهنية** (Mental State) تتراكم وتتحدّث. هذا بالضبط ما تحاول الشبكات العصبية المتكررة (RNN) محاكاته رياضيًا.

لكن كما سنكتشف، هذه الفكرة البسيطة والأنيقة تصطدم بعقبة رياضية خطيرة جدًا — مشكلة **تلاشي التدرّج** (Vanishing Gradient) — التي تجعل الشبكة "تنسى" المعلومات البعيدة بسرعة مذهلة. وحلّ هذه المشكلة بالذات، عبر بنيتي LSTM وGRU، هو أحد أذكى الاختراعات الهندسية في تاريخ الشبكات العصبية. وفهم *لماذا* فشلت RNN البسيطة هو المفتاح الذي سيجعلك تُقدّر، في الفصول القادمة، لماذا كانت المحوّلات (Transformers) ثورة حقيقية لا مجرد تحسين تدريجي.

---

3. 🟡 الشرح المتدرّج: من الحدس إلى الرياضيات إلى الكود

3.1 الفكرة الأساسية: حالة مخفية تتراكم عبر الزمن

لنبدأ بالحدس البسيط. لدينا تسلسل من المدخلات $x_1, x_2, \ldots, x_T$ (كل x_t هو تضمين كلمة، مثلًا من Word2Vec). نريد شبكة تُنتج، عند كل خطوة زمنية t، **حالة مخفية** h_t تُلخّص "كل ما رأته الشبكة حتى الآن".

الفكرة الرياضية الأنيقة: اجعل h_t دالة في شيئين فقط: المدخل الحالي x_t، والحالة المخفية السابقة h_{t-1}:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

لاحظ الأناقة هنا: **نفس المصفوفتين** W_{xh} و W_{hh} (ونفس التحيّز b_h) تُستخدَم في **كل** خطوة زمنية، بغض النظر عن طول الجملة! هذا يُسمى **مشاركة الأوزان عبر الزمن** (Parameter Sharing Across Time) — وهو ما يسمح للشبكة بالتعامل مع تسلسلات من **أي طول** بعدد ثابت من المعاملات (Parameters)، تمامًا كما تستخدم عينيك نفس "آلية القراءة" سواء كانت الجملة من 3 كلمات أو 300 كلمة.

ثم، إن أردنا مخرجًا فعليًا عند كل خطوة (مثلًا توقّع الكلمة التالية، أو تصنيف)، نُضيف طبقة إخراج:

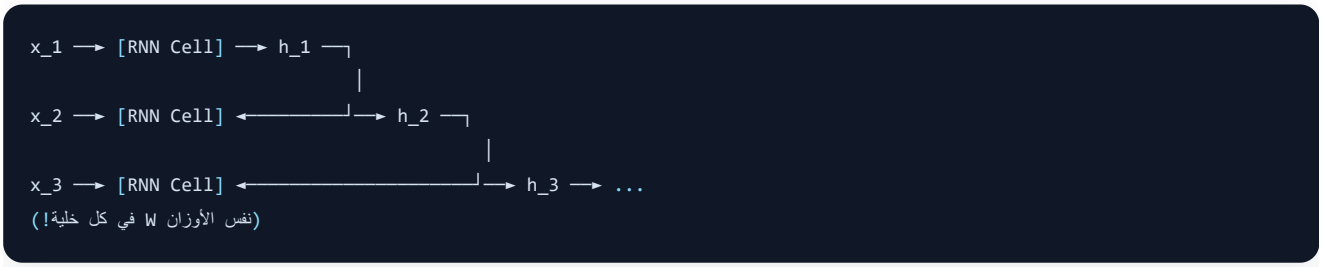
$$\hat{y}_t = \text{softmax}(W_{hy} h_t + b_y)$$

الحالة الابتدائية h_0 عادة تُهيّأ بأصفار (أو تُتعلّم كمعامل إضافي قابل للتدريب في بعض التطبيقات المتقدمة).

> 🟢 **تخيّل الصورة التالية لترسيخ الفهم:** تخيّل RNN كشخص يقرأ كتابًا ويُمسك بملاحظة واحدة فقط على ورقة صغيرة (الحالة المخفية h_t). بعد كل صفحة (كلمة)، يمحو بعض الملاحظة القديمة ويكتب ملاحظة جديدة مبنية على القديمة + ما قرأه للتو. المشكلة: الورقة **محدودة الحجم** (الحالة المخفية ذات أبعاد ثابتة)، فمع تقدّم القراءة، تفاصيل من الصفحات الأولى جدًا تبدأ بالتلاشي والانمحاء تدريجيًا لإفساح المجال للمعلومات الجديدة.

3.2 الانتشار الأمامي عبر الزمن (Unrolling RNN)

من المفيد جدًا "فكّ لفّ" (Unroll) الشبكة المتكررة عبر الزمن لنراها كشبكة عميقة عادية، حيث كل "طبقة" تُمثّل خطوة زمنية واحدة:



هذا المنظور حاسم لفهم التدريب: بمجرد أن "نفكّ" RNN عبر T خطوة زمنية، تصبح رياضيًا **مكافئة تمامًا لشبكة تغذية أمامية عميقة جدًا بعمق T طبقة**، مع قيد إضافي أن كل الطبقات تتشارك نفس الأوزان. وهذا المنظور بالذات هو ما يُفسّر لنا لاحقًا، بشكل طبيعي جدًا، لماذا تحدث مشكلة تلاشي التدرّج — فهي نفس المشكلة التي ناقشناها في الفصل الثالث بخصوص الشبكات العميقة جدًا، لكن هنا تتضخّم بشكل خاص.

3.3 خوارزمية Backpropagation Through Time (BPTT)

لتدريب RNN، نحتاج حساب التدرّج (Gradient) لدالة الخسارة بالنسبة لكل الأوزان. بما أن نفس الأوزان W_{hh} تُستخدم في كل خطوة زمنية، فإن التدرّج الكلي هو **مجموع** مساهمات كل خطوة زمنية على حدة:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W_{hh}}$$

لكن احسب $\partial L_t / \partial W_{hh}$ بعناية — الخسارة عند الزمن t لا تعتمد فقط على h_t، بل تعتمد عبر سلسلة من الاعتماديات على $h_{t-1}, h_{t-2}, \ldots, h_1$ أيضًا (لأن h_t نفسها دالة في h_{t-1}، وهكذا تنازليًا). بتطبيق قاعدة السلسلة (Chain Rule) بعناية:

$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^{t} \frac{\partial L_t}{\partial h_t} \left(\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W_{hh}}$$

توقّف عند هذا الحدّ الأوسط — حاصل الضرب $\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$ — لأنه قلب المشكلة بأكملها.

3.4 مشكلة تلاشي/انفجار التدرّج: التشريح الرياضي الكامل

بما أن $h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$، فإن:

$$\frac{\partial h_j}{\partial h_{j-1}} = \operatorname{diag}\left(\tanh'(\cdot) \right) \cdot W_{hh}^{\top}$$

والآن تأمّل حاصل الضرب عبر عدة خطوات زمنية متتالية (لنقل من $k+1$ إلى t، وليكن الفرق $t - k = n$ خطوة):

$$\prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \approx (W_{hh}^{\top})^n \cdot \prod \operatorname{diag}(\tanh')$$

هذا **حاصل ضرب لنفس المصفوفة W_{hh} مع نفسها n مرة متتالية**! والآن اسأل نفسك: ماذا يحدث لمصفوفة عندما تُضرب في نفسها مرات عديدة متتالية؟

من الجبر الخطي، إذا كانت λ هي أكبر قيمة ذاتية (Eigenvalue) لمصفوفة W_{hh}:

- إذا كانت $|\lambda| < 1$: فإن $\lambda^n \to 0$ بسرعة أُسّية (Exponentially) كلما زاد n ⟹ **تلاشي التدرّج (Vanishing Gradient)**. التدرّج القادم من خطوة بعيدة في الزمن يصبح صغيرًا جدًا لدرجة أنه لا يُحدث أي تحديث فعّال على الأوزان — الشبكة "تنسى" عمليًا كل ما حدث قبل بضع خطوات زمنية فقط.
- إذا كانت $|\lambda| > 1$: فإن $\lambda^n \to \infty$ بسرعة أُسّية ⟹ **انفجار التدرّج (Exploding Gradient)**. التحديثات تصبح ضخمة وغير مستقرة، وقد تُنتج قيم `NaN`.

ثم أضف إلى ذلك أن $\tanh'(x) \in (0, 1]$ دائمًا (أقصى قيمة له هي 1 عند $x = 0$، وتتناقص بسرعة كلما ابتعدنا عن الصفر) — وهذا يعني أن **كل** حدّ في حاصل الضرب يحمل عامل تصغير إضافي من مشتقة tanh، مما يزيد مشكلة

التلاشي سوءًا بشكل خاص (في الممارسة العملية، مشكلة التلاشي أكثر شيوعًا وإزعاجًا من الانفجار في RNN البسيطة، لأن مشتقة tanh تدفع الأمور نحو الصفر أكثر مما تدفعها نحو اللانهاية).

> ⚠ **لماذا هذه المشكلة أخطر في RNN منها في الشبكات العميقة العادية؟** في شبكة تغذية أمامية عادية بعمق L طبقة، كل طبقة عادة تملك **مصفوفة وزن مختلفة**، فاحتمالية أن تتسق كل القيم الذاتية باتجاه واحد (كلها > 1 أو كلها < 1) أقل. لكن في RNN، **نفس** المصفوفة W_{hh} تُضرب في نفسها حرفيًا n مرة متتالية — فإذا كانت قيمتها الذاتية الكبرى أقل قليلًا من 1، فالتلاشي الأُسّي **مضمون رياضيًا** ومتسارع جدًا مع طول الجملة. هذا ليس مجرد مشكلة عملية عرضية؛ إنه نتيجة حتمية من بنية الشبكة نفسها.

النتيجة العملية: RNN البسيطة تستطيع "تذكّر" سياق قصير فقط (عمليًا نحو 5-10 كلمات بشكل موثوق)، وتفشل تمامًا في ربط معلومات متباعدة، مثل الجملة:

> "الطالب **الذي** حصل على المركز الأول في المسابقة العلمية الدولية التي أقيمت العام الماضي في باريس بمشاركة أكثر من خمسين دولة **نجح** بتفوّق."

الفعل "نجح" في آخر الجملة يجب أن "يتذكّر" أن الفاعل هو "الطالب" (مفرد)، لكن بينهما عشرات الكلمات — RNN البسيطة ستفشل غالبًا في الحفاظ على هذه الإشارة النحوية عبر كل هذه المسافة.

3.5 الحل: LSTM (Long Short-Term Memory) — هوشرايتر وشميدهوبر، 1997

الفكرة الثورية لـ LSTM: بدلًا من حالة مخفية واحدة فقط تمر عبر تحويل غير خطي (tanh) في كل خطوة (مما يُسبب التلاشي كما رأينا)، أضف **مسارًا ثانويًا منفصلًا** يُسمى **حالة الخلية** (Cell State) C_t، يمرّ عبر الزمن بعمليات **جمع وضرب خطية بسيطة فقط** (بلا tanh مباشر على المسار الرئيسي)، فتسمح للتدرّج بالتدفق خلفيًا عبر الزمن بثبات أكبر بكثير — تمامًا مثل "طريق سريع" (Highway) للمعلومة، لا يُصفّى بقوة في كل محطة.

لتحقيق ذلك، يستخدم LSTM ثلاث **بوابات** (Gates) — كل بوابة هي طبقة sigmoid تُنتج قيمًا بين 0 و1 تُحدّد "كم تسمح بالمرور":

1. بوابة النسيان (Forget Gate) — تُقرّر ماذا نحذف من الذاكرة القديمة:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. بوابة الإدخال (Input Gate) — تُقرّر ما الجديد الذي نُضيفه للذاكرة:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad \text{(المرشح الجديد للمعلومة)}$$

3. تحديث حالة الخلية (العملية الحاسمة — جمع وضرب خطي بسيط فقط!):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

4. بوابة الإخراج (Output Gate) — تُقرّر ماذا نُخرج من الذاكرة كحالة مخفية:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

حيث $\odot$ تعني الضرب العنصري (Element-wise)، و $[h_{t-1}, x_t]$ تعني دمج (Concatenation) المتجهين.

لماذا تحلّ هذه البنية مشكلة تلاشي التدرّج؟ انظر بعناية إلى معادلة $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$. عندما نحسب $\partial C_t / \partial C_{t-1}$ ، نحصل ببساطة على f_t (بوابة النسيان) — **بلا أي ضرب بمصفوفة وزن ثابتة متكررة** n **مرة!** إذا تعلّمت الشبكة أن $f_t \approx 1$ لخطوات زمنية معينة (أي: "احتفظ بهذه المعلومة، لا تنسها")، فإن التدرّج يتدفّق عبر هذا المسار بثبات تقريبي، بلا تلاشي أُسّي. هذا بالضبط ما يُسمى أحيانًا مسار **Constant Error Carousel** في الورقة الأصلية — "دوّامة خطأ ثابتة" تسمح بحفظ الإشارة عبر مئات الخطوات الزمنية.

> 💡 **تشبيه عملي:** تخيّل C_t كصندوق بريد طويل الأمد (الذاكرة طويلة المدى)، و h_t كملخّص سريع تكتبه كل يوم (الذاكرة قصيرة المدى/العاملة). بوابة النسيان تُقرّر أي رسائل قديمة في الصندوق تُمزّق، بوابة الإدخال تُقرّر أي رسائل جديدة تُضاف، وبوابة الإخراج تُقرّر ماذا تقتبس من الصندوق لتكتب ملخص اليوم.

3.6 GRU (Gated Recurrent Unit) — تشو وآخرون، 2014: تبسيط أنيق لـ LSTM

جاء GRU كاقتراح أبسط بوابتين فقط بدلًا من ثلاث، وبدون حالة خلية منفصلة عن الحالة المخفية — دمج الاثنين في متغير واحد:

1. بوابة إعادة الضبط (Reset Gate) — تُحدّد كم من الماضي نتجاهل عند حساب المرشح الجديد:

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

2. بوابة التحديث (Update Gate) — تُحدّد التوازن بين الحفاظ على القديم واستقبال الجديد (تلعب دور بوابتي forget وinput في LSTM معًا):

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

3. المرشح الجديد و التحديث النهائي:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

لاحظ الأناقة الرياضية في السطر الأخير: إنه **متوسط مرجّح** (Weighted Average) بين الحالة القديمة والمرشح الجديد، يُتحكّم فيه بـ z_t بالكامل. إذا كانت $z_t \approx 0$، تحافظ الشبكة على h_{t-1} كاملًا (تجاهل المدخل الجديد تمامًا)؛ وإذا كانت $z_t \approx 1$، تستبدل الحالة بالكامل بالمرشح الجديد. وبنفس منطق LSTM، هذا الجمع الخطي البسيط يسمح بتدفّق تدرّج أكثر استقرارًا عبر الزمن.

3.7 LSTM مقابل GRU: أيهما تختار؟

GRU له معاملات أقل (بوابتان بدلًا من ثلاث، وبدون حالة خلية منفصلة)، مما يجعله أسرع تدريبًا وأقل عرضة للإفراط في المطابقة على بيانات صغيرة. تجريبيًا، أداؤهما **متقارب جدًا** في معظم المهام العملية (Chung et al., 2014)، ولا يوجد فائز حاسم بينهما بشكل عام — الاختيار غالبًا تجريبي حسب المهمة وحجم البيانات، رغم أن LSTM يُفضَّل أحيانًا في المهام التي تتطلب ذاكرة طويلة المدى معقدة جدًا بفضل حالة الخلية المنفصلة.

3.8 الكود: بناء RNN وLSTM من الصفر بـ PyTorch (للفهم)، ثم استخدام الوحدات المُدمَجة (للإنتاج)

لنبدأ أولًا ببناء خلية RNN وخلية LSTM **من الصفر يدويًا** لفهم الآلية الداخلية تمامًا، ثم ننتقل لاستخدام `nn.LSTM` و`nn.GRU` الجاهزتين والمُحسَّنتين في PyTorch للاستخدام الفعلي.

```
"""
rnn_lstm_from_scratch.py
بناء خلايا RNN وLSTM من الصفر لفهم الآلية الداخلية بعمق،
ثم مقارنتها بالوحدات المدمجة في PyTorch.
Python 3.13+ | PyTorch 2.x
"""
from __future__ import annotations
```

```python
import torch
from torch import nn, Tensor

torch.manual_seed(42)



# ======================================================================
# 1) خلية RNN بسيطة (Vanilla RNN) مبنية يدويًا
# ======================================================================
class VanillaRNNCell(nn.Module):
    def __init__(self, input_size: int, hidden_size: int) -> None:
        super().__init__()
        self.hidden_size = hidden_size
        self.W_xh = nn.Linear(input_size, hidden_size, bias=False)
        self.W_hh = nn.Linear(hidden_size, hidden_size, bias=True)

    def forward(self, x_t: Tensor, h_prev: Tensor) -> Tensor:
        """
        x_t:    (batch, input_size)
        h_prev: (batch, hidden_size)
h_t (batch, hidden_size) :يُعيد
        """
        return torch.tanh(self.W_xh(x_t) + self.W_hh(h_prev))


class VanillaRNN(nn.Module):
"""يُطبّق الخلية عبر كامل التسلسل الزمني (يدوي Unrolling)."""

    def __init__(self, input_size: int, hidden_size: int) -> None:
        super().__init__()
        self.hidden_size = hidden_size
        self.cell = VanillaRNNCell(input_size, hidden_size)

    def forward(self, x: Tensor, h0: Tensor | None = None) -> tuple[Tensor, Tensor]:
        """
        x: (batch, seq_len, input_size)
(batch, seq_len, hidden_size) يُعيد: (كل الحالات المخفية, آخر حالة مخفية)
        """
        batch_size, seq_len, _ = x.shape
        h_t = h0 if h0 is not None else torch.zeros(batch_size, self.hidden_size, device=x.device)

        outputs: list[Tensor] = []
        for t in range(seq_len):
            h_t = self.cell(x[:, t, :], h_t)
            outputs.append(h_t)

        return torch.stack(outputs, dim=1), h_t


# ======================================================================
# 2) خلية LSTM مبنية يدويًا بكل البوابات الأربع
# ======================================================================
class LSTMCellFromScratch(nn.Module):
    def __init__(self, input_size: int, hidden_size: int) -> None:
        super().__init__()
```

```
        self.hidden_size = hidden_size
        combined_size = input_size + hidden_size

# نُجمّع كل البوابات في مصفوفة واحدة كبيرة لكفاءة حسابية أعلى
# (بدلًا من 4 طبقات Linear منفصلة -- حيلة شائعة في التطبيقات الفعلية)
        self.gates = nn.Linear(combined_size, 4 * hidden_size)

    def forward(
        self, x_t: Tensor, state: tuple[Tensor, Tensor]
    ) -> tuple[Tensor, Tensor]:
        """
        x_t: (batch, input_size)
        state: (h_prev, C_prev), كل منهما (batch, hidden_size)
(h_t, C_t) :يُعيد
        """
        h_prev, c_prev = state
        combined = torch.cat([x_t, h_prev], dim=1)

        gates = self.gates(combined)  # (batch, 4*hidden_size)
        f_gate, i_gate, g_candidate, o_gate = gates.chunk(4, dim=1)

        f_t = torch.sigmoid(f_gate)        # بوابة النسيان
        i_t = torch.sigmoid(i_gate)        # بوابة الإدخال
        c_tilde = torch.tanh(g_candidate)  # المرشح الجديد
        o_t = torch.sigmoid(o_gate)        # بوابة الإخراج

        c_t = f_t * c_prev + i_t * c_tilde     # تحديث حالة الخلية (المسار الخطي الحاسم!)
        h_t = o_t * torch.tanh(c_t)             # الحالة المخفية الجديدة

        return h_t, c_t


class LSTMFromScratch(nn.Module):
    def __init__(self, input_size: int, hidden_size: int) -> None:
        super().__init__()
        self.hidden_size = hidden_size
        self.cell = LSTMCellFromScratch(input_size, hidden_size)

    def forward(
        self, x: Tensor, init_state: tuple[Tensor, Tensor] | None = None
    ) -> tuple[Tensor, tuple[Tensor, Tensor]]:
        batch_size, seq_len, _ = x.shape
        device = x.device

        if init_state is None:
            h_t = torch.zeros(batch_size, self.hidden_size, device=device)
            c_t = torch.zeros(batch_size, self.hidden_size, device=device)
        else:
            h_t, c_t = init_state

        outputs: list[Tensor] = []
        for t in range(seq_len):
            h_t, c_t = self.cell(x[:, t, :], (h_t, c_t))
            outputs.append(h_t)
```

```
        return torch.stack(outputs, dim=1), (h_t, c_t)


# ======================================================================
# 3) التحقق العددي: هل تطبيقنا اليدوي يُطابق سلوك PyTorch المدمج منطقيًا؟
# ======================================================================
def sanity_check_shapes() -> None:
    batch, seq_len, input_size, hidden_size = 4, 10, 16, 32

    x = torch.randn(batch, seq_len, input_size)

    my_rnn = VanillaRNN(input_size, hidden_size)
    outputs, h_last = my_rnn(x)
    assert outputs.shape == (batch, seq_len, hidden_size)
    assert h_last.shape == (batch, hidden_size)
    print(f"✅ VanillaRNN من الصفر: outputs={outputs.shape}, h_last={h_last.shape}")

    my_lstm = LSTMFromScratch(input_size, hidden_size)
    outputs, (h_last, c_last) = my_lstm(x)
    assert outputs.shape == (batch, seq_len, hidden_size)
    print(f"✅ LSTM من الصفر: outputs={outputs.shape}, h_last={h_last.shape}, c_last={c_last.shape}")

# الوحدة المدمجة الرسمية في PyTorch -- للاستخدام الفعلي في الإنتاج
    official_lstm = nn.LSTM(input_size, hidden_size, batch_first=True)
    outputs_official, (h_n, c_n) = official_lstm(x)
    assert outputs_official.shape == (batch, seq_len, hidden_size)
    print(f"✅ nn.LSTM الرسمية: outputs={outputs_official.shape}")


if __name__ == "__main__":
    sanity_check_shapes()
```

💡 **لاحظ حيلة هندسية مهمة** في `LSTMCellFromScratch`: بدلًا من تعريف 4 طبقات `Linear` منفصلة لكل بوابة (forget, input, candidate, output)، جمعناها في طبقة واحدة `nn.Linear(combined_size, 4*hidden_size)` ثم قسّمناها بـ `.chunk(4, dim=1)`. هذا ليس مجرد اختصار في الكتابة؛ إنه **أسرع فعليًا** لأن عملية ضرب مصفوفة واحدة كبيرة على GPU أكفأ من أربع عمليات ضرب صغيرة منفصلة (استغلال أفضل للتوازي الحسابي). هذه بالضبط الطريقة التي تُطبَّق بها `nn.LSTM` الرسمية داخليًا.

3.9 استخدام LSTM/GRU في الإنتاج الفعلي: نموذج تصنيف نصوص كامل

الآن لنبنِ تطبيقًا عمليًا كاملًا: مصنّف مشاعر (Sentiment Classifier) باستخدام LSTM ثنائي الاتجاه (Bidirectional) متعدد الطبقات — وهو إعداد قياسي شائع جدًا في التطبيقات الحقيقية قبل عصر المحوّلات، وما زال مفيدًا في سياقات خفيفة الموارد اليوم.

```python
"""
lstm_sentiment_classifier.py
.Bidirectional Multi-layer LSTM مصنّف مشاعر كامل باستخدام
Python 3.13+ | PyTorch 2.x
"""
from __future__ import annotations

import torch
from torch import nn, Tensor
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

torch.manual_seed(42)


class BiLSTMSentimentClassifier(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        embedding_dim: int = 128,
        hidden_size: int = 256,
        num_layers: int = 2,
        num_classes: int = 3,       # مثال: سلبي / محايد / إيجابي
        pad_idx: int = 0,
        dropout: float = 0.3,
    ) -> None:
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=pad_idx)

        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=True,     # يقرأ الجملة من اليمين لليسار ومن اليسار لليمين معًا
            dropout=dropout if num_layers > 1 else 0.0,  # Dropout بين الطبقات فقط
        )

# ثنائي الاتجاه ⟹ حجم الإخراج = hidden_size * 2 (دمج الاتجاهين)
        self.classifier = nn.Sequential(
            nn.Linear(hidden_size * 2, hidden_size),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_size, num_classes),
        )

    def forward(self, input_ids: Tensor, lengths: Tensor) -> Tensor:
        """
        input_ids: (batch, seq_len) -- مُرمّزة ومُحشوّة (padded)
        lengths:   (batch,) -- الطول الحقيقي لكل جملة قبل الحشو
logits (batch, num_classes) :يُعيد
        """
        embedded = self.embedding(input_ids)  # (batch, seq_len, embedding_dim)
```

```
        # حيلة حاسمة للأداء والصحة: تجاهل خطوات الحشو (padding) في حساب LSTM
        packed = pack_padded_sequence(
            embedded, lengths.cpu(), batch_first=True, enforce_sorted=False
        )
        packed_output, (h_n, c_n) = self.lstm(packed)
        # لا نحتاج فك الحزمة هنا لأننا سنستخدم h_n مباشرة

        # h_n شكلها: (num_layers * num_directions, batch, hidden_size)
        # نأخذ آخر طبقة فقط، لكلا الاتجاهين، وندمجهما
        forward_last = h_n[-2, :, :]   # آخر حالة من الاتجاه الأمامي
        backward_last = h_n[-1, :, :]  # آخر حالة من الاتجاه الخلفي
        final_hidden = torch.cat([forward_last, backward_last], dim=1)

        return self.classifier(final_hidden)


def demo_training_step() -> None:
    """مثال توضيحي لخطوة تدريب واحدة كاملة."""
    vocab_size, batch_size, max_seq_len = 5000, 8, 20

    model = BiLSTMSentimentClassifier(vocab_size=vocab_size, num_classes=3)
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
    criterion = nn.CrossEntropyLoss()

    # بيانات وهمية للتوضيح: جمل بأطوال مختلفة ومُحشوّة بـ 0
    input_ids = torch.randint(1, vocab_size, (batch_size, max_seq_len))
    lengths = torch.randint(5, max_seq_len + 1, (batch_size,))
    labels = torch.randint(0, 3, (batch_size,))

    optimizer.zero_grad()
    logits = model(input_ids, lengths)
    loss = criterion(logits, labels)
    loss.backward()

    # حيلة لا غنى عنها مع RNN/LSTM: Gradient Clipping لمنع الانفجار
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)

    optimizer.step()
    print(f"Loss: {loss.item():.4f} | logits shape: {logits.shape}")


if __name__ == "__main__":
    demo_training_step()
```

💡 **لماذا `pack_padded_sequence` ضروري هنا وليس مجرد تحسين اختياري؟** عندما تُجمّع جملًا بأطوال مختلفة في Batch واحد، يجب حشوها (Padding) بنفس الطول لتتمكن من تكديسها في Tensor واحد. لكن دون `pack_padded_sequence`، ستُعالج LSTM رموز الحشو (عادة أصفار) **كأنها كلمات حقيقية**، فتُفسد الحالة المخفية بمعلومات زائفة لا معنى لها، وتُهدر حسابات على خطوات زمنية فارغة. `pack_padded_sequence` يُخبر PyTorch صراحة بالطول الحقيقي لكل تسلسل، فتتوقف المعالجة عند الطول الفعلي لكل جملة بالضبط — توفير حسابي ودقة أعلى معًا.

⚠️ **لماذا Gradient Clipping إلزامي شبه دائمًا مع RNN/LSTM تحديدًا؟** كما ناقشنا في القسم 3.4، حتى مع حلول LSTM/GRU الذكية لمشكلة التلاشي، تبقى مشكلة **انفجار** التدرّج واردة، خاصة في التسلسلات الطويلة جدًا أو معدلات تعلّم مرتفعة. لهذا السبب تجد `clip_grad_norm_` دائمًا تقريبًا في أي كود تدريب RNN/LSTM جاد، بخلاف شبكات التغذية الأمامية حيث هي أقل إلزامية (رغم أنها ممارسة جيدة بشكل عام كما رأينا في الفصل الثالث).

---

4. 🧪 مثال محلول خطوة بخطوة (End-to-End)

لنتتبّع **بالأرقام الفعلية** كيف تتدفّق المعلومة عبر خلية LSTM واحدة، لجملة قصيرة مُبسَّطة جدًا، بأبعاد صغيرة جدًا كي نستطيع الحساب يدويًا.

الإعداد: `hidden_size=2`، `input_size=2`. لنفترض قيمًا مُبسَّطة (وليست مُتعلَّمة فعليًا، فقط للتوضيح الحسابي):

الحالة الابتدائية: $h_0 = [0, 0]$ ، $C_0 = [0, 0]$

المدخل عند $t = 1$: $x_1 = [1.0, 0.5]$ (تمثيل مُبسّط لكلمة "الطالب")

لنفترض بعد تطبيق الأوزان أن القيم الخام (قبل التفعيل) للبوابات كانت:

البوابة	القيمة الخام	بعد sigmoid/tanh
(forget) f_1	$[2.0, -1.0]$	$\sigma \rightarrow [0.88, 0.27]$
(input) i_1	$[0.5, 1.5]$	$\sigma \rightarrow [0.62, 0.82]$
(candidate) $\tilde{C}_1$	$[0.8, -0.3]$	$\tanh \rightarrow [0.66, -0.29]$
(output) o_1	$[1.0, 1.0]$	$\sigma \rightarrow [0.73, 0.73]$

تحديث حالة الخلية:

$$C_1 = f_1 \odot C_0 + i_1 \odot \tilde{C}_1 = [0.88, 0.27] \odot [0, 0] + [0.62, 0.82] \odot [0.66, -0.29]$$

$$C_1 = [0, 0] + [0.409, -0.238] = [0.409, -0.238]$$

لاحظ أن C_0 كانت صفرًا (بداية الجملة)، لذا حدّ النسيان لم يُضف شيئًا هنا — هذا طبيعي في أول خطوة زمنية.

الحالة المخفية الجديدة:

$$h_1 = o_1 \odot \tanh(C_1) = [0.73, 0.73] \odot \tanh([0.409, -0.238])$$

$$= [0.73, 0.73] \odot [0.388, -0.234] = [0.283, -0.171]$$

الآن عند $t = 2$ (كلمة "نجح")، لنفترض بوابة النسيان أصبحت $f_2 = [0.95, 0.90]$ (الشبكة "قرّرت" الاحتفاظ بمعظم معلومات C_1 لأنها مهمة نحويًا — مثلًا معلومة أن الفاعل مفرد):

$$C_2 = f_2 \odot C_1 + i_2 \odot \tilde{C}_2 = [0.95, 0.90] \odot [0.409, -0.238] + (\text{مساهمة جديدة})$$

$$= [0.389, -0.214] + (\text{مساهمة جديدة})$$

لاحظ الأمر الجوهري هنا: حوالي 95% و90% من قيمة C_1 انتقلت بشكل **شبه خطي مباشر** إلى C_2 (لم تُسحق عبر tanh على المسار الرئيسي). هذا هو بالضبط آلية "الطريق السريع" التي شرحناها نظريًا — وهذا بالضبط ما يسمح لإشارة "الفاعل مفرد" بالبقاء محفوظة عبر عشرات الخطوات الزمنية اللاحقة حتى تصل لحظة حساب الفعل "نجح" في نهاية الجملة الطويلة.

الكود المقابل للتحقق من هذا الحساب رقميًا:

```
import torch

h0 = torch.zeros(1, 2)
c0 = torch.zeros(1, 2)

f1 = torch.sigmoid(torch.tensor([[2.0, -1.0]]))
i1 = torch.sigmoid(torch.tensor([[0.5, 1.5]]))
c_tilde1 = torch.tanh(torch.tensor([[0.8, -0.3]]))
o1 = torch.sigmoid(torch.tensor([[1.0, 1.0]]))

c1 = f1 * c0 + i1 * c_tilde1
h1 = o1 * torch.tanh(c1)

print(f"f1={f1}")           # tensor([[0.8808, 0.2689]])
print(f"i1={i1}")           # tensor([[0.6225, 0.8176]])
print(f"c_tilde1={c_tilde1}") # tensor([[0.6640, -0.2913]])
print(f"C1={c1}")           # tensor([[0.4133, -0.2382]])
print(f"h1={h1}")           # tensor([[0.2855, -0.1717]])
```

5. 💡 نصائح وحيل (Tips & Tricks)

1. **ابدأ دائمًا بـ Bidirectional عند توفّر كامل الجملة مسبقًا:** إذا كانت مهمتك تصنيفًا أو ترميزًا لكامل الجملة (وليس توليدًا تنبؤيًا للكلمة التالية حيث المستقبل غير متاح)، استخدم `bidirectional=True` دائمًا — فهم السياق من الاتجاهين يُحسّن الأداء بشكل شبه مضمون تقريبًا بتكلفة إضافية بسيطة.
2. **تهيئة بوابة النسيان بانحياز موجب:** ممارسة شائعة جدًا في LSTM المُحسَّن هي تهيئة تحيّز بوابة النسيان b_f بقيمة موجبة (مثل 1.0) بدلًا من الصفر، لدفع الشبكة في البداية نحو "تذكّر كل شيء" افتراضيًا، ثم تتعلّم تدريجيًا متى تنسى. هذا يُحسّن استقرار التدريب المبكر بشكل ملحوظ (Jozefowicz et al., 2015).
3. **استخدم `pack_padded_sequence` دائمًا مع دفعات متغيّرة الطول:** كما أوضحنا، تجاهلها يُفسد النتائج ويُهدر الحساب.
4. **Gradient Clipping بقيمة `max_norm` بين 1.0 و5.0** هو إعداد افتراضي آمن تقريبًا لكل تدريب RNN/LSTM/GRU.
5. **كدّس الطبقات (Stacking) بحذر:** طبقتان أو ثلاث طبقات LSTM عادة كافية لمعظم المهام العملية؛ أكثر من ذلك نادرًا ما يُحسّن الأداء بشكل ملموس ويزيد كلفة التدريب وخطر الإفراط في المطابقة.
6. **راقب معيار التدرّج (Gradient Norm) أثناء التدريب:** إذا رأيت قيمته تقترب من الصفر بسرعة، فأنت تواجه تلاشي التدرّج فعليًا؛ إذا قفزت لقيم ضخمة فجأة (`NaN` أحيانًا)، فأنت تواجه انفجارًا — وهذا تشخيص عملي مباشر لمشكلات التدريب في RNN.

---

6. ⚠️ أخطاء شائعة ومصائد

❌ الخطأ 1: استخدام `nn.LSTM` دون `batch_first=True` ثم الخلط بين الأبعاد

```
lstm = nn.LSTM(input_size=128, hidden_size=256)  # ❌ الافتراضي: (seq_len, batch, features)!
x = torch.randn(32, 50, 128)  # تتوقع (batch, seq_len, features) لكن هذا ليس الافتراضي
output, _ = lstm(x)  # سيعمل بدون خطأ لكن بترتيب أبعاد خاطئ تمامًا! خطأ صامت خطير
```

✅ **الصحيح:** احرص دائمًا على تحديد `batch_first=True` صراحة، فهو يتوافق مع ترتيب الأبعاد الأكثر شيوعًا وبديهية `(batch, seq_len, features)` المستخدم في بقية مكتبة PyTorch:

```
lstm = nn.LSTM(input_size=128, hidden_size=256, batch_first=True)  # ✅
```

❌ الخطأ 2: نسيان أن `bidirectional=True` يُضاعف حجم الإخراج

```
lstm = nn.LSTM(128, 256, bidirectional=True, batch_first=True)
classifier = nn.Linear(256, 10)  # ❌ خطأ في الأبعاد! الإخراج الفعلي حجمه 512 وليس 256
```

✅ **الصحيح:** تذكّر دائمًا أن `hidden_size * 2` هو الحجم الفعلي للإخراج عندما يكون ثنائي الاتجاه:

```
classifier = nn.Linear(256 * 2, 10)  # ✅
```

❌ الخطأ 3: الخلط بين `h_n` الأخيرة و`outputs` الكاملة عند الحاجة لمعلومات من كل الخطوات

```
outputs, (h_n, c_n) = lstm(x)
# إذا احتجت الانتباه (Attention) على كل كلمة، استخدام h_n فقط خطأ شائع
result = classifier(h_n[-1])  # ❌ إذا كنت تحتاج معلومات من كل الخطوات الزمنية، فقدتها هنا
```

✅ **الصحيح:** إذا كنت ستبني لاحقًا آلية انتباه فوق مخرجات LSTM (كما كان شائعًا قبل عصر المحوّلات الكامل، في نماذج مثل Bahdanau Attention التي سنراها في الفصل القادم)، يجب الاحتفاظ بـ `outputs` الكاملة (كل الخطوات الزمنية)، لا فقط `h_n` الأخيرة.

❌ الخطأ 4: تجاهل تأثير التهيئة العشوائية على استقرار تدريب RNN

RNN وLSTM أكثر حساسية للتهيئة العشوائية مقارنة بالشبكات التلافيفية مثلًا، بسبب تكرار نفس الأوزان عبر الزمن. تهيئة سيّئة (مثل انحراف معياري كبير جدًا) قد تُسبب انفجار تدرّج من الخطوة الأولى. ✅ اعتمد على التهيئات الافتراضية المدروسة في PyTorch (Xavier/Orthogonal) لمصفوفات `W_hh` تحديدًا شائعة جدًا وموصى بها في الأدبيات لأنها تحافظ على معيار (norm) المتجه عبر الضرب المتكرر).

---

7. ⚖️ جدول مقارنة

المعيار	Vanilla RNN	LSTM	GRU
عدد البوابات	0	3 (forget, input, output)	2 (reset, update)
حالة منفصلة للذاكرة طويلة المدى	لا	نعم (C_t Cell State)	لا (مدمجة في h_t)
عدد المعاملات (تقريبيًا)	$O(d^2)$	$\approx 4 \times O(d^2)$	$\approx 3 \times O(d^2)$
مقاومة تلاشي التدرّج	ضعيفة جدًا	قوية	قوية
سرعة التدريب	الأسرع (لكن عديم الفائدة لتسلسلات طويلة)	الأبطأ نسبيًا	أسرع من LSTM قليلًا
الأداء على تسلسلات طويلة (>50 كلمة)	فشل شبه مضمون	جيد	جيد
الأداء على بيانات صغيرة	—	قد يُفرط في المطابقة	أفضل قليلًا (معاملات أقل)
قابلية التوازي على GPU	ضعيفة (تسلسلية إجبارية)	ضعيفة (تسلسلية إجبارية)	ضعيفة (تسلسلية إجبارية)

المعيار	Vanilla RNN	LSTM	GRU
الاستخدام اليوم	نادر جدًا (تعليمي أساسًا)	ما زال يُستخدم في سياقات خفيفة الموارد	ما زال يُستخدم، خاصة Edge Devices

---

8. ✅ قائمة التحقق من أفضل الممارسات

- [ ] هل استخدمت `batch_first=True` صراحة لتجنّب الالتباس في ترتيب الأبعاد؟
- [ ] هل استخدمت `pack_padded_sequence` عند التعامل مع دفعات بأطوال جمل متغيّرة؟
- [ ] هل طبّقت `Gradient Clipping` (`clip_grad_norm_`) في حلقة التدريب؟
- [ ] هل ضبطت حجم طبقة التصنيف بشكل صحيح إذا كنت تستخدم `bidirectional=True` (الحجم المضاعف)؟
- [ ] هل اخترت LSTM أو GRU بدلًا من Vanilla RNN لأي مهمة تتجاوز بضع كلمات في التسلسل؟
- [ ] هل راقبت معيار التدرّج (Gradient Norm) أثناء التدريب المبكر للتأكد من عدم وجود تلاشي/انفجار؟
- [ ] هل فكّرت في تهيئة بوابة النسيان بانحياز موجب لتحسين استقرار التدريب المبكر؟
- [ ] عند التعامل مع تسلسلات طويلة جدًا (>200 كلمة)، هل فكّرت في أن LSTM/GRU نفسها قد لا تكون كافية فعليًا، وأن المحوّلات قد تكون خيارًا أفضل؟

---

9. 🎯 أسئلة مقابلات شخصية + إجابات نموذجية

س1: اشرح بالتفصيل لماذا تحدث مشكلة تلاشي التدرّج في RNN البسيطة، وكيف تحلّها LSTM رياضيًا؟

> في RNN البسيطة، حساب التدرّج عبر الزمن (BPTT) يتطلب ضرب مصفوفة W_{hh} في نفسها بشكل متكرر n مرة (عدد الخطوات الزمنية الفاصلة)، مضروبة أيضًا بمشتقات tanh المتتالية التي قيمتها القصوى 1 دائمًا. إذا كانت أكبر قيمة ذاتية لـ W_{hh} أقل من 1، فإن هذا الضرب المتكرر يتلاشى أسّيًا نحو الصفر مع ازدياد n، مما يعني أن التدرّج القادم من خطوات بعيدة في الماضي يُصبح ضئيلًا جدًا فلا يُحدِث تحديثًا فعليًا للأوزان. LSTM تحلّ هذا عبر إدخال حالة خلية C_t منفصلة تتحدّث بمعادلة خطية بسيطة $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$، فتُصبح مشتقة $\partial C_t / \partial C_{t-1} = f_t$ فقط — بلا أي ضرب متكرر بمصفوفة وزن ثابتة. إذا تعلّمت بوابة النسيان أن تكون قريبة من 1، يتدفّق التدرّج بثبات نسبي عبر عدد كبير من الخطوات الزمنية.

س2: لماذا نستخدم `pack_padded_sequence` عند تدريب LSTM على دفعات (batches) من جمل بأطوال مختلفة؟

لأن الجمل المختلفة الطول يجب حشوها (Padding) بنفس الطول الأقصى في الدفعة لتُجمَّع في Tensor واحد. دون `pack_padded_sequence`، ستُعالج LSTM رموز الحشو (عادة أصفار) كأنها كلمات حقيقية فتُدخل معلومات زائفة في الحالة المخفية، وتُهدر حسابًا على خطوات زمنية لا معنى لها، مما يُفسد جودة النموذج ويُبطئ التدريب. `pack_padded_sequence` يُعلم PyTorch بالطول الحقيقي لكل جملة فيتوقف الحساب عند الحد الصحيح لكل عيّنة.

س3: ما الفرق بين LSTM وGRU من حيث البنية، ومتى تختار أحدهما؟

LSTM تستخدم ثلاث بوابات (forget, input, output) بالإضافة إلى حالة خلية (Cell State) منفصلة عن الحالة المخفية، بينما GRU تستخدم بوابتين فقط (reset, update) وتدمج الذاكرة طويلة وقصيرة المدى في متغير واحد h_t. هذا يجعل GRU أخفّ وزنًا (معاملات أقل، تدريب أسرع)، بينما LSTM أكثر تعبيرية بفضل التحكم المنفصل الأدق في الذاكرة. تجريبيًا أداؤهما متقارب في معظم المهام؛ أفضّل GRU عند محدودية البيانات أو الحاجة لسرعة تدريب أعلى، وأفضّل LSTM عند التعامل مع تبعيات طويلة المدى معقدة جدًا حيث التحكم الدقيق الإضافي قد يُحدث فرقًا.

س4: ما القيد البنيوي الأساسي المشترك بين RNN وLSTM وGRU الذي لم تحلّه أي منها، وكيف حلّته المحوّلات لاحقًا؟

القيد هو **المعالجة التسلسلية الإجبارية** (Sequential Processing): حساب h_t يتطلب بالضرورة أن تكون h_{t-1} قد حُسبت أولًا، مما يمنع أي توازي حقيقي عبر خطوات الزمن المختلفة أثناء التدريب على GPU — حتى لو كان لديك آلاف وحدات المعالجة المتوازية، ستبقى مضطرًا لحساب خطوة واحدة تلو الأخرى. هذا القيد يُبطئ التدريب بشكل كبير خاصة على تسلسلات طويلة وبيانات ضخمة. المحوّلات (Transformers) حلّت هذا جذريًا عبر آلية Self-Attention التي تسمح لكل موضع في التسلسل برؤية كل المواضع الأخرى **دفعة واحدة ومتوازية بالكامل**، دون أي اعتماد تسلسلي إجباري أثناء التدريب — وهذا هو السبب الجوهري الذي مكّن المحوّلات من التوسّع لمليارات المعاملات وتريليونات التوكنز بكفاءة غير ممكنة مع RNN/LSTM.

س5: لماذا نستخدم LSTM ثنائي الاتجاه (Bidirectional) في مهام التصنيف، ولماذا لا نستخدمه في توليد النصوص؟

في مهام التصنيف أو الترميز (Encoding)، كامل الجملة متاحة لنا مسبقًا، فالاستفادة من السياق في كلا الاتجاهين (قبل وبعد كل كلمة) تُحسّن جودة التمثيل — مثلًا فهم كلمة "بنك" يتطلب أحيانًا رؤية ما بعدها لتحديد المعنى الصحيح. لكن في توليد النصوص (Autoregressive Generation)، نحن نتنبأ بالكلمة التالية خطوة بخطوة، والمستقبل (الكلمات التي لم تُولَّد بعد) غير موجود أصلًا وقت التنبؤ — فاستخدام معلومات من

> "المستقبل" في هذه الحالة يُمثّل تسريبًا للمعلومة (Data Leakage) يجعل التدريب عديم المعنى عمليًا ومستحيل التطبيق وقت الاستدلال الفعلي.

---

10. 🧪 تمرين عملي (Mini-Project)

المهمة: بناء نموذج لغوي بسيط على مستوى الحرف (Character-Level Language Model) باستخدام LSTM

الهدف: درّب LSTM على التنبؤ بالحرف التالي في نص عربي، حرفًا بحرف، ثم استخدمه لتوليد نص جديد تلقائيًا (نفس الفكرة المُستخدمة في الفصل الرابع لتوضيح Overfitting، لكن هنا بـ LSTM بدلًا من نموذج بسيط).

هيكل بداية الكود:

```
"""
char_level_lstm_generator.py
.LSTM نموذج لغوي على مستوى الحرف لتوليد نص عربي باستخدام
"""
from __future__ import annotations

import torch
from torch import nn, Tensor
import torch.nn.functional as F

torch.manual_seed(42)


class CharLSTM(nn.Module):
    def __init__(
        self, vocab_size: int, embedding_dim: int = 64, hidden_size: int = 256, num_layers: int = 2
    ) -> None:
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(
            embedding_dim, hidden_size, num_layers=num_layers, batch_first=True
        )
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(
        self, x: Tensor, state: tuple[Tensor, Tensor] | None = None
    ) -> tuple[Tensor, tuple[Tensor, Tensor]]:
        embedded = self.embedding(x)
        output, new_state = self.lstm(embedded, state)
        logits = self.fc(output)
        return logits, new_state


# TODO 1: حمّل نصًا عربيًا طويلًا (رواية، مجموعة مقالات) وابنِ قاموس حرف↔رقم
```

```python
def build_char_vocab(text: str) -> tuple[dict[str, int], dict[int, str]]:
    raise NotImplementedError


# TODO 2: قسّم النص إلى تسلسلات تدريب بطول ثابت (مثلًا 100 حرف) بأسلوب Sliding Window
def create_training_sequences(
    encoded_text: list[int], seq_length: int = 100
) -> list[tuple[list[int], list[int]]]:
"""كل عيّنة: (تسلسل مدخل، نفس التسلسل مُزاح بخطوة واحدة = الهدف)"""
    raise NotImplementedError


# TODO 3: اكتب حلقة تدريب كاملة (استلهم من full_training_loop.py في الفصل 3)


# TODO 4: اكتب دالة التوليد بأسلوب Autoregressive مع Temperature Sampling
@torch.no_grad()
def generate_text(
    model: CharLSTM,
    start_text: str,
    char2idx: dict[str, int],
    idx2char: dict[int, str],
    length: int = 300,
    temperature: float = 0.8,
) -> str:
    """
يبدأ من start_text، ويُولّد حرفًا تلو الآخر، معيدًا تغذية الحرف المُولَّد
كمدخل للخطوة التالية (Autoregressive)، مع التحكم بالعشوائية عبر temperature.
    """
    raise NotImplementedError

# تحدٍّ إضافي: قارن جودة النص المُولَّد بين temperature=0.3 (محافظ)
# و temperature=1.2 (جريء وعشوائي أكثر)، وناقش المقايضة بينهما.
```

معيار النجاح: يجب أن ينتج النموذج، بعد تدريب كافٍ، نصًا يحترم بنية الكلمات العربية الأساسية (مسافات في أماكن معقولة، تسلسلات حروف تُشبه كلمات حقيقية)، حتى لو لم يكن المعنى الدلالي متماسكًا بالكامل — وهذا بالضبط ما يُظهر أن LSTM تعلّمت نمط اللغة الإحصائي على مستوى الحرف.

---

11. 🔗 جسر للفصل التالي

لقد وصلنا الآن لفهم عميق لكيفية معالجة RNN وLSTM وGRU للتسلسلات، وكيف تحافظ البوابات الذكية على المعلومة عبر الزمن. لكن بقيت مشكلة لم نحلّها بعد، حتى مع كل حيل LSTM: عندما تريد شبكة **ترجمة آلية** (مثلًا من العربية إلى الإنجليزية)، تواجه معمارية الترميز-فك الترميز (Encoder-Decoder) عنق زجاجة خطير — **كل** معلومات الجملة المصدر، بغض النظر عن طولها، يجب ضغطها في **متجه واحد ثابت الحجم** (آخر حالة مخفية للمُرمِّز)، ثم يُطلب من فك المُرمِّز أن يُعيد بناء الجملة كاملة انطلاقًا من هذا المتجه الوحيد فقط!

تخيّل أنك تحاول تلخيص رواية كاملة من 300 صفحة في جملة واحدة فقط، ثم تُطلب من شخص آخر أن يُعيد كتابة الرواية بالكامل معتمدًا فقط على تلك الجملة الواحدة. هل تتوقع نتيجة دقيقة؟ بالطبع لا — وهذا بالضبط ما يحدث في الجمل الطويلة: الأداء ينهار بشكل حاد كلما طالت الجملة المصدر.

في **الفصل التاسع**، سنكتشف كيف حلّ الباحثون (Bahdanau وآخرون، 2015؛ ثم Luong وآخرون) هذه المشكلة بفكرة ثورية: بدلًا من إجبار فك المُرمِّز على الاعتماد على متجه واحد ثابت، لماذا لا نسمح له "بالنظر مجددًا" (Attend) إلى **كل** الحالات المخفية للمُرمِّز في كل خطوة توليد، ويُحدّد ديناميكيًا أيّ أجزاء من الجملة المصدر هي الأكثر صلة الآن؟ هذه الفكرة البسيطة في جوهرها ستكون البذرة المباشرة التي ستُثمر، بعد تطوّرات متتالية، إلى آلية **Self-Attention** الكاملة التي درسناها بعمق رياضي شامل في الفصل العاشر. الدائرة ستكتمل: ستفهم أخيرًا *من أين* جاءت فكرة الانتباه، لا فقط *كيف* تعمل رياضيًا.

---

12. 📚 مراجع للتعمّق

المرجع	الوصف
Hochreiter & Schmidhuber, 1997 — *"Long Short-Term Memory"*	الورقة الأصلية التي قدّمت LSTM بكامل بنيتها
Cho et al., 2014 — *"Learning Phrase Representations using RNN Encoder-Decoder..."*	الورقة الأصلية التي قدّمت GRU ومعمارية Encoder-Decoder
Chung et al., 2014 — *"Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling"*	مقارنة تجريبية شاملة بين LSTM وGRU
Bengio, Simard & Frasconi, 1994 — *"Learning long-term dependencies with gradient descent is difficult"*	أول تحليل رياضي رسمي لمشكلة تلاشي/انفجار التدرّج في RNN
Pascanu, Mikolov & Bengio, 2013 — *"On the difficulty of training Recurrent Neural Networks"*	تحليل عميق إضافي + اقتراح Gradient Clipping كحل عملي
Jozefowicz, Zaremba & Sutskever, 2015 — *"An Empirical Exploration of Recurrent Network Architectures"*	دراسة واسعة لمتغيرات LSTM، بما فيها فائدة تهيئة بوابة النسيان بانحياز موجب
Graves, 2013 — *"Generating Sequences With Recurrent Neural Networks"*	تطبيقات توليدية رائدة لـ LSTM (أساس تمرين هذا الفصل)
Christopher Olah — *"Understanding LSTM Networks"* (مدونة)	أفضل شرح بصري وبديهي لآلية عمل LSTM متاح مجانًا على الإنترنت

==========